openstaadpy

Contents

- Module contents

Module contents

`openstaadpy.os_analytical.connect(filePath: str = None)` [source]

Connect to STAAD.Pro via OpenSTAAD Python API. Returns an OSRoot object for all STAAD operations.

Root

- OSRoot
  - OSRoot.Analyze()
  - OSRoot.AnalyzeEx()
  - OSRoot.AnalyzeModel()
  - OSRoot.CloseSTAADFile()
  - OSRoot.GetAnalysisStatus()
  - OSRoot.GetApplicationVersion()
  - OSRoot.GetBaseUnit()
  - OSRoot.GetErrorMessage()
  - OSRoot.GetFullJobInfo()
  - OSRoot.GetInputUnitForForce()
  - OSRoot.GetInputUnitForLength()
  - OSRoot.GetMainWindowHandle()
  - OSRoot.GetProcessHandle()
  - OSRoot.GetProcessId()

- OSRoot.GetSTAADFile()
- OSRoot.GetSTAADFileFolder()
- OSRoot.GetShortJobInfo()
- OSRoot.IsAnalyzing()
- OSRoot.IsPhysicalModel()
- OSRoot.NewSTAADFile()
- OSRoot.OpenSTAADFile()
- OSRoot.Quit()
- OSRoot.SaveModel()
- OSRoot.SetFullJobInfo()
- OSRoot.SetInputUnitForForce()
- OSRoot.SetInputUnitForLength()
- OSRoot.SetInputUnits()
- OSRoot.SetShortJobInfo()
- OSRoot.SetSilentMode()
- OSRoot.ShowApplication()
- OSRoot.UpdateStructure()
- OSRoot.__init__()

- getActiveObject()

Geometry

- OSGeometry
  - OSGeometry.AddBeam()
  - OSGeometry.AddCircularRegionToSurface()
  - OSGeometry.AddDensityLineToSurface()
  - OSGeometry.AddDensityPointToSurface()
  - OSGeometry.AddMultipleBeams()
  - OSGeometry.AddMultipleNodes()
  - OSGeometry.AddMultiplePlates()
  - OSGeometry.AddMultipleSolids()
  - OSGeometry.AddNode()
  - OSGeometry.AddParametricSurfaceToModel()

OSGeometry.AddPlate()

OSGeometry.AddPolygonalRegionToSurface()

OSGeometry.AddSolid()

OSGeometry.BreakBeamsAtSpecificNodes()

OSGeometry.ClearMemberSelection()

OSGeometry.ClearNodeSelection()

OSGeometry.ClearPhysicalMemberSelection()

OSGeometry.ClearPlateSelection()

OSGeometry.ClearSolidSelection()

OSGeometry.CommitParametricSurfaceMesh()

OSGeometry.CreateBeam()

OSGeometry.CreateGroup()

OSGeometry.CreateGroupEx()

OSGeometry.CreateMultipleBeams()

OSGeometry.CreateMultipleNodes()

OSGeometry.CreateMultiplePlates()

OSGeometry.CreateNode()

OSGeometry.CreatePhysicalMember()

OSGeometry.CreatePlate()

OSGeometry.CreateSolid()

OSGeometry.DefineParametricSurface()

OSGeometry.DeleteBeam()

OSGeometry.DeleteGroup()

OSGeometry.DeleteNode()

OSGeometry.DeletePhysicalMember()

OSGeometry.DeletePlate()

OSGeometry.DeleteSolid()

OSGeometry.DoTranslationalRepeat()

OSGeometry.GetAnalyticalMemberCountForPhysicalMember()

OSGeometry.GetAnalyticalMembersForPhysicalMember()

OSGeometry.GetAreaOfPlates()

OSGeometry.GetBeamLength()

OSGeometry.GetBeamList()

OSGeometry.GetBeamsConnectedAtNode()

OSGeometry.GetCountOfBreakableBeamsAtSpecificNodes()

OSGeometry.GetFlagForHiddenEntities()

OSGeometry.GetGeneratedQuadPanelIncidences()

OSGeometry.GetGroupCount()

OSGeometry.GetGroupCountAll()

OSGeometry.GetGroupEntities()

OSGeometry.GetGroupEntityCount()

OSGeometry.GetGroupNames()

OSGeometry.GetIntersectBeamsCount()

OSGeometry.GetLastBeamNo()

OSGeometry.GetLastNodeNo()

OSGeometry.GetLastPhysicalMemberNo()

OSGeometry.GetLastPlateNo()

OSGeometry.GetLastSolidNo()

OSGeometry.GetMemberCount()

OSGeometry.GetMemberIncidence()

OSGeometry.GetMemberIncidence_CIS2()

OSGeometry.GetMemberUniqueID()

OSGeometry.GetNoOfBeamsConnectedAtNode()

OSGeometry.GetNoOfGeneratedQuadPanels()

OSGeometry.GetNoOfSelectedBeams()

OSGeometry.GetNoOfSelectedNodes()

OSGeometry.GetNoOfSelectedPhysicalMembers()

OSGeometry.GetNoOfSelectedPlates()

OSGeometry.GetNoOfSelectedSolids()

OSGeometry.GetNodeCoordinates()

OSGeometry.GetNodeCount()

OSGeometry.GetNodeDistance()

OSGeometry.GetNodeIncidence()

OSGeometry.GetNodeIncidence_CIS2()

OSGeometry.GetNodeList()

OSGeometry.GetNodeNumber()

OSGeometry.GetNodeUniqueID()

OSGeometry.GetPID()

OSGeometry.GetPMemberCount()

OSGeometry.GetParametricSurfaceCount()

OSGeometry.GetParametricSurfaceInfo()

OSGeometry.GetParametricSurfaceInfoEx()

OSGeometry.GetParametricSurfaceMeshData()

OSGeometry.GetParametricSurfaceMeshInfo()

OSGeometry.GetParametricSurfaceSubType()

OSGeometry.GetParametricSurfaceUniqueID()

OSGeometry.GetPhysicalMemberCount()

OSGeometry.GetPhysicalMemberList()

OSGeometry.GetPhysicalMemberUniqueID()

OSGeometry.GetPlateCount()

OSGeometry.GetPlateIncidence()

OSGeometry.GetPlateIncidence_CIS2()

OSGeometry.GetPlateList()

OSGeometry.GetPlateNodeCount()

OSGeometry.GetPlateUniqueID()

OSGeometry.GetSelectedBeams()

OSGeometry.GetSelectedNodes()

OSGeometry.GetSelectedPhysicalMembers()

OSGeometry.GetSelectedPlates()

OSGeometry.GetSelectedSolids()

OSGeometry.GetSolidCount()

OSGeometry.GetSolidIncidence()

OSGeometry.GetSolidIncidence_CIS2()

OSGeometry.GetSolidList()

OSGeometry.GetSolidUniqueID()

OSGeometry.IntersectBeams()

OSGeometry.IsBeam()

OSGeometry.IsColumn()

OSGeometry.IsOrphanNode()

OSGeometry.IsZUp()

OSGeometry.MergeBeams()

OSGeometry.MergeNodes()

OSGeometry.RemoveParametricSurfaceMesh()

OSGeometry.RenumberBeam()

OSGeometry.SelectBeam()

OSGeometry.SelectMultipleBeams()

OSGeometry.SelectMultipleNodes()

OSGeometry.SelectMultiplePhysicalMembers()

OSGeometry.SelectMultiplePlates()

OSGeometry.SelectMultipleSolids()

OSGeometry.SelectNode()

OSGeometry.SelectPhysicalMember()

OSGeometry.SelectPlate()

OSGeometry.SelectSolid()

OSGeometry.SetCheckForIdenticalEntity()

OSGeometry.SetFlagForHiddenEntities()

OSGeometry.SetMemberUniqueID()

OSGeometry.SetNodeCoordinate()

OSGeometry.SetNodeUniqueID()

OSGeometry.SetPID()

OSGeometry.SetParametricSurfaceSubType()

OSGeometry.SetParametricSurfaceUniqueID()

OSGeometry.SetPhysicalMemberUniqueID()

OSGeometry.SetPlateUniqueID()

- OSGeometry.SetSolidUniqueID()
- OSGeometry.SplitBeam()
- OSGeometry.SplitBeamInEqlParts()
- OSGeometry.UpdateGroup()
- OSGeometry.__init__()

Load

- OSLoad
  - OSLoad.AddAutoCombinationRepeat()
  - OSLoad.AddAutoLoadCombinations()
  - OSLoad.AddDirectAnalysisDefinitionParameter()
  - OSLoad.AddElementHydrostaticPressure()
  - OSLoad.AddElementPressure()
  - OSLoad.AddElementTrapPressureEx()
  - OSLoad.AddLoadAndFactorToCombination()
  - OSLoad.AddLoadCasesToEnvelop()
  - OSLoad.AddMemberAreaLoad()
  - OSLoad.AddMemberConcForce()
  - OSLoad.AddMemberConcMoment()
  - OSLoad.AddMemberFixedEnd()
  - OSLoad.AddMemberFloorLoad()
  - OSLoad.AddMemberFloorLoadEx()
  - OSLoad.AddMemberLinearVari()
  - OSLoad.AddMemberTrapezoidal()
  - OSLoad.AddMemberUniformForce()
  - OSLoad.AddMemberUniformMoment()
  - OSLoad.AddNodalLoad()
  - OSLoad.AddNotionalLoad()
  - OSLoad.AddRSLoad()
  - OSLoad.AddReferenceLoad()
  - OSLoad.AddRepeatLoad()
  - OSLoad.AddResponseSpectrumLoadEx()

OSLoad.AddSeismicDefElementWeight()

OSLoad.AddSeismicDefFloorWeight()

OSLoad.AddSeismicDefJointWeight()

OSLoad.AddSeismicDefMemberWeight()

OSLoad.AddSeismicDefSelfWeight()

OSLoad.AddSeismicDefWallArea()

OSLoad.AddSeismicDefinition()

OSLoad.AddSeismicLoad()

OSLoad.AddSelfWeightInXYZ()

OSLoad.AddSelfWeightInXYZToGeometry()

OSLoad.AddStrainLoad()

OSLoad.AddSupportDisplacement()

OSLoad.AddTemperatureLoad()

OSLoad.AddWindDefinition()

OSLoad.AddWindDefinitionASCE7Parameters()

OSLoad.AddWindExposure()

OSLoad.AddWindIntensity()

OSLoad.AddWindLoad()

OSLoad.BeginLoadMerging()

OSLoad.ClearPrimaryLoadCase()

OSLoad.ClearReferenceLoadCase()

OSLoad.ComputeWallWindPressureProfile()

OSLoad.ComputeWallWindPressureProfileASCE72016()

OSLoad.CreateLoadEnvelop()

OSLoad.CreateLoadList()

OSLoad.CreateNewLoadCombination()

OSLoad.CreateNewPrimaryLoad()

OSLoad.CreateNewPrimaryLoadEx()

OSLoad.CreateNewPrimaryLoadEx2()

OSLoad.CreateNewReferenceLoad()

OSLoad.DeleteDirectAnalysisDefinition()

OSLoad.DeleteDirectAnalysisDefinitionParameter()

OSLoad.DeleteLoadEnvelop()

OSLoad.DeleteLoadList()

OSLoad.DeletePrimaryLoadCases()

OSLoad.DeleteReferenceLoadCases()

OSLoad.DeleteWindDefinition()

OSLoad.EndLoadMerging()

OSLoad.GetActiveLoad()

OSLoad.GetAssignmentListForLoadType()

OSLoad.GetAttribute()

OSLoad.GetBeamCountAtFloor()

OSLoad.GetConcForceCount()

OSLoad.GetConcForces()

OSLoad.GetConcMomentCount()

OSLoad.GetConcMoments()

OSLoad.GetElementConcLoadCount()

OSLoad.GetElementConcLoads()

OSLoad.GetElementLoadInfo()

OSLoad.GetElementPressureLoadCount()

OSLoad.GetElementPressureLoads()

OSLoad.GetEnvelopeCount()

OSLoad.GetEnvelopeIDs()

OSLoad.GetInfluenceArea()

OSLoad.GetLinearVaryingLoadCount()

OSLoad.GetLinearVaryingLoads()

OSLoad.GetListSizeForLoadType()

OSLoad.GetLoadAndFactorForCombination()

OSLoad.GetLoadCaseTitle()

OSLoad.GetLoadCombinationCaseCount()

OSLoad.GetLoadCombinationCaseNumbers()

OSLoad.GetLoadCountInLoadList()

OSLoad.GetLoadEnvelopeDetails()

OSLoad.GetLoadItemType()

OSLoad.GetLoadItemsCount()

OSLoad.GetLoadListCount()

OSLoad.GetLoadListfromLoadEnvelope()

OSLoad.GetLoadType()

OSLoad.GetLoadTypeCount()

OSLoad.GetLoadsInLoadList()

OSLoad.GetMemberLoadInfo()

OSLoad.GetNoLoadFactorDirectionInNotionalLoad()

OSLoad.GetNoLoadFactorInRepeatLoad()

OSLoad.GetNoOfLoadAndFactorPairsForCombination()

OSLoad.GetNoOfSetsInReferenceLoad()

OSLoad.GetNodalLoadCount()

OSLoad.GetNodalLoadInfo()

OSLoad.GetNodalLoads()

OSLoad.GetNotionalLoadByIndex()

OSLoad.GetNotionalLoadCount()

OSLoad.GetPrimaryLoadCaseCount()

OSLoad.GetPrimaryLoadCaseNumbers()

OSLoad.GetReferenceLoadByIndex()

OSLoad.GetReferenceLoadCaseCount()

OSLoad.GetReferenceLoadCaseNumbers()

OSLoad.GetReferenceLoadCaseTitle()

OSLoad.GetReferenceLoadCount()

OSLoad.GetReferenceLoadType()

OSLoad.GetRepeatLoadByIndex()

OSLoad.GetRepeatLoadCount()

OSLoad.GetTrapLoadCount()

OSLoad.GetTrapLoads()

OSLoad.GetUDLLoadCount()

- OSLoad.GetUDLLoads()
- OSLoad.GetUNIMomentCount()
- OSLoad.GetUNIMoments()
- OSLoad.IsCombinationCase()
- OSLoad.IsDynamicLoadIncluded()
- OSLoad.MergeLoadsOnBeam()
- OSLoad.ModifySeismicDefinitionParams()
- OSLoad.RemoveAttribute()
- OSLoad.RemoveLoadCasesFromEnvelop()
- OSLoad.SetASDLoadAttribute()
- OSLoad.SetLSDLoadAttribute()
- OSLoad.SetLoadActive()
- OSLoad.SetLoadType()
- OSLoad.SetReferenceLoadActive()
- OSLoad.SplitLoadsOnBeam()
- OSLoad.__init__()

Property

- OSProperty
  - OSProperty.AddControlDependentRelation()
  - OSProperty.AddUPTPropertyANGLE()
  - OSProperty.AddUPTPropertyCHANNEL()
  - OSProperty.AddUPTPropertyDOUBLEANGLE()
  - OSProperty.AddUPTPropertyGENERAL()
  - OSProperty.AddUPTPropertyISECTION()
  - OSProperty.AddUPTPropertyPIPE()
  - OSProperty.AddUPTPropertyPRISMATIC()
  - OSProperty.AddUPTPropertyTEE()
  - OSProperty.AddUPTPropertyTUBE()
  - OSProperty.AddUPTPropertyWIDEFLANGE()
  - OSProperty.AddUPTPropertyWIDEFLANGECOMPOSITE()
  - OSProperty.AddUPTPropertyWIDEFLANGEUNEQUAL()

OSProperty.AssignBeamProperty()

OSProperty.AssignBetaAngle()

OSProperty.AssignElementAttribute()

OSProperty.AssignElementSpecToPlate()

OSProperty.AssignMaterialToMember()

OSProperty.AssignMaterialToPlate()

OSProperty.AssignMaterialToSolid()

OSProperty.AssignMemberAttribute()

OSProperty.AssignMemberSpecToBeam()

OSProperty.AssignPlateThickness()

OSProperty.CreateAnglePropertyFromTable()

OSProperty.CreateAssignProfileProperty()

OSProperty.CreateBeamPropertyFromTable()

OSProperty.CreateBeamPropertyFromTableComposite()

OSProperty.CreateBeamPropertyFromTableEx()

OSProperty.CreateBeamPropertyFromTableWithCoverPlates()

OSProperty.CreateChannelPropertyFromTable()

OSProperty.CreateElementAttribute()

OSProperty.CreateElementIgnoreInplaneRotnSpec()

OSProperty.CreateElementLocalZOffsetSpec()

OSProperty.CreateElementNodeReleaseSpec()

OSProperty.CreateElementOffsetSpec()

OSProperty.CreateElementPlaneStressSpec()

OSProperty.CreateIsotropicMaterialAluminum()

OSProperty.CreateIsotropicMaterialConcrete()

OSProperty.CreateIsotropicMaterialProperties()

OSProperty.CreateIsotropicMaterialPropertiesEx()

OSProperty.CreateIsotropicMaterialSteel()

OSProperty.CreateIsotropicMaterialTimber()

OSProperty.CreateMemberAttribute()

OSProperty.CreateMemberCableSpec()

OSProperty.CreateMemberCableSpecEx()

OSProperty.CreateMemberCompressionSpec()

OSProperty.CreateMemberFireProofingSpec()

OSProperty.CreateMemberIgnoreStiffSpec()

OSProperty.CreateMemberInactiveSpec()

OSProperty.CreateMemberOffsetSpec()

OSProperty.CreateMemberPartialReleaseSpec()

OSProperty.CreateMemberReleaseSpec()

OSProperty.CreateMemberTensionSpec()

OSProperty.CreateMemberTrussSpec()

OSProperty.CreateParametricSurfaceThicknessProperty()

OSProperty.CreatePipePropertyFromTable()

OSProperty.CreatePlateThicknessProperty()

OSProperty.CreatePrismaticCircleProperty()

OSProperty.CreatePrismaticGeneralProperty()

OSProperty.CreatePrismaticRectangleProperty()

OSProperty.CreatePrismaticTeeProperty()

OSProperty.CreatePrismaticTrapezoidalProperty()

OSProperty.CreatePropertyFromUPTTable()

OSProperty.CreatePropertyFromUserTable()

OSProperty.CreateTaperedIProperty()

OSProperty.CreateTaperedTubeProperty()

OSProperty.CreateTeePropertyFromTable()

OSProperty.CreateTubePropertyFromTable()

OSProperty.CreateUPTTable()

OSProperty.CreateUPTTableEx()

OSProperty.CreateWideFlangePropertyFromTable()

OSProperty.DeleteAllControlDependentRelations()

OSProperty.DeleteElementAttribute()

OSProperty.DeleteMaterial()

OSProperty.DeleteMemberAttribute()

OSProperty.DeleteMemberReleaseSpec()

OSProperty.DeleteMemberSpec()

OSProperty.DeleteProperty()

OSProperty.GetAlphaAngleForSection()

OSProperty.GetAssignedAttributeByIndex()

OSProperty.GetAssignedAttributeCount()

OSProperty.GetBeamConstants()

OSProperty.GetBeamMaterialName()

OSProperty.GetBeamProperty()

OSProperty.GetBeamPropertyAll()

OSProperty.GetBeamSectionDisplayName()

OSProperty.GetBeamSectionName()

OSProperty.GetBeamSectionPropertyRefNo()

OSProperty.GetBeamSectionPropertyTypeNo()

OSProperty.GetBeamSectionPropertyValuesEx()

OSProperty.GetBetaAngle()

OSProperty.GetCentroidLocationForSection()

OSProperty.GetCountofSectionPropertyValuesEx()

OSProperty.GetCountryTableNo()

OSProperty.GetDefaultStandardProfileDBFolder()

OSProperty.GetElementCountByAttribute()

OSProperty.GetElementGlobalOffSet()

OSProperty.GetElementListByAttribute()

OSProperty.GetElementLocalOffset()

OSProperty.GetElementMaterialName()

OSProperty.GetElementOffSetSpec()

OSProperty.GetElementOffsetSpecCount()

OSProperty.GetFireProofDataForBeam()

OSProperty.GetFireProofedBeamCount()

OSProperty.GetFireProofedBeamList()

OSProperty.GetFireProofingSpecAssignedBeamCount()

OSProperty.GetFireProofingSpecAssignedBeamList()

OSProperty.GetFireProofingSpecCount()

OSProperty.GetFireProofingSpecDetails()

OSProperty.GetInactiveMemberCount()

OSProperty.GetInactiveMemberList()

OSProperty.GetIsotropicMaterialAssignedBeamCount()

OSProperty.GetIsotropicMaterialAssignedBeamList()

OSProperty.GetIsotropicMaterialAssignedPlateCount()

OSProperty.GetIsotropicMaterialAssignedPlateList()

OSProperty.GetIsotropicMaterialAssignedSolidCount()

OSProperty.GetIsotropicMaterialAssignedSolidList()

OSProperty.GetIsotropicMaterialCount()

OSProperty.GetIsotropicMaterialProperties()

OSProperty.GetIsotropicMaterialPropertiesAssigned()

OSProperty.GetIsotropicMaterialPropertiesEx()

OSProperty.GetMaterialProperty()

OSProperty.GetMaterialPropertyEx()

OSProperty.GetMemberAttributeCount()

OSProperty.GetMemberAttributeList()

OSProperty.GetMemberCountByAttribute()

OSProperty.GetMemberGlobalOffSet()

OSProperty.GetMemberListByAttribute()

OSProperty.GetMemberLocalOffSet()

OSProperty.GetMemberReleaseSpec()

OSProperty.GetMemberReleaseSpecEx()

OSProperty.GetMemberSpecCode()

OSProperty.GetOrthotropic2DMaterialCount()

OSProperty.GetOrthotropic2DMaterialProperties()

OSProperty.GetOrthotropic3DMaterialCount()

OSProperty.GetOrthotropic3DMaterialProperties()

OSProperty.GetPlateMaterialName()

OSProperty.GetPlateSectionPropertyRefNo()

OSProperty.GetPlateThickness()

OSProperty.GetPropertyUniqueID()

OSProperty.GetPublishedProfileName()

OSProperty.GetRecordForSection()

OSProperty.GetSTAADProfileName()

OSProperty.GetSectionPropertyAssignedBeamCount()

OSProperty.GetSectionPropertyAssignedBeamList()

OSProperty.GetSectionPropertyCount()

OSProperty.GetSectionPropertyCountry()

OSProperty.GetSectionPropertyList()

OSProperty.GetSectionPropertyName()

OSProperty.GetSectionPropertyType()

OSProperty.GetSectionPropertyValues()

OSProperty.GetSectionPropertyValuesEx()

OSProperty.GetSectionTableNo()

OSProperty.GetShapeCode()

OSProperty.GetSolidMaterialName()

OSProperty.GetStandardProfileDBFolder()

OSProperty.GetStandardSectionDatabaseName()

OSProperty.GetStandardSectionName()

OSProperty.GetStandardSectionTableName()

OSProperty.GetThicknessPropertyAssignedPlateCount()

OSProperty.GetThicknessPropertyAssignedPlateList()

OSProperty.GetThicknessPropertyCount()

OSProperty.GetThicknessPropertyList()

OSProperty.GetThicknessPropertyValues()

OSProperty.GetTypeForIsotropicMaterial()

OSProperty.GetUptGeneralProfileBoundaryPoints()

OSProperty.GetUptGeneralProfilePointsCount()

OSProperty.GetUptGeneralStressLocationPoints()

OSProperty.GetUserProvidedTableCount()

OSProperty.GetUserProvidedTableList()

OSProperty.GetUserProvidedTableNo()

OSProperty.GetUserProvidedTableSectionCount()

OSProperty.GetUserProvidedTableSectionList()

OSProperty.GetUserProvidedTableSectionProperties()

OSProperty.GetUserProvidedTableSectionPropertyCount()

OSProperty.GetUserProvidedTableSectionType()

OSProperty.IsStandardDatabaseSection()

OSProperty.RemoveAllElementNodeReleaseSpec()

OSProperty.RemoveAllElementOffsetSpec()

OSProperty.RemoveAttribute()

OSProperty.RemoveElementIgnoreInplaneRotnSpecFromPlate()

OSProperty.RemoveElementNodeReleaseSpecFromPlate()

OSProperty.RemoveElementPlaneStressSpecFromPlate()

OSProperty.RemoveMaterialFromBeam()

OSProperty.RemoveMaterialFromPlate()

OSProperty.RemoveMaterialFromSolid()

OSProperty.RemoveMemberCableSpecFromBeam()

OSProperty.RemoveMemberCompressionSpecFromBeam()

OSProperty.RemoveMemberFireProofingSpecFromBeam()

OSProperty.RemoveMemberIgnoreStiffSpecFromBeam()

OSProperty.RemoveMemberInactiveSpecFromBeam()

OSProperty.RemoveMemberOffsetSpecFromBeam()

OSProperty.RemoveMemberReleaseSpecFromBeam()

OSProperty.RemoveMemberTensionSpecFromBeam()

OSProperty.RemoveMemberTrussSpecFromBeam()

OSProperty.RemovePropertyFromBeam()

OSProperty.RemovePropertyFromPlate()

OSProperty.RemovePropertyFromUPTTable()

OSProperty.RemoveUPTTable()

- OSProperty.SetMaterialName()
- OSProperty.SetPropertyUniqueID()
- OSProperty.SetStandardProfileDBFolder()
- OSProperty.SetTypeToIsotropicMaterial()
- OSProperty.UpdatePropertiesToDesignSection()
- OSProperty.__init__()

Support

- OSSupport
  - OSSupport.AssignSupportToEntityList()
  - OSSupport.AssignSupportToNode()
  - OSSupport.CreateElasticFooting()
  - OSSupport.CreateElasticMat()
  - OSSupport.CreateInclinedSupport()
  - OSSupport.CreatePlateMat()
  - OSSupport.CreateSupportFixed()
  - OSSupport.CreateSupportFixedBut()
  - OSSupport.CreateSupportPinned()
  - OSSupport.DeleteSupport()
  - OSSupport.GetCountOfElasticFooting()
  - OSSupport.GetCountOfElasticMat()
  - OSSupport.GetCountOfPlateMat()
  - OSSupport.GetElasticFootingAssignmentList()
  - OSSupport.GetElasticFootingDetail()
  - OSSupport.GetElasticMatAssignmentList()
  - OSSupport.GetElasticMatDetail()
  - OSSupport.GetPlateMatAssignmentList()
  - OSSupport.GetPlateMatDetail()
  - OSSupport.GetPlateMatSupportId()
  - OSSupport.GetSupportCount()
  - OSSupport.GetSupportInformation()
  - OSSupport.GetSupportInformationEx()

- OSSupport.GetSupportName()
- OSSupport.GetSupportNodes()
- OSSupport.GetSupportType()
- OSSupport.GetSupportUniqueID()
- OSSupport.RemoveElasticFooting()
- OSSupport.RemoveElasticFootingFromNode()
- OSSupport.RemoveElasticMat()
- OSSupport.RemoveElasticMatFromNode()
- OSSupport.RemovePlateMat()
- OSSupport.RemovePlateMatFromPlate()
- OSSupport.RemoveSupportFromNode()
- OSSupport.SetSupportUniqueID()
- OSSupport.__init__()

View

- OSView
  - OSView.CloseActiveWindow()
  - OSView.CopyPicture()
  - OSView.CreateNewViewForSelections()
  - OSView.CreateNewViewForSelectionsEx()
  - OSView.DetachView()
  - OSView.ExportView()
  - OSView.GetApplicationDesktopSize()
  - OSView.GetBeamsInView()
  - OSView.GetInterfaceMode()
  - OSView.GetNoOfBeamsInView()
  - OSView.GetScaleCount()
  - OSView.GetScaleValueByType()
  - OSView.GetScaleValues()
  - OSView.GetWindowCount()
  - OSView.GetWindowTitle()
  - OSView.HideAllMembers()

OSView.HideEntity()

OSView.HideMember()

OSView.HideMembers()

OSView.HidePlate()

OSView.HideSolid()

OSView.HideSurface()

OSView.OpenView()

OSView.RefreshView()

OSView.RenameView()

OSView.RotateDown()

OSView.RotateLeft()

OSView.RotateRight()

OSView.RotateUp()

OSView.SaveView()

OSView.SelectByItemList()

OSView.SelectByMissingAttribute()

OSView.SelectEntitiesConnectedToMember()

OSView.SelectEntitiesConnectedToNode()

OSView.SelectEntitiesConnectedToPlate()

OSView.SelectEntitiesConnectedToSolid()

OSView.SelectGroup()

OSView.SelectInverse()

OSView.SelectMembersParallelTo()

OSView.SetActiveWindow()

OSView.SetBeamAnnotationMode()

OSView.SetDesignResults()

OSView.SetDiagramMode()

OSView.SetInterfaceMode()

OSView.SetLabel()

OSView.SetModeSectionPage()

OSView.SetNodeAnnotationMode()

- OSView.SetReactionAnnotationMode()
- OSView.SetScaleValueByType()
- OSView.SetScaleValues()
- OSView.SetSectionView()
- OSView.SetUnits()
- OSView.SetWindowPosition()
- OSView.ShowAllMembers()
- OSView.ShowBack()
- OSView.ShowBottom()
- OSView.ShowFront()
- OSView.ShowIsometric()
- OSView.ShowLeft()
- OSView.ShowMember()
- OSView.ShowMembers()
- OSView.ShowPlan()
- OSView.ShowRight()
- OSView.SpinLeft()
- OSView.SpinRight()
- OSView.ZoomAll()
- OSView.ZoomExtentsMainView()
- OSView.__init__()

Command

- OSCommand
  - OSCommand.CreateSteelDesignCommand()
  - OSCommand.DeleteAllAnalysisCommands()
  - OSCommand.DeleteCheckIrregularitiesCommand()
  - OSCommand.DeleteCheckSoftStoryCommand()
  - OSCommand.DeleteFloorDiaphragmBaseCommand()
  - OSCommand.PerformAnalysis()
  - OSCommand.PerformBucklingAnalysis()
  - OSCommand.PerformBucklingAnalysisEx()

- OSCommand.PerformCableAnalysis()
- OSCommand.PerformCableAnalysisEx()
- OSCommand.PerformDirectAnalysis()
- OSCommand.PerformNonlinearAnalysisEx()
- OSCommand.PerformPDeltaAnalysisEx()
- OSCommand.PerformPDeltaAnalysisNoConverge()
- OSCommand.SetCheckIrregularitiesCommand()
- OSCommand.SetCheckSoftStoryCommand()
- OSCommand.SetFloorDiaphragmBaseCommand()
- OSCommand.__init__()

Output

- OSOutput
  - OSOutput.AreResultsAvailable()
  - OSOutput.GetAllPlateCenterForces()
  - OSOutput.GetAllPlateCenterMoments()
  - OSOutput.GetAllPlateCenterPrincipalStressesAndAngles()
  - OSOutput.GetAllPlateCenterPrincipalStressesAndAnglesEx()
  - OSOutput.GetAllPlateCenterStressesAndMoments()
  - OSOutput.GetAllSolidNormalStresses()
  - OSOutput.GetAllSolidPrincipalStresses()
  - OSOutput.GetAllSolidShearStresses()
  - OSOutput.GetAllSolidVonMisesStresses()
  - OSOutput.GetBasePressures()
  - OSOutput.GetBucklingFactor()
  - OSOutput.GetBucklingModeDisplacementAtNode()
  - OSOutput.GetIntermediateDeflectionAtDistance()
  - OSOutput.GetIntermediateMemberAbsTransDisplacements()
  - OSOutput.GetIntermediateMemberForcesAtDistance()
  - OSOutput.GetIntermediateMemberTransDisplacements()
  - OSOutput.GetMatInfluenceAreas()
  - OSOutput.GetMaxBeamStresses()

OSOutput.GetMaxSectionDisplacement()

OSOutput.GetMemberDesignSectionName()

OSOutput.GetMemberEndDisplacements()

OSOutput.GetMemberEndForces()

OSOutput.GetMemberSteelDesignMaxFailureRatio()

OSOutput.GetMemberSteelDesignMinFailureRatio()

OSOutput.GetMemberSteelDesignRatio()

OSOutput.GetMemberSteelDesignResults()

OSOutput.GetMinMaxAxialForce()

OSOutput.GetMinMaxBendingMoment()

OSOutput.GetMinMaxShearForce()

OSOutput.GetModalDisplacementAtNode()

OSOutput.GetModalMassParticipationFactors()

OSOutput.GetModeFrequency()

OSOutput.GetMultipleMemberSteelDesignMaxRatio()

OSOutput.GetMultipleMemberSteelDesignRatio()

OSOutput.GetMultipleMemberSteelDesignResults()

OSOutput.GetNLLoadStep()

OSOutput.GetNLNodeDisplacements()

OSOutput.GetNoOfBucklingFactors()

OSOutput.GetNoOfModesExtracted()

OSOutput.GetNodeDisplacements()

OSOutput.GetOutputUnitForDensity()

OSOutput.GetOutputUnitForDimension()

OSOutput.GetOutputUnitForDisplacement()

OSOutput.GetOutputUnitForDistForce()

OSOutput.GetOutputUnitForDistMoment()

OSOutput.GetOutputUnitForForce()

OSOutput.GetOutputUnitForMoment()

OSOutput.GetOutputUnitForRotation()

OSOutput.GetOutputUnitForSectArea()

OSOutput.GetOutputUnitForSectDimension()

OSOutput.GetOutputUnitForSectInertia()

OSOutput.GetOutputUnitForSectModulus()

OSOutput.GetOutputUnitForStress()

OSOutput.GetPMemberEndForces()

OSOutput.GetPMemberIntermediateForcesAtDistance()

OSOutput.GetPlateCenterNormalPrincipalStresses()

OSOutput.GetPlateCenterVonMisesStresses()

OSOutput.GetPlateCornerForces()

OSOutput.GetPlateStressAtPoint()

OSOutput.GetResultantForceAlongLineForParametricSurface()

OSOutput.GetResultantForceAlongLineForPlateList()

OSOutput.GetStaticCheckResult()

OSOutput.GetSteelDesignParameterBlockCount()

OSOutput.GetSteelDesignParameterBlockNameByIndex()

OSOutput.GetSupportReactions()

OSOutput.GetTimeHistoryIntegrationStepInfo()

OSOutput.GetTimeHistoryResponse()

OSOutput.GetTimeHistoryResponseAtTime()

OSOutput.GetTimeHistoryResponseMinMax()

OSOutput.IsBucklingAnalysisResultsAvailable()

OSOutput.IsMultipleMemberSteelDesignResultsAvailable()

OSOutput.__init__()

Errors

Table

OSTable

OSTable.AddTable()

OSTable.CreateReport()

OSTable.DeleteReport()

OSTable.DeleteTable()

OSTable.GetCellValue()

- OSTable.GetReportCount()
- OSTable.GetTableCount()
- OSTable.RenameTable()
- OSTable.ResizeTable()
- OSTable.SaveReport()
- OSTable.SaveReportAll()
- OSTable.SaveTable()
- OSTable.SetCellTextBold()
- OSTable.SetCellTextColor()
- OSTable.SetCellTextHorzAlignment()
- OSTable.SetCellTextItalic()
- OSTable.SetCellTextSize()
- OSTable.SetCellTextSizeAll()
- OSTable.SetCellTextUnderline()
- OSTable.SetCellTextVertAlignment()
- OSTable.SetCellValue()
- OSTable.SetColumnHeader()
- OSTable.SetColumnUnitString()
- OSTable.SetRowHeader()
- OSTable.__init__()

Design

- OSDesign
  - OSDesign.AssignDesignCommand()
  - OSDesign.AssignDesignGroup()
  - OSDesign.AssignDesignParameter()
  - OSDesign.CreateDesignBrief()
  - OSDesign.GetDesignBriefCode()
  - OSDesign.GetMemberDesignParameters()
  - OSDesign.__init__()

Root

Contents

- OSRoot
- getActiveObject()

class openstaadpy.os_analytical.openstaadroot.OSRoot [source]

Bases: object

Analyze() [source]

Analyze the currently opened .STD file.

Return type:

None

> **See also**
>
> AnalyzeEx
>
> For more options.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.Analyze()
```

AnalyzeEx(*silentMode: int, hiddenMode: int, waitTillComplete: int*) [source]

Analyze the model with extended options.

Notes

- This extended method analyzes the currently opened .STD file. This method is equivalent to running analysis from user interface.
- However, it has additional three arguments to specify whether to run the analysis in silent or hidden mode.
- The third parameter specifies whether the method should wait for the analysis to finish or return immediately.
- This method may be used in conjunction with SetSilentMode(), if one wants to suppress all dialog boxes displayed from the application during running of analysis.

Parameters:

- **silentMode** (*int*) – Integer value to enable silent mode. [1 = Enable, 0 otherwise]. Enabling silent mode will suppress all dialog boxes in the engine which requires user input. The analysis dialog box however will be displayed and close automatically on completion.
- **hiddenMode** (*int*) – Integer value to enable hidden mode. [1 = Enable, 0 = Disable]. Enabling hidden mode will suppress the display of analysis dialog. The analysis dialog box will not be displayed.
- **waitTillComplete** (*int*) – Integer value to specify whether to wait for the analysis process to finish or return immediately. [1 to wait , 0 otherwise]

Returns:

Returns -1 if Analysis Terminated. Returns 0 if General Error. Returns 1 if Analysis is in progress. Returns 2 if Analysis completed without errors or warnings. Returns 3 if Analysis completed with warnings but without errors. Returns 4 if Analysis completed with errors. Returns 5 if Analysis has not been performed.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> status = staad_obj.AnalyzeEx(1, 0, 1)
```

AnalyzeModel()

[source]

Analyze the current model.

Return type:

None

> **See also**
>
> `AnalyzeEx`
>
> For more options.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.AnalyzeModel()
```

CloseSTAADFile()

[source]

Close the currently open .STD file.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.CloseSTAADFile()
```

GetAnalysisStatus(*modelPath: str = None*)

[source]

Get analysis status for the open STAAD Model.

Parameters:

modelPath (*str, optional*) – The full path of the STAAD model. If not provided, the currently open model will be used.

Returns:

A dictionary containing the analysis status with following keys:

- **'ReturnValue' : *int***

  Status code of the analysis:

- **'ReturnString' : *str***

  Description corresponding to the *ReturnValue*.

- **'NoOfWarnings' : *int***

  Number of warnings generated during the analysis.

- **'NoOfErrors' : *int***

  Number of errors generated during the analysis.

- **'CPUTime' : *int***

  CPU time taken for the analysis in seconds.

Return type:

dict

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> status_dict = staad_obj.GetAnalysisStatus() # or staad_obj.GetAn
>>> print(f"Return Value: {status_dict['ReturnValue']}")
>>> print(f"Return String: {status_dict['ReturnString']}")
>>> print(f"No Of Warnings: {status_dict['NoOfWarnings']}")
>>> print(f"No Of Errors: {status_dict['NoOfErrors']}")
>>> print(f"CPU Time (sec): {status_dict['CPUTime']}")
```

`GetApplicationVersion()` [source]

Get the current application version as a string.

Returns:

The application version in the format 'X.X.X.X'.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> version = staad_obj.GetApplicationVersion()
>>> print(version)
```

GetBaseUnit()

[source]

Get the base unit for the currently open .STD file.

- For English system of units (The values that are derived from a length unit, e.g. dimensions, areas, stresses, will be based on inches, 'in'. All values derived from a force unit, e.g. Axial force, moments, stresses, etc, will be based on kilopounds, 'KIP').
- For Metric system of units (The values that are derived from a length unit, will be based on Meters, 'm'. All values derived from a force unit, will be based on kilo newtons, 'kNs').

Returns:

'English' or 'Metric'.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> base_unit = staad_obj.GetBaseUnit()
>>> print(base_unit)
```

GetErrorMessage()

[source]

Returns error messages thrown by OpenSTAAD (e.g. - for unavailability of license, unavailability of required named view)

Returns:

The error message string.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> msg = staad_obj.GetErrorMessage()
```

GetFullJobInfo() [source]

Get full job information for the current model.

Returns:

[job name, job client, engineer's name, engineer date, job number, revision, part name, reference, checker name, checker date, approver name, approval date, comments]

Return type:

list of 14 Strings

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> info = staad_obj.GetFullJobInfo()
```

GetInputUnitForForce() [source]

Retrieve the input unit of force of the currently open .STD file.

Returns:

The force unit name. ('Kilopound', 'Pound', 'Kilogram', 'Metric Ton', 'Newton', 'KiloNewton', 'MegaNewton', 'DecaNewton').

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> force_unit = staad_obj.GetInputUnitForForce()
>>> print(force_unit)
```

GetInputUnitForLength() [source]

Retrieve the input unit of length of the currently open .STD file.

Returns:

The length unit name. ('Inch', 'Feet', 'Feet', 'CentiMeter', 'Meter', 'MilliMeter', 'DeciMeter', 'KiloMeter').

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> length_unit = staad_obj.GetInputUnitForLength()
>>> print(length_unit)
```

GetMainWindowHandle() [source]

Get the main window handle of the STAAD.Pro application.

Returns:

Window handle.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> hwnd = staad_obj.GetMainWindowHandle()
```

GetProcessHandle() [source]

Retrieve the current STAAD.Pro process handle.

Returns:

The process handle.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> handle = staad_obj.GetProcessHandle()
>>> print(handle)
```

GetProcessId() [source]

Retrieve the current STAAD.Pro process ID.

Returns:

The process ID.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> pid = staad_obj.GetProcessId()
>>> print(pid)
```

GetSTAADFile(*bFullPath: bool = True*) [source]

Retrieve the path or the name of the current .STD file.

Parameters:

bFullPath (*bool, optional*) – If True, returns the full path. If False, returns only the file name. Default is True.

Returns:

The file path or name.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> file_path = staad_obj.GetSTAADFile() # or staad_obj.GetSTAADFile
>>> print(file_path)
```

GetSTAADFileFolder() [source]

Retrieve the folder path of the current STAAD file.

Returns:

The folder path.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> folder = staad_obj.GetSTAADFileFolder()
>>> print(folder)
```

GetShortJobInfo() [source]

Get short job information for the current model.

Returns:

(job name, job ID, job status)

Return type:

tuple of 3 Strings

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> job_name, job_id, job_status = staad_obj.GetShortJobInfo()
>>> print(f"Job Name: {job_name}")
>>> print(f"Job ID: {job_id}")
>>> print(f"Job Status: {job_status}")
```

IsAnalyzing() [source]

Specify whether the analysis is running or not.

Returns:

True if analysis is running, False otherwise.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.IsAnalyzing()
```

IsPhysicalModel()

[source]

Check if the current model is a physical model.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.IsPhysicalModel()
```

NewSTAADFile(*fileName: str, lengthUnit: int, forceUnit: int*)

Create a .STD file with specified length and force units. [source]

Parameters:

- **fileName** (*str*) – The file name to save.
- **lengthUnit** (*int*) – Integer from 0 to 7 representing length unit (0-Inch, 1-Feet, 2-Feet, 3-CentiMeter, 4-Meter, 5-MilliMeter, 6-DeciMeter, 7-KiloMeter).
- **forceUnit** (*int*) – Integer from 0 to 7 representing force unit (0-Kilopound, 1-Pound, 2-Kilogram, 3-Metric Ton, 4-Newton, 5-Kilo Newton, 6-Mega Newton, 7-DecaNewton).

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.NewSTAADFile("C:/Models/new_model.std", 4, 5)
```

OpenSTAADFile(*file: str*) [source]

Open the specified .STD file.

Parameters:

file (*str*) – Path to the .STD file to open.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.OpenSTAADFile("C:/Models/structure.std")
```

Quit() [source]

Close the STAAD.Pro application environment.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.Quit()
```

SaveModel(*saveSilent: bool = False*) [source]

Save the current structure with optional silent mode.

Parameters:

saveSilent (*bool, optional*) – If True, saves the model silently. Default is False.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.SaveModel()
>>> staad_obj.SaveModel(saveSilent=True)
```

SetFullJobInfo(*job_name: str, job_client: str = '', eng_name: str = '', eng_date: str = '', job_number: str = '', revision: str = '', part_name: str = '', reference: str = '', checker_name: str = '', checker_date: str = '', approver_name: str = '', approval_date: str = '', comments: str = ''*) [source]

Set full job information for the current model.

Parameters:

- **job_name** (*str*) – Name of the job.
- **job_client** (*str*) – Client name for the job.
- **eng_name** (*str*) – Engineer's name.
- **eng_date** (*str*) – Engineer's date.
- **job_number** (*str*) – Job number.
- **revision** (*str*) – Revision number.
- **part_name** (*str*) – Part name.
- **reference** (*str*) – Reference.
- **checker_name** (*str*) – Checker name.
- **checker_date** (*str*) – Checker date.

- **approver_name** (*str*) – Approver name.
- **approval_date** (*str*) – Approval date.
- **comments** (*str*) – Comments.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.SetFullJobInfo("Full info")
```

SetInputUnitForForce(*forceUnit: int*) [source]

Set the input unit of force of the currently open .STD file.

Parameters:

forceUnit (*int*) – Integer from 0 to 7 representing force unit. (0- Kilopound, 1- Pound, 2- Kilogram, 3-Metric Ton, 4- Newton, 5-Kilo Newton, 6- Mega Newton, 7- DecaNewton).

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.SetInputUnitForForce(4) # Setting force unit to Newton
```

SetInputUnitForLength(*lengthUnit: int*) [source]

Set the input unit of length of the currently open .STD file.

Parameters:

lengthUnit (*int*) – Integer from 0 to 7 representing length unit. (0- Inch, 1- Feet, 2- Feet, 3- CentiMeter, 4- Meter, 5- MilliMeter, 6- DeciMeter, 7- KiloMeter).

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.SetInputUnitForLength(4)
```

SetInputUnits(*lengthUnit: int, forceUnit: int*) [source]

Set the input units of length and force of the currently open .STD file.

Parameters:

- **lengthUnit** (*int*) – Integer from 0 to 7 representing length unit. (0- Inch, 1- Feet, 2- Feet, 3- CentiMeter, 4- Meter, 5- MilliMeter, 6 - DeciMeter, 7 - KiloMeter).
- **forceUnit** (*int*) – Integer from 0 to 7 representing force unit. (0- Kilopound, 1- Pound, 2- Kilogram, 3-Metric Ton, 4- Newton, 5-Kilo Newton, 6- Mega Newton, 7- DecaNewton).

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.SetInputUnits(4, 5)
```

SetShortJobInfo(*job_name: str, job_id: str, job_status: str*)

Set short job information for the current model. [source]

Parameters:

- **job_name** (*str*) – Job name.
- **job_id** (*str*) – Job ID.
- **job_status** (*str*) – Job status.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.SetShortJobInfo("NewJob", "12345", "In Progress")
```

SetSilentMode(*silent: bool*) [source]

Set silent mode for the application.

Parameters:

silent (*bool*)

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.SetSilentMode(True)
```

ShowApplication() [source]

Bring the STAAD.Pro Application to the foreground.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.ShowApplication()
```

UpdateStructure() [source]

Update the structure in the STAAD.Pro application.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> if staad_obj is None:
...     print("staad object not found")
...     exit()
>>> staad_obj.UpdateStructure()
```

__init__(*filePath=None*) [source]

openstaadpy.os_analytical.openstaadroot.**getActiveObject**(*filePath: str = ''*) [source]

Geometry

Contents

- OSGeometry

class openstaadpy.os_analytical.osgeometry.**OSGeometry** [source]

Bases: object

AddBeam(*nNodeStart: int, nNodeEnd: int*) [source]

Add a beam/member with specified nodes and return the assigned beam number.

Parameters:

- **nNodeStart** (*int*) – ID of the starting node.
- **nNodeEnd** (*int*) – ID of the ending node.

Returns:

Beam number assigned.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_no = staad_obj.Geometry.AddBeam(1, 2)
>>> print(beam_no)
```

AddCircularRegionToSurface(*surfaceNo: int, x: float, y: float, z: float, radius: float, divisions: int, density: int, is_opening: bool = False*) [source]

Add a circular region or opening to a surface.

Parameters:

- **surfaceNo** (*int*) – Surface ID of the parametric surface to which the circular region will be added.
- **x** (*float*) – Global X coordinate of the center of the circular region.
- **y** (*float*) – Global Y coordinate of the center of the circular region.
- **z** (*float*) – Global Z coordinate of the center of the circular region.
- **radius** (*float*) – Radius of the circular region.
- **divisions** (*int*) – Number of divisions along the circular region.
- **density** (*int*) – Density of the circular region.
- **is_opening** (*bool*) – Whether the circular region is an opening or not.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.AddCircularRegionToSurface(1, 5.0, 5
>>> print(result)
```

`AddDensityLineToSurface(surfaceNo: int, x1: float, y1: float, z1: float, density1: int, x2: float, y2: float, z2: float, density2: int, divisions: int)` [source]

Add a density line to a surface.

Parameters:

- **surfaceNo** (*int*) – Surface ID of the parametric surface to which the density line will be added.
- **x1** (*float*) – Global X coordinate of the start point of the density line.
- **y1** (*float*) – Global Y coordinate of the start point of the density line.
- **z1** (*float*) – Global Z coordinate of the start point of the density line.
- **density1** (*int*) – Density at the start point of the density line.
- **x2** (*float*) – Global X coordinate of the end point of the density line.
- **y2** (*float*) – Global Y coordinate of the end point of the density line.
- **z2** (*float*) – Global Z coordinate of the end point of the density line.
- **density2** (*int*) – Density at the end point of the density line.

- **divisions** (*int*) – Number of divisions along the density line.

Returns:

index (0 based) of the density line added.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.AddDensityLineToSurface(1, 0.0, 0.0,
>>> print(result)
```

AddDensityPointToSurface(*surfaceNo: int, pointData*) [source]

Add a density point to a surface.

Parameters:

- **surfaceNo** (*int*)
- **pointData** (*object*)

Return type:

None

AddMultipleBeams(*incidences*) [source]

Add multiple beams at once.

Parameters:

incidences (*list of lists containing int*) – List of lists containing start and end node numbers for each beam. [[start1, end1], [start2, end2], ...]

Returns:

List of beam numbers assigned to the added beams.

Return type:

List

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.AddMultipleBeams([[1,2],[2,3]])
>>> print(beam_ids)
```

AddMultipleNodes(*coordinates*) [source]

Add multiple nodes at once.

Parameters:

coordinates (*list of lists containing float or int*) – List of lists containing x, y, z coordinates for each node. [[x1, y1, z1], [x2, y2, z2], ...]

Returns:

List

Return type:

List of node numbers assigned to the added nodes.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> node_ids = staad_obj.Geometry.AddMultipleNodes([[0.0,0.0,0.0],[1
>>> print(node_ids)
```

AddMultiplePlates(*incidences*) [source]

Add multiple plates at once.

Parameters:

incidences (*list*) – List of lists containing nodeA, nodeB, nodeC, nodeD for each plate. [[nodeA1, nodeB1, nodeC1, nodeD1], [nodeA2, nodeB2, nodeC2, nodeD2], ...]

Returns:

List

Return type:

List of plate numbers assigned to the added plates.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plate_ids = staad_obj.Geometry.AddMultiplePlates([[1,2,3,4],[2,3
>>> print(plate_ids)
```

AddMultipleSolids(*incidences*) [source]

Add multiple solids at once.

Parameters:

incidences (*list*) – List of lists containing nodeA, nodeB, nodeC, nodeD, nodeE, nodeF, nodeG, nodeH for each solid. [[nodeA1, nodeB1, nodeC1, nodeD1, nodeE1, nodeF1, nodeG1, nodeH1], [nodeA2, nodeB2, nodeC2, nodeD2, nodeE2, nodeF2, nodeG2, nodeH2], ...]

Returns:

List

Return type:

List of solid numbers assigned to the added solids.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> solid_ids = staad_obj.Geometry.AddMultipleSolids([[1,2,3,4,5,6,7
>>> print(solid_ids)
```

AddNode(*x: float, y: float, z: float*) [source]

Add a node with specified coordinates and return the assigned node number.

Parameters:

- **x** (*float*) – X coordinate.
- **y** (*float*) – Y coordinate.
- **z** (*float*) – Z coordinate.

Returns:

Node number assigned.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> node_no = staad_obj.Geometry.AddNode(0.0, 0.0, 0.0)
>>> print(node_no)
```

AddParametricSurfaceToModel(*surfaceNo: int*) [source]

Add definition of the specified parametric surface to the model.

Parameters:

surfaceNo (*int*)

Returns:

True if successful, False otherwise.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> surface_id = staad_obj.Geometry.AddParametricSurfaceToModel(1)
>>> print(surface_id)
```

AddPlate(*nNodeA: int, nNodeB: int, nNodeC: int, nNodeD: int = 0*) [source]

Add a plate with specified nodes and return the assigned plate number.

Parameters:

- **nNodeA** (*int*) – Node A for plate connectivity.
- **nNodeB** (*int*) – Node B for plate connectivity.
- **nNodeC** (*int*) – Node C for plate connectivity.
- **nNodeD** (*int*) – Node D for plate connectivity.

Returns:

Plate number assigned.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plate_no = staad_obj.Geometry.AddPlate(1, 2, 3, 4)
>>> print(plate_no)
```

AddPolygonalRegionToSurface(*surfaceNo: int, regionData*)

Add a polygonal region to a surface. [source]

Parameters:

- **surfaceNo** (*int*)
- **regionData** (*object*)

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.AddPolygonalRegionToSurface(1, regionData)
```

AddSolid(*nodeA: int, nodeB: int, nodeC: int, nodeD: int, nodeE: int, nodeF: int, nodeG: int = 0, nodeH: int = 0*)

[source]

Add a solid element.

Parameters:

- **nodeA** (*int*) – ID of node A.
- **nodeB** (*int*) – ID of node B.
- **nodeC** (*int*) – ID of node C.
- **nodeD** (*int*) – ID of node D.

- **nodeE** (*int*) – ID of node E.
- **nodeF** (*int*) – ID of node F.
- **nodeG** (*int*) – ID of node G.
- **nodeH** (*int*) – ID of node H.

Returns:

ID number of the added solid.

Return type:

Int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> solidID = staad_obj.Geometry.AddSolid(1, 2, 3, 4, 5, 6, 7, 8)
>>> print(solidID)
```

BreakBeamsAtSpecificNodes(*nodeList: list*) [source]

Breaks beams that passes through the specified list of nodes and assigns same attributes to the newly added beams.

Parameters:

nodeList (*list of int*) – List of node IDs where beams should be broken.

Returns:

1. List of int : IDs of the broken beams.
2. List of int : IDs of the newly created beams.

Return type:

tuple of 2 lists

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> new_beams = staad_obj.Geometry.BreakBeamsAtSpecificNodes([1,2,3]
>>> print(new_beams)
```

ClearMemberSelection() [source]

Clear the current member selection.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.ClearMemberSelection()
```

ClearNodeSelection() [source]

Clear the current node selection.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.ClearNodeSelection()
```

ClearPhysicalMemberSelection() [source]

Clears the current selection of physical members.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.ClearPhysicalMemberSelection()
```

ClearPlateSelection() [source]

Clear the current plate selection.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.ClearPlateSelection()
```

ClearSolidSelection() [source]

Clear the current solid selection.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.ClearSolidSelection()
```

CommitParametricSurfaceMesh(*surfaceNo: int*) [source]

Merges the specified parametric mesh with the model

Parameters:

surfaceNo (*int*) – surface ID of the parametric surface to be merged with the model.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.CommitParametricSurfaceMesh(1)
>>> print(result)
```

CreateBeam(*nBeamNo: int, nNodeStart: int, nNodeEnd: int*)

Create a beam/member with specified nodes. [source]

Parameters:

- **nBeamNo** (*int*) – Member number ID to assign.
- **nNodeStart** (*int*) – ID of the starting node.
- **nNodeEnd** (*int*) – ID of the ending node.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.CreateBeam(5, 1, 2)
```

CreateGroup(*group_type: int, group_name: str*) [source]

Create a new group with the specified name and type.

Parameters:

- **group_type** (*int*) –

  Type of the group:

Index	Group Type
1	Nodes
2	Members
3	Plates
4	Solids
5	Geometry (Members, Plates and Solids)
6	Floor (Floor beam)

- **group_name** (*str*) – Name of the group to create.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.CreateGroup(1, "MyGroup")
```

`CreateGroupEx(groupType: int, groupName: str, entityList: list)`

Create a group with extended options. [source]

Parameters:

- **groupType** (*int*) –

  The int representating the corresponding group type as show in below table:

Index	Group Type
1	Nodes
2	Members
3	Plates
4	Solids
5	Geometry (Members, Plates and Solids)
6	Floor (Floor beam)

- **groupName** (*str*) – Name of the group.
- **entityList** (*list of int*) – List of entity IDs to include in the group.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.CreateGroupEx(1, "GroupA", [1,2,3])
```

CreateMultipleBeams(*beam_ids: list, beam_incidences: list*)

Create multiple beams. [source]

Parameters:

- **beam_ids** (*list of int*) – Beam IDs for each beam. [BeamID1, BeamID2, BeamID3, ...]
- **beam_incidences** (*list of lists*) – List of [start_node, end_node] for each beam. [[start1, end1], [start2, end2], ...]

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.CreateMultipleBeams(beam_ids, beam_incidences
```

CreateMultipleNodes(*node_ids: list, nodeCoordinates: list*)

Create multiple nodes. [source]

Parameters:

- **node_ids** (*list of int*) – Node IDs for each node. [NodeID1, NodeID2, NodeID3, ...]
- **nodeCoordinates** (*list of lists*) – List of [x, y, z] coordinates for each node. [[x1, y1, z1], [x2, y2, z2], ...]

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.CreateMultipleNodes(node_ids,nodeCoordinates)
```

CreateMultiplePlates(*plate_ids: list | int, plate_incidences: list*)

Create multiple plates. [source]

Parameters:

- **plate_ids** (*list of int or int*) – plate IDs for each plate. [PlateID1, PlateID2, PlateID3, ...]
- **plate_incidences** (*list of lists*) – List of lists containing incidences for each plate. [[NodeA1, NodeB1, NodeC1, NodeD1], [NodeA2, NodeB2, NodeC2, NodeD2], ...]

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.CreateMultiplePlates(plateIds, plateIncidence
```

CreateNode(*nNodeNo: int, x: float, y: float, z: float*) [source]

Create a node with specified coordinates and node number.

Parameters:

- **nNodeNo** (*int*) – Node number ID to assign.
- **x** (*float*) – X coordinate.
- **y** (*float*) – Y coordinate.
- **z** (*float*) – Z coordinate.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.CreateNode(10, 0.0, 0.0, 0.0)
```

CreatePhysicalMember(*memberList: list*) [source]

Create a physical member from the currently selected members.

Parameters:

- **memberList** (*list of int*) – List of member IDs to include in the physical member.
- **physicalMemberName** (*str*) – Name of the physical member to create.

Returns:

Id of the created physical member.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.CreatePhysicalMember([1,2,3], "MyPhy
>>> print(result)
```

CreatePlate(*nPlateNo: int, nNodeA: int, nNodeB: int, nNodeC: int, nNodeD: int = 0*) [source]

Create a plate with specified nodes.

Parameters:

- **nPlateNo** (*int*) – Plate number ID to assign.
- **nNodeA** (*int*) – Node A for plate connectivity.
- **nNodeB** (*int*) – Node B for plate connectivity.
- **nNodeC** (*int*) – Node C for plate connectivity.
- **nNodeD** (*int*) – Node D for plate connectivity.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.CreatePlate(1, 1, 2, 3, 4)
```

CreateSolid(*solidNo: int, nodeA: int, nodeB: int, nodeC: int, nodeD: int, nodeE: int, nodeF: int, nodeG: int = 0, nodeH: int = 0*) [source]

Create a solid element.

Parameters:

- **solidNo** (*int*) – Solid number ID to assign.
- **nodeA** (*int*) – ID of node A.
- **nodeB** (*int*) – ID of node B.
- **nodeC** (*int*) – ID of node C.
- **nodeD** (*int*) – ID of node D.

- **nodeE** (*int*) – ID of node E.
- **nodeF** (*int*) – ID of node F.
- **nodeG** (*int*) – ID of node G.
- **nodeH** (*int*) – ID of node H.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.CreateSolid(4, 1, 2, 3, 4, 5, 6, 7, 8)
```

`DefineParametricSurface(name: str, type: int, origin_Node: int, x_vertex_node: int, y_vertex_node: int, vertices_list: list[int], auto_generate: bool)` [source]

Define a parametric surface.

Parameters:

- **name** (*str*) – Name of the parametric surface.
- **type** (*int*) –

  Type of the parametric surface:

value	Surface Type
0	None
1	Wall
2	Slab

- **origin_Node** (*int*) – Node number defining the origin of the parametric surface.
- **x_vertex_node** (*int*) – Node number defining the local X axis of the parametric surface.
- **y_vertex_node** (*int*) – Node number defining the local Y axis of the parametric surface.

- **vertices_list** (*list[int]*) – List of vertices of the parametric surface. (must lie in same plane)
- **auto_generate** (*bool*) – Specifies whether to auto-generate boundary points and density objects for the parametric surface (True = Auto-generate, False = Otherwise)

Returns:

The ID of the created parametric surface.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> surface_id = staad_obj.Geometry.DefineParametricSurface("Surface
>>> print(surface_id)
```

DeleteBeam(*BeamNo: int*) [source]

Delete a specified beam.

Parameters:

BeamNo (*int*) – Beam number to delete.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.DeleteBeam(5)
```

DeleteGroup(*groupName: str*) [source]

Delete a group.

Parameters:

groupName (*str*)

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.DeleteGroup("GroupA")
```

DeleteNode(*nNodeNo: int*) [source]

Delete a specified node.

Parameters:

nNodeNo (*int*) – Node number to delete.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.DeleteNode(10)
```

DeletePhysicalMember(*physicalMemberId: int*) [source]

Delete a physical member.

Parameters:

physicalMemberId (*int*) – ID of the physical member to delete.

Returns:

True if successful, False otherwise.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.DeletePhysicalMember(1)
>>> print(result)
```

DeletePlate(*nPlateNo: int*) [source]

Delete a specified plate.

Parameters:

nPlateNo (*int*) – Plate number to delete.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.DeletePlate(1)
```

DeleteSolid(*solidID*) [source]

Delete a specified solid.

Parameters:

solidID (*int*) – ID of solid to delete.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.DeleteSolid(1)
```

DoTranslationalRepeat(*link_bays: bool, open_base: bool, axis_dir: int, spacing_list: list[float], no_of_bays: int,*

renumber_bays: bool, renumber_list: list[int], geometry_only_flag: bool) [source]

Perform a translational repeat operation.

Parameters:

- **link_bays** (*bool*) – specifies whether to generate new members between each step in the direction of the repeat (True = Link Bays, False = Otherwise)
- **open_base** (*bool*) – specifies not to generate linking members at the base of the structure (i.e., the lowest nodes in the selection) (True = Open base, False = Otherwise)
- **axis_dir** (*int*) – value to specify direction in global axis along which translational repeat operation is to be performed (0 = GX, 1 = GY, 2 = GZ)
- **spacing_list** (*list[float]*) – List of spacing distances.
- **no_of_bays** (*int*) – specifies number of generated bays (maximum no of bays that can be generated single call of the API = 100)
- **renumber_bays** (*bool*) – specifies whether to use a user-specified starting number of the members generated in each newly generated bay (True = Renumber, False = Otherwise)
- **renumber_list** (*list[int]*) – specify starting member numbers for each newly generated bays (length of list = no_of_bays). if renumber_bays = False, this parameter is ignored.
- **geometry_only_flag** (*bool*) – specifies whether only geometry data is to be copied (True = Copy geometry only, False = Copy all)

Returns:

Result

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.DoTranslationalRepeat(True, True, 3,
>>> print(result) # True if successful, False otherwise
```

GetAnalyticalMemberCountForPhysicalMember(*physicalMemberId: int*) [source]

Get the count of analytical members in a physical member.

Parameters:

physicalMemberId (*int*) – ID of the physical member.

Returns:

Count of analytical members in the physical member.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Geometry.GetAnalyticalMemberCountForPhysicalMe
>>> print(count)
```

GetAnalyticalMembersForPhysicalMember(*physicalMemberId: int*)

Get the analytical members in a physical member. [source]

Parameters:

physicalMemberId (*int*) – ID of the physical member.

Returns:

List of analytical member IDs in the physical member.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> members = staad_obj.Geometry.GetAnalyticalMembersForPhysicalMemb
>>> print(members)
```

GetAreaOfPlates(*plateList*) [source]

Get the area of plates.

Parameters:

plateList (*list of int*)

Returns:

list of area of each plate in the list.

Return type:

List

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> area_list = staad_obj.Geometry.GetAreaOfPlates([1,2,3])
>>> print(area_list)
```

GetBeamLength(*beam: int*) [source]

Get the length of a beam.

Parameters:

beam (*int*) – Beam number.

Returns:

Length of the beam.

Return type:

float

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> length = staad_obj.Geometry.GetBeamLength(1)
>>> print(length)
```

GetBeamList() [source]

Get the list of all beam numbers.

Returns:

List of beam numbers.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beams = staad_obj.Geometry.GetBeamList()
>>> print(beams)
```

GetBeamsConnectedAtNode(*node*) [source]

Get the list of beams connected at a node.

Parameters:

node (*int*) – Node number.

Returns:

Beam numbers connected at the node.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beams = staad_obj.Geometry.GetBeamsConnectedAtNode(1)
>>> print(beams)
```

GetCountOfBreakableBeamsAtSpecificNodes(*nodeList: list*) [source]

Get number of beams that can be broken based on the list of node Ids.

Parameters:

nodeList (*list of int*) – List of node IDs to check for breakable beams.

Returns:

Count of breakable beams.

Return type:

int

> See also
>
> `BreakBeamsAtSpecificNodes`

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Geometry.GetCountOfBreakableBeamsAtSpecificNod
>>> print(count)
```

`GetFlagForHiddenEntities()` [source]

Get the flag specified for consideration of hidden entities (nodes and plates) while getting count or list of those entities

Returns:

All entities = 0 (Default option), Ignore Hidden entities = 1, Only hidden entities = 2

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> flag = staad_obj.Geometry.GetFlagForHiddenEntities()
>>> print(flag)
```

`GetGeneratedQuadPanelIncidences()` [source]

Get the incidences of generated quad panels for selected beams.

Returns:

List of 4 lists containing NodeAs, NodeBs, NodeCs, NodeDs in respective order. [[A1, A2, ...], [B1, B2, ...], [C1, C2, ...], [D1, D2, ...]]

Return type:

List of lists of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> incidences = staad_obj.Geometry.GetGeneratedQuadPanelIncidences(
```

GetGroupCount(*grouptype*) [source]

Get the number of groups of a given type.

Parameters:

grouptype (*int*) –

Index	Group Type
1	Nodes
2	Members
3	Plates
4	Solids
5	Geometry (Members, Plates and Solids)
6	Floor (Floor beam)

Returns:

Number of groups.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Geometry.GetGroupCount(1) # For Node groups
>>> print(count)
```

GetGroupCountAll() [source]

Get the total number of groups.

Returns:

Total group count.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> total_groups = staad_obj.Geometry.GetGroupCountAll()
>>> print(total_groups)
```

GetGroupEntities(*group_name*) [source]

Get the list of entities in a group.

Parameters:

group_name (*str*) – Name of the group.

Returns:

Entity numbers in the group.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> entities = staad_obj.Geometry.GetGroupEntities("Group1")
>>> print(entities)
```

GetGroupEntityCount(*group_name*) [source]

Get the number of entities in a group.

Parameters:

group_name (*str*) – Name of the group.

Returns:

Number of entities in the group.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Geometry.GetGroupEntityCount("Group1")
>>> print(count)
```

GetGroupNames(*grouptype*) [source]

Get the names of all groups of a given type.

Parameters:

grouptype (*int*) –

Index	Group Type
1	Nodes
2	Members
3	Plates
4	Solids
5	Geometry (Members, Plates and Solids)
6	Floor (Floor beam)

Returns:

List of group names.

Return type:

list of str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> names = staad_obj.Geometry.GetGroupNames(1)
>>> print(names)
```

GetIntersectBeamsCount(*beamList: list, tolerance: float*)

Get the count of intersecting beams. [source]

Parameters:

- **beamList** (*list of int*) – list of beam IDs to check for intersection. if it is empty, all beams in the model are considered.
- **tolerance** (*float*) – Tolerance to be used for finding beam intersection, should not be negative value, meter for Metric and inch for English in Base Units.

Returns:

Number of intersecting beams.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Geometry.GetIntersectBeamsCount([1,2,3])
```

GetLastBeamNo()

[source]

Get the last beam ID.

Returns:

Last beam number.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> last_beam = staad_obj.Geometry.GetLastBeamNo()
>>> print(last_beam)
```

GetLastNodeNo()

[source]

Get the last node number.

Returns:

The last node number. - 1 : General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> last_node = staad_obj.Geometry.GetLastNodeNo()
>>> print(last_node)
```

GetLastPhysicalMemberNo()

[source]

Get the last physical member number.

Returns:

Last physical member number.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> last_no = staad_obj.Geometry.GetLastPhysicalMemberNo()
>>> print(last_no)
```

GetLastPlateNo()

[source]

Get the last plate number.

Returns:

Last plate number.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> last_plate = staad_obj.Geometry.GetLastPlateNo()
>>> print(last_plate)
```

GetLastSolidNo()

[source]

Returns the solid number of the last solid created in the model.

Returns:

Last solid number.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> last_solid = staad_obj.Geometry.GetLastSolidNo()
```

GetMemberCount()

[source]

Get number of beam.

Returns:

Number of beams.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_count = staad_obj.Geometry.GetMemberCount()
>>> print(beam_count)
```

GetMemberIncidence(*beam*) [source]

Get the start and end node numbers of a beam.

Parameters:

beam (*int*) – Beam number.

Returns:

(start_node, end_node)

Return type:

tuple of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> start, end = staad_obj.Geometry.GetMemberIncidence(1)
>>> print(f"Start Node: {start}, End Node: {end}")
```

GetMemberIncidence_CIS2(*memberId: int*) [source]

Get the incidence of a member in CIS/2 format.

Parameters:

memberId (*int*) – ID of the member.

Returns:

(unique_str_id, start_node, end_node) where: unique_str_id : Unique string ID of the member. (str) start_node : Start node ID of the member. (int) end_node : End node ID of the member. (int)

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> incidence = staad_obj.Geometry.GetMemberIncidence_CIS2(1)
>>> print(incidence)
```

GetMemberUniqueID(*memberNo: int*) [source]

Get the unique ID of a member.

Parameters:

memberNo (*int*)

Returns:

Unique ID of the member.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> uid = staad_obj.Geometry.GetMemberUniqueID(1)
>>> print(uid)
```

GetNoOfBeamsConnectedAtNode(*node*) [source]

Get the number of beams connected at a node.

Parameters:

node (*int*) – Node number.

Returns:

Number of beams connected at the node.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> connected_beam_count = staad_obj.Geometry.GetNoOfBeamsConnectedA
>>> print(connected_beam_count)
```

GetNoOfGeneratedQuadPanels() [source]

Get the number of generated quad panels for selected beams.

Returns:

Number of generated quad panels.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> quad_panel_count = staad_obj.Geometry.GetNoOfGeneratedQuadPanels
>>> print(quad_panel_count)
```

GetNoOfSelectedBeams() [source]

Get the number of selected beams.

Returns:

Number of selected beams.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> selected_beam_count = staad_obj.Geometry.GetNoOfSelectedBeams()
>>> print(selected_beam_count)
```

GetNoOfSelectedNodes()

[source]

Get the number of selected nodes.

Returns:

Number of selected nodes.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> no_of_selected_nodes = staad_obj.Geometry.GetNoOfSelectedNodes()
>>> print(no_of_selected_nodes)
```

GetNoOfSelectedPhysicalMembers()

[source]

Get the number of selected physical members.

Returns:

Number of selected physical members.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Geometry.GetNoOfSelectedPhysicalMembers()
>>> print(count)
```

GetNoOfSelectedPlates()

[source]

Return the number of selected plates.

Returns:

Number of selected plates.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> selected_plate_count = staad_obj.Geometry.GetNoOfSelectedPlates(
>>> print(selected_plate_count)
```

GetNoOfSelectedSolids() [source]

Get the number of selected solids.

Returns:

Number of selected solids.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> n_solids = staad_obj.Geometry.GetNoOfSelectedSolids()
>>> print(n_solids)
```

GetNodeCoordinates(*node: int*) [source]

Get the coordinates of a node.

Parameters:

node (*int*) – Node number.

Returns:

(x, y, z) coordinates.

Return type:

tuple of float

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> coords = staad_obj.Geometry.GetNodeCoordinates(1)
>>> print(coords)
```

GetNodeCount()

[source]

Get the total number of nodes.

Returns:

Number of nodes.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Geometry.GetNodeCount()
>>> print(count)
```

GetNodeDistance(*nodeA*, *nodeB*)

[source]

Get the distance between two nodes.

Parameters:

- **nodeA** (*int*) – First node number.
- **nodeB** (*int*) – Second node number.

Returns:

Distance between the nodes.

Return type:

float

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> dist = staad_obj.Geometry.GetNodeDistance(1, 2)
>>> print(dist)
```

GetNodeIncidence(*node*) [source]

Get the incidence (coordinates) of a node.

Parameters:

node (*int*) – Node number.

Returns:

(x, y, z) Coordinates of the node.

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> inc = staad_obj.Geometry.GetNodeIncidence(1)
>>> print(inc)
```

GetNodeIncidence_CIS2(*nodeId: int*) [source]

Get the incidence of a node in CIS/2 format.

Parameters:

nodeId (*int*) – ID of the node.

Returns:

(unique_str_id, x, y, z) where: unique_str_id : Unique string ID of the node. (str) x : X-coordinate of the node. (float) y : Y-coordinate of the node. (float) z : Z-coordinate of the node. (float)

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> incidence = staad_obj.Geometry.GetNodeIncidence_CIS2(1)
>>> print(incidence)
```

GetNodeList() [source]

Get the list of all node numbers.

Returns:

List of node numbers.

Return type:

list

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> nodes = staad_obj.Geometry.GetNodeList()
>>> print(nodes)
```

GetNodeNumber(*x_y_z_coordinates: tuple*) [source]

Get the node number from coordinates.

Parameters:

x_y_z_coordinates (*tuple of float*) – (x, y, z) coordinates.

Returns:

Node number.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> node_no = staad_obj.Geometry.GetNodeNumber((0.0, 0.0, 0.0))
>>> print(node_no)
```

GetNodeUniqueID(*nodeNo: int*) [source]

Get the unique ID of a node.

Parameters:

nodeNo (*int*)

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> uid = staad_obj.Geometry.GetNodeUniqueID(1)
>>> print(uid)
```

GetPID(*EntityNo: int, EntityType: int*) [source]

Get the property ID of a member.

Parameters:

- **EntityNo** (*int*) – ID of the entity.
- **EntityType** (*int*) – Type of the entity. (1 for Node, 2 for Beam, 3 for Plate, 4 for Solid, 5 for Surface).

Returns:

Property ID of the member.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> pid = staad_obj.Geometry.GetPID(1)
>>> print(pid)
```

GetPMemberCount() [source]

Get the count of physical members in the model.

Returns:

Count of physical members.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Geometry.GetPMemberCount()
>>> print(count)
```

GetParametricSurfaceCount()

[source]

Get the number of parametric surfaces.

Returns:

Number of parametric surfaces.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> num_surfaces = staad_obj.Geometry.GetParametricSurfaceCount()
```

GetParametricSurfaceInfo(*surfaceNo: int*)

[source]

Get extended information about a parametric surface.

Parameters:

surfaceNo (*int*)

Returns:

1. Surface Name (str) : Name of the Mesh
2. Surface Type (str) : Type of the Mesh (e.g., Slab, Wall)
3. boundary points count (int) : Number of boundary points
4. density points count (int) : Number of density points
5. opening count (int) : Number of openings
6. region count (int) : Number of regions

Return type:

tuple of various details about the surface

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> surface_name, surface_type, boundary_points, density_points, ope
```

GetParametricSurfaceInfoEx(*surfaceNo: int*) [source]

Get information about a parametric surface.

Parameters:

surfaceNo (*int*)

Returns:

1. Surface Name (str) : Name of the Mesh
2. Surface Type (int) : (0: None, 1: Wall, 2: Slab)
3. Surface sub-type (str) : Sub type of the surface
4. number of vertices (int) : Number of vertices after meshing
5. Mesh Size (float) : Target mesh size
6. Divisions (int) : Number of divisions along the boundary
7. Meshing method (int) : (0: Basic, 1: Advanced)
8. isQuad (bool) : Whether the mesh is Quad or Triangular (True = Quad, False = Triangular)
9. Origin Node (int) : Origin Node ID
10. X Node (int) : Node ID on X axis to determine x axis
11. Y Node (int) : Node ID towards positive Y axis
12. Number of Circular Openings (int) : Number of circular openings
13. Number of Polygonal Openings (int) : Number of polygonal openings
14. Number of Circular Regions (int) : Number of circular regions
15. Number of Polygonal Regions (int) : Number of polygonal regions
16. Number of Density Points (int) : Number of density points
17. Number of Density Lines (int) : Number of density lines

Return type:

tuple containing various details about the surface

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> surface_name, surface_type, surface_sub_type, number_of_vertices
```

GetParametricSurfaceMeshData(*surfaceNo: int*) [source]

Gets data about specified parametric surface available in the currently loaded model

Parameters:

surfaceNo (*int*)

Returns:

1. Nodes (list) : List of generated node ids
2. Elements (list) : List of generated element(plate) ids

Return type:

tuple containing mesh data

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> mesh_data = staad_obj.Geometry.GetParametricSurfaceMeshData(1)
>>> print(mesh_data)
```

GetParametricSurfaceMeshInfo(*surfaceNo: int*) [source]

Gets information about specified parametric surface available in the currently loaded model.

Parameters:

surfaceNo (*int*) – Surface ID of the parametric surface.

Returns:

1. Node count (int) : Number of nodes in the mesh
2. Element count (int) : Number of elements in the mesh

Return type:

tuple containing mesh details

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> nodes, elements = staad_obj.Geometry.GetParametricSurfaceMeshInf
>>> print(f"Nodes: {nodes}, Elements: {elements}")
```

GetParametricSurfaceSubType(*surfaceName: str*) [source]

Get the subtype of a parametric surface.

Parameters:

surfaceNo (*int*)

Returns:

str

Return type:

subtype information

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> subtype = staad_obj.Geometry.GetParametricSurfaceSubType("SECOND
>>> print(subtype)
```

GetParametricSurfaceUniqueID(*surface_name: str*) [source]

Get the unique ID of a parametric surface.

Parameters:

surface_name (*str*)

Returns:

Unique ID of the parametric surface.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> uid = staad_obj.Geometry.GetParametricSurfaceUniqueID("SECOND_FL
>>> print(uid)
```

GetPhysicalMemberCount()

[source]

Get the count of physical members in the model.

Returns:

Count of physical members.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Geometry.GetPhysicalMemberCount()
>>> print(count)
```

GetPhysicalMemberList()

[source]

Get the list of physical members in the model.

Returns:

List of physical member IDs.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> members = staad_obj.Geometry.GetPhysicalMemberList()
>>> print(members)
```

GetPhysicalMemberUniqueID(*physicalMemberId: int*)

[source]

Get the unique ID of a physical member.

Parameters:

physicalMemberId (*int*) – ID of the physical member.

Returns:

Unique ID of the physical member.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> uid = staad_obj.Geometry.GetPhysicalMemberUniqueID(1)
>>> print(uid)
```

`GetPlateCount()` [source]

Returns the number of plates.

Returns:

Number of plates.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plate_count = staad_obj.Geometry.GetPlateCount()
>>> print(plate_count)
```

`GetPlateIncidence(plateNo: int)` [source]

Get the node incidences A, B, C, D for a plate.

Parameters:

plateNo (*int*) – Plate number.

Returns:

4 end node IDs for the plate (A, B, C, D).

Return type:

tuple of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> a, b, c, d = staad_obj.Geometry.GetPlateIncidence(1)
>>> print(a, b, c, d)
```

GetPlateIncidence_CIS2(*plateId: int*) [source]

Get the incidence of a plate in CIS/2 format.

Parameters:

plateId (*int*) – ID of the plate.

Returns:

(unique_str_id, nodeA, nodeB, nodeC, nodeD) where: unique_str_id : Unique string ID of the plate. (str) nodeA : Node A ID of the plate. (int) nodeB : Node B ID of the plate. (int) nodeC : Node C ID of the plate. (int) nodeD : Node D ID of the plate. (int)

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> incidence = staad_obj.Geometry.GetPlateIncidence_CIS2(1)
>>> print(incidence)
```

GetPlateList() [source]

Returns the list of all plate numbers.

Returns:

List of plate numbers.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plates = staad_obj.Geometry.GetPlateList()
>>> print(plates)
```

GetPlateNodeCount(*plateNo: int*) [source]

Get the number of nodes in a plate.

Parameters:

plateNo (*int*) – Plate number.

Returns:

Number of nodes in the plate.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> node_count = staad_obj.Geometry.GetPlateNodeCount(1)
>>> print(node_count)
```

GetPlateUniqueID(*plateNo: int*) [source]

Get the unique ID of a plate.

Parameters:

plateNo (*int*)

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> uid = staad_obj.Geometry.GetPlateUniqueID(1)
>>> print(uid)
```

GetSelectedBeams()

[source]

Get the list of selected beam numbers.

Returns:

Selected beam numbers.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> selected = staad_obj.Geometry.GetSelectedBeams()
>>> print(selected)
```

GetSelectedNodes()

[source]

Get the list of selected node numbers.

Returns:

Selected node numbers.

Return type:

list

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> selected = staad_obj.Geometry.GetSelectedNodes()
>>> print(selected)
```

GetSelectedPhysicalMembers()

[source]

Get the list of selected physical members.

Returns:

List of selected physical member IDs.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> members = staad_obj.Geometry.GetSelectedPhysicalMembers()
>>> print(members)
```

GetSelectedPlates(*isSorted: bool = False*) [source]

return a list of selected plate numbers.

Parameters:

isSorted (*bool optional*) – If True, the plate numbers will be sorted. The default is False. (in the selected order)

Returns:

Selected plate numbers.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plates = staad_obj.Geometry.GetSelectedPlates(True)
>>> print(plates)
```

GetSelectedSolids(*isSorted: bool = False*) [source]

Get the list of selected solid numbers.

Parameters:

isSorted (*bool optional*) – If True, the solid numbers will be sorted. The default is False. (in the selected order)

Returns:

Selected solid numbers.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> solids = staad_obj.Geometry.GetSelectedSolids(True)
>>> print(solids)
```

GetSolidCount()

[source]

Returns the number of solids.

Returns:

Number of solids.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> solid_count = staad_obj.Geometry.GetSolidCount()
>>> print(solid_count)
```

GetSolidIncidence(*solidNo*)

[source]

Get the node incidences for a solid.

Returns:

8 end node IDs for the solid. (A, B, C, D, E, F, G, H)

Return type:

tuple of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> nodes = staad_obj.Geometry.GetSolidIncidence(1)
>>> print(nodes)
```

GetSolidIncidence_CIS2(*solidId: int*) [source]

Get the incidence of a solid in CIS/2 format.

Parameters:

solidId (*int*) – ID of the solid.

Returns:

(unique_str_id, nodeA, nodeB, nodeC, nodeD, nodeE, nodeF, nodeG, nodeH) where: unique_str_id : Unique string ID of the solid. (str) nodeA : Node A ID of the solid. (int) nodeB : Node B ID of the solid. (int) nodeC : Node C ID of the solid. (int) nodeD : Node D ID of the solid. (int) nodeE : Node E ID of the solid. (int) nodeF : Node F ID of the solid. (int) nodeG : Node G ID of the solid. (int) nodeH : Node H ID of the solid. (int)

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> incidence = staad_obj.Geometry.GetSolidIncidence_CIS2(1)
>>> print(incidence)
```

GetSolidList() [source]

Get the list of all solid numbers.

Returns:

List of solid numbers.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> solids = staad_obj.Geometry.GetSolidList()
>>> print(solids)
```

GetSolidUniqueID(*solidNo: int*) [source]

Get the unique ID of a solid.

Parameters:

solidNo (*int*)

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> uid = staad_obj.Geometry.GetSolidUniqueID(1)
>>> print(uid)
```

IntersectBeams(*method: int, beamList: list, tolerance: float*)

Intersect beams. [source]

Parameters:

- **method** (*int*) –

Index	Method
1	Highlight
2	Intersect

- **beamList** (*list of int*) – list of beam IDs to intersect. if it is empty, all beams in the model are considered.
- **tolerance** (*float*) – Tolerance to be used for finding beam intersection, should not be negative value, meter for Metric and inch for English in Base Units.

Returns:

IDs of the beams that have been changed and added, only used for intersect method.

Return type:

List of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> new_Ids = staad_obj.Geometry.IntersectBeams([1,2,3])
>>> print(new_Ids)
```

IsBeam(*beam_no: int, tol_angle: float*) [source]

Returns True if the angle of inclination for specified BEAM member is not more than given tolerance angle (for small angle only).

Parameters:

- **beam_no** (*int*)
- **tol_angle** (*float*)

Returns:

True if the beam is within the tolerance angle, False otherwise.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> is_beam = staad_obj.Geometry.IsBeam(1, 1)
>>> print(is_beam)
```

IsColumn(*column_no: int, tol_angle: float*) [source]

Returns True if the angle of inclination for specified COLUMN member is not more than given tolerance angle (for small angle only).

Parameters:

- **column_no** (*int*)
- **tol_angle** (*float*)

Returns:

True if the column is within the tolerance angle, False otherwise.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> is_column = staad_obj.Geometry.IsColumn(1, 1)
>>> print(is_column)
```

IsOrphanNode(*nodeNo: int*) [source]

Check if a node is an orphan.

Parameters:

nodeNo (*int*)

Returns:

True if orphan, False otherwise.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> is_orphan = staad_obj.Geometry.IsOrphanNode(1)
>>> print(is_orphan)
```

IsZUp() [source]

Check if the Z axis is up.

Returns:

True if Z is up, False otherwise.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> is_z_up = staad_obj.Geometry.IsZUp()
>>> print(is_z_up)
```

`MergeBeams(beamList: list, newId: int, property_id: int, beta_angle: float, material_name: str)` [source]

Merge beams.

Parameters:

- **beamList** (*list of int*) – List of beam IDs to merge.
- **newId** (*int*) – New ID for the merged beam.
- **property_id** (*int*) – Property ID to assign to the merged beam.
- **beta_angle** (*float*) – Beta angle for the merged beam.
- **material_name** (*str*) – Material name for the merged beam.

Returns:

True if successful, False otherwise.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.MergeBeams([1,2,3], 4, 5, 30.0, "Ste
>>> print(result)
```

`MergeNodes(new_Id: int, nodeList: list)` [source]

Merge nodes.

Parameters:

- **new_Id** (*int*) – New ID for the merged node.

- **nodeList** (*list of int*) – List of node IDs to merge.

Returns:

True if successful, False otherwise.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.MergeNodes(4, [1,2,3])
>>> print(result)
```

RemoveParametricSurfaceMesh(*surfaceNo: int*) [source]

Remove the specified parametric mesh from the model.

Parameters:

surfaceNo (*int*) – surface ID of the parametric surface to be delete from model.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.RemoveParametricSurfaceMesh(1)
>>> print(result)
```

RenumberBeam(*oldBeamNo: int, newBeamNo: int*) [source]

Renumber a beam.

Parameters:

- **oldBeamNo** (*int*)
- **newBeamNo** (*int*)

Returns:

True if successful, False otherwise.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.RenumberBeam(1, 10)
>>> print(result)
```

SelectBeam(*beamID*) [source]

Select a beam.

Parameters:

beamID (*int*) – beam number to select.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.SelectBeam(1)
>>> print(result)
```

SelectMultipleBeams(*beam_ids: list*) [source]

Select multiple beams.

Parameters:

beam_ids (*list of int*) – List of beam numbers to select.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SelectMultipleBeams([1, 2, 3])
```

SelectMultipleNodes(*nodes: list*) [source]

Select multiple nodes.

Parameters:

nodes (*list*) – node numbers to select.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.SelectMultipleNodes([1,2,3])
>>> print(result)
```

SelectMultiplePhysicalMembers(*physicalMemberList: list*)

Select multiple physical members. [source]

Parameters:

physicalMemberList (*list of int*) – List of physical member IDs to select.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SelectMultiplePhysicalMembers([1,2,3])
```

SelectMultiplePlates(*plates: list*) [source]

Select multiple plates.

Parameters:

plates (*list*) – Plate numbers to select.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.SelectMultiplePlates([1,2,3])
>>> print(result)
```

SelectMultipleSolids(*solids: list*) [source]

Select multiple solids.

Parameters:

solids (*list*) – Solid numbers to select.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.SelectMultipleSolids([1,2,3])
>>> print(result)
```

SelectNode(*nodeID*) [source]

Select a node.

Parameters:

nodeID (*int*) – node number to select.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.SelectNode(1)
>>> print(result)
```

SelectPhysicalMember(*physicalMemberId: int*) [source]

Select a physical member.

Parameters:

physicalMemberId (*int*) – ID of the physical member to select.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SelectPhysicalMember(1)
```

SelectPlate(*nPlateNo: int*) [source]

Select a plate by its number.

Parameters:

nPlateNo (*int*) – Plate number to select.

Returns:

Status of selection

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.SelectPlate(1)
>>> print(result) # True if selected, False otherwise
```

SelectSolid(*solidID*) [source]

Select a solid.

Parameters:

solidID (*int*) – solid number to select.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Geometry.SelectSolid(1)
>>> print(result)
```

SetCheckForIdenticalEntity(*entityType: int, checkFlag: bool*) [source]

This API will set whether to enable checking for existing identical entities (beam, plate, node etc.) or not. If set is enabled, time taken by the corresponding add/create multiple entities APIs will take longer time, otherwise if set is disabled, time taken by corresponding APIs will be comparatively less.

Parameters:

- **entityType** (*int*) – Type of the entity. (1 for Node, 2 for Beam, 3 for Plate, 4 for Solid, 5 for Surface).
- **checkFlag** (*bool*) – Flag to check for identical entities.

Returns:

True if successful, False otherwise.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SetCheckForIdenticalEntity(1, True)
```

SetFlagForHiddenEntities(*flag: int*) [source]

Set the flag specified for consideration of hidden entities (nodes and plates) while getting count or list of those entities

Parameters:

flag (*int*) – All entities = 0 (Default option), Ignore Hidden entities = 1, Only hidden entities = 2

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SetFlagForHiddenEntities(1)
```

SetMemberUniqueID(*beamNo: int, uniqueID: str*) [source]

Set a unique ID for a member.

Parameters:

- **beamNo** (*int*) – Beam number to set the unique ID for.
- **uniqueID** (*str*) – unique identifier for the member.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SetMemberUniqueID(1, "beam-uuid")
```

SetNodeCoordinate(*nodeNo: int, x: float, y: float, z: float*)

Set the coordinates of a node. [source]

Parameters:

- **nodeNo** (*int*)

- **x** (*float*)
- **y** (*float*)
- **z** (*float*)

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SetNodeCoordinate(1, 0.0, 0.0, 0.0)
```

SetNodeUniqueID(*nodeNo: int, uniqueID: str*) [source]

Set a unique ID for a node.

Parameters:

- **nodeNo** (*int*) – Node number to set the unique ID for.
- **uniqueID** (*str*) – Unique identifier for the node.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SetNodeUniqueID(1, "node-uuid")
```

SetPID(*EntityNo: int, EntityType: int, PropertyID: int*) [source]

Set the property ID of a member.

Parameters:

- **EntityNo** (*int*) – ID of the entity.
- **EntityType** (*int*) – Type of the entity. (1 for Node, 2 for Beam, 3 for Plate, 4 for Solid, 5 for Surface).
- **PropertyID** (*int*) – Property ID to set for the member.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SetPID(1, 2, 5)
```

SetParametricSurfaceSubType(*surfaceName: str, subType: str*)

Set the subtype for a parametric surface. [source]

Parameters:

- **surfaceName** (*str*) – Name of the parametric surface.
- **subType** (*str*) – Sub-type of surface.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SetParametricSurfaceSubType("SECOND_FLOOR_SLA
```

SetParametricSurfaceUniqueID(*surface_name: str, unique_id: str*)

[source]

Set a unique ID for a parametric surface.

Parameters:

- **surface_name** (*str*)
- **unique_id** (*str*)

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SetParametricSurfaceUniqueID("SECOND_FLOOR_SL
```

SetPhysicalMemberUniqueID(*physicalMemberId: int*, *uniqueId: str*) [source]

Set the unique ID for a physical member.

Parameters:

- **physicalMemberId** (*int*) – ID of the physical member.
- **uniqueId** (*str*) – Unique ID to set for the physical member.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SetPhysicalMemberUniqueID(1, "physical-member
```

SetPlateUniqueID(*plateNo: int*, *uniqueID: str*) [source]

Set a unique ID for a plate.

Parameters:

- **plateNo** (*int*) – plate number to set the unique ID for.
- **uniqueID** (*str*) – unique identifier for the plate.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SetPlateUniqueID(1, "plate-uuid")
```

SetSolidUniqueID(*solidNo: int, uniqueID: str*) [source]

Set a unique ID for a solid.

Parameters:

- **solidNo** (*int*) – Solid number to set the unique ID for.
- **uniqueID** (*str*) – Unique identifier for the solid.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SetSolidUniqueID(1, "solid-uuid")
```

SplitBeam(*beamNo: int, nodes: int, distToNodes: list*) [source]

Split a beam into parts.

Parameters:

- **beamNo** (*int*) – Beam ID to split.
- **nodes** (*int*) – The number of node(s) to be inserted in the beam.
- **distToNodes** (*list*) – List of distances in from the start of the beam to each new node.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SplitBeam(1, 2, [1.0, 2.0])
```

SplitBeamInEqlParts(*nBeamNo: int, nParts: int*) [source]

Split a beam into equal parts.

Parameters:

- **nBeamNo** (*int*) – Beam number to split.
- **nParts** (*int*) – Number of equal parts to split the beam into.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.SplitBeamInEqlParts(1, 3)
```

UpdateGroup(*groupName: str, update_option: int, entityList: list[int]*) [source]

Updates (replaces, removes, adds) entities to a specified group.

Parameters:

- **groupName** (*str*)
- **update_option** (*int*) –

Index	Update Option
0	Replace entities
1	Remove entities
2	Add entities

- **entityList** (*list of int*)

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Geometry.UpdateGroup("GroupA", [1,2,3])
```

__init__(*staadObj*) [source]

Load

Contents

- OSLoad

class openstaadpy.os_analytical.osload.OSLoad [source]

Bases: object

AddAutoCombinationRepeat(*varCode: str, varCategory: str, varLoadList: list, varStartLoadCaseNo: int, varGeneratedLCS: int, bVarReference: bool, bVarNotional: bool, dVarNotionalLoadFactor: float, bVarGB50017: bool, nVarFloor: int, bVarX: bool, bVarNegtiveX: bool, bVarZ: bool, bVarNegtiveZ: bool*)

Automatically adds repeat load based on assigned design code and Category. [source]

Parameters:

- **varCode** (*bool*) – Load Combination Code string name (refer to “Codes.ini”)
- **varCategory** (*bool*) – Load Combination Category string name (refer to corresponding rule ini file defined in “Codes.ini”)
- **varLoadList** (*List of int*) – List of Load case reference IDs. If the array is either null or empty then all load cases in current model will be considered.
- **varStartLoadCaseNo** (*int*) – (Repeat Load) Load case reference ID with which automatically generation starts. If nStartLoadCaseNo is valid, auto repeat load will be created from the provided ID. If nStartLoadCaseNo is invalid Load Case ID/already present Load Case ID, the repeat load would automatically generated from next available Load Case ID and nStartLoadCaseNo will be returned/updated with this ID.
- **varGeneratedLCS** (*int*) – (Repeat Load) The counts of automatically generated repeat loads.
- **bVarReference** (*bool*) – Whether include Reference load
- **bVarNotional** (*bool*) – Whether include Notional load. If it’s True but all Directions are Flase, return -1.
- **dVarNotionalLoadFactor** (*float*) – If bVarNotional is valid, the value of Notional load factor
- **bVarGB50017** (*bool*) – Consider Notional load factor per GB 50017 Design code
- **nVarFloor** (*int*) – The count of floor, it is valid when bVarGB50017 is True only
- **bVarX** (*bool*) – Consider X Direction of Notional Load
- **bVarNegtiveX** (*bool*) – Consider -X Direction of Notional Load
- **bVarZ** (*bool*) – Consider Z Direction of Notional Load
- **bVarNegtiveZ** (*bool*) – Consider -Z Direction of Notional Load

Returns:

- *int* – Returns 0 if successful. Returns -1 if unsuccessful
- *Notes* –
  - The default path of Codes.ini under “%localappdata%BentleyEngineeringSTAAD.Pro 2023DefaultLanguageen”.

AddAutoLoadCombinations(*loadCombCode: str, loadCombCategory: str, loadList: list*) [source]

Automatically adds load combination based on assigned design code and Category.

Parameters:

- **loadCombCode** (*str*) – Load Combination Code string name (refer to “Codes.ini”)
- **loadCombCategory** (*str*) – Load Combination Category string name (refer to corresponding rule ini file defined in “Codes.ini”)
- **loadList** (*list of int*) – Load case reference ID(s), Array of Load case numbers. If the array is either null or empty then all load cases in current model will be considered

Returns:

- *int* – Returns load case reference ID with which automatically load combination generation starts. If nStartLoadCaseNo is valid, auto load combinations will be created from the provided ID. IfnStartLoadCaseNo is

invalid Load Case ID already present Load Case ID, load combinations would automatically generated from next available Load Case ID and nStartLoadCaseNo will be returned/updatedwith this ID.

- *>>> from openstaadpy import os_analytical*
- *>>> staad_obj = os_analytical.connect()*
- *>>> staad_obj.Load.AddAutoLoadCombinations("AISC 9th Ed","2.3 LRFD General", [1, 2, 3])*

`AddDirectAnalysisDefinitionParameter(pParamType: int, members: list, param: float)` [source]

Adds Direct Analysis Definition (FLEX,AXIAL parameters).

Parameters:

- **pParamType** (*int*) –

  Integer indicating type of direct analysis parameter to be added. Integer value should be taken from following table :

Value	AnalysisCommand
FLEX = 0	DirectAnalysisParameterTypes.FLEX
AXIAL = 2	DirectAnalysisParameterTypes.AXIAL

- **members** (*list of int*) – List of Member IDs
- **param** (*float*) – FLEX parameter value. [For AXIAL this value is Not Applicable. Should pass 0(zero)]

Returns:

- *bool* – Returns TRUE if successful Returns FALSE if unsuccessful
- *>>> from openstaadpy import os_analytical*
- *>>> staad_obj = os_analytical.connect()*
- *>>> staad_obj.Load.AddDirectAnalysisDefinitionParameter(0, [1, 2], 0.5)*
- *True*

`AddElementHydrostaticPressure(plateIds: list, varLoadDirection: int, varInterpolateDirection: int, varMinLoad: float, varMaxLoad: float)` [source]

Adds Hydrostatic pressure loading to plate elements.

Parameters:

- **plateIds** (*list of int*) – List of plate IDs.
- **varLoadDirection** (*int*) – Load direction: (= 3 to 6 for LocalZ, GlobalX, GlobalY, GlobalZ, respectively)
- **varInterpolateDirection** (*int*) – Interpolate along Global Axis(Int or Long), valid direction codes are 1, 2, 3 for Interpolate along Global X, Y, Z. No other direction is a valid input.
- **varMinLoad** (*float*) – Minimum Pressure load
- **varMaxLoad** (*float*) – Maximum Pressure load

Returns:

True if successfully. False if an error occurred.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddElementHydrostaticPressure([1], 3, 1, 0.0, 10.0)
```

`AddElementPressure(plateIds: list, varDirection: int, varPressure: float, varX1: float, varY1: float, varX2: float, varY2: float)` [source]

Adds pressure load to plate elements.

Parameters:

- **plateIds** (*list of int*) – List of plate IDs.
- **varDirection** (*int*) – Load direction: (1 to 9 for LocalX, LocalY, LocalZ, GlobalX, GlobalY, GlobalZ, ProjectedX, ProjectedY and ProjectedZ respectively).
- **varPressure** (*float*) – Magnitude of the pressure or concentrate load on the element.
- **varX1** (*float*) – Top-Left coordinate X (local).
- **varY1** (*float*) – Top-Left coordinate Y (local).
- **varX2** (*float*) – Bottom-Right coordinate X (local).
- **varY2** (*float*) – Bottom-Right coordinate Y (local).
- **Notes**
- **X1** (*- If*)
- **Y1** (*Pressure applied over the area between (X1 , Y1) and (X2 , Y2) measured from the center of plate(s) in the local axis system.*)
- **0** (*but X2 and Y2 are*)
- **element.** (*the pressure is applied over the full area of the*)
- **X1**
- **Y1**
- **0**
- **0**
- **0**

Returns:

Returns 0 if OK. Returns -1 if General error. Returns -8001 if Load direction is invalid.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddElementPressure([1], 3, 5.0, 0.0, 0.0, 1.0, 1.0)
```

`AddElementTrapPressureEx(PlateIDs: list, LoadDirection: int, LoadVaryDirection: int, StartPressure: float, EndPressure: float, Pressure3: float, Pressure4: float)` [source]

Adds trapezoidal pressure loading to plate elements.

Parameters:

- **PlateNo** (*list of int*) – List of Plate IDs.
- **LoadDirection** (*int*) – Load direction: (= 3 to 6 for LocalZ, GlobalX, GlobalY, GlobalZ, respectively)
- **LoadVaryDirection** (*int*) – Load varying direction: (= 1, 2 ,3 for X, Y and JOINT respectively)
- **StartPressure** (*float*) – Pressure at loading starting point.(Node1 when JOINT is selected)
- **EndPressure** (*float*) – Pressure at loading ending point.(Node2 when JOINT is selected)
- **Pressure3** (*float*) – Pressure at loading point.(applicable only when JOINT is selected)
- **Pressure4** (*float*) – Pressure at loading point.(applicable only when JOINT is selected)

Returns:

True if OK. False if any General error.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddElementTrapPressureEx([1], 3, 1, 100.0, 200.0, 150.0, 250.0)
```

AddLoadAndFactorToCombination(*loadCombNo: int, loadNo: int, factor: float*) [source]

Adds a primary load case with specified multiplication factor to an existing load combination.

Parameters:

- **loadCombNo** (*int*) – Load Combination Number.
- **loadNo** (*int*) – Load Case Reference ID.
- **factor** (*float*) – Multiplication factor for the specified primary load case.

Returns:

Returns 0 OK. Returns -1 General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddLoadAndFactorToCombination(1, 2, 1.0)
```

AddLoadCasesToEnvelop(*varEnvNo: int, varLoadCaseList: list*) [source]

Adds a list of primary load case(s) to an existed load envelop.

Parameters:

- **varEnvNo** (*int*) – Load Envelop reference ID
- **varLoadCaseList** (*list of int*) – Load cases reference IDs list.

Returns:

- *int* – Returns 1 if OK. Returns 0 if general error.
- *>>> from openstaadpy import os_analytical*
- *>>> staad_obj = os_analytical.connect()*
- *>>> staad_obj.Load.AddLoadCasesToEnvelop(1, [1, 2, 3])*

AddMemberAreaLoad(*beamIds: list, load: float*) [source]

Adds AREA LOAD to beams.

Parameters:

- **beamIds** (*list of int*) – List of Beam IDs.
- **load** (*float*) – Magnitude of the load value.

Returns:

Returns 0 if OK. Returns -1 if General error.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddMemberAreaLoad([1], 5.0)
```

AddMemberConcForce(*beamIds: list, varDirection: int, varForce: float, varD1: float, varD2: float*)

Adds a concentrated force to the specified beams. [source]

Parameters:

- **beamIds** (*list of int*) – List of beam IDs.
- **varDirection** (*int*) – Load direction (1 to 6 for LocalX, LocalY, LocalZ, GlobalX, GlobalY, GlobalZ respectively).
- **varForce** (*float*) – Magnitude of the concentrate force in current units.
- **varD1** (*float*) – Distance from the start of the member to concentrated force.
- **varD2** (*float*) – Perpendicular distance from the member shear center to the local plane of loading.

Returns:

Returns 0 OK. Returns -1 General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.AddMemberConcForce([1], 1, 20.0, 2.5, 0)
>>> print(result)
```

`AddMemberConcMoment(beamIds: list, varDirection: int, varMoment: float, varD1: float, varD2: float)`

Adds a concentrated moment to the specified beams. [source]

Parameters:

- **beamIds** (*list of int*) – List of beam IDs.
- **varDirection** (*int*) – Load direction (1 to 6 for LocalX, LocalY, LocalZ, GlobalX, GlobalY, GlobalZ respectively).
- **varMoment** (*float*) – Magnitude of the concentrate moment in current units.
- **varD1** (*float*) – Distance from the start of the member to concentrated moment.
- **varD2** (*float*) – Perpendicular distance from the member shear center to the local plane of loading.

Returns:

Returns True if successful.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.AddMemberConcMoment([1], 2, 15.0, 3.0, 0)
>>> print(result)
```

`AddMemberFixedEnd(beamIds: list, loadStart: float, loadEnd: float)` [source]

Adds FIXED END LOAD to beams.

Parameters:

- **beamIds** (*list of int*) – List of Beam IDs.
- **loadStart** (*list of float*) – Load at starting point in form of array containing 6 elements corresponding to FX, FY, FZ, MX, MY & MZ respective to each index.
- **loadEnd** (*list of float*) – Load at end point in form of array containing 6 elements corresponding to FX, FY, FZ, MX, MY & MZ respective to each index.

Returns:

Returns True if successful.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddMemberFixedEnd([1], [1.0, 1.0, 1.0, 0, 0, 0], [1.0, 1.0, 1.0, 0, 0, 0])
```

`AddMemberFloorLoad(varPressure: float, varYMIN: float, varYMAX: float, varZMIN: float, varZMAX: float, varXMIN: float, varXMAX: float)` [source]

Automatically finds enclosed panels in the given boundary (specified using max and min X, Y, Z range inputs) and adds a FLOOR LOAD. Generated floor load is applied only in the Global X direction with YRANGE option.

Parameters:

- **varPressure** (*float*) – Magnitude of the pressure or concentrate load on the element.
- **varYMIN** (*float*) – Y range from which the load start (in global coordinate).
- **varYMAX** (*float*) – Y range at which the load end (in global coordinate).
- **varZMIN** (*float*) – Z range from which the load start (in global coordinate).
- **varZMAX** (*float*) – Z range at which the load end (in global coordinate).
- **varXMIN** (*float*) – X range from which the load start (in global coordinate).
- **varXMAX** (*float*) – X range at which the load end (in global coordinate).

Returns:

Returns 1 OK. Returns 0 General error. Returns -8001 Load direction is invalid.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddMemberFloorLoad(5.0, 0.0, 10.0, 0.0, 10.0, 0.0, 10.0)
```

`AddMemberFloorLoadEx(rangeType: int, loadDirection: int, pressure: float, grpOrOneWay: int, yMIN: float, yMAX: float, zMIN: float, zMAX: float, xMIN: float, xMAX: float)` [source]

Automatically finds enclosed panels in the given boundary (specified using max and min of X/Y /Z range inputs and varRange) and if varRange is 3 adds member group FLOOR LOAD (specified by GrpOrOneWay input). Otherwise adds a FLOOR LOAD with pressure (dPressure) in the Global X/Y/Z direction (as specified by Direction input) with RANGE option.

Parameters:

- **rangeType** (*int*) –

  Type of the Range :

Value	Range Type
0	X-RANGE
1	Y-RANGE
2	Z-RANGE
3	Group Load

- **loadDirection** (*int*) –

  Load direction :

Value	Direction
0	Global X
1	Global Y
2	Global Z

- **pressure** (*float*) – Magnitude of the pressure or concentrate load on the elemen
- **grpOrOneWay** (*int*) – One-Way Load (if it is either "" or "0") or corresponding group name to add Floor Group Load (if it contains Group string name). Notes: - Group name should be of FLOOR group type.
- **yMIN** (*float*) – Y range from which the load start(in global coordinate).
- **yMAX** (*float*) – Y range at which the load end(in global coordinate).
- **zMIN** (*float*) – Z range from which the load start(in global coordinate).
- **zMAX** (*float*) – Z range at which the load end(in global coordinate).
- **xMIN** (*float*) – X range from which the load start(in global coordinate).
- **xMAX** (*float*) – X range at which the load end(in global coordinate).

Returns:

Returns 1 OK. Returns 0 General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddMemberFloorLoadEx(0, 0, 5.0, 0, 0.0, 10.0, 0.0, 10.0, 0.0, 10.0)
```

`AddMemberLinearVari(memberIds: list[int], varDirection: int, varW1: float, varW2: float, varW3: float)` [source]

Adds LINEARLY VARYING load to beams.

Parameters:

- **memberIds** (*list of int*) – List of member IDs.
- **varDirection** (*int*) – Load direction (1 to 3 for LocalX, LocalY, LocalZ respectively).
- **varW1** (*float*) – Load at the start of the member.
- **varW2** (*float*) – Load at the end of the member.
- **varW3** (*float*) – Load in the middle of the member (for triangular load).

Returns:

Returns 0 OK. Returns -1 General error. Returns -8001 Load direction is invalid.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.AddMemberLinearVari([1], 2, 2.0, 0.0, 0.0)
>>> print(result)
```

`AddMemberTrapezoidal(memberIds: list, varDirection: int, varW1: float, varW2: float, varD1: float, varD2: float)` [source]

Adds trapezoidal linearly varying load to beams.

Parameters:

- **memberIds** (*list of int*) – List of member IDs.
- **varDirection** (*int*) – Load direction (1 to 9 for LocalX, LocalY, LocalZ, GlobalX, GlobalY, GlobalZ, ProjectedX, ProjectedY, ProjectedZ respectively).
- **varW1** (*float*) – Load at the start of the member.
- **varW2** (*float*) – Load at the end of the member.
- **varD1** (*float*) – Distance from the start of the member to loading starting point.
- **varD2** (*float*) –

  Distance from the start of the member to loading stopping point.
  Notes: - If varD1 and varD2 are not given, the load is assumed to cover the full member length.

Returns:

Returns 0 OK. Returns -1 General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.AddMemberTrapezoidal([1], 1, 5.0, 10.0, 0, 5)
>>> print(result)
```

`AddMemberUniformForce(beamIds: list, varDirection: int, varForce: float, varD1: float, varD2: float, varD3: float)` [source]

Adds a uniform force to the specified beams.

Parameters:

- **beamIds** (*list of int*) – List of beam IDs.
- **varDirection** (*int*) – Load direction (1 to 9 for LocalX, LocalY, LocalZ, GlobalX, GlobalY, GlobalZ, ProjectedX, ProjectedY, ProjectedZ respectively).
- **varForce** (*float*) – Magnitude of the uniform force.
- **varD1** (*float*) – Distance from the start of the member to the start of the load.
- **varD2** (*float*) – Distance from the start of the member to the end of the load.
- **varD3** (*float*) – Perpendicular distance from the member shear center to the local plane of loading.

Returns:

True if the uniform force was added successfully. False if an error occurred.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.AddMemberUniformForce([1,2], 1, 5.0, 0, 5, 0)
>>> print(result)
```

`AddMemberUniformMoment(beamIds: list, varDirection: int, varMoment: float, varD1: float, varD2: float, varD3: float)` [source]

Adds a uniform moment to the specified beams.

Parameters:

- **beamIds** (*list of int*) – List of beam IDs.
- **varDirection** (*int*) – Load direction (1 to 9 for LocalX, LocalY, LocalZ, GlobalX, GlobalY, GlobalZ, ProjectedX, ProjectedY, ProjectedZ).

- **varMoment** (*float*) – Magnitude of the uniform moment.
- **varD1** (*float*) – Distance from the start of the member to the start of the load.
- **varD2** (*float*) – Distance from the start of the member to the end of the load.
- **varD3** (*float*) – Perpendicular distance from the member shear center to the local plane of loading.

Returns:

True if the uniform moment was added successfully.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.AddMemberUniformMoment([1], 2, 10.0, 0, 5, 0)
>>> print(result)
```

`AddNodalLoad(nodeIds: list, forceInXDir: float, forceInYDir: float, forceInZDir: float, momentInXDir: float, momentInYDir: float, momentInZDir: float)` [source]

Adds joint load to the specified node numbers.

Parameters:

- **nodeIds** (*list of int*) – List of node IDs to apply the joint load.
- **forceInXDir** (*float*) – Force in the X direction.
- **forceInYDir** (*float*) – Force in the Y direction.
- **forceInZDir** (*float*) – Force in the Z direction.
- **momentInXDir** (*float*) – Moment in the X direction.
- **momentInYDir** (*float*) – Moment in the Y direction.
- **momentInZDir** (*float*) – Moment in the Z direction.

Returns:

True if the joint load was added successfully. False if an error occurred.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.AddNodalLoad([1,2], 10, 0, 0, 0, 0, 0)
>>> print(result)
```

`AddNotionalLoad(varPrimaryLoadCaseList: list[int], varPLFactorList: list[float], varPLDirectionList: list[int], varReferenceLoadCaseList: list[int], varRLFactorList: list[float], varRLDirectionList: list[int])` [source]

Creates a Notional load case using combinations of previously defined primary load cases and Reference load cases.

Parameters:

- **varPrimaryLoadCaseList** (*list of int*) – List of Primary load case reference number IDs
- **varPLFactorList** (*list of int*) – List of Multiplication factor of Primary load cases
- **varPLDirectionList** (*list of int*) –

  List of Directions of Primary load cases. Directions can be passed as following:

Direction	Integer
X OR GlobalLoadDirection X	1 4
Y OR GlobalLoadDirection Y	2 5
Z OR GlobalLoadDirection Z	3 6

- **varReferenceLoadCaseList** (*list of int*) – List of Reference load case reference number IDs
- **varRLFactorList** (*list of int*) – List of Multiplication factor of Reference load cases
- **varRLDirectionList** (*list of int*) –

  List of Directions of Reference load cases. Directions can be passed as following:

Direction	Integer
X OR GlobalLoadDirection X	1 4
Y OR GlobalLoadDirection Y	2 5
Z OR GlobalLoadDirection Z	3 6

Returns:

- *bool* – return True if successful
- *>>> from openstaadpy import os_analytical*
- *>>> staad_obj = os_analytical.connect()*
- *>>> staad_obj.Load.AddNotionalLoad([1, 2], [1.0, 1.2], [1, 2], [3], [0.8], [3])*

`AddRSLoad(varType: int, varFactArray: list, varAccOrDis: int, varScale: float, varDampType: int, varDampFact: float, varLinOrLog: int, varMis: int, varMisFact: float, varZpa: int, varZpaFact: float, varFf1: int, varFf1Fact: float, varFf2: int, varFf2Fact: float, varSaveFlag: int, varPairs: int, varDispOrAccSet: list, varVals: list)` [source]

Add Response Spectrum Load

Parameters:

- **varType** (*int*) – Response Spectrum Load type(1=Srss, 2=Cqc, 3=Absolute, 4=Asce, 5=Ten, 6=Csm)
- **varFactArray** (*list of float*) – Factor List.
- **varAccOrDis** (*int*) – 1 for Acceleration or 0 for Displacement.
- **varScale** (*float*) – Scale
- **varDampType** (*int*) – Damp Type(1=DAMP, 2=CDAMP, 3 = MDAMP)
- **varDampFact** (*float*) – Damp Factor
- **varLinOrLog** (*int*) – Interpolation Type: 1 for Logarithmic or 0 for Linear.
- **varMis** (*int*) – Missing Mass(1 for checked, 0 for unchecked)
- **varMisFact** (*float*) – Missing Mass Factor
- **varZpa** (*int*) – ZPA(1 for checked, 0 for unchecked)
- **varZpaFact** (*float*) – ZPA Factor
- **varFf1** (*int*) – Ff1(1 for checked, 0 for unchecked)
- **varFf1Fact** (*float*) – Ff1 Factor
- **varFf2** (*int*) – Ff2(1 for checked, 0 for unchecked)
- **varFf2Fact** (*float*) – Ff2 Factor
- **varSaveFlag** (*int*) – Save Flag(1 for checked, 0 for unchecked)
- **varPairs** (*int*) – Disp Or Acc Set pair count
- **varDispOrAccSet** (*List of float*) – Disp or Acc list.
- **varVals** (*List of float*) – Value list.

Returns:

- *int* – Returns 1 if is successfully adds response spectrum load. Returns 0 if general error.

- *>>> from openstaadpy import os_analytical*
- *>>> staad_obj = os_analytical.connect()*
- *>>> staad_obj.Load.AddRSLoad(1, [1.0, 2.0, 3.0], 1, 1.0, 1, 1.0, 1, 1, 1.0, 1, 1.0, 1, 1.0, 1, 1.0, 1, 1, [1.0, 2.0, 3.0], [1.0, 2.0, 3.0])*

`AddReferenceLoad(varRefLoadCaseNoIds: list[int], varFactorList: list[float])` [source]

Adds a reference load item to the currently active load case.

Parameters:

- **varRefLoadCaseNoIds** (*list of int*) – List of reference load case number IDs from Reference Load Definitions.
- **varFactorList** (*list of float*) – List of corresponding load factors.

Returns:

- *int* – Reference load case number ID.
- *>>> from openstaadpy import os_analytical*
- *>>> staad_obj = os_analytical.connect()*
- *>>> staad_obj.Load.AddReferenceLoad([1, 2, 3], [1.0, 2.0, 3.0])*

`AddRepeatLoad(varLoadCaseList: list, varFactorList: list)` [source]

Creates a primary load case using combinations of previously defined primary load cases.

Parameters:

- **varLoadCaseList** (*list of int*) – (Primary) load case reference number ID(s) array.
- **varFactorList** (*list of float*) – Multiplication factor array.

Returns:

- *int* – Returns 1 if Load Case is added successfully, 0 otherwise.
- *>>> from openstaadpy import os_analytical*
- *>>> staad_obj = os_analytical.connect()*
- *>>> staad_obj.Load.AddRepeatLoad([1, 2, 3], [1.0, 2.0, 3.0])*

`AddResponseSpectrumLoadEx(rsaCode: int, rsaCombination: int, varSet1Names: list, varSet1Vals: list, varSet2Names: list, varSet2Vals: list, varDataPairs: list)` [source]

Adds Response Spectrum load item to the currently active load case.

Parameters:

- **rsaCode** (*int*) – Response Spectrum Loading Code. Refer to the following table for the integers corresponding to different codes.
- **rsaCombination** (*int*) – Modal combination rule. (SRSS = 0, ABS = 1, CQC = 2, ASCE = 3, TEN = 4, CSM = 5, GRP = 6)
- **varSet1Names** (*list of string*) – List of string containing parameter key words. Refer to the Technical Reference sections as indicated below.
- **varSet1Vals** (*list of float*) – List of Parameters values corresponding to the keywords supplied in varSet1Names array.
- **varSet2Names** (*list of string*) – List of string containing parameter key words for the spectrum generation data command. NULL can be used if not needed.
- **varSet2Vals** (*list of float*) – List of Parameters values corresponding to the keywords supplied in varSet1Names array. NULL can be used if not needed.
- **varDataPairs** (*list of float*) – List of containing pairs of time period and acceleration data. NULL can be used if not needed. Inputs (varSet2Names, varSet2Vals) and (varDataPairs) are mutually exclusive, i.e. if one is specified, other should not specified
- **Notes**
- **sections** (- *Techincal Reference*) –

nTypeNo	Seismic Code	Parameters	Ref. Sec.
0	Generic or Custom	DEC, ECC, X, Y, Z, ACC, DIS, SCA, DAM, CDA, MDA, LIN, LOG, MIS, ZPA, FF1, FF2, DOM, SIG, SAV, IMR, STA	TR.32.10.1.1
1	IS:1893 Part 1 2002	TOR, DEC, ECC, X, Y, Z, ACC, DIS, SCA, DAM, CDA, MDA, MIS, ZPA, IGN, DOM, SIG, SAV, IMR, STA SOI, CHE, RF	TR.32.10.1.7
2	IS:1893 2016	TOR, DEC, ECC, X, Y, Z, ACC, DIS, SCA, DAM, CDA, MDA, LIN, LOG, MIS, ZPA, IGN, DOM, SIG, SAV, IMR, STA SOI, CHE, RF	TR.32.10.1.8
4	ENV 1998-1:1994	ELA, DES, X, Y, Z, ACC, DAM, CDA, MDA, LIN, LOG, MIS, ZPA, DOM, SIG, SAV, IMR, STA SOI, ALP, Q	TR.32.10.1.4
5	EN 1998-1:2004	ELA, DES, RS1, RS2, X, Y, Z, ACC, DAM, CDA, MDA, LIN, LOG, MIS, ZPA, DOM, SIG, SAV, IMR, STA SOI, ALP, Q	TR.32.10.1.5
6	IBC 2006	X, Y, Z, ACC, DAM, CDA, MDA, LIN, LOG, MISC, ZPA, DOM, SIG, SAV IMR, STA ZIP, LAT, LON, SS, S1, SCA, FA, FV, TL	TR.32.10.1.10
7	IBC 2012	X, Y, Z, ACC, DAM, CDA, MDA, LIN, LOG, MISC, ZPA, DOM, SIG, SAV IMR, STA ZIP, LAT, LON, SS, S1, SCA, FA, FV, TL	TR.32.10.1.11
8	IBC 2015	X, Y, Z, ACC, DAM, CDA, MDA, LIN, LOG, MISC, ZPA, DOM, SIG, SAV IMR, STA ZIP, LAT, LON, SS, S1, SCA, FA, FV, TL	TR.32.10.1.12
10	SNiP II-7-81	A, X, KWX, KX1, Y, KWY, KY1, Z, KWZ, KZ1, ACC, SCA, DAM, CDA, MDA, LIN, LOG, MIS, ZPA, DOM, SIG, SOI, SAV	TR.32.10.1.14
11	SP 14.13330.2011	ECC, A, X, Y, Z, ACC, SCA, DAM, LOG, MIS, ZPA, DOM, SIG, SOI	TR.32.10.1.15
12	CANADIAN: NRC-2005	TOR, DEC, ECC, X, Y, Z, ACC, DIS, SCA, DAM, CDA, MDA, LIN, LOG, MIS, ZPA, DOM, SIG, SAV, IMR, STA	TR.32.10.1.2
13	CANADIAN: NRC-2010	TOR, DEC, ECC, X, Y, Z, ACC, DIS, SCA, DAM, CDA, MDA, LIN, LOG, MIS, ZPA, DOM, SIG, SAV, IMR, STA	TR.32.10.1.3
14	GB 50011 2010	X, Y, Z, ALP, DAM, CDA, MDA, LIN, LOG, MISS, ZPA, DOM, SIG, INT, FRE, FOR,RAR, GRO, SCL	TR.32.10.1.6

- **code** (- *Following values should be specified for INT parameter of GB 50011 2010*) –

Fortification Intensity	Value
6	0
7	1
7A	2
8	3
8A	4
9	5

Returns:

Returns TRUE if successful Returns FALSE if unsuccessful

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddResponseSpectrumLoadEx(0, 0, ['DEC', 'ECC'], [1.0, 2.0], [], [], [])
```

`AddSeismicDefElementWeight(pressure: float, elementList: list)` [source]

Adds a pressure to Seismic Definition.

Parameters:

- **pressure** (*float*) – Pressure Value
- **elementList** (*List of int*) – 'List of element ID list.

Returns:

- *int* – True if successful. False if unsucessful
- *>>> from openstaadpy import os_analytical*
- *>>> staad_obj = os_analytical.connect()*
- *>>> staad_obj.Load.AddSeismicDefElementWeight(1.0, [1, 2, 3])*

`AddSeismicDefFloorWeight(rangeType: int, loadDirection: int, pressure: float, grpOrOneWay: int, yMIN: float, yMAX: float, zMIN: float, zMAX: float, xMIN: float, xMAX: float)` [source]

Adds a floor weight to Seismic Definition.

Parameters:

- **rangeType** (*int*) –

  Type of the Range :

Value	Range Type
0	X-RANGE
1	Y-RANGE
2	Z-RANGE
3	Group Load

- **loadDirection** (*int*) –

  Load direction :

Value	Direction
0	Global X
1	Global Y
2	Global Z

- **pressure** (*float*) – Magnitude of the pressure or concentrate load on the elemen
- **grpOrOneWay** (*int*) – One-Way Load (if it is either "" or "0") or corresponding group name to add Floor Group Load (if it contains Group string name). Notes: - Group name should be of FLOOR group type.
- **yMIN** (*float*) – Y range from which the load start(in global coordinate).
- **yMAX** (*float*) – Y range at which the load end(in global coordinate).
- **zMIN** (*float*) – Z range from which the load start(in global coordinate).
- **zMAX** (*float*) – Z range at which the load end(in global coordinate).
- **xMIN** (*float*) – X range from which the load start(in global coordinate).
- **xMAX** (*float*) – X range at which the load end(in global coordinate).

Returns:

- *int* – True if successful. False if unsucessful

- >>> *from openstaadpy import os_analytical*
- >>> *staad_obj = os_analytical.connect()*
- >>> *staad_obj.Load.AddSeismicDefFloorWeight(0, 1, 2.0, 3, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0)*

`AddSeismicDefJointWeight(weight: float, nodeList: list)` [source]

> Adds joint self weight to Seismic Definition.

Parameters:

- **weight** (*float*) – Weight value.
- **nodeList** (*list of int*) – List of Node number IDs

Returns:

- *int* – Returns 0 if OK . Returns -1 if General error. Returns -100 if Invalid argument. Returns -106 if 1 dimensional array of long expected. Returns -113 if Integer array/Integer expected. Returns -2006 if Invalid Node Number. Returns -8034 if Seisemic Code not found.
- >>> *from openstaadpy import os_analytical*
- >>> *staad_obj = os_analytical.connect()*
- >>> *staad_obj.Load.AddSeismicDefJointWeight(1.0, [1, 2, 3])*

`AddSeismicDefMemberWeight(varSeismicType: int, loadType: int, weight: float, startDist: float, endDist: float, memberList: list)` [source]

Adds member concentrated/uniform weight to Seismic Definition.

Parameters:

- **varSeismicType** (*int*) –

  Type of seismic code:

Value	Seismic Code
0	AUTO DETECT
1	ALGERIAN: RPA
2	CANADIAN: NRC-1995
3	CANADIAN: NRC-2005
4	CANADIAN: NRC-2010
5	CANADIAN: NRC-2020
6	Chinese: GB50011-2001
7	Chinese: GB50011-2010
8	COLOMBIAN: NSR 95
9	COLOMBIAN: NSR 2010
10	IBC 2000
11	IBC 2003 ASCE 7-02
12	IBC 2006/2009 ASCE 7-05
13	IBC 2012 ASCE 7-10
14	IBC 2015 ASCE 7-10
15	IBC 2018 ASCE 7-16
16	Indian: IS 1893-1984
17	Indian: IS 1893-2002/2005
18	Indian: IS 1893-2016
19	Indian: IS 1893(Part4) 2015
20	JAPANESE (AIJ)
21	MEX: CFE-1993
22	MEX: NTC-1987
23	TURKISH
24	UBC 1985
25	UBC 1994
26	UBC 1997

- **loadType** (*int*) – 1 for uniform loadType and 2 for concentrated loadType
- **weight** (*float*) – Uniform weight.
- **startDist** (*float*) – Starting distance(= distance from member starting node to weight starting point)
- **endDist** (*float*) – Ending distance(= distance from member starting node to weight ending point)
- **memberList** (*list of int*) – List of Member ID to add member concentrated/uniform weight

Returns:

- *bool* – True if successful adds member concentrated/uniform weight to Seismic Definition. False if unsucessful
- *>>> from openstaadpy import os_analytical*
- *>>> staad_obj = os_analytical.connect()*
- *>>> staad_obj.Load.AddSeismicDefMemberWeight(1, 2, 3.0, 4.0, 5.0, [6, 7, 8])*

`AddSeismicDefSelfWeight(varWeightFactor: float)` [source]

Adds self weight to Seismic Definition.

Parameters:

varWeightFactor (*float*) – Weight Factor to add to self weight

Returns:

- *bool* – True if successful. False if unsucessful
- *>>> from openstaadpy import os_analytical*
- *>>> staad_obj = os_analytical.connect()*
- *>>> staad_obj.Load.AddSeismicDefSelfWeight(1.0)*

`AddSeismicDefWallArea(nTypeNo: int, direction: str, sizeArray: list)` [source]

Adds wall area to Seismic Definition. .. note:: - Wall Area is only available in IS1893-2016 seismic code.

Parameters:

- **nTypeNo** (*int*) –

  Type of seismic code:

Value	Seismic Code
15	Indian: IS 1893-2016

- **direction** (*string*) – Direction value. [X directin or Z direction]
- **sizeArray** (*List of float*) – Length and Width list consisting of consecutive length and width measurements

Returns:

- *int* – Returns 0 if OK . Returns -1 if General error. Returns -107 if 1 dimensional array of long expected. Returns -8034 if Invalid seismic code. Returns -8038 if Invalid Direction.
- *>>> from openstaadpy import os_analytical*
- *>>> staad_obj = os_analytical.connect()*
- *>>> staad_obj.Load.AddSeismicDefWallArea(15, "X", [10.0, 20.0])*

`AddSeismicDefinition(varType: int, varAccidental: int)` [source]

Adds a Seismic Definition with default parameters.

Parameters:

- **varType** (*int*) –

  Type of seismic code:

Value	Seismic Code
0	UBC 1985
1	UBC 1994
2	UBC 1997
3	Indian: IS 1893-1984
4	Indian: IS 1893-2002/2005
5	IBC 2000
6	IBC 2003
7	COLOMBIAN: NSR 98
8	JAPANESE (AIJ)
9	ALGERIAN: RPA
10	MEX: CFE-1993
11	MEX: NTC-1987
12	Indian: IS 1893-2016
13	Indian: IS 1893(Part4) 2015
14	IBC 2006
15	IBC 2012
16	IBC 2015
17	IBC 2018
18	CANADIAN: NRC-2005
19	CANADIAN: NRC-2010
20	CANADIAN: NRC-1995
21	COLOMBIAN: NSR 2010
22	Chinese: GB50011-2001
23	Chinese: GB50011-2010
24	TURKISH

- **varAccidental** (*int*) – '1' to consider accidental torsion else '0' ignore

Returns:

True if successful. False if unsucessful

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddSeismicDefinition(1, 0)
```

`AddSeismicLoad(loadDirection: int, factor: float)` [source]

Adds a Seismic Definition with default parameters.

Parameters:

- **loadDirection** (*int*) – Load direction: (= 0 to 2 for global X, Y and Z, respectively).
- **factor** (*float*) – 'Multiplication factor to be used to multiply the seismic load.

Returns:

- *int* – Returns 0 if OK. Returns -1 if General error. Returns -8001 if Load direction is invalid.
- *>>> from openstaadpy import os_analytical*
- *>>> staad_obj = os_analytical.connect()*
- *>>> staad_obj.Load.AddSeismicLoad(0, 1.0)*

AddSelfWeightInXYZ(*varInDirection: int, varLoadFactor: float*) [source]

Adds self-weight to the active load case for all entities (beams, plates, solids).

Parameters:

- **varInDirection** (*int*) – Direction index for self-weight (1 = X, 2 = Y, 3 = Z).
- **varLoadFactor** (*float*) – Multiplying factor for self-weight.

Returns:

True if self-weight was added successfully. False if an error occurred.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.AddSelfWeightInXYZ(2, 1.0)
>>> print(result)
```

AddSelfWeightInXYZToGeometry(*varGeomNumberIDs: list, varInDirection: int, varLoadFactor: float*)

Adds self-weight to specified geometry entities in the active load case. [source]

Parameters:

- **varGeomNumberIDs** (*list of int*) – List of beam, plate, or solid number IDs.
- **varInDirection** (*int*) – Direction index for self-weight (1 = X, 2 = Y, 3 = Z).
- **varLoadFactor** (*float*) – Multiplying factor for self-weight.

Returns:

True if self-weight was added to the specified geometries. False if an error occurred.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.AddSelfWeightInXYZToGeometry([1,2], 3, 0.9)
>>> print(result)
```

AddStrainLoad(*elementIds: list, varAxialElong: float*) [source]

Adds STRAIN LOAD to beam or plate elements.

Parameters:

- **elementIds** (*list of int*) – List of element IDs.
- **varAxialElong** (*float*) – Initial axial elongation (+)/ shrinkage (-) in member due to misfit, etc.

Returns:

Returns 0 OK. Returns -1 General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddStrainLoad([1], 0.001)
```

`AddSupportDisplacement(nodeIds: list, varDirection: int, varDispValue: float)` [source]

Adds support displacement to one or more nodes.

Parameters:

- **nodeIds** (*list of int*) – List of node IDs.
- **varDirection** (*int*) – Direction index (1 = X, 2 = Y, 3 = Z).
- **varDispValue** (*float*) – Displacement value in the specified direction.

Returns:

True if the support displacement was added successfully. False if an error occurred.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.AddSupportDisplacement([1], 1, 5.0)
>>> print(result)
```

`AddTemperatureLoad(elementIds: list, varTempAxialElong: float, varTempDiffTopAndBtm: float, varTemDiffSide: float)` [source]

Adds TEMPERATURE LOAD to beam or plate elements.

Parameters:

- **elementIds** (*list of int*) – List of element IDs.
- **varTempAxialElong** (*float*) – Change in temperature.
- **varTempDiffTopAndBtm** (*float*) – Temperature difference from the top to the bottom of the element (for calculating bending).
- **varTemDiffSide** (*float*) – Temperature difference from side to side of the element (local Z axis).

Returns:

Returns 0 OK. Returns -1 General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddTemperatureLoad([1], 20.0, 30.0, 40.0)
```

`AddWindDefinition(varTypeNo: int, varTypeName: str)` [source]

Adds a Wind Definition named “varTypeName” with number ID varTypeNo.

Parameters:

- **varTypeNo** (*int*) – Wind Definition Type number ID.
- **varTypeName** (*string*) – String name of this new type.

Returns:

Returns 0 OK. Returns -1 General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddWindDefinition(1, "Wind Load 1")
```

`AddWindDefinitionASCE7Parameters(varTypeNo: int, code: int, windSpeed: float, heightAboveSeaLvl: float, bldgclass: int, bldgtype: int, expCat: int, varEscarpment: bool, wallType: int, varIsFlexible: bool, varEscarpmentData: list, varbldgData: list, varUnitsData: list, varFactorsUserInput: list, varFactors: list)` [source]

Generates the Wind Definition Parameters using ASCE CODE.

Parameters:

- **varTypeNo** (*int*) – Wind Definition Type number ID
- **code** (*int*) –

Value
0
1
2
3

- **windSpeed** (*float*) – Wind speed.
- **heightAboveSeaLvl** (*float*) – Ground Height above sea level. [Required only for ASCE7-2016. For other versions give 0.0]
- **bldgclass** (*int*) –

value	Buil
0	Cate
1	Cate
2	Cate
3	Cate

- **bldgtype** (*int*) –

Value	Building
0	Building
1	Chimney
2	Solid Sig
3	Open Sig
4	Lattice Fr
5	Trussed T

- **expCat** (*int*) –

Value
0
1
2
3

- **varEscarpment** (*bool*) – Consider Wind Speed-up over Hills (FALSE) or Escarpment (TRUE).
- **wallType** (*int*) –

Value
0
1
2

- **varIsFlexible** (*bool*) – Consider structure is Flexible (TRUE) or RIGID (FALSE).
- **varEscarpmentData** (*List of float*) – Float list of size 4 containing information describing Hills or Escarpment +——-+————
- **varbldgData** (*List of float*) –

  Float list of size 7 containing information describing the building based on structure type :

  - **Building Data :**

Index	Item
0	**Enclosure Classification :** ■ Before 2016 : Open Building (0)/ Partially Enclosed (1)/ E ■ [2016] : Open Building (0)/ Partially Open (1)/ Partially Er
1	Building Height
2	Building length long the direction of Wind (L)
3	Building length normal to the direction of Wind (B)
4	Building Natural Frequency
5	Building Damping Ratio

  - **Tank Data :**

Index	Item
0	**Horizontal Cross-section Type :** ■ Before 2016 : Square (0)/ Square Diagonal (1)/ Hexagonal or Octagonal (2)/ Rou ■ [2016] : Square (0)/ Square Diagonal (1)/ Hexagonal (2)/ Octagonal Non-axisym
1	Tank Height
2	Least Horizontal Dimension (W)
3	Depth of producing elements like Spoilers and Ribs (D')
4	Structure Natural Frequency
5	Structure Damping Ratio

  - **Solid Sign Data :**

Index
0
1
2
3
4

- **Open Sign/Lattice Framework Data :**

Index	It
0	O
1	H
2	W
3	Di
4	Ra
5	St
6	St

- **Trussed Tower Data :**

Index	Item
0	Horizontal Cross Sectio Typ
1	Tank Height (H)
2	Width
3	Ratio of Solid Area to Gross
4	Structure Natural Frequenc
5	Structure Damping Ratio

- **varUnitsData** (*List of int*) – Float list of size 7 containing Units of data inputs +——-+——————————————————————————————+=======+==========
  | 0 | Unit of Wind Speed {mph(VelocityUnit::mph or 0) or m/sec(VelocityUnit::metersec or 1) or cm/sec(VelocityUnit::cmsec or 2) o
  {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M o
  ——————————————————————————————

- >>> *staad_obj = os_analytical.connect()*
- >>> *staad_obj.Load.AddWindDefinitionASCE7Parameters(1, "ASCE7-10", 100.0, 10.0, 1, 1, 1, 1, [1.0, 2.0, 3.0], [1.0, 2.0, 3.0])*

`AddWindExposure(varTypeNo: int, varExposureFactor: float, varNodeArray: list)` [source]

Adds Wind Exposures factor to Wind Definitions and assign to nodes.

Parameters:

- **varTypeNo** (*int*) – Wind Definition Type number ID.
- **varExposureFactor** (*float*) – Exposure factor.
- **varNodeArray** (*list of int*) – Node number ID list.

Returns:

Returns 0 OK. Returns -1 General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddWindExposure(1, 1.0, [1,2,3])
```

`AddWindIntensity(varTypeNo: int, varIntensity: list, varHeight: list)` [source]

Adds to Wind Definitions Wind Intensity by giving Intensity vs. Height.

Parameters:

- **varTypeNo** (*int*) – Wind Definition Type number ID.
- **varIntensity** (*list of float*) – Intensity values float list
- **varHeight** (*list of float*) – Height value float list.

Returns:

Returns 0 OK. Returns -1 General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddWindIntensity(1,[5.2], [10.0])
```

`AddWindLoad(varTypeNo: int, varDirection: int, dFraction: float, varOpenStructure: int, dYMIN: float, dYMAX: float, dZMIN: float, dZMAX: float, dXMIN: float, dXMAX: float)` [source]

Adds a wind load.

Parameters:

- **varTypeNo** (*int*) – Wind Definition Type number ID.
- **varDirection** (*int*) –

  Wind load direction:

Value	Direction
1	Global X
3	Global Z
4	Global -X
6	Global -Z

- **dFraction** (*float*) – Factor to be used to multiply the wind loads. Negative signs may be used to indicate opposite direction of resulting load (default=1.0).
- **varOpenStructure** (*int*) – For Open-type of structure enter 1 , closed-type of structure 0
- **dYMIN** (*float*) – Ymin of GLOBAL Y range in which Wind load applied (assume Y axis is vertical).
- **dYMAX** (*float*) – Ymax of GLOBAL Y range in which Wind load applied (assume Y axis is vertical).
- **dZMIN** (*float*) – Zmin of GLOBAL Z range in which Wind load applied (assume Y axis is vertical).
- **dZMAX** (*float*) – Zmax of GLOBAL Z range in which Wind load applied (assume Y axis is vertical).
- **dXMIN** (*float*) – Xmin of GLOBAL X range in which Wind load applied (assume Y axis is vertical).
- **dXMAX** (*float*) – Xmax of GLOBAL X range in which Wind load applied (assume Y axis is vertical).

Returns:

Returns 0 OK. Returns -1 General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.AddWindLoad(1, 3, 1.0, 1, 0.0, 10.0, 0.0, 10.0, 0.0, 10.0)
```

`BeginLoadMerging()` [source]

Begin Load Merging

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.BeginLoadMerging()
```

`ClearPrimaryLoadCase(varLoadCaseNos: list, isReferenceLoad: bool)` [source]

Clears the load items in a specified Primary Load cases or Reference Load cases.

Parameters:

- **varLoadCaseNos** (*list*) – Primary load case reference ID(s) list.
- **isReferenceLoad** (*bool*) – If reference load case(s): True or False.

Returns:

Returns 1 if OK Returns 0 if failed to delete load(s)

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.ClearPrimaryLoadCase([1, 2, 3], False)
```

ClearReferenceLoadCase(*varLoadCaseNos: list*) [source]

Clears the load items in a specified Primary Load cases or Reference Load cases.

Parameters:

varLoadCaseNos (*list*) – Primary load case reference ID(s) list.

Returns:

Returns 1 if OK Returns 0 if failed to delete load(s)

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.ClearReferenceLoadCase([1, 2, 3])
```

ComputeWallWindPressureProfile(*loadingCode: int, windSpeed: float, bldgClass: int, bldgType: int, expCat: int, bEscarpment: bool, varUnitsData: list, varEscarpmentData: list, varBldgData: list, wallType: int*) [source]

Generates the wall wind pressure profile using ASCE CODE.

Parameters:

- **loadingCode** (*int*) –

  ASCE CODE:

Value	ASCE CODE
ASCE7Y95 = 0	ACSE 7-1995
ACSE702 = 1	ACSE 7-2002
ACSE705_10 = 2	ACSE 7-2010

- **windSpeed** (*float*) – Wind speed. Default value 85 mph.
- **bldgClass** (*int*) –

  Building Classification Category:

Value	Building Classification Category
TypeI = 0	Category I
TypeII = 1	Category II
TypeIII = 2	Category III
TypeIV = 3	Category IV

- **bldgtype** (*int*) –

  Structure Type:

Value	Structure Type
Building = 0	Building Structures
Chimney = 1	Chimney, Tank and similar structures
Solidsign = 2	Solid Signs
Opensign = 3	Open Signs
Laticeframe = 4	Lattice Framework
Trusstower = 5	Trussed Tower

- **expCat** (*int*) –

  Exposure Category:

Value	Exposure Category
ExpA = 0	Exposure A
ExpB = 1	Exposure B
ExpC = 2	Exposure C
ExpD = 3	Exposure D

- **bEscarpment** (*bool*) – Consider Wind Speed-up over Hills (FALSE) or Escarpment (TRUE).
- **varUnitsData** (*list of long*) –

  Integer list of size 8 containing Units of data inputs:

Index	Data
0	Unit of Wind Speed {mph(VelocityUnit::mph or 0) or m/sec(VelocityUnit::metersec or 1) or cm/sec(VelocityUnit::cmsec or 2) or mm/sec(VelocityUnit::mmsec or 3) or kmph(VelocityUnit::kmph or 4) or in/sec(VelocityUnit::inchsec or 5) or ft/sec(VelocityUnit::ftsec or 6) or Yd/sec(VelocityUnit::yardsec or 7)}
1	Unit of Ground height above sea level {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M or 4) or mm(LengthUnit::Mm or 5) or dm(LengthUnit::Dm or 6) or Km(LengthUnit::Km or 7) or yard(LengthUnit::Yd or 8) or mile(LengthUnit::mil or 9)}
2	[Escarpment] Unit of Height (H) {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M or 4) or mm(LengthUnit::Mm or 5) or dm(LengthUnit::Dm or 6) or Km(LengthUnit::Km or 7) or yard(LengthUnit::Yd or 8) or mile(LengthUnit::mil or 9)}
3	[Escarpment] Unit of Distance upwind of crest (Lh) {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M or 4) or mm(LengthUnit::Mm or 5) or dm(LengthUnit::Dm or 6) or Km(LengthUnit::Km or 7) or yard(LengthUnit::Yd or 8) or mile(LengthUnit::mil or 9)}
4	[Escarpment] Unit of Distance from the crest to the building (x) {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M or 4) or mm(LengthUnit::Mm or 5) or dm(LengthUnit::Dm or 6) or Km(LengthUnit::Km or 7) or yard(LengthUnit::Yd or 8) or mile(LengthUnit::mil or 9)}
5	[Building]Unit of Height/ [Tank]Unit of Height/ [Solid Sign]Unit of Height/ [Open Sign/Lattice]Unit of Height/ [Trusses Tower]Unit of Height {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M or 4) or mm(LengthUnit::Mm or 5) or dm(LengthUnit::Dm or 6) or Km(LengthUnit::Km or 7) or yard(LengthUnit::Yd or 8) or mile(LengthUnit::mil or 9)}
6	[Building]Unit of Length/ [Tank]Unit of Width/ [Solid Sign]Unit of M dimension/ [Open Sign/Lattice]Unit of Width/ [Trusses Tower]Unit of Width {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M or 4) or mm(LengthUnit::Mm or 5) or dm(LengthUnit::Dm or 6) or Km(LengthUnit::Km or 7) or yard(LengthUnit::Yd or 8) or mile(LengthUnit::mil or 9)}
7	[Building]Unit of Width/ [Tank]Unit of Depth/ [Solid Sign]Unit of N dimension/ [Open Sign/Lattice]Unit of Diameter/ [Trusses Tower]Not Applicable {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M or 4) or mm(LengthUnit::Mm or 5) or dm(LengthUnit::Dm or 6) or Km(LengthUnit::Km or 7) or yard(LengthUnit::Yd or 8) or mile(LengthUnit::mil or 9)}

- **varescarpmentData** (*list*) –

  Information describing Hills or Escarpment:

Index	Data
0	Type: 2D Ridge (0), 2D Escarpment (1), 3D Escarpment (2)
1	Height (H)
2	Distance upwind of crest (Lh)
3	Distance from the crest to the building (x)

- **varbldgData** (*list*) –

  List of size 7 containing information describing the building based on structure type.

  - **Building Data :**

Index	Item
0	Enclosure Classification: Open Building (0)/ Partially Enclosed (1)/ Enclosed Building (2)
1	Building Height
2	Building length long the direction of Wind (L)
3	Building length normal to the direction of Wind (B)
4	Building Natural Frequency
5	Building Damping Ratio

- **OR Tank Data :**

Index	Item
0	Horizontal Cross-section Type:- Square (0)/ Square Diagonal (1)/ Hexagonal or Octagonal (2)/ Round (3)
1	Tank Height (H)
2	Least Horizontal Dimension (W)
3	Depth of producing elements like Spoilers and Ribs (D')
4	Structure Natural Frequency
5	Structure Damping Ratio

- **OR Solid Sign Data :**

Index	Item
0	Solid Sign Height (H)
1	Solid Sign M Dimension (M)
2	Solid Sign N Dimension (N)
3	Structure Natural Frequency
4	Structure Damping Ratio

- **OR Open Sign/Lattice Framework Data :**

Index	Item
0	Orientation Type: Flat (0)/ Rounded (1)
1	Height (H)
2	Width
3	Diameter of typical round member
4	Structure Natural Frequency
5	Structure Damping Ratio
6	Ratio of Solid Area to Gross Area

- **OR Trussed Tower Data :**

Index	Item
0	Horizontal Cross Sectio Type: Triangle (0)/ Square (1)
1	Height (H)
2	Width
3	Structure Natural Frequency
4	Structure Damping Ratio
5	Ratio of Solid Area to Gross Area(in percetage)

- **wallType** (*int*) –

  Building wall to generate Wind Load on:

Value	Wall Type
WindWard = 0	WindWard
LeeWard = 1	Leeward
SideWall = 2	SideWall

  - (0 to 2 for WindWard, Leeward and SideWall, respectively).

Returns:

Returns number of Height or Intensity data.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.ComputeWallWindPressureProfile(2, 90.0, 1, 0, 2, False, [0]*8, [0.0]*4, [0.0]*7, 0)
```

`ComputeWallWindPressureProfileASCE72016`(*windSpeed: float, heightAboveSeaLvl: float, bldgClass: int, bldgType: int, expCat: int, bEscarpment: bool, varUnitsData: list, varEscarpmentData: list, varBldgData: list, wallType: int*) [source]

Modifies or adds a seismic parameter in the existing seismic definition.

Parameters:

- **windSpeed** (*float*) – Wind speed. Default value 85 mph.
- **heightAboveSeaLvl** (*float*) – Ground height above sea level. Used only for ASCE7-2016 Wind. Default value 0.0 ft.
- **bldgClass** (*int*) –

  Building Classification Category:

Value	Building Classification Category
TypeI = 0	Category I
TypeII = 1	Category II
TypeIII = 2	Category III
TypeIV = 3	Category IV

- **bldgtype** (*int*) –

  Structure Type:

Value	Structure Type
Building = 0	Building Structures
Chimney = 1	Chimney, Tank and similar structures
Solidsign = 2	Solid Signs
Opensign = 3	Open Signs
Laticeframe = 4	Lattice Framework
Trusstower = 5	Trussed Tower

- **expCat** (*int*) –

  Exposure Category:

Value	Exposure Category
ExpA = 0	Exposure A
ExpB = 1	Exposure B
ExpC = 2	Exposure C
ExpD = 3	Exposure D

- **bEscarpment** (*bool*) – Consider Wind Speed-up over Hills (FALSE) or Escarpment (TRUE).
- **varUnitsData** (*list of long*) –

  Integer list of size 8 containing Units of data inputs:

Index	Data
0	Unit of Wind Speed {mph(VelocityUnit::mph or 0) or m/sec(VelocityUnit::metersec or 1) or cm/sec(VelocityUnit::cmsec or 2) or mm/sec(VelocityUnit::mmsec or 3) or kmph(VelocityUnit::kmph or 4) or in/sec(VelocityUnit::inchsec or 5) or ft/sec(VelocityUnit::ftsec or 6) or Yd/sec(VelocityUnit::yardsec or 7)}
1	Unit of Ground height above sea level {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M or 4) or mm(LengthUnit::Mm or 5) or dm(LengthUnit::Dm or 6) or Km(LengthUnit::Km or 7) or yard(LengthUnit::Yd or 8) or mile(LengthUnit::mil or 9)}
2	[Escarpment] Unit of Height (H) {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M or 4) or mm(LengthUnit::Mm or 5) or dm(LengthUnit::Dm or 6) or Km(LengthUnit::Km or 7) or yard(LengthUnit::Yd or 8) or mile(LengthUnit::mil or 9)}
3	[Escarpment] Unit of Distance upwind of crest (Lh) {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M or 4) or mm(LengthUnit::Mm or 5) or dm(LengthUnit::Dm or 6) or Km(LengthUnit::Km or 7) or yard(LengthUnit::Yd or 8) or mile(LengthUnit::mil or 9)}
4	[Escarpment] Unit of Distance from the crest to the building (x) {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M or 4) or mm(LengthUnit::Mm or 5) or dm(LengthUnit::Dm or 6) or Km(LengthUnit::Km or 7) or yard(LengthUnit::Yd or 8) or mile(LengthUnit::mil or 9)}
5	[Building]Unit of Height/ [Tank]Unit of Height/ [Solid Sign]Unit of Height/ [Open Sign/Lattice]Unit of Height/ [Trusses Tower]Unit of Height {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M or 4) or mm(LengthUnit::Mm or 5) or dm(LengthUnit::Dm or 6) or Km(LengthUnit::Km or 7) or yard(LengthUnit::Yd or 8) or mile(LengthUnit::mil or 9)}
6	[Building]Unit of Length/ [Tank]Unit of Width/ [Solid Sign]Unit of M dimension/ [Open Sign/Lattice]Unit of Width/ [Trusses Tower]Unit of Width {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M or 4) or mm(LengthUnit::Mm or 5) or dm(LengthUnit::Dm or 6) or Km(LengthUnit::Km or 7) or yard(LengthUnit::Yd or 8) or mile(LengthUnit::mil or 9)}
7	[Building]Unit of Width/ [Tank]Unit of Depth/ [Solid Sign]Unit of N dimension/ [Open Sign/Lattice]Unit of Diameter/ [Trusses Tower]Not Applicable {inch(LengthUnit::In or 0) or ft(LengthUnit::Ft or 1) or foot(LengthUnit::foot or 2) or cm(LengthUnit::cm or 3) or m(LengthUnit::M or 4) or mm(LengthUnit::Mm or 5) or dm(LengthUnit::Dm or 6) or Km(LengthUnit::Km or 7) or yard(LengthUnit::Yd or 8) or mile(LengthUnit::mil or 9)}

- **varescarpmentData** (*list*) –

  Information describing Hills or Escarpment:

Index	Data
0	Type: 2D Ridge (0), 2D Escarpment (1), 3D Escarpment (2)
1	Height (H)
2	Distance upwind of crest (Lh)
3	Distance from the crest to the building (x)

- **varbldgData** (*list*) –

  List of size 7 containing information describing the building based on structure type.

  - **Building Data :**

Index	Item
0	Enclosure Classification: Open Building (0)/ Partially Enclosed (1)/ Enclosed Building (2)
1	Building Height
2	Building length long the direction of Wind (L)
3	Building length normal to the direction of Wind (B)
4	Building Natural Frequency
5	Building Damping Ratio

- **OR Tank Data :**

Index	Item
0	Horizontal Cross-section Type:- Square (0)/ Square Diagonal (1)/ Hexagonal or Octagonal (2)/ Round (3)
1	Tank Height (H)
2	Least Horizontal Dimension (W)
3	Depth of producing elements like Spoilers and Ribs (D')
4	Structure Natural Frequency
5	Structure Damping Ratio

- **OR Solid Sign Data :**

Index	Item
0	Solid Sign Height (H)
1	Solid Sign M Dimension (M)
2	Solid Sign N Dimension (N)
3	Structure Natural Frequency
4	Structure Damping Ratio

- **OR Open Sign/Lattice Framework Data :**

Index	Item
0	Orientation Type: Flat (0)/ Rounded (1)
1	Height (H)
2	Width
3	Diameter of typical round member
4	Structure Natural Frequency
5	Structure Damping Ratio
6	Ratio of Solid Area to Gross Area

- **OR Trussed Tower Data :**

Index	Item
0	Horizontal Cross Sectio Type: Triangle (0)/ Square (1)
1	Height (H)
2	Width
3	Structure Natural Frequency
4	Structure Damping Ratio
5	Ratio of Solid Area to Gross Area(in percetage)

- **wallType** (*int*) –

  Building wall to generate Wind Load on:

Value	Wall Type
WindWard = 0	WindWard
LeeWard = 1	Leeward
SideWall = 2	SideWall

  - (0 to 2 for WindWard, Leeward and SideWall, respectively).

Returns:

Returns number of Height or Intensity data.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.ComputeWallWindPressureProfileASCE72016(100, 10, 1, 1, 1, 1, [1], [1.0], [1.0], 0)
```

`CreateLoadEnvelop(envelopNumber: int, envelopType: int, loadCaseList: list)` [source]

Creates a Load Envelop with specified primary load case(s) and envelop type.

Parameters:

- **envelopNumber** (*int*) – Load Envelop reference ID
- **envelopType** (*int*) –

  Type of the load envelop:

Value	Load Envelop Type
0	NONE
1	STRESS
2	SERVICEABILITY
3	COLUMN
4	CONNECTION
5	STRENGTH
6	TEMPORARY

- **loadCaseList** (*list of int*) – Load Case IDs for which to create a load envelop

Returns:

True OK. False General error.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.CreateLoadEnvelop(1, 1, [1,2,3])
>>> print(result)
```

CreateLoadList(*listType: int, loadCaseList: list[int]*) [source]

Creates a load list.

Parameters:

- **listType** (*int*) – Load list type: 0 and 1 for load list and load envelope list, respectively.
- **loadCaseList** (*list of int*) – Load Case reference IDs for which to create a load envelop

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.CreateLoadList(0, [1,2])
>>> print(result)
```

CreateNewLoadCombination(*loadCombTitle: str, loadCombNo: int*) [source]

Creates a new load combination case.

Parameters:

- **loadCombTitle** (*str*) – Title of the load combination.
- **loadCombNo** (*int*) – Load combination number.

Returns:

Load number ID assigned to the load combination. Returns -1 in case of an error. Returns -8004 if it fails to create the load.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> comb_id = staad_obj.Load.CreateNewLoadCombination("DL+LL", 2)
>>> print(comb_id)
```

CreateNewPrimaryLoad(*primaryLoadTitle: str*) [source]

Creates a new PRIMARY load case.

Parameters:

primaryLoadTitle (*str*) – Title of the primary load case.

Returns:

Load number ID if the load case is created successfully. Returns -1 in case of a general error. Returns -8004 if the load case creation fails specifically.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> load_id = staad_obj.load.CreateNewPrimaryLoad("Dead Load")
>>> print(load_id)
```

`CreateNewPrimaryLoadEx(primaryLoadTitle: str, loadType: int)` [source]

Creates new PRIMARY load case.

Parameters:

- **primaryLoadTitle** (*string*) – The primary load case string title.
- **loadType** (*int*) –

  Type of the load:

Value	Load Type	Value	Load Type
0	Dead	12	Traffic
1	Live	13	Temp
2	Roof Live	14	Imperfection
3	Wind	15	Accidental
4	Seismic-H	16	Flood
5	Seismic-V	17	Ice
6	Snow	18	Wind Ice
7	Fluids	19	Crane Hook
8	Soil	20	Mass
9	Rain	21	Gravity
10	Ponding	22	Push
11	Dust	23	None

Returns:

Returns load Number of newly created Primary load Case. Returns -1 if general error. Returns -8004 if fail to create load.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> load_id = staad_obj.Load.CreateNewPrimaryLoadEx("Live Load", 1)
>>> print(load_id)
```

`CreateNewPrimaryLoadEx2(primaryLoadTitle: str, loadType: int, loadCaseNo: int)` [source]

Creates new PRIMARY load case.

Parameters:

- **primaryLoadTitle** (*string*) – The primary load case string title.
- **loadType** (*int*) –

  Type of the load:

Value	Load Type	Value	Load Type
0	Dead	12	Traffic
1	Live	13	Temp
2	Roof Live	14	Imperfection
3	Wind	15	Accidental
4	Seismic-H	16	Flood
5	Seismic-V	17	Ice
6	Snow	18	Wind Ice
7	Fluids	19	Crane Hook
8	Soil	20	Mass
9	Rain	21	Gravity
10	Ponding	22	Push
11	Dust	23	None

- **loadCaseNo** (*int*) – The load case number.

Returns:

Returns load Case number of newly created Primary load Case. Returns 0 if not successfully

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> load_id = staad_obj.Load.CreateNewPrimaryLoadEx2("Wind", 3, 5)
>>> print(load_id)
```

`CreateNewReferenceLoad(nodeNo: int, referenceLoadCaseTitle: str, loadType: int)` [source]

Creates a new reference load case.

Parameters:

- **nodeNo** (*int*) – Reference ID to be assigned to the new reference load case.
- **referenceLoadCaseTitle** (*str*) – Title of the reference load case.
- **loadType** (*int*) – Type of load.

Returns:

Reference load case number ID. Returns -1 in case of an error. Returns -8004 if it fails to create the load.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> ref_id = staad_obj.Load.CreateNewReferenceLoad(1, "Ref Load", 0)
>>> print(ref_id)
```

DeleteDirectAnalysisDefinition() [source]

Deletes whole Direct Analysis Definition.

Returns:

Returns True if successful. Returns False if unsuccessful.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.DeleteDirectAnalysisDefinition()
```

DeleteDirectAnalysisDefinitionParameter(*pParamType: int*) [source]

Deletes respective parameters from Direct Analysis Definition based on the Parameter Type passed as argument (FLEX/AXIAL).

Parameters:

pParamType (*int*) –

Integer indicating type of direct analysis parameter to be added. Integer value should be taken from following table :

Value	AnalysisCommand
FLEX = 0	DirectAnalysisParameterTypes.FLEX
AXIAL = 2	DirectAnalysisParameterTypes.AXIAL

Returns:

Returns True if successful. Returns False if unsuccessful.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.DeleteDirectAnalysisDefinitionParameter(0)
```

DeleteLoadEnvelop(*varEnvNo: int*) [source]

Deletes a specified load envelop.

Parameters:

varEnvNo (*int*) – Load Envelop reference ID.

Returns:

Returns 0 if OK Returns -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.DeleteLoadEnvelop(1)
```

DeleteLoadList(*varLoadListIndex: int*) [source]

Deletes specified load list.

Parameters:

varLoadListIndex (*int*) – Load list index.

Returns:

Returns 0 if OK Returns -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.DeleteLoadList(1)
```

DeletePrimaryLoadCases(*varLoadCaseNos: list, varIsReferenceLoads: bool*) [source]

Deletes specified Primary/Reference Load Cases.

Parameters:

- **varLoadCaseNos** (*List of int*) – List of Primary/Reference load case reference ID.
- **varIsReferenceLoads** (*bool*) – If reference load case(s): TRUE or FALSE

Returns:

Returns 0 if OK Returns -1 if failed to delete load(s)

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.DeletePrimaryLoadCases([1, 2, 3], False)
```

DeleteReferenceLoadCases(*varLoadCaseNos: list*) [source]

Deletes specified Reference Load Cases.

Parameters:

varLoadCaseNos (*List of int*) – List of Reference load case reference ID.

Returns:

Returns 0 if OK Returns -1 if failed to delete load(s)

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.DeleteReferenceLoadCases([1, 2, 3])
```

DeleteWindDefinition(*nTypeNo: int*) [source]

Deletes Wind definition. All defintions will be deleted if this input is set as 0.

Parameters:

nTypeNo (*int*) – Type of Wind.

Returns:

Returns 0 if OK Returns -8039 if Invalid load definition.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.DeleteWindDefinition(1)
```

EndLoadMerging()

[source]

End Load Merging

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.EndLoadMerging()
```

GetActiveLoad()

[source]

Returns the current load case number.

Returns:

Returns active load case number ID. Else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetActiveLoad()
```

GetAssignmentListForLoadType(*loadType: int*, *loadIndex: int*)

[source]

Return the list of entities that have been assigned to a load command in the active load case (see function SetLoadActive). This command is identified as the LoadType, which has a defined reference number (see below) and an index number starting from 0. That is, if a load command has been defined 10 times in the load case, then index 0 identifies the first instance of the command and index 9 identifies the 10th.

Parameters:

- **loadType** (*int*) –

Value	LoadType	○ Value | LoadType	
4000	SelfWeight	3275	Uniform Force (Physical)
3110	Nodal Load (Node)	3280	Uniform Moment (Physical)
3120	Nodal Load (Inclined)	3285	Concentrated Force (Physical)
3910	Nodal Load (Support Displacement)	3290	Concentrated Moment (Physical)
3210	Uniform Force	3295	Trapezoidal (Physical)
3220	Uniform Moment	3310	Pressure on full plate
3230	Concentrated Force	3310	Concentrated Load (Plate)
3240	Concentrated Moment	3310	Partial plate pressure load
3250	Linear Varying	3320	Trapezoidal (Plate)
3260	Trapezoidal	3322	Solid
3260	Hydrostatic	3710	Temperature
3620	Pre/Post Stress	3720	Strain
3810	Fixed End	3721	Strain Rate
3530	FloorLoadGroup	3410	Area
3554	OneWayFloorLoadGroup		

- **loadIndex** (*int*) – Load item index of specified load type (Zero based). Program returns the first one start from loadIndex if exist the same load type in specified load case.

Returns:

Returns List of Entities number ID(s)

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetAssignmentListForLoadType(1, 0)
```

`GetAttribute(lLoadCase: int)` [source]

Gets load attribute information of specified load case.

Parameters:

lLoadCase (*int*) – Load case reference ID.

Returns:

Returns 0 if OK. Returns -1 if General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetAttribute(1)
```

GetBeamCountAtFloor(*varfMinX: float, varfMaxX: float, varfMinY: float, varfMaxY: float, varfMinZ: float, varfMaxZ: float, varnDirection: int*) [source]

Get the beam count at the specific floor.

Parameters:

- **varfMinX** (*float*) – varfMinX X range start (in global coordinate)
- **varfMaxX** (*float*) – varfMaxX X range end (in global coordinate)
- **varfMinY** (*float*) – varfMinY Y range start (in global coordinate)
- **varfMaxY** (*float*) – varfMaxY Y range end (in global coordinate)
- **varfMinZ** (*float*) – varfMinZ Z range start (in global coordinate)
- **varfMaxZ** (*float*) – varfMaxZ Z range end (in global coordinate)
- **varnDirection** (*int*) – varnDirection Direction(1 for XRange, 2 for YRange, 3 for ZRange).

Returns:

The beam count at the specific floor.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetBeamCountAtFloor(0, 10, 0, 10, 0, 10, 1)
```

GetConcForceCount(*nBeamNo: int*) [source]

Get number of concentrated force(s) present for the specified beam.

Parameters:

nBeamNo (*int*) – Beam number ID.

Returns:

Returns the number of concentrated force item(s) applied. Returns -1 if general error

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetConcForceCount(1)
```

GetConcForces(*nBeamNo: int*) [source]

Returns the concentrated force(s) with all the parameters for the specified member.

Parameters:

nBeamNo (*int*) – Beam number ID.

Returns:

Return a list of tuple in which tuple consist of load direction, Magnitude of the concentrate force, distance from the start of the member to concentrated force or moment, Perpendicular distance from the member shear center to the local plane of loading respectively. Load direction will be represented numerically - 1 to 6 for LocalX, LocalY, LocalZ, GlobalX, GlobalY and GlobalZ, respectively

Return type:

List of tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetConcForces(1)
```

GetConcMomentCount(*nBeamNo: int*) [source]

Gets number of concentrated moment(s) present for the specified beam.

Parameters:

nBeamNo (*int*) – Beam number ID.

Returns:

Returns the number of concentrated moment item(s) applied. Returns -1 if general error

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetConcMomentCount(1)
```

GetConcMoments(*nBeamNo: int*) [source]

Returns the concentrated moments(s) with all the parameters for the specified member.

Parameters:

nBeamNo (*int*) – Beam number ID.

Returns:

Return a list of tuple in which tuple consist of load direction, Magnitude of the concentrate moment, distance from the start of the member to concentrated force or moment, Perpendicular distance from the member shear center to the local plane of loading respectively. Load direction will be represented numerically - 1 to 6 for LocalX, LocalY, LocalZ, GlobalX, GlobalY and GlobalZ, respectively

Return type:

List of tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetConcMoments(1)
```

GetElementConcLoadCount(*varPlateNo: int*) [source]

Returns the number of concentrated load for specified plate.

Parameters:

varPlateNo (*int*) – Plate number ID.

Returns:

Returns the number of concentrated load on specified plate. Returns -1 if General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetElementConcLoadCount(1)
```

GetElementConcLoads(*varPlateNo: int*) [source]

Returns the concentrated load(s) with all the parameters for the specified plate.

Parameters:

varPlateNo (*int*) – Plate number ID.

Returns:

Returns a list of tuple in which tuple consist of load direction, pressure, the first d x coordinate, the first d y coordinate respectively. Load direction will be represented numerically - 1 to 9 for LocalX, LocalY, LocalZ, GlobalX, GlobalY, GlobalZ, ProjectedX, ProjectedY and ProjectedZ respectively.

Return type:

List of tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetElementConcLoads(1)
```

GetElementLoadInfo(*loadIndex: int*) [source]

Gets element load information generated by specified load item in specified load case. Please select the loadCase from UI (ElementLoadPressure only supported).

Parameters:

loadIndex (*int*) – Load item index (Zero based)

Returns:

Returns a list of tuple in which tuple consist of load direction, element pressures : dW1, dW2, dW3 and dW4, element force distances: dX1, dY1, dX2 and dY2 respectively. Load direction will be represented numerically - = 0 to 8 for LocalX, LocalY, LocalZ, GlobalX, GlobalY and GlobalZ, ProjectedX, ProjectedY, ProjectedZ respectively.

Return type:

List of tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetElementLoadInfo(1)
```

GetElementPressureLoadCount(*varPlateNo: int*) [source]

Gets the number pressure load(s) for the specified plate.

Parameters:

varPlateNo (*int*) – Plate number ID.

Returns:

Returns the number of pressure load(s). Returns -1 if General error.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetElementPressureLoadCount(1)
```

GetElementPressureLoads(*varPlateNo: int*) [source]

Returns the pressure load(s) with all the parameters for the specified plate.

Parameters:

varPlateNo (*int*) – Plate number ID.

Returns:

Returns a list of tuple in which tuple consist of load direction, Magnitude of the pressure load(s), Top-Left coordinate X (local), Top-Left coordinate Y (local), Bottom-Right coordinate X (local) and Bottom-Right coordinate Y (local) respectively. Load direction will be represented numerically - 1 to 9 for LocalX, LocalY, LocalZ, GlobalX, GlobalY, GlobalZ, ProjectedX, ProjectedY and ProjectedZ respectively.

Return type:

List of tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetElementPressureLoads(1)
```

GetEnvelopeCount() [source]

Returns number of Envelopes defined.

Returns:

Total Number of load Envelopes present.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetEnvelopeCount()
```

GetEnvelopeIDs() [source]

Gets the list of Loads Envelope IDs present in the staad file.

Returns:

Envelope ID(s)

Return type:

List of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetEnvelopeIDs()
```

GetInfluenceArea(*varfMinX: float, varfMaxX: float, varfMinY: float, varfMaxY: float, varfMinZ: float, varfMaxZ: float, varnDirection: int*) [source]

Returns a dictionary of beam to influence area at the specific floor.

Parameters:

- **varfMinX** (*float*) – X range start (in global coordinate).
- **varfMaxX** (*float*) – X range end (in global coordinate).
- **varfMinY** (*float*) – Y range start (in global coordinate).

- **varfMaxY** (*float*) – Y range end (in global coordinate).
- **varfMinZ** (*float*) – Z range start (in global coordinate).
- **varfMaxZ** (*float*) – Z range end (in global coordinate).
- **varnDirection** (*int*) – Direction(1 for XRange, 2 for YRange, 3 for ZRange).

Returns:

Returns dictionary have beam id to influence area data.

Return type:

Dictionary

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetInfluenceArea(0, 10, 0, 10, 0, 10, 1)
```

GetLinearVaryingLoadCount(*nBeamNo: int*) [source]

Returns number of linear varying load(s) present for the specified beam.

Parameters:

nBeamNo (*int*) – Beam number ID.

Returns:

Returns the number of linear varying load item(s) applied. Returns -1 if General Error

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetLinearVaryingLoadCount(1)
```

GetLinearVaryingLoads(*nBeamNo: int*) [source]

Returns parameters for defining linear varying loads for specified beam.

Parameters:

nBeamNo (*int*) – Beam number ID.

Returns:

Returns a list of tuple in which tuple consist of load direction, load at the start of the member, load at the end of the member, Load in the middle of the member (for triangular load) respectively. Load direction will be represented numerically - 1 to 3 for local X, Y and Z, respectively

Return type:

List of tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetLinearVaryingLoads(1)
```

GetListSizeForLoadType(*loadType: int, loadIndex: int*) [source]

Gets number of entities to vwhich specified Load Type and load index.

Parameters:

- **loadType** (*int*) –

Value	LoadType	○	Value | LoadType	
4000	SelfWeight		3275	Uniform Force (Physical)
3110	Nodal Load (Node)		3280	Uniform Moment (Physical)
3120	Nodal Load (Inclined)		3285	Concentrated Force (Physical)
3910	Nodal Load (Support Displacement)		3290	Concentrated Moment (Physical)
3210	Uniform Force		3295	Trapezoidal (Physical)
3220	Uniform Moment		3310	Pressure on full plate
3230	Concentrated Force		3310	Concentrated Load (Plate)
3240	Concentrated Moment		3310	Partial plate pressure load
3250	Linear Varying		3320	Trapezoidal (Plate)
3260	Trapezoidal		3322	Solid
3260	Hydrostatic		3710	Temperature
3620	Pre/Post Stress		3720	Strain
3810	Fixed End		3721	Strain Rate
3530	FloorLoadGroup		3410	Area
3554	OneWayFloorLoadGroup			

- **loadIndex** (*int*) – Load item index of specified load type (Zero based). Program returns the first one start from loadIndex if exist the same load type in specified load case.

Returns:

Returns the number of entities.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetListSizeForLoadType(1, 0)
```

`GetLoadAndFactorForCombination(varLoadCombNo: int)` [source]

Get number of concentrated force(s) present for the specified beam.

Parameters:

varLoadCombNo (*int*) – Combination Load case reference number ID.

Returns:

Returns a Tuple consisting of a list of load case reference number IDs and list of multiplication factors. For SRSS, in multiplication factor list, an extra element is added, at the end. This factor represents overall multiplication factor for SRSS combination.

Return type:

Tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetLoadAndFactorForCombination(1)
```

GetLoadCaseTitle(*varLoadNo: int*) [source]

Returns title of the specified load case as a text string. Input 0 to retrieve title of current active load case or reference load case

Parameters:

varLoadNo (*int*) – The load case string title.

Returns:

Returns the load case string title. Returns “NONE” if load case varLoadNo not found.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetLoadCaseTitle(1)
```

GetLoadCombinationCaseCount() [source]

Returns the total number of load combination cases in the current structure.

Returns:

Total number of load combination cases.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetLoadCombinationCaseCount()
```

GetLoadCombinationCaseNumbers() [source]

Retrieves all load combination case numbers.

Returns:

List of load case reference number IDs.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetLoadCombinationCaseNumbers()
```

GetLoadCountInLoadList(*varLoadListIndex: int*) [source]

Gets the number of load case(s) in specified load list.

Parameters:

varLoadListIndex (*int*) – Load list index.

Returns:

The number of Load Case(s) in specified Load List.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetLoadCountInLoadList(1)
```

GetLoadEnvelopeDetails(*EnvNo: int*) [source]

Returns

Parameters:

EnvNo (*int*) – Load Envelope reference ID.

Returns:

Returns a tuple containing EnvelopeType and NumberofLoadCasesInEnvelope information respectively. Type of Load Envelope +——-+————————+ | Value | Load Envelop Type | +=======+=====================+ | 0 | NONE | +——-+————————+ | 1 | STRESS | +——-+————————+ | 2 | SERVICEABILITY | +——-+————————+ | 3 | COLUMN | +——-+————————+ | 4 | CONNECTION | +——-+————————+ | 5 | STRENGTH | +——-+————————+ | 6 | TEMPORARY | +——-+————————+

Return type:

Tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetEnvelopeDetails(1)
```

GetLoadItemType(*loadCaseNo: int, loadItemIndex: int*) [source]

Returns the load item type for the specified loadIndex and loadCase.

Parameters:

- **loadCaseNo** (*int*) – Load case number.
- **loadItemIndex** (*int*) – Load item index (Zero based).

Returns:

Returns LoadItemType for the specified loadIndex and loadCase. Returns 0 if LoadCase/LoadItemIndex Not Found. Return Values a LoadItem Type +——-+——————————————+—+——-+——————————————+ | Value | LoadItem Type | * | Value | LoadIte +=======+==+===+=======+==========================
| 4000 | SelfWeight | | 3520 | FloorLoadZrange | +——-+——————————————+—+——-+——————————————+ | 3110 | N (Node) | | 3530 | FloorLoadGroup | +——-+——————————————+—+——-+——————————————+ | 3120 | Nodal Load (In 3551 | OneWayFloorLoadXrange | +——-+——————————————+—+——-+——————————————+ | 3910 | Nodal Load (Su Displacement) | | 3552 | OneWayFloorLoadYrange | +——-+——————————————+—+——-+——————————————+ | 3312 Load (Region node load) | | 3553 | OneWayFloorLoadZrange | +——-+——————————————+—+——-+—————————— 3210 | Uniform Force | | 3554 | OneWayFloorLoadGroup | +——-+——————————————+—+——-+—————————————— Uniform Moment | | 3310 | Pressure on full plate | +——-+——————————————+—+——-+——————————————+ | 3230 | Concentrated Force | | 3311 | Concentrated Load (Plate) | +——-+——————————————+—+——-+—————————————— Concentrated Moment | | 3312 | Partial plate pressure load | +——-+——————————————+—+——-+—————————— | Linear Varying | | 3320 | Trapezoidal (Plate) | +——-+——————————————+—+——-+——————————————+ | 3260 | Trap 3322 | Solid | +——-+——————————————+—+——-+——————————————+ | 3261 | Hydrostatic | | 3710 | Temperature | ——————————————+—+——-+——————————————+ | 3620 | Pre/Post Stress | | 3720 | Strain | +——-+———————————— +—+——-+——————————————+ | 3810 | Fixed End | | 3721 | Strain Rate | +——-+——————————————+—+——-+ ——————————————+ | 3275 | Uniform Force (Physical) | | 4400 | UBC Load | +——-+——————————————+—+——-+ ——————————————+ | 3280 | Uniform Moment (Physical) | | 4600 | Wind Load | +——-+——————————————+—+——-+ ——————————————+ | 3285 | Concentrated Force (Physical) | | 4610 | Wind Load Dynamic | +——-+———————————————— ——-+——————————————+ | 3290 | Concentrated Moment (Physical) | | 4405 | IbcLoad | +——-+——————————————+— +————————————————+ | 3295 | Trapezoidal (Physical) | | 4410 | 1893Load | +——-+——————————————+—+——-+ ——————————————+ | 3410 | Area | | 4500 | AijLoad | +——-+——————————————+—+——-+—————————————— FloorLoadYrange | | 4510 | ColombianLoad | +——-+——————————————+—+——-+——————————————+ | 3511 |

FloorLoadXrange | | 4520 | CFELoad | +———-+————————————+—+———-+————————————+ | 4570 | TurkishLoad RPALoad | +———-+————————————+—+———-+————————————+ | 4575 | GB50011Load | | 4540 | NTCLoad | +———————————+—+———-+————————————+ | 4576 | Colombian2010Load | | 4550 | NRCLoad | +———-+ ———————————+—+———-+————————————+ | 4820 | TimeHistoryLoad | | 4560 | NRCLoad2005 | +———-+ ———————————+—+———-+————————————+ | 4651 | Snow Load Data | | 4561 | NRCLoad2010 | +———-+ ———————————+—+———-+————————————+ | 4201 | Repeat load data | | 4100 | Spectrum Load | +———-+ ———————————+—+———-+————————————+ | 4223 | Notional Load Data | | 4700 | Calulate Natural Frequency | ———————————+—+———-+————————————+ | 4220 | Reference Load | | 4710 | Modal Calculation Requested | +———————————+—+———-+————————————+ | 4101 | Spectrum Data | | 4222 | Notional Load | +———-+ ———————————+—+———-+————————————+ | 4701 | Calulate Rayleigh Frequency | | 4650 | Snow Load | +———- ———————————+—+———-+————————————+ | 4200 | Repeat load | | | | +———-+————————————+—+—————————+

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetLoadItemType(1, 1)
```

GetLoadItemsCount(*loadCaseNo: int*) [source]

Returns the number of loaditems in the specified load case.

Parameters:

loadCaseNo (*int*) – Load case number.

Returns:

Returns the number of loaditems in the specified load case. Returns -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetLoadItemsCount(1)
```

GetLoadListCount() [source]

Gets the number of existing load list(s)

Returns:

Returns the number of load list(s). Returns -1 if General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetLoadListCount()
```

GetLoadListfromLoadEnvelope(*EnvNo: int*) [source]

Gets the list of primary load case reference Ids present in the load envelope passed.

Parameters:

EnvNo (*int*) – Load Envelope reference ID

Returns:

(Primary) load case(s) reference ID(s).

Return type:

List of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetLoadListfromLoadEnvelope(1)
```

GetLoadType(*varLoadNo: int*) [source]

Returns primary load case category(s) as an long value.

Parameters:

varLoadNo (*int*) – Primary load case reference ID. Pass in 0 to get information about current active load case or reference load case

Returns:

Returns 0 if Dead. Returns 1 if Live. Returns 2 if Roof Live. Returns 3 if Wind. Returns 4 if Seismic-H. Returns 5 if Seismic-V. Returns 6 if Snow. Returns 7 if Fluids. Returns 8 if Soil. Returns 9 if Rain. Returns 10 if Ponding. Returns 11 if Dust. Returns 12 if Traffic. Returns 13 if Temperature. Returns 14 if Imperfection. Returns 15 if Accidental. Returns 16 if Flood. Returns 17 if Ice. Returns 18 if Wind Ice. Returns 19 if Crane Hook. Returns 20 if Mass. Returns 21 if Gravity. Returns 22 if Push. Returns 23 if None. Returns -1 if General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetLoadType(1)
```

GetLoadTypeCount(*loadType: int*) [source]

Gets the number of load(s) with specified Load Type in active Load Case.

Parameters:

loadType (*int*) –

Value	LoadType	•	Value | LoadType	
4000	SelfWeight		3275	Uniform Force (Physical)
3110	Nodal Load (Node)		3280	Uniform Moment (Physical)
3120	Nodal Load (Inclined)		3285	Concentrated Force (Physical)
3910	Nodal Load (Support Displacement)		3290	Concentrated Moment (Physical)
3210	Uniform Force		3295	Trapezoidal (Physical)
3220	Uniform Moment		3310	Pressure on full plate
3230	Concentrated Force		3310	Concentrated Load (Plate)
3240	Concentrated Moment		3310	Partial plate pressure load
3250	Linear Varying		3320	Trapezoidal (Plate)
3260	Trapezoidal		3322	Solid
3260	Hydrostatic		3710	Temperature
3620	Pre/Post Stress		3720	Strain
3810	Fixed End		3721	Strain Rate
3530	FloorLoadGroup		3410	Area
3554	OneWayFloorLoadGroup			

Returns:

Returns the number of load(s). Returns 0 if loadCaseNo not found.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetLoadTypeCount(1)
```

GetLoadsInLoadList(*varLoadListIndex: int*) [source]

Gets the load case(s) in specified load list.

Parameters:

varLoadListIndex (*int*) – Load list index(Starts from one).

Returns:

Load Case reference IDs list.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetLoadsInLoadList(1)
```

GetMemberLoadInfo(*loadIndex: int*) [source]

Gets member load(s) information generated by specified load item in specified load case.

Parameters:

loadIndex (*int*) – Load item index (Zero based)

Returns:

- direction: int (Load direction will be represented numerically - 1 to 9 for LocalX, LocalY, LocalZ, GlobalX, GlobalY, GlobalZ, ProjectedX, ProjectedY, ProjectedZ, respectively.)
- member force parameters: List [dW1, dW2, dW3]
- member force distances: List [dD1, dD2, dD3]

Return type:

tuple containing

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetMemberLoadInfo(1)
```

GetNoLoadFactorDirectionInNotionalLoad(*nIndex: int*) [source]

Gets the no of factor for specified Notional load.

Parameters:

nIndex (*int*) – The index for Notional load.

Returns:

Returns the factor for specified Notional load. Returns -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetNoLoadFactorDirectionInNotionalLoad(1)
```

GetNoLoadFactorInRepeatLoad(*nIndex: int*) [source]

Returns the number of load and factor pairs associated with a given repeat load command in the active load case.

Parameters:

nIndex (*int*) – The index(One based) for repeat load.

Returns:

Returns number of load and factor pairs associated with a given repeat load command in the active load case. Returns -1 if case of invalid repeat load index.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetNoLoadFactorInRepeatLoad(1)
```

GetNoOfLoadAndFactorPairsForCombination(*varLoadCombNo: int*) [source]

Gets the number of load case(s) applied with multiplication factor in specified load combination.

Parameters:

varLoadCombNo (*int*) – Combination Load case reference number ID.

Returns:

Returns the number of load cases in specified load combination.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetNoOfLoadAndFactorPairsForCombination(1)
```

GetNoOfSetsInReferenceLoad(*nIndex: int*) [source]

Returns the number of reference load case-factor sets in a specified reference load item.

Parameters:

nIndex (*int*) – Index of the reference load case item.

Returns:

Number of sets in the reference load item. Returns -1 in case of an error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetNoOfSetsInReferenceLoad(1)
```

GetNodalLoadCount(*nNodeNo: int*) [source]

Returns number of nodal loads present for the specified node.

Parameters:

nNodeNo (*int*) – Node Id

Returns:

Returns the number of node(s). Else -1 if general error (perhaps load case not found).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetNodalLoadCount(1)
```

GetNodalLoadInfo(*loadIndex: int*) [source]

Gets nodal load(s) generated by specified load item in specified load case.

Parameters:

loadIndex (*int*) – Load item index.

Returns:

Returns a list of 5 nodal forces - FX, FY, FZ, MX, MY and MZ which are placed respectively.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetNodalLoadInfo(1)
```

GetNodalLoads(*nNodeNo: int*) [source]

Returns tuple of list of forces in X direction, forces in Y direction, forces in Z direction, moments in X direction, moments in Y direction and moments in Z direction respectively.

Parameters:

nNodeNo (*int*) – Node Id

Returns:

Returns a tuple of list of forces in X direction, forces in Y direction, forces in Z direction, moments in X direction, moments in Y direction and moments in Z direction respectively.

Return type:

Tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetNodalLoads(1)
```

GetNotionalLoadByIndex(*nIndex: int*) [source]

Gets load case(s), direction(s) and factor(s) for specified Notional load.

Parameters:

nIndex (*int*) – The index for Notional load.

Returns:

Returns a list of tuple in which tuple consist of load direction, load case reference ID(s) in VARIANT array - (if +ve values = Primary Load Cases or -ve values = Reference Load Cases) , load factors respectively. Load direction will be represented numerically - = 1 to 3 for X, Y and Z direction respectively.

Return type:

List of tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetNotionalLoadByIndex(1)
```

GetNotionalLoadCount() [source]

Returns the number of Notional load.

Returns:

Returns the number of Notional load Returns -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetNotionalLoadCount()
```

GetPrimaryLoadCaseCount() [source]

Returns the total number of primary load cases in the current structure.

Returns:

Total number of primary load cases.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetPrimaryLoadCaseCount()
```

GetPrimaryLoadCaseNumbers() [source]

Retrieves all primary load case numbers.

Returns:

List of load case reference number IDs.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetPrimaryLoadCaseNumbers()
```

GetReferenceLoadByIndex(*nIndex: int*) [source]

Retrieves a dictionary of load case numbers and their corresponding factors for a given reference load case.

Parameters:

nIndex (*int*) – Index of the reference load.

Returns:

tuple of load case number factor lists. [[loadcase1, loadcase2, ...], [factor1, factor2, ...]]

Return type:

tuple of lists

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetReferenceLoadByIndex(1)
```

GetReferenceLoadCaseCount() [source]

Returns the number of reference load case items in the currently active load case.

Returns:

Number of reference load case items. Returns -1 in case of an error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetReferenceLoadCaseCount()
```

GetReferenceLoadCaseNumbers()

[source]

Retrieves reference load case number IDs from Reference Load Definitions.

Returns:

List of reference load case IDs.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetReferenceLoadCaseNumbers()
```

GetReferenceLoadCaseTitle(*varLoadNo: int*)

[source]

Returns the title of a reference load case.

Parameters:

varLoadNo (*int*) – Reference load number.

Returns:

Title of the reference load case.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetReferenceLoadCaseTitle(1)
```

GetReferenceLoadCount()

[source]

Returns the number of reference load items in the currently active load case.

Returns:

Number of reference load items. Returns -1 in case of an error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetReferenceLoadCount()
```

GetReferenceLoadType(*varLoadNo: int*)

[source]

Returns the type of a reference load.

Parameters:

varLoadNo (*int*) – Reference load number.

Returns:

Reference load type (0 to 23). Returns -1 in case of an error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetReferenceLoadType(1)
```

GetRepeatLoadByIndex(*nIndex: int*) [source]

Returns the dictionary of load case IDs to load factors for a given repeat load command in the active load case.

Parameters:

nIndex (*int*) – The index(One based) for repeat load.

Returns:

Returns a dictionary of load case ID to load factor.

Return type:

dictionary

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetRepeatLoadByIndex(1)
```

GetRepeatLoadCount() [source]

Returns the number of repeat load commands in the active load case.

Returns:

Returns the number of repeat load commands in the active load case. Returns 0 if General Error

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetRepeatLoadCount()
```

GetTrapLoadCount(*nBeamNo: int*) [source]

Returns number of trapezoidal load(s) present for the specified beam.

Parameters:

nBeamNo (*int*) – Beam number ID.

Returns:

Returns the number of trapezoidal load item(s) applied. Returns -1 if general error

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetTrapLoadCount(1)
```

GetTrapLoads(*nBeamNo: int*) [source]

Returns the trapezoidal load(s) with all the parameters for the specified member.

Parameters:

nBeamNo (*int*) – Beam number ID.

Returns:

Return a list of tuple in which tuple consist of load direction, Load at the start of the member, Load at the end of the member, distance from the start of the member to loading starting point, distance from the end of the member to loading stopping point respectively. Load direction will be represented numerically - 1 to 9 for LocalX, LocalY, LocalZ, GlobalX, GlobalY, GlobalZ, ProjectedX, ProjectedY, ProjectedZ, respectively.

Return type:

List of tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetTrapLoads(1)
```

GetUDLLoadCount(*nBeamNo: int*) [source]

Returns the number of uniformly distributed load(s) present for the specified beam.

Parameters:

nBeamNo (*int*) – Beam number ID.

Returns:

Returns the number of uniformly distributed load item(s) applied. Returns -1 if general error

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetUDLLoadCount(1)
```

GetUDLLoads(*nBeamNo: int*) [source]

Gets the uniformly distributed load(s) with all the parameters for the specified member.

Parameters:

nBeamNo (*int*) – Beam number ID.

Returns:

Return a tuple of lists in which each list consist of - load directions (1 to 9 for LocalX, LocalY, LocalZ, GlobalX, GlobalY, GlobalZ, ProjectedX, ProjectedY, ProjectedZ, respectively.) - magnitude of uniform force - distance from start of member to the start of load - distance from start of member to the end of load - perpendicular distance from the member shear center to the local plane of loading respectively. [[dirL1, dirL2,..], [forceL1, forceL2,..], [dst_startL1, dst_startL2,..], [dst_endL1, dst_endL2,..], [dst_perpendicularL1, dst_perpendicularL2,..]]

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetUDLLoads(1)
```

GetUNIMomentCount(*nBeamNo: int*) [source]

Returns the count of uniformly distributed (UNI) moment applied to the specified member.

Parameters:

nBeamNo (*int*) – Beam number ID.

Returns:

Returns the number of uniformly distributed (UNI) moment item(s) applied. Returns -1 if general error

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetUNIMomentCount(1)
```

GetUNIMoments(*nBeamNo: int*) [source]

Returns the uniformly distributed (UNI) moments with all the parameters for the specified member.

Parameters:

nBeamNo (*int*) – Beam number ID.

Returns:

Return a list of tuple in which tuple consist of load direction, magnitude of uniform moment, distance from start of member to the start of load, distance from start of member to the end of load, perpendicular distance from the member shear center to the local plane of loading respectively. Load direction will be represented numerically - 1 to 9 for LocalX, LocalY, LocalZ, GlobalX, GlobalY, GlobalZ, ProjectedX, ProjectedY, ProjectedZ, respectively.

Return type:

List of tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.GetUNIMoments(1)
```

IsCombinationCase(*nLoadCase: int*) [source]

Checks if specified load case is combination load case.

Parameters:

nLoadCase (*int*) – Load case reference ID

Returns:

Returns 1 if YES Returns 0 if NO Returns -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.IsCombinationCase(1)
```

IsDynamicLoadIncluded(*nLoadCase: int*) [source]

Checks if dynamic load included in specified load case.

Parameters:

nLoadCase (*int*) – Load case reference ID

Returns:

Returns 1 if YES Returns 0 if NO Returns -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.IsDynamicLoadIncluded(1)
```

MergeLoadsOnBeam(*varBeamToKeep: int, varBeamToMerge: int*) [source]

Merge Load from beam to merge.

Parameters:

- **varBeamToKeep** (*int*) – Beam Id where load to not merge.
- **varBeamToMerge** (*int*) – Beam Id to where load to merge.

Returns:

Returns 1 (TRUE) if Successful Returns 0 (FALSE) if General Error.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.MergeLoadsOnBeam(1, 2)
```

ModifySeismicDefinitionParams(*varParamName: str, varValue: float*) [source]

Modifies or adds a seismic parameter in the existing seismic definition.

Parameters:

- **varParamName** (*string*) – Parameter name for the corresponding code in the seismic definition.
- **varValue** (*float*) –

  Value corresponding to the above parameter:

Seismic Code	Parameters
ALGERIAN: RPA	A Q RX RZ STYPE CT CRDAMP PX PZ
CANADIAN: NRC-1995	V ZA ZV RX RZ I F CT PX PZ
CANADIAN: NRC-2005	SA1 SA2 SA3 SA4 IE SCLASS MVX MVZ JX JZ RDX RDZ ROX ROZ CT PX PZ FA FV
CANADIAN: NRC-2010	SA1 SA2 SA3 SA4 I SCLASS MVX MVZ RDX RDZ ROX ROZ CTX CTZ PX PZ FA FV STX STZ MD
CHINESE: GB50011-2001	INTENSITY FREQUENT RARE GROUP SCLASS DAMP DELN SF PX PZ GFACTOR Note: For CHINESE: GB50011-2001 FREQUENT/RARE parameter value will be 0, 1 respectively
CHINESE: GB50011-2010	INTENSITY FREQUENT FORTIFIED RARE GROUP SCLASS DAMP GFACTOR DELN SF PX PZ Note: For CHINESE: GB50011-2010 FREQUENT/FORTIFIED/RARE parameter value will be 0, 1 and 2 respectively
COLOMBIAN: NSR 98	ZONE I S
COLOMBIAN: NSR 2010	AA AV FA FV I CT PX PZ ALPHA
IBC 2000	SDS SD1 S1 I RX RZ SCLASS CT PX PZ
IBC 2003	SDS SD1 S1 I RX RZ SCLASS CT PX PZ
IBC 2006	SS S1 ZIP I RX RZ SCLASS CTX CTZ PX PZ LAT LONG TL FA FV XX XZ
IBC 2012	SS S1 ZIP I RX RZ SCLASS CTX CTZ PX PZ LAT LONG TL FA FV XX XZ
IBC 2015	SS S1 ZIP I RX RZ SCLASS CTX CTZ PX PZ LAT LONG TL FA FV XX XZ
IBC 2018	SS S1 ZIP I RX RZ SCLASS CTX CTZ PX PZ LAT LONG TL FA FV XX XZ Note: For IBC 2006 - 2018 Please provide any one of ZIP OR LAT LONG OR SS S1
INDIAN: IS 1893-1984	ZONE K I B PX PZ
INDIAN: IS 1893-2002/2005	ZONE RF I SS ST DM PX PZ DT GL SA DF CS AX ES CV DV
INDIAN: IS 1893-2016	ZONE RF I SS ST DM PX PZ DT GL SA DF HT DX DZ
INDIAN: IS 1893(Part4) 2015	ZONE RF I SS ST DM PX PZ SA DF
JAPANESE (AIJ)	ZONE CO TC ALPHA
MEX: CFE-1993	ZONE QX QZ GROUP STYPE REGULAR TS PX PZ
MEX: NTC-1987	ZONE QX QZ GROUP SHADOWED REGULAR REDUCE PX PZ Note: For SHADOWED, REGULAR and REDUCE parameter value will be 0 or 1 respectively
TURKISH	A TA TB I RX RZ CT PX PZ
UBC 1985	ZONE I K TS
UBC 1994	ZONE I RWX RWZ S CT PX PZ
UBC 1997	ZONE I RWX RWZ STYPE CT PX PZ NA NV

Returns:

Returns 0 if OK Returns -1 if general error.

Return type:

int

Example

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.ModifySeismicDefinitionParams('ZONE', 0.2)
```

`RemoveAttribute(lLoadCase: int)` [source]

Removes the load attribute specified by lLoadCase.

Parameters:

lLoadCase (*int*) – Load case reference ID

Returns:

Returns 0 if OK Returns -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.RemoveAttribute(1)
```

`RemoveLoadCasesFromEnvelop(varEnvNo: int, varLoadCaseList: list)` [source]

Removes a list of primary load case(s) from an existed load envelop.

Parameters:

- **varEnvNo** (*int*) – Load Envelop reference ID
- **varLoadCaseList** (*list of int*) – Load cases reference IDs list.

Returns:

Returns 0 if OK Returns -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.RemoveLoadCasesFromEnvelop(1, [2, 3])
```

`SetASDLoadAttribute(loadCaseRefID: int, strengthType: int, allowStressIncrease: bool)` [source]

Sets Allowable Stress Design (ASD) load attribute.

Parameters:

- **loadCaseRefID** (*int*) – Load case reference ID.
- **strengthType** (*int*) –

  Strength Type :

Value	Integer
STRENGTH_TYPE_NONE	0
NORMAL_ASD_WORKING_STRESS_FORCES_WITHOUT_P_DELTA	1
NORMAL_ASD_WORKING_STRESS_FORCES_WITH_P_DELTA	2
STRENGTH_TYPE_OF_FORCES_WITHOUT_P_DELTA	3
STRENGTH_TYPE_OF_FORCES_WITH_P_DELTA	4
COLUMN_ONLY_STRENGTH_TYPE_OF_FORCES_WITHOUT_P_DELTA	5
COLUMN_ONLY_STRENGTH_TYPE_OF_FORCES_WITH_P_DELTA	6

- **allowStressIncrease** (*bool*) – Allow 1/3 stress increase in ASD.

Returns:

Returns 0 OK. Returns -1 General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.SetASDLoadAttribute(1, 1, True)
>>> print(result)
```

SetLSDLoadAttribute(*loadCaseRefID: int*) [source]

Sets Limit State Design (LSD) load attribute.

Parameters:

loadCaseRefID (*int*) – Load case reference ID.

Returns:

Returns 0 OK. Returns -1 General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.SetLSDLoadAttribute(1)
>>> print(result)
```

SetLoadActive(*loadNumber: int*) [source]

Activates the specified load number to allow adding or removing load items.

Parameters:

loadNumber (*int*) – Load case reference number ID.

Returns:

True if the load case was successfully activated. False if an error occurred.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.SetLoadActive(1)
>>> print(result)
```

SetLoadType(*loadCaseNumber: int, loadType: int*) [source]

Set load type to load case for considering load combination.

Parameters:

- **loadCaseNumber** (*int*) – The load case reference number ID.
- **loadType** (*int*) –

  Type of the load.:

Value	Load Type	Value	Load Type
0	Dead	12	Traffic
1	Live	13	Temp
2	Roof Live	14	Imperfection
3	Wind	15	Accidental
4	Seismic-H	16	Flood
5	Seismic-V	17	Ice
6	Snow	18	Wind Ice
7	Fluids	19	Crane Hook
8	Soil	20	Mass
9	Rain	21	Gravity
10	Ponding	22	Push
11	Dust	23	None

Returns:

Returns 0 OK. Returns -1 General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.SetLoadType(1, 0)
>>> print(result)
```

SetReferenceLoadActive(*nLoadCaseNo: int*) [source]

Activates a reference load case to allow operations on its items.

Parameters:

nLoadCaseNo (*int*) – Reference load case ID in Load Case Details.

Returns:

Reference load case number ID. Returns -1 in case of an error. Returns -8002 if the load case is not found.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Load.SetReferenceLoadActive(2)
>>> print(result)
```

SplitLoadsOnBeam(*varBeamOld: int, varBeamNew: int*) [source]

Split Load from BeamOld to BeamNew.

Parameters:

- **varBeamOld** (*int*) – Old Beam Id
- **varBeamNew** (*int*) – New Beam Id

Returns:

Returns 1 (TRUE) if Successful Returns 0 (FALSE) if General Error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Load.SplitLoadsOnBeam(1, 2)
```

__init__(*staadObj*) [source]

Property

Contents

- OSProperty

class openstaadpy.os_analytical.osproperty.OSProperty [source]

Bases: object

AddControlDependentRelation(*control_node: int, rigid_type: int, fx: int, fy: int, fz: int, mx: int, my: int, mz: int, dependent_node_list: list*) [source]

Add a control/dependent joint specification to specified node(s).

Parameters:

- **control_node** (*int*) – Set node (number ID) control node.
- **rigid_type** (*int*) – Set plate rigid: all directions rigid (= 0), XY plate rigid (= 1), YZ plate rigid (= 2), ZX plate rigid (= 3), specific define FX, FY, FZ, MX, MY, MZ rigid (= others)
- **fx** (*int*) – Rigid in X direction translation (Rigid = 1, Not Rigid = 0)
- **fy** (*int*) – Rigid in Y direction translation (Rigid = 1, Not Rigid = 0)
- **fz** (*int*) – Rigid in Z direction translation (Rigid = 1, Not Rigid = 0)
- **mx** (*int*) – Rigid in X direction rotation (Rigid = 1, Not Rigid = 0)
- **my** (*int*) – Rigid in Y direction rotation (Rigid = 1, Not Rigid = 0)
- **mz** (*int*) – Rigid in Z direction rotation (Rigid = 1, Not Rigid = 0)
- **dependent_node_list** (*list*) – Nodes number ID list

Returns:

Returns 0 if successful.

Returns -106 if list of long expected.

Returns -6029 if Library Error: Unable to create CONTROL/DEPENDENT specification.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.AddControlDependentRelation(3, -1, 1, 1, 0, 0, 0, 1, [
```

AddUPTPropertyANGLE(*table_reference_id: int, section_name: str, depth_of_angle: float, width_of_angle: float, flange_thickness: float,*

gyration_radius: float, shear_area_y: float, shear_area_z: float) [source]

Add angle type to an defined UPT section.

Parameters:

- **table_reference_id** (*int*) – The existing table number ID.
- **section_name** (*str*) – UPT section string name.
- **depth_of_angle** (*float*) – Depth of angle.
- **width_of_angle** (*float*) – Width of angle.
- **flange_thickness** (*float*) – Thickness of flange (TF).
- **gyration_radius** (*float*) – Radius of gyration about principal axis.
- **shear_area_y** (*float*) – Shear area in local y-axis. If zero, shear deformation is ignored in the analysis.
- **shear_area_z** (*float*) – Shear area in local z-axis. If zero, shear deformation is ignored in the analysis.

Returns:

Returns 0 if OK.
Returns -6032 if the section with section_name cannot be added to the UPT table using table_reference_id.
Returns -6045 if a section with the same section_name already exists in the UPT table for the given table_reference_id.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_num_id = staad_obj.Property.CreateUPTTable(3)
>>> status = staad_obj.Property.AddUPTPropertyANGLE(upt_num_id, "UPT_Channel1", 12.32,
```

AddUPTPropertyCHANNEL(*table_reference_id: int, stn_name: str, cro_sec_area: float, sectn_depth: float, web_Thickness: float, top_flange_width: float, top_flange_thickness: float, torsional_constant: float, moi_l_y: float, moi_l_z: float, c_z: float, shear_area_y: float, shear_area_z: float*) [source]

Add channel type to an defined UPT section.

Parameters:

- **table_reference_id** (*int*) – The existing table number ID.
- **stn_name** (*str*) – UPT section string name.
- **cro_sec_area** (*float*) – Cross section area.
- **sectn_depth** (*float*) – Depth of the section.
- **web_Thickness** (*float*) – Thickness of web.
- **top_flange_width** (*float*) – Width of the top flange.

- **top_flange_thickness** (*float*) – Thickness of top flange.
- **torsional_constant** (*float*) – Torsional constant.
- **moi_l_y** (*float*) – Moment of inertia about local y-axis.
- **moi_l_z** (*float*) – Moment of inertia about local z-axis.
- **c_z** (*float*) – CZ value.
- **shear_area_y** (*float*) – Shear area in local y-axis. If zero, shear deformation is ignored in the analysis.
- **shear_area_z** (*float*) – Shear area in local z-axis. If zero, shear deformation is ignored in the analysis.

Returns:

Returns 0 if OK. Returns -6032 if unable to add section stn_name in UPT table_reference_id. Returns -6045 if a section with the same section_name already exists in the UPT table for the given table_reference_id.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_num_id = staad_obj.Property.CreateUPTTable(2)
>>> result = staad_obj.Property.AddUPTPropertyCHANNEL(upt_num_id, "VJG20-2", 1.1, 2.0,
```

`AddUPTPropertyDOUBLEANGLE(table_reference_id: int, section_name: str, depth_angle: float, width_angle: float, flanges_thickness: float, distance_between_two_angles: float, torsional_constant: float, moi_y: float, moi_z: float, dist_z_top_section: float, shear_area_y: float, shear_area_z: float)` [source]

Add double angle type to an defined UPT section.

Parameters:

- **table_reference_id** (*int*) – The existing table number ID.
- **section_name** (*str*) – UPT section string name.
- **depth_angle** (*float*) – Depth of angle.
- **width_angle** (*float*) – Width of angle.
- **flanges_thickness** (*float*) – Thickness of flanges.
- **distance_between_two_angles** (*float*) – Distance between two angles.
- **torsional_constant** (*float*) – Torsional constant
- **moi_y** (*float*) – Moment of inertia about local y-axis.
- **moi_z** (*float*) – Moment of inertia about local z-axis.
- **dist_z_top_section** (*float*) – Distance from z axis to the top of section.

- **shear_area_y** (*float*) – Shear area in local y-axis. If zero, shear deformation is ignored in the analysis (AY).
- **shear_area_z** (*float*) – Shear area in local z-axis. If zero, shear deformation is ignored in the analysis (AZ).

Returns:

Returns 0 if OK.
Returns -6032 if the section with section_name cannot be added to the UPT table using table_reference_id.
Returns -6045 if a section with the same section_name already exists in the UPT table for the given table_reference_id.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_num_id = staad_obj.Property.CreateUPTTable(4)
>>> result = staad_obj.Property.AddUPTPropertyDOUBLEANGLE(upt_num_id, "VJG20-2", 1.2,
```

`AddUPTPropertyGENERAL(table_reference_id: float, section_name: float, cross_section_area: float, section_depth: float, thickness_parallel_depth: float, width_of_section: float, thickness_parallel_flange: float, torsional_constant: float, moi_y: float, moi_z: float, section_modulus_z: float, section_modulus_y: float, shear_area_y: float, shear_area_z: float, plastic_modulus_z: float, plastic_modulus_y: float, warping_constant: float, depth_of_web: float)` [source]

Add general type to an defined UPT section.

Parameters:

- **table_reference_id** (*int*) – The existing table number ID.
- **section_name** (*str*) – UPT section string name.
- **cross_section_area** (*float*) – Cross section area (AX).
- **section_depth** (*float*) – Depth of the section (D).
- **thickness_parallel_depth** (*float*) – Thickness associated with section element parallel to depth (TD).
- **width_of_section** (*float*) – Width of section (B).
- **thickness_parallel_flange** (*float*) – Thickness associated with section element parallel to flange(TB).
- **torsional_constant** (*float*) – Torsional constant (IZ).
- **moi_y** (*float*) – Moment of inertia about local y-axis (IY).
- **moi_z** (*float*) – Moment of inertia about local z-axis (IZ).
- **section_modulus_z** (*float*) – Section modulus about local Z-axis (SZ).

- **section_modulus_y** (*float*) – Section modulus about local Y-axis (SY).
- **shear_area_y** (*float*) – Shear area for shear parallel to local Y-axis (AY).
- **shear_area_z** (*float*) – Shear area for shear parallel to local Z-axis (AZ).
- **plastic_modulus_z** (*float*) – Plastic modulus about local Z-axis (PZ).
- **plastic_modulus_y** (*float*) – Plastic modulus about local Y-axis (PY).
- **warping_constant** (*float*) – Warping constant for lateral torsional buckling calculations (HSS).
- **depth_of_web** (*float*) – Depth of web. For rolled sections, distance between fillets should be provided (DEE).

Returns:

Returns 0 OK.
Returns -6032 if the section with section_name cannot be added to the UPT table using table_reference_id.
Returns -6045 if a section with the same section_name already exists in the UPT table for the given table_reference_id.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_num_id = staad_obj.Property.CreateUPTTable(8)
>>> result = staad_obj.Property.AddUPTPropertyGENERAL(upt_num_id, "VJG20", 4.6, 12.1,
```

`AddUPTPropertyISECTION(table_reference_id: int, section_name: str, depth_of_web: float, thickness_of_web: float, depth_of_web1: float, width_of_top_flange: float, thickness_of_top_flange: float, width_of_bottom_flange: float, thickness_of_bottom_flange: float, shear_area_y: float, shear_area_z: float, torsional_constant: float)` [source]

Add I type to an defined UPT section.

Parameters:

- **table_reference_id** (*int*) – The existing table number ID.
- **section_name** (*str*) – UPT section string name.
- **depth_of_web** (*float*) – Depth of section at start node(DWW).
- **thickness_of_web** (*float*) – Thickness of web(TWW).
- **depth_of_web1** (*float*) – Depth of section at end node(DWW1).
- **width_of_top_flange** (*float*) – Width of top flange(BFF).
- **thickness_of_top_flange** (*float*) – Thickness of top flange(TFF).
- **width_of_bottom_flange** (*float*) – Width of bottom flange(BFF1).
- **thickness_of_bottom_flange** (*float*) – Thickness of bottom flange(TFF1).

- **shear_area_y** (*float*) – Shear area for shear parallel to Y-axis(AYF).
- **shear_area_z** (*float*) – Shear area for shear parallel to Z-axis(AZF).
- **torsional_constant** (*float*) – Torsional constant (XIF).

Returns:

Returns 0 OK.
Returns -6032 if the section with section_name cannot be added to the UPT table using table_reference_id.
Returns -6045 if a section with the same section_name already exists in the UPT table for the given table_reference_id.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_num_id = staad_obj.Property.CreateUPTTable(9)
>>> result = staad_obj.Property.AddUPTPropertyISECTION(upt_num_id, "VJG20", 9.2, 15.4,
```

`AddUPTPropertyPIPE(table_reference_id: int, section_name: str, out_diameter: float, in_diameter: float, shear_area_y: float, shear_area_z: float)` [source]

Add pipe type to a defined UPT section.

Parameters:

- **table_reference_id** (*int*) – The existing table number ID.
- **section_name** (*str*) – UPT section string name.
- **out_diameter** (*float*) – Outer diameter (OD).
- **in_diameter** (*float*) – Inner diameter (ID).
- **shear_area_y** (*float*) – Shear area in local y-axis. If zero, shear deformation is ignored in the analysis (AY).
- **shear_area_z** (*float*) – Shear area in local z-axis. If zero, shear deformation is ignored in the analysis (AZ).

Returns:

Return 0 if ok.
Return -6032 if the section with section_name cannot be added to the UPT table using table_reference_id.
Return -6045 if a section with the same section_name already exists in the UPT table for the given table_reference_id.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_num_id = staad_obj.Property.CreateUPTTable(6)
>>> result = staad_obj.Property.AddUPTPropertyPIPE(upt_num_id, "VJG20-2", 1.2, 2.2, 4.
```

AddUPTPropertyPRISMATIC(*table_reference_id: int, section_name: str, cross_section_area: float, torsional_constant: float, moment_of_inertia_y: float, moment_of_inertia_z: float, shear_area_y: float, shear_area_z: float, depth_y: float, depth_z: float*) [source]

Add PRISMATIC type to an defined UPT section.

Parameters:

- **table_reference_id** (*int*) – The existing table number ID.
- **section_name** (*str*) – UPT section string name.
- **cross_section_area** (*float*) – Cross section area (AX).
- **torsional_constant** (*float*) – Torsional constant (IZ).
- **moment_of_inertia_y** (*float*) – Moment of inertia about local y-axis (IY).
- **moment_of_inertia_z** (*float*) – Moment of inertia about local z-axis (IZ).
- **shear_area_y** (*float*) – Shear area for shear parallel to local Y-axis (AY).
- **shear_area_z** (*float*) – Shear area for shear parallel to local Z-axis (AZ).
- **depth_y** (*float*) – Depth of the section in the direction of the local Y-axis (YD).
- **depth_z** (*float*) – Depth of the section in the direction of the local Z-axis (ZD).

Returns:

Returns 0 OK.

Returns -6032 if the section with section_name cannot be added to the UPT table using table_reference_id.

Returns -6045 if a section with the same section_name already exists in the UPT table for the given table_reference_id.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_num_id = staad_obj.Property.CreateUPTTable(10)
>>> result = staad_obj.Property.AddUPTPropertyPRISMATIC(upt_num_id, "VJG20", 7.2, 2.2,
```

AddUPTPropertyTEE(*table_reference_id: int, section_name: str, cross_section_area: float, section_depth: float, top_flange_width: float, top_flange_thickness: float, web_thickness: float, torsional_constant: float,*

moi_y: float, moi_z: float, dist_z_top_section: float, shear_area_y: float, shear_area_z: float) [source]

Add tee type to a defined UPT section.

Parameters:

- **table_reference_id** (*int*) – The existing table number ID.
- **section_name** (*str*) – UPT section string name.
- **cross_section_area** (*float*) – Cross section area (AX).
- **section_depth** (*float*) – Depth of the section (D).
- **top_flange_width** (*float*) – Width of the top flange (WF).
- **top_flange_thickness** (*float*) – Thickness of top flange (TF).
- **web_thickness** (*float*) – Thickness of web (TW).
- **torsional_constant** (*float*) – Torsional constant (IZ).
- **moi_y** (*float*) – Moment of inertia about local y-axis (IY).
- **moi_z** (*float*) – Moment of inertia about local z-axis (IZ).
- **dist_z_top_section** (*float*) – Distance from z axis to the top of section.
- **shear_area_y** (*float*) – Shear area in local Y-axis. If zero, shear deformation is ignored in the analysis (AY).
- **shear_area_z** (*float*) – Shear area in local Z-axis. If zero, shear deformation is ignored in the analysis (AZ).

Returns:

Returns 0 if OK.
Returns -6032 if the section with section_name cannot be added to the UPT table using table_reference_id.
Returns -6045 if a section with the same section_name already exists in the UPT table for the given table_reference_id.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_num_id = staad_obj.Property.CreateUPTTable(5)
>>> result = staad_obj.Property.AddUPTPropertyTEE(upt_num_id, "VJG20-2", 1.2, 2.2, 4.5
```

AddUPTPropertyTUBE(*table_reference_id: int, section_name: str, cross_section_area: float, section_depth: float, top_flange_width: float, top_flange_thickness: float, torsional_constant: float, moi_y: float, moi_z: float, shear_area_y: float, shear_area_z: float*) [source]

Add tube type to an defined UPT section.

Parameters:

- **table_reference_id** (*int*) – The existing table number ID.
- **section_name** (*str*) – UPT section string name.
- **cross_section_area** (*float*) – Cross section area (AX).
- **section_depth** (*float*) – Depth of the section (D).
- **top_flange_width** (*float*) – Width of the top flange (WF).
- **top_flange_thickness** (*float*) – Thickness of top flange (TF).
- **torsional_constant** (*float*) – Torsional constant (Iz).
- **moi_y** (*float*) – Moment of inertia about local y-axis (IY).
- **moi_z** (*float*) – Moment of inertia about local z-axis (IX).
- **shear_area_y** (*float*) – Shear area in local y-axis. If zero, shear deformation is ignored in the analysis (AY).
- **shear_area_z** (*float*) – Shear area in local z-axis. If zero, shear deformation is ignored in the analysis (AZ).

Returns:

Returns 0 OK.
Returns -6032 if unable to add section section_name in upt table_reference_id
Returns -6045 if a section with the same section_name already exists in the UPT table for the given table_reference_id.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_num_id = staad_obj.Property.CreateUPTTable(7)
>>> result = staad_obj.Property.AddUPTPropertyTUBE(upt_num_id, "VJG20-2", 1.2, 2.2, 4.
```

`AddUPTPropertyWIDEFLANGE(table_ref_id: int, stn_name: str, cro_sec_area: float, sectn_depth: float, web_Thickness: float, top_flange_width: float, top_flange_thickness: float, torsional_constant: float, moi_l_y: float, moi_l_z: float, shear_area_y: float, shear_area_z: float)` [source]

Add wide flange type to an defined UPT section.

Parameters:

- **table_ref_id** (*int*) – The existing table number ID.
- **stn_name** (*str*) – UPT section string name.
- **cro_sec_area** (*float*) – Cross section area.
- **sectn_depth** (*float*) – Depth of the section.
- **web_Thickness** (*float*) – Thickness of web.
- **top_flange_width** (*float*) – Width of the top flange.
- **top_flange_thickness** (*float*) – Thickness of top flange.

- **torsional_constant** (*float*) – Torsional constant.
- **moi_l_y** (*float*) – Moment of inertia about local y-axis.
- **moi_l_z** (*float*) – Moment of inertia about local z-axis.
- **shear_area_y** (*float*) – Shear area in local y-axis. If zero, shear deformation is ignored in the analysis.
- **shear_area_z** (*float*) – Shear area in local z-axis. If zero, shear deformation is ignored in the analysis.

Returns:

Returns 0 if OK.
Returns -6032 if unable to add section stn_name in UPT table_reference_id.
Returns -6045 if a section with the same section_name already exists in the UPT table for the given table_reference_id.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_num_id = staad_obj.Property.CreateUPTTable(1)
>>> result = staad_obj.Property.AddUPTPropertyWIDEFLANGE(upt_num_id, "VJG20-2",  1.1,
```

`AddUPTPropertyWIDEFLANGECOMPOSITE(table_reference_id: int, section_name: str, profile_spec_list: list)` [source]

Add wide flange type with additional composite and bottom steel plate to a defined UPT section.

Parameters:

- **table_reference_id** (*int*) – The existing table number ID.
- **section_name** (*str*) – UPT section string name.
- **profile_spec_list** (*List*) –

  Float list consisting of Profile Specifications data of size 12 (without additional composite flange & bottom plate):

Index	Data
0	Cross section area (AX).
1	Depth of the section (D).
2	Thickness of web (TW).
3	Width of the top flange (WF).
4	Thickness of top flange (TF).
5	Torsional constant (IZ).
6	Moment of inertia about local y-axis (IY).
7	Moment of inertia about local z-axis (IX).
8	Shear area in local y-axis. If zero, shear deformation is ignored in the analysis (AY).
9	Shear area in local z-axis. If zero, shear deformation is ignored in the analysis (AZ).
10	Width of the bottom flange (WF1).
11	Thickness of bottom flange (TF1).
12	(for additional composite flange) Width of the composite slab to the right of the web center line (CFR).
13	(for additional composite flange) Width of the composite slab to the left of the web center line (CFL).
14	(for additional composite flange) Thickness of the composite slab (CFT).
15	(for additional composite flange) Modular ratio of the concrete in the composite slab (MR).
16	(for additional bottom plate) Width of the additional bottom flange plate to the right of the web center line (BPR).
17	(for additional bottom plate) Width of the additional bottom flange plate to the right of the web center line (BPL).
18	(for additional bottom plate) Thickness of the additional bottom flange plate (BPT).

Returns:

Returns ‘True’ if OK, else ‘False’ if Error.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_num_id = staad_obj.Property.CreateUPTTable(1)
>>> result = staad_obj.Property.AddUPTPropertyWIDEFLANGECOMPOSITE(upt_num_id, "UPT_Wid
```

AddUPTPropertyWIDEFLANGEUNEQUAL(*table_reference_id: int, section_name: str, profile_spec_list: list*) [source]

Add unequal wide flange to a defined UPT section.

Parameters:

- **table_reference_id** (*int*) – The existing table number ID.
- **section_name** (*str*) – UPT section string name.
- **profile_spec_list** (*List*) –

  Profile specification list which consists of the following corresponding 12 value:

Index	Prop Spec Value
0	Ax
1	D
2	TW
3	WF
4	TF
5	IZ
6	IY
7	IX
8	AY
9	AZ
10	WF1
11	TF1

Returns:

Returns ‘True’ if add unequal wide flange successful.

Returns ‘False’ if it encounters generate error.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_num_id = staad_obj.Property.CreateUPTTable(1)
>>> result = staad_obj.Property.AddUPTPropertyWIDEFLANGEUNEQUAL(upt_num_id, "VJG20-2",
```

AssignBeamProperty(*beam_ids: list | int, property_id: int*) [source]

Assign beam property to a single or multiple beams.

Parameters:

- **beam_ids** (*list of int or int*) – List of beam ids or a single beam id to which the property will be assigned.
- **property_id** (*int*) – ID of the property to assign.

Returns:

Status code indicating the result of the operation:

- 0 : Success
- -106 : BeamNo array dimension error.
- -3006 : Invalid member number ID(s).
- -6001 : Invalid section The assigned section property ID.
- -6002 : Library Error: Property Assign.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> country_code = 6
>>> section_name = "HE100A"
>>> type_spec = 0
>>> add_spec_1 = 0.0
>>> add_spec_2 = 0.0
>>> property_id = staad_obj.Property.CreateBeamPropertyFromTable(country_code, sectior
>>> beam_ids = staad_obj.Geometry.GetBeamList()    # Getting all beam Ids
>>> output = staad_obj.Property.AssignBeamProperty(beam_ids[0:3], property_id)    # As
>>> output = staad_obj.Property.AssignBeamProperty(4, property_id)    # Assigning beam
```

AssignBetaAngle(*beam_ids: list, beta_angle: float*) [source]

Assign beta angle to beam(s).

Parameters:

- **beam_ids** (*list*) – List of beam ids.
- **beta_angle** (*float*) – The beta angle in degrees.

Returns:

Status code indicating the result of the operation:

- 1 : OK
- 0 : General error
- 0 : List of long expected.
- 0 : Library Error of being unable to assign BETA angle.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> output = staad_obj.Property.AssignBetaAngle([beam_ids[0], beam_ids[1]], 90.0) # As
>>> output = staad_obj.Property.AssignBetaAngle(beam_ids[2], 90.0) # Assign beta angle
```

AssignElementAttribute(*attribute_name: str, str_Value: str, element_list: list | int*) [source]

Assign element(s) to an attribute.

Parameters:

- **attribute_name** (*str*) – Name of the attribute.
- **str_Value** (*str*) – A string value
- **element_list** (*list of int or int*) – Element(s) number ID list.

Returns:

Returns 0 if successful else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.AssignElementAttribute("MEMBTYPE", "BRACE", [5, 7, 3])
>>> result = staad_obj.Property.AssignElementAttribute("MEMBTYPE", "BRACE", 6) # Assig
```

AssignElementSpecToPlate(*plate_ids: list, spec_no: int*) [source]

Assign specifications to plate(s).

Parameters:

- **plate_ids** (*list*) – The plate number ID(s) list

- **spec_no** (*int*) – The specification number ID.

Returns:

Status code indicating the result of the operation:

- 0 : OK
- -106 : List of long expected.
- -6017 : Library Error: Unable to assign specification.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plate_ids = staad_obj.Geometry.GetPlateList()
>>> node_ids = staad_obj.Geometry.GetNodeList()
>>> specification_number = staad_obj.Property.CreateElementNodeReleaseSpec(node_ids[0]
>>> output = staad_obj.Property.AssignElementSpecToPlate(plate_ids[0:2], specificatior
>>> output = staad_obj.Property.AssignElementSpecToPlate(plate_ids[3], specification_r
```

AssignMaterialToMember(*material_name: str, member_ids: list*) [source]

Assign a material property to specified members.

Parameters:

- **material_name** (*str*) – The ID of the material property.
- **member_ids** (*list of int*) – List of member numbers to assign the material to.

Returns:

- 'True' if it succeeds
- 'False' if it fails

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.AssignMaterialToMember("CONCRETE1", [5, 6, 7])
```

AssignMaterialToPlate(*material_name: str, plate_ids: list | int*) [source]

Assign a material property to specified plates.

Parameters:

- **material_name** (*str*) – The ID of the material property.
- **plate_ids** (*list of int*) – List of plate numbers to assign the material to.

Returns:

Status code indicating the result of the operation:

- 0 : Success
- -113 : Invalid data type(Long or Int Expected)
- -4009 : All the plate numbers are invalid.
- -4008 : Some of the plate numbers are invalid.
- -6023 : Material not found.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plate_list = staad_obj.Geometry.GetPlateList()
>>> status = staad_obj.Property.AssignMaterialToPlate("CONCRETE1", plate_list)
```

`AssignMaterialToSolid(material_name: str, solid_ids: list)` [source]

Assign material to solid.

Parameters:

- **material_name** (*str*) – Identification title of material.
- **solid_ids** (*list*) – List of integers containing solid numbers.

Returns:

Returns '1' if True else '0' if False.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> solid_ids = staad_obj.Geometry.GetSolidList()
>>> result = staad_obj.Property.AssignMaterialToSolid("CONCRETE1", solid_ids)
```

`AssignMemberAttribute(attribute_name: str, str_Value: str, member_list: list | int)` [source]

Assign member(s) to an attribute.

Parameters:

- **attribute_name** (*str*) – Name of the attribute.
- **str_Value** (*str*) – A string value

- **member_list** (*list of int or int*) – Member number ID or ID List.

Returns:

Returns 0 if successful else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIdList = staad_obj.Geometry.GetBeamList()
>>> result = staad_obj.Property.AssignMemberAttribute("MEMBTYPE", "BRACE", beamIdList[
>>> result = staad_obj.Property.AssignMemberAttribute("MEMBTYPE", "BRACE", beamIdList[
```

AssignMemberSpecToBeam(*beam_ids: list | int, spec_id: int*) [source]

Assign a member specification to specified beams.

Parameters:

- **beam_ids** (*list of int or int*) – List of member numbers to assign the specification to.
- **spec_id** (*int*) – The ID of the member specification.

Returns:

Status code indicating the result of the operation:

- 0 : Success
- -106 : List of long expected.
- -6017 : Library Error: Unable to assign specification.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> status = staad_obj.Property.AssignMemberSpecToBeam(beam_ids, 2)
```

AssignPlateThickness(*plate_ids: list, thickness_property_id: int*) [source]

Assigns a plate thickness property to the specified plates.

Parameters:

- **plate_ids** (*list of int*) – List of plate numbers to which the thickness property will be assigned.
- **thickness_property_id** (*int*) – ID of the plate thickness property to assign.

Returns:

Status code indicating the result of the operation:

- 0 : Success
- -1 : General error.
- -106 : Plate number array dimension error.
- -113 : Plate number type error (expected int or long).
- -4009 : All provided plate numbers are invalid.
- -4008 : Some of the plate numbers provided are invalid.
- -6001 : The specified thickness property ID is invalid.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> property_id = staad_obj.Property.CreatePlateThicknessProperty([2, 2, 1.5, 1.5])
>>> plate_list = staad_obj.Geometry.GetPlateList()
>>> status = staad_obj.Property.AssignPlateThickness(plate_list, property_id)
```

`CreateAnglePropertyFromTable(country_code: int, section_name: str, specification_type_no: int, add_spec: float)` [source]

Create a new angle property from a table.

Parameters:

- **country_code** (*int*) – Code resembling specific country.
- **section_name** (*str*) – The section name in the table.
- **specification_type_no** (*int*) –

  Specification type to use while creating angle property. [Please refer to enum 'ProfileType']:

Value	Spec Type	Description
0	ST	Single section from the standard built-in tables.
1	RA	Single angle with reverse Y-Z axes (refer to G.4.2 Local Coordinate System).
3	LD	Double angle with long legs back-to-back.
4	SD	Double angle with short legs back-to-back
12	SA	Double angle in a star arrangement (heel to heel)[for Aluminum]

- **add_spec** (*float*) –

  Additional Specification Value :

Spec Value	Specification Description
WP TH	for TC and BC
WP TH BW BT	for TB / WP TH for TB
CT FC	for CM
SP	for D, BA and FR
SP	for LD and SD
TH WT DT	for Tube define
OD ID	for Pipe define

Returns:

The ID of the created beam property if successful else returns 0 if it is unable to create property.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> openstaad = os_analytical.connect()
>>> country_code = 6
>>> section_name = "HE100A"
>>> specification_type_no = 0
>>> add_spec = 0.0
>>> property_id = openstaad.Property.CreateAnglePropertyFromTable(country_code, sectio
```

`CreateAssignProfileProperty(profile_type: int)` [source]

Create “Assign Profile” property.

Parameters:

profile_type (*int*) –

Profile type number ID:

Type of Profile	Value
Angle	0
Double Angle	1
Beam	2
Column	3
Channel	4

Returns:

Returns the assigned section property ID if successful.

Else returns status code indicating the result of the operation:

- 0 : Library Error of being unable to create property.
- -6008 : Invalid assign profile type.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateAssignProfileProperty(2)
```

`CreateBeamPropertyFromTable(country_code: int, section_name: str, type_spec: int, add_spec_1: float, add_spec_2: float)` [source]

Create a new beam property from a table.

Parameters:

- **country_code** (*int*) –

  Code resembling specific country:

Country Code	Country
1	American
2	Australian
3	British
4	Canadian
5	Chinese
6	Dutch
7	European
8	French
9	German
10	Indian
11	Japanese
12	Russian
13	SouthAfrican
14	Spanish
15	Venezuelan
16	Korean

- **section_name** (*str*) – Name of the section
- **type_spec** (*int*) –

  Specification Type Number:

Value	Type Spec.	Description
0	ST	
2	D	Double profile.
5	T	Tee section cut from I shaped section (for aluminium)

- **add_spec_1** (*float*) – Clear Spacing for Double profile.
- **add_spec_2** (*float*) – Please set it with 0.0.

Returns:

The ID of the created beam property if successful else returns 0 if it is unable to create property.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> openstaad = os_analytical.connect()
>>> country_code = 6
>>> section_name = "HE100A"
>>> type_spec = 0
>>> add_spec_1 = 0.0
>>> add_spec_2 = 0.0
>>> property_id = openstaad.Property.CreateBeamPropertyFromTable(country_code, sectior
```

`CreateBeamPropertyFromTableComposite`(*country_code: int, section_name: str, spec_type: int, additional_spec_list: list*) [source]

Creates beam property from table composite.

Parameters:

- **country_code** (*int*) – The value for the specified country
- **section_name** (*str*) – Name of the section.
- **spec_type** (*int*) –

  The specification type number:

Index	Spec Type
-1	Define
0	ST
1	RA
2	D
3	LD
4	SD
5	T (for aluminum)
6	CM
7	TC
8	BC
9	TB
10	BA (for aluminum)
11	FR
12	SA (for aluminum)

- **additional_spec_list** (*List*) –

  List of additional specification values:

Spec Value | Specification Description	
WP TH | for TC and BC	
WP TH BW BT	for TB / WP TH for TB
CT FC	for CM
SP	for D, BA and FR
SP	for LD and SD
TH WT DT	for Tube define
OD ID	for Pipe define

Returns:

Returns the assigned section property ID if successful else returns a status from below:

- 0 : Library Error: Unable to create property.

- -6004 : Section is not found in profile database.
- -6005 : Section data for a section is not found.
- -6006 : Invalid section type.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.reateBeamPropertyFromTableComposite(10, strInput, 2, [
```

`CreateBeamPropertyFromTableEx(country_code: int, section_name: str, solid_shape_type: int)` [source]

Creates beam property from table.

Parameters:

- **country_code** (*int*) –

  The value for the specified country::

Country Code	Country
1	American
2	Australian
3	British
4	Canadian
5	Chinese
6	Dutch
7	European
8	French
9	German
10	Indian
11	Japanese
12	Russian
13	SouthAfrican
14	Spanish
15	Venezuelan
16	Korean

- **section_name** (*str*) – Name of the section.
- **solid_shape_type** (*int*) –

  The specification type number:

Solid Shape ID	The shape of section
1	Plate Strip
2	Solid Rect
3	Solid Round
4	Round
5	Cable

Returns:

Returns the assigned section property ID else returns 0 if library error: Unable to create property.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> nCountryCode = 6
>>> strSectionName = "HE100A"
>>> typeSolidShape = 1
>>> result = staad_obj.Property.CreateBeamPropertyFromTableEx(nCountryCode, strSectior
```

`CreateBeamPropertyFromTableWithCoverPlates(country_code: int, section_name: str, spec_type: int, additional_spec_list: list)` [source]

Creates beam property from table with cover plates.

Parameters:

- **country_code** (*int*) – The value for the specified country
- **section_name** (*str*) – Name of the section.
- **spec_type** (*int*) –

  The specification type number:

Index	Spec Type
-1	Define
0	ST
1	RA
2	D
3	LD
4	SD
5	T (for aluminum)
6	CM
7	TC
8	BC
9	TB
10	BA (for aluminum)
11	FR
12	SA (for aluminum)

- **additional_spec_list** (*List*) –

  list of additional specification values:

Spec Value | Specification Description	
WP TH | for TC and BC	
WP TH BW BT	for TB / WP TH for TB
CT FC	for CM
SP	for D, BA and FR
SP	for LD and SD
TH WT DT	for Tube define
OD ID	for Pipe define

Returns:

Returns the assigned section property ID if successful else returns status code from below:

- 0 : Library Error: Unable to create property.

- -6004 : Section is not found in profile database.
- -6005 : Section data for a section is not found.
- -6006 : Invalid section type.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.CreateBeamPropertyFromTableWithCoverPlates(4, "ISMB600
```

`CreateChannelPropertyFromTable`(*country_code: int, section_name: str, spec_type: int, additional_spec_1: float*) [source]

Creates channel property from database.

Parameters:

- **country_code** (*int*) – The value for the specified country
- **section_name** (*str*) – Name of the section
- **spec_type** (*int*) –

  The specification type number:

Index	Spec Type
-1	Define
0	ST
1	RA
2	D
3	LD
4	SD
5	T (for aluminum)
6	CM
7	TC
8	BC
9	TB
10	BA (for aluminum)
11	FR
12	SA (for aluminum)

- **additional_spec_1** (*float*) –

  Additional specification value:

Spec Value	Specification Description
WP TH	for TC and BC
WP TH BW BT	for TB / WP TH for TB
CT FC	for CM
SP	for D, BA and FR
SP	for LD and SD
TH WT DT	for Tube define
OD ID	for Pipe define

Returns:

Returns assigned section property ID if successful.

Else returns a status code indicating the result of the operation:

- 0 : Library Error: Unable to create property.

- -6004 : Section is not found in profile database.
- -6005 : Section data for a section is not found.
- -6006 : Invalid section type.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateChannelPropertyFromTable(10, "ISMC200", 0, 0.0)
```

CreateElementAttribute(*attribute_name: str, str_value: str*) [source]

Create element attribute by name.

Parameters:

- **attribute_name** (*str*) – Name of the attribute.
- **str_value** (*str*) – A string value

Returns:

Returns 0 if successful else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.CreateElementAttribute("MEMBTYPE", "BRACE")
```

CreateElementIgnoreInplaneRotnSpec() [source]

Create MEMBER INPLANE ROTATION specification.

Returns:

Returns the assigned specification number ID if successful.

Else returns status code '-6019' if library error (Unable to create ELEMENT INPLANE ROTATION specification).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateElementIgnoreInplaneRotnSpec()
```

CreateElementLocalZOffsetSpec(*node1_localz_offset: float, node2_localz_offset: float, node3_localz_offset: float, node4_localz_offset: float*) [source]

Create ELEMENT OFFSET specification (Z-Offset).

Parameters:

- **node1_localz_offset** (*float*) – The offset at Node 1 for local-Z offset.
- **node2_localz_offset** (*float*) – The offset at Node 2 for local-Z offset.
- **node3_localz_offset** (*float*) – The offset at Node 3 for local-Z offset.
- **node4_localz_offset** (*float*) – The offset at Node 4 for local-Z offset.

Returns:

Returns the assigned specification number ID if successful else -6016 if Library Error(Unable to create OFFSET specification).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateElementLocalZOffsetSpec(3.6, 17.0, 8.6, 10.0)
```

CreateElementNodeReleaseSpec(*node_id: int, dof_release: list*) [source]

Creates ELEMENT NODE RELEASE specification.

Parameters:

- **node_id** (*int*) – The node number ID to be released.
- **dof_release** (*list*) – Degrees of freedom: No Release (=0) or Release (=1) for FX, FY, FZ, MX, MY and MZ.

Returns:

Gives specification id if successful, else gives status code indicating the result of the operation:

- -106 : List of long type elements for dof_release parameter expected.
- -108 : Array size is smaller than expected (size should be 6).
- -6020 : Library Error of being unable to create ELEMENT NODE RELEASE specification.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateElementNodeReleaseSpec(5, [0, 0, 1, 0, 1, 0])
```

CreateElementOffsetSpec(*offset_direction: int, plate_node_index: int, x_offset: float, y_offset: float, z_offset: float*) [source]

Create ELEMENT OFFSET specification.

Parameters:

- **offset_direction** (*int*) – The offset direction at Local (= 0) or Global (= 1) of the element.
- **plate_node_index** (*int*) – The Node index at which the offset is to be applied for local and global directions (1/2/3/4).
- **x_offset** (*float*) – The offset x coordinate.
- **y_offset** (*float*) – The offset y coordinate.
- **z_offset** (*float*) – The offset z coordinate.

Returns:

Returns the assigned specification number ID if successful else status code -6016 if library error (Unable to create OFFSET specification).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateElementOffsetSpec(7, 3, 9.9, 4.6, 14.7)
```

CreateElementPlaneStressSpec() [source]

Create MEMBER PLANE STRESS specification.

Returns:

Returns the assigned specification number ID if successful.
Else returns status code '-6018' if library error (Unable to create ELEMENT PLANE STRESS specification).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateElementPlaneStressSpec()
```

CreateIsotropicMaterialAluminum(*material_name: str, elasticity_mod: float, poisson: float, shear_mod: float, density: float, thermal_exp: float, damping_ratio: float, physical_flag: int*) [source]

Creates isotropic aluminum material.

Parameters:

- **material_name** (*str*) – Material name.
- **elasticity_mod** (*float*) – Modulus of elasticity (E).
- **poisson** (*float*) – Poisson's ratio.
- **shear_mod** (*float*) – Shear modulus (G).
- **density** (*float*) – Weight density.
- **thermal_exp** (*float*) – Coefficient of thermal expansion.
- **damping_ratio** (*float*) – Damping ratio.
- **physical_flag** (*int*) – Flag indicating physical-member material (nonzero = physical).

Returns:

Status code:

- 1 : Material updated (name existed).
- 0 : Material created.
- -1 : General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.CreateIsotropicMaterialAluminum("AluminumM124", 6.8, 0
```

CreateIsotropicMaterialConcrete(*name: str, elasticity: float, poisson: float, shear_modulus: float, density: float, alpha: float, damping_ratio: float, compressive_strength: float, physical: int*) [source]

Create isotropic concrete material.

Parameters:

- **name** (*str*) – Material name.
- **elasticity** (*float*) – Modulus of elasticity (E).
- **poisson** (*float*) – Poisson's ratio.

- **shear_modulus** (*float*) – Shear modulus (G).
- **density** (*float*) – Weight density.
- **alpha** (*float*) – Coefficient of thermal expansion.
- **damping_ratio** (*float*) – Damping ratio.
- **compressive_strength** (*float*) – Compressive strength (Fcu).
- **physical** (*int*) – Flag indicating physical-member material (nonzero = physical).

Returns:

Status code:

- 1 : Material updated as a material with that name was already present.
- 0 : Material created
- -1 : General error

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.CreateIsotropicMaterialConcrete("CONCRETE1", 453600.0,
```

`CreateIsotropicMaterialProperties(material_name: str, elasticity_mod: float, poisson: float, shear_mod: float, density: float, coef_thermal_exp: float, damp_ratio: float)` [source]

Creates isotropic material properties.

Parameters:

- **material_name** (*str*) – Material Name
- **elasticity_mod** (*float*) – Modulus of elasticity List (of size 3).
- **poisson** (*float*) – Poisson's ratio List (of size 3).
- **shear_mod** (*float*) – Shear modulus List (of size 3).
- **density** (*float*) – Weight density List (of size 3).
- **coef_thermal_exp** (*float*) – Coefficient of thermal expansion List (of size 3).
- **damp_ratio** (*float*) – Damping ratio List (of size 3).

Returns:

Returns 1 if Material is updated as a material with that name was already present. Returns 0 if Material is created. Returns -1 if General Error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.CreateIsotropicMaterialProperties("MATERIAL1", 262040,
```

CreateIsotropicMaterialPropertiesEx(*material_name: str, elasiticity: float, poisson: float, shear_modulus: float, density: float, alpha: float, damping_ratio: float, fy: float, fu: float, ry: float, rt: float, fcu: float*) [source]

Creates isotropic material property extended.

Parameters:

- **material_name** (*str*) – Identification title of material of material.
- **elasiticity** (*float*) – Modulus of elasticity (E).
- **poisson** (*float*) – Poisson's ratio (POI).
- **shear_modulus** (*float*) – Shear modulus (G).
- **density** (*float*) – Weight density (DEN).
- **alpha** (*float*) – Coefficient of thermal expansion (ALP).
- **damping_ratio** (*float*) – Damping ratio (DAMP).
- **fy** (*float*) – Yield stress (Fy)
- **fu** (*float*) – Tensile strength (Fu).
- **ry** (*float*) – Yield strength ratio (Ry).
- **rt** (*float*) – Tensile strength ratio (Rt).
- **fcu** (*float*) – Compressive strength (Fcu).

Returns:

Status code:

- 1 : Material is updated as a material with that name was already present.
- 0 : Material is created.
- -1 : General Error

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.CreateIsotropicMaterialPropertiesEx("STEEL1", 2.0, 7.6
```

CreateIsotropicMaterialSteel(*name: str, elasticity_mod: float, poisson_ratio: float, shear_modulus: float, density: float, thermal_expansion: float, damping_ratio: float, tensile_strength: float, yield_strength: float, tensile_ratio: float, yield_ratio: float, is_physical: int*) [source]

Creates isotropic material steel.

Parameters:

- **name** (*str*) – Identification title of material.
- **elasticity_mod** (*float*) – Modulus of elasticity (E).
- **poisson_ratio** (*float*) – Poisson's ratio (POI).
- **shear_modulus** (*float*) – Shear modulus (G).
- **density** (*float*) – Weight density (DEN).
- **thermal_expansion** (*float*) – Coefficient of thermal expansion (ALP).
- **damping_ratio** (*float*) – Damping ratio (DAMP).
- **tensile_strength** (*float*) – Tensile strength (Fu).
- **yield_strength** (*float*) – Yield stress (Fy).
- **tensile_ratio** (*float*) – Tensile strength ratio (Rt).
- **yield_ratio** (*float*) – Yield strength ratio (Ry).
- **is_physical** (*int*) – Identifies if the material is for physical member (flag/int).

Returns:

Status code from below:

- 1 : Material is updated as a material with that name was already present.
- 0 : Material is created.
- -1 : General Error

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.Property.CreateIsotropicMaterialSteel("STEEL1", 11.5, 3.5, 11.1
```

`CreateIsotropicMaterialTimber(material_name: str, elasticity: float, poisson: float, shear_modulus: float, density: float, thermal_expansion: float, damping_ratio: float, physical_flag: int)` [source]

Creates isotropic timber material.

Parameters:

- **material_name** (*str*) – Identification title of the material.
- **elasticity** (*float*) – Modulus of elasticity (E).
- **poisson** (*float*) – Poisson's ratio (POI).
- **shear_modulus** (*float*) – Shear modulus (G).
- **density** (*float*) – Weight density (DEN).
- **thermal_expansion** (*float*) – Coefficient of thermal expansion (ALP).

- **damping_ratio** (*float*) – Damping ratio (DAMP).
- **physical_flag** (*int*) – Flag indicating if the material is for physical members (nonzero = physical).

Returns:

Status code:

- 1 : Material updated (a material with that name already existed).
- 0 : Material created.
- -1 : General error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateIsotropicMaterialTimber("TIMBER1", 18.0, 6.0, 1.
```

`CreateMemberAttribute(attribute_name: str, str_Value: str)` [source]

Create member attribute by name.

Parameters:

- **attribute_name** (*str*) – Name of the attribute.
- **str_Value** (*str*) – A string value

Returns:

Returns 0 if successful else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.CreateMemberAttribute("MEMBTYPE", "BRACE")
```

`CreateMemberCableSpec(tension_or_unstressed_len: int, spec_value: float)`

Create MEMBER CABLE specification. [source]

Parameters:

- **tension_or_unstressed_len** (*int*) –

  Specify additional information about the cable:

  - 0 = Initial TENSION of Value in the cable to be considered.

- 1 = Unstressed LENGTH of Value to be considered.

- **spec_value** (*float*) – Value for TENSION or Unstressed LENGTH

Returns:

Returns the assigned specification number ID if successful.
Else returns status code -6015 if library error (Unable to create MEMBER CABLE specification).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateMemberCableSpec(0, 4.5)
```

`CreateMemberCableSpecEx(tension_or_unstressed_len: int, spec_value: float, tension_end_node_indicator: int, self_weight_factor_x: float, self_weight_factor_y: float, self_weight_factor_z: float)` [source]

Create MEMBER CABLE specification.

Parameters:

- **tension_or_unstressed_len** (*int*) –

  Specify additional information about the cable:

  - 0 = Initial TENSION of Value in the cable to be considered.
  - 1 = Unstressed LENGTH of Value to be considered.

- **spec_value** (*float*) – Value for TENSION or Unstressed LENGTH
- **tension_end_node_indicator** (*int*) –

  Initial tension end for TENSION. To be used for Advanced Cable Analysis:

  - 0 = cable start or end node will not be considered.
  - 1 = cable start node to be considered.
  - 2 = cable end node to be considered.

- **self_weight_factor_x** (*float*) – Multiplying factor on self weight component applied in the global X.
- **self_weight_factor_y** (*float*) – Multiplying factor on self weight component applied in the global Y.
- **self_weight_factor_z** (*float*) – Multiplying factor on self weight component applied in the global Z.

Returns:

Returns the assigned specification number id if successful, else returns status code '-6015' if Library Error (Unable to create MEMBER CABLE specification).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plate_ids = staad_obj.Geometry.GetPlateIDs()
>>> specification_id = staad_obj.Property.CreateMemberCableSpecEx(1, 16.6, 2, 2.1, 17.
```

CreateMemberCompressionSpec() [source]

Create MEMBER COMPRESSION specification.

Returns:

Returns the assigned specification number ID if successful.
Else returns status code '-6013' if library error (Unable to create MEMBER COMPRESSION specification).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateMemberCompressionSpec()
```

CreateMemberFireProofingSpec(*fire_proof_type: int, thickness_value: float, density: float*) [source]

Create MEMBER FIREPROOFING specification.

Parameters:

- **fire_proof_type** (*int*) –

  Specify type of fire proofing:

  - 1 = BFP Block Fireproofing.
  - 2 = CFP Contour Fireproofing.

- **thickness_value** (*float*) – Thickness of the Fireproofing
- **density** (*float*) – Density of the Fireproofing material

Returns:

Returns zero based index for the newly created specification if successful else -1 if unable to create MEMBER FIREPROOFING specification.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.CreateMemberFireProofingSpec(1, 3.5, 6.4)
```

CreateMemberIgnoreStiffSpec() [source]

Create MEMBER IGNORE STIFFNESS specification.

Returns:

Returns the assigned specification number ID if successful.
Else returns status code '-6014' if library error (Unable to create IGNORE STIFFNESS specification).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateMemberIgnoreStiffSpec()
```

CreateMemberInactiveSpec() [source]

Create MEMBER INACTIVE specification.

Returns:

Returns the assigned specification number ID if successful.
Else returns status code '-6011' if it encounters library error(Unable to create MEMBER INACTIVE specification).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateMemberInactiveSpec()
```

CreateMemberOffsetSpec(*offset_location: int, offset_with_respect_to: int, offset_x: float, offset_y: float, offset_z: float*) [source]

Create a member offset specification.

Parameters:

- **offset_location** (*int*) – Sets Offset Location at start if passed '0' else at the end if passed '1'
- **offset_with_respect_to** (*int*) – Sets Offset with respect to Global Axis if passed '0' else to Local Axis if passed '1'
- **offset_x** (*float*) – The offset x coordinate.
- **offset_y** (*float*) – The offset y coordinate.
- **offset_z** (*float*) – The offset z coordinate.

Returns:

The id of the created member offset specification if successful else returns 0 if it is unable to create property.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> spec_id = staad_obj.Property.CreateMemberOffsetSpec(0, 0, 0.5, 0.0, 0.0)
```

CreateMemberPartialReleaseSpec(*location: int, dof_release: list, factor: list*) [source]

Creates MEMBER RELEASE specification.

Parameters:

- **location** (*int*) – The offset location at START (= 0) or END (= 1) of the member.
- **dof_release** (*list*) – Degrees of freedom: No Release (= 0) or Release (= 1) for FX, FY, FZ.
- **factor** (*list*) – List of partial release factors arranged in respective DOFs.

Returns:

Status code indicating the result of the operation:

- 0 : OK
- -106 : List of long for dof_release and list of double for factor expected.
- -108 : Array size is smaller than expected (size should be 6).
- -6020 : Library Error: Unable to create MEMBER RELEASE specification.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateMemberPartialReleaseSpec(1, [0, 0, 1], [0.0, 0.5
```

CreateMemberReleaseSpec(*offset_location: int, dof_values: list, spring_constant_values: list*) [source]

Create a member release specification.

Parameters:

- **offset_location** (*int*) – The offset location at START (= 0) or END (= 1) of the member.
- **dof_values** (*list of int*) – Degrees of freedom: No Release (= 0) or Release (= 1) for FX, FY, FZ, MX, MY and MZ.
- **spring_constant_values** (*list of float*) – The variable spring constants KFX, KFY, KFZ, KMX, KMY and KMZ.

Returns:

Returns the ID of the created member release specification if successful.
Returns -106 if list of long for dof_values and list of double for spring_constant_values expected.
Returns -108 if array size is smaller than expected (size should be 6).
Returns -6020 if library error: unable to create member release specification.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> property_id = staad_obj.Property.CreateMemberReleaseSpec(0, [0, 1, 0, 0, 0, 1], [0
```

CreateMemberTensionSpec() [source]

Create MEMBER TENSION specification.

Returns:

Returns the assigned specification number ID if successful.
Else returns status code '-6012' if it encounters library error (Unable to create MEMBER TENSION specification).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateMemberTensionSpec()
```

CreateMemberTrussSpec() [source]

Create MEMBER TRUSS specification.

Returns:

Returns the assigned specification number ID. Else returns status code -6010 for unable to create MEMBER TRUSS specification (Library Error).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateMemberTrussSpec()
```

`CreateParametricSurfaceThicknessProperty(node_thickness_list: list)` [source]

Creates plate uniform or nonuniform thickness property.

Parameters:

node_thickness_list (*List*) – List consisting of thickness for all nodes.

Returns:

Returns the assigned section property ID if successful,

Else returns status code from below:

- -106 : node_thickness_list gives dimensional array error.
- -6003 : Library error being unable to create property.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.CreateParametricSurfaceThicknessProperty([4.3, 1.7, 10
```

`CreatePipePropertyFromTable(country_code: int, section_name: str, spec_type: int, additional_spec_1: float, additional_spec_2: float)` [source]

Creates pipe property from database.

Parameters:

- **country_code** (*int*) – The value for the specified country
- **section_name** (*str*) – Name of the section
- **spec_type** (*int*) –

  The specification type number:

Index	Spec Type
-1	Define
0	ST
1	RA
2	D
3	LD
4	SD
5	T (for aluminum)
6	CM
7	TC
8	BC
9	TB
10	BA (for aluminum)
11	FR
12	SA (for aluminum)

- **additional_spec_1** (*float*) –

  Additional specification value

Spec Value	Specification Description
WP TH	for TC and BC
WP TH BW BT	for TB / WP TH for TB
CT FC	for CM
SP	for D, BA and FR
SP	for LD and SD
TH WT DT	for Tube define
OD ID	for Pipe define

- **additional_spec_2** (*float*) –

  Additional specification value

Spec Value	Specification Description
WP TH	for TC and BC
WP TH BW BT	for TB / WP TH for TB
CT FC	for CM
SP	for D, BA and FR
SP	for LD and SD
TH WT DT	for Tube define
OD ID	for Pipe define

Returns:

Returns assigned section property ID if successful.

Else returns a status code indicating the result of the operation:

- 0 : Library Error: Unable to create property.
- -6004 : Section is not found in profile database.
- -6005 : Section data for a section is not found.
- -6006 : Invalid section type.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreatePipePropertyFromTable(17, "0.500PipeX5", 0, 0.0,
```

`CreatePlateThicknessProperty(thickness_list: list)` [source]

Create a new plate thickness property.

Parameters:

thickness_list (*list of float*) – The thickness value for the plate.

Returns:

Returns id of the created plate thickness property if successful. Returns -1 if it encounters an issue regarding the thickness list. Returns -6003 if it is unable to create property (library error).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> propertyId = staad_obj.Property.CreatePlateThicknessProperty([2, 2, 1.5, 1.5])
```

CreatePrismaticCircleProperty(*circle_diameter: float*) [source]

Creates prismatic circle property.

Parameters:

circle_diameter (*float*) – The circle diameter.

Returns:

Returns the assigned section property ID else '0' if it gets Library Error: Unable to create property.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreatePrismaticCircleProperty(0.25)
```

CreatePrismaticGeneralProperty(*property_value_list: list*) [source]

Creates prismatic general property.

Parameters:

property_value_list (*list*) –

The property values in double type list:

Index	Property Type	Description
0	AX	Cross section area
1	AY	Shear area in local Y-axis. If zero, shear deformation is ignored in the analysis.
2	AZ	Shear area in local Z-axis. If zero, shear deformation is ignored in the analysis.
3	IX	Torsional constant
4	IY	Moment of inertia about local Y-axis
5	IZ	Moment of inertia about local Z-axis
6	YD	Depth of the section in the direction of the local Y-axis.
7	ZD	Depth of the section in the direction of the local Z-axis.
8	YB	Depth of stem (T-beams); width of section at top fiber (trapezoidal beams)
9	ZB	Width of stem (T-beams); width of section at bottom fiber (trapezoidal beams)

Returns:

Returns the assigned section property ID if successful.

Else returns status code indicating the result of the operation:

- 0 : Library Error: Unable to create property.
- -106 : One dimensional array of double expected.
- -108 : Array size is smaller than expected.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreatePrismaticGeneralProperty([216, 216, 216, 6096, 2
```

`CreatePrismaticRectangleProperty(depth_along_y_axis: float, depth_along_z_axis: float)` [source]

Creates prismatic rectangle property.

Parameters:

- **depth_along_y_axis** (*float*) – The depth along the local Y-axis.
- **depth_along_z_axis** (*float*) – The width along the local Z-axis.

Returns:

Returns the assigned section property ID else '0' if it gets library Error (Unable to create property).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreatePrismaticRectangleProperty(0.5, 0.25)
```

CreatePrismaticTeeProperty(*total_section_depth: float, flange_width: float, stem_depth: float, stem_width: float*) [source]

Creates prismatic tee property.

Parameters:

- **total_section_depth** (*float*) – Total depth of section (top fiber of flange to bottom fiber of web).
- **flange_width** (*float*) – Width of flange.
- **stem_depth** (*float*) – Depth of stem.
- **stem_width** (*float*) – Width of stem.

Returns:

Returns the assigned section property ID else '0' if it gets library Error (Unable to create property).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreatePrismaticTeeProperty(0.5, 0.25, 0.4, 0.1)
```

CreatePrismaticTrapezoidalProperty(*section_depth: float, top_fiber_section_width: float, bottom_fiber_section_width: float*) [source]

Creates prismatic trapezoidal section property.

Parameters:

- **section_depth** (*float*) – Total depth of section.
- **top_fiber_section_width** (*float*) – Width of section at top fiber.
- **bottom_fiber_section_width** (*float*) – Width of section at bottom fiber.

Returns:

Returns the assigned section property ID else '0' if it gets library Error (Unable to create property).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreatePrismaticTrapezoidalProperty(0.5, 0.25, 0.2)
```

CreatePropertyFromUPTTable(*table_id: int, section_name: str*) [source]

Creates a section property from User Provided Table (UPT).

Parameters:

- **table_id** (*int*) – The existing table number ID.
- **section_name** (*str*) – UPT section string name.

Returns:

Returns section property number ID if successful else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.CreatePropertyFromUPTTable(2, "UPT VJG50-2")
```

CreatePropertyFromUserTable(*section_name: str, table_no: int*) [source]

Create a section Property from User Table.

Parameters:

- **section_name** (*str*) – Section name
- **table_no** (*int*) – Table Id

Returns:

Returns section property reference number if successful. Zero if not found.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.CreatePropertyFromUserTable("H600X300X12X20", 1)
```

CreateTaperedIProperty(*property_value_list: list*) [source]

Creates tapered I property.

Parameters:

property_value_list (*list*) –

Arrange the values with respect to following table in provided property_value_list:

Array Index	Property Type	Description
0	F1	Depth of section at start node.
1	F2	Thickness of web.
2	F3	Depth of section at end node.
3	F4	Width of top flange.
4	F5	Thickness of top flange.
5	F6	Width of bottom flange. Defaults to F4 if left out.
6	F7	Thickness of bottom flange. Defaults to F5 left out.

Returns:

Returns the assigned section property ID if successful.

Else returns status code indicating the result of the operation:

- 0 : Library Error: Unable to create property.
- -106 : List of double expected.
- -108 : Length of list is smaller than expected.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateTaperedIProperty([13.98, 0.285, 13.98, 13.98, 6.
```

CreateTaperedTubeProperty(*tube_type: int, start_member_section_depth: float, end_member_section_depth: float, section_thickness: float*) [source]

Creates tapered tube property.

Parameters:

- **tube_type** (*int*) –

  Type of the tube:

Type of Tube	Value
Round	0
HexDecagonal	1
Dodecagonal	2
Octagonal	3
Hexagonal	4
Square	5

- **start_member_section_depth** (*float*) – Depth of section at start of member.
- **end_member_section_depth** (*float*) – Depth of section at end of member.
- **section_thickness** (*float*) – Thickness of section (constant throughout the member length).

Returns:

Returns the assigned section property ID if successful.

Else returns status code indicating the result of the operation:

- 0 : Library Error of being unable to create property.
- -6008 : Invalid assign profile type.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateTaperedTubeProperty(0, 0.5, 0.4, 0.01)
```

CreateTeePropertyFromTable(*country_code: int, section_name: str, spec_type: int*) [source]

Creates Tee property from database.

Parameters:

- **country_code** (*int*) – The value for the specified country.
- **section_name** (*str*) – Name of the section.
- **spec_type** (*int*) –

  The specification type number:

Index	Spec Type
-1	Define
0	ST
5	T From Wide Flange

Returns:

Returns the assigned section property ID if successful else returns status code from below:

- 0 : Library error: unable to create property.
- -6004 : Section is not found in profile database.
- -6005 : Section data for a section is not found.
- -6006 : Invalid section type.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.CreateTeePropertyFromTable(7, "ISNT20", 9)
```

`CreateTubePropertyFromTable(country_code: int, section_name: str, spec_type: int, add_spec_1: float, add_spec_2: float, add_spec_3: float)` [source]

Creates tube property from database.

Parameters:

- **country_code** (*int*) – The value for the specified country
- **section_name** (*str*) – Name of the section
- **spec_type** (*int*) –

  The specification type number:

Index	Spec Type
-1	Define
0	ST
1	RA
2	D
3	LD
4	SD
5	T (for aluminum)
6	CM
7	TC
8	BC
9	TB
10	BA (for aluminum)
11	FR
12	SA (for aluminum)

- **add_spec_1** (*float*) –

  Additional specification value:

Spec Value	Specification Description
WP TH	for TC and BC
WP TH BW BT	for TB / WP TH for TB
CT FC	for CM
SP	for D, BA and FR
SP	for LD and SD
TH WT DT	for Tube define
OD ID	for Pipe define

- **add_spec_2** (*float*) –

  Additional specification value:

Spec Value	Specification Description
WP TH	for TC and BC
WP TH BW BT	for TB / WP TH for TB
CT FC	for CM
SP	for D, BA and FR
SP	for LD and SD
TH WT DT	for Tube define
OD ID	for Pipe define

- **add_spec_3** (*float*) –

  Additional specification value:

Spec Value	Specification Description
WP TH	for TC and BC
WP TH BW BT	for TB / WP TH for TB
CT FC	for CM
SP	for D, BA and FR
SP	for LD and SD
TH WT DT	for Tube define
OD ID	for Pipe define

Returns:

Status code indicating the result of the operation:

- 0 : Library Error: Unable to create property.
- -6004 : Section is not found in profile database.
- -6005 : Section data for a section is not found.
- -6006 : Invalid section type.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.CreateTubePropertyFromTable(10, "TUB30302.6", 0, 0.0,
```

`CreateUPTTable(table_type: int)` [source]

Creates user provided table (UPT).

Parameters:

table_type (*int*) –

Type of the table:

No.	Table Type
1	scUserTableWideFlangeTitle
2	scUserTableChannelTitle
3	scUserTableAngleTitle
4	scUserTableDoubleAngleTitle
5	scUserTableTeeTitle
6	scUserTablePipeTitle
7	scUserTableTubeTitle
8	scUserTableGeneralTitle
9	scUserTableIsectionTitle
10	scUserTablePrismaticTitle

Returns:

Returns User Provided Table (UPT) number id else -6031 if it is unable to create UPT table (unknown table type specified).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.CreateUPTTable(1)
```

`CreateUPTTableEx(table_ref_id: int, table_type: int)` [source]

Create User Provided Table (UPT) specified by table number ID and Table Type.

Parameters:

- **table_ref_id** (*int*) – A new table number ID.
- **table_type** (*int*) –

  Type of the table:

No.	Table Type
1	scUserTableWideFlangeTitle
2	scUserTableChannelTitle
3	scUserTableAngleTitle
4	scUserTableDoubleAngleTitle
5	scUserTableTeeTitle
6	scUserTablePipeTitle
7	scUserTableTubeTitle
8	scUserTableGeneralTitle
9	scUserTableIsectionTitle
10	scUserTablePrismaticTitle

Returns:

Returns table number ID if successful else '0' if create new User Provided Table encountered generate Error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.CreateUPTTableEx(6, 7) // Create User Provided Table w
```

`CreateWideFlangePropertyFromTable(country_code: int, section_name: str, spec_type: str, specs_list: list)` [source]

Creates wide flange member property from table with data for all specs.

Parameters:

- **country_code** (*int*) –

  The country CODE:

Country Code	Country
1	American
2	Australian
3	British
4	Canadian
5	Chinese
6	Dutch
7	European
8	French
9	German
10	Indian
11	Japanese
12	Russian
13	SouthAfrican
14	Spanish
15	Venezuelan
16	Korean

- **section_name** (*str*) – Name of the section.
- **spec_type** (*str*) –

  The specification type number:

Specification Type	Specification Number
ST	0
D	2
T	5
CM	6
TC	7
BC	8
TB	9

- **specs_list** (*list*) –

  The specification values corresponding to type shown in the table below:

Array Index	Spec Type	Description
0	SP/CT/WP	○ SP: Spacing for double-I, double-C, double-L ○ CT: Conc. thickness for composite-I ○ WP: Width of top cover plate for TC,TB and bottom cover plate for BC
1	FC/TH	○ FC: Concrete grade for composite-I ○ TH: Thickness of top cover plate for TC,TB and bottom cover plate for BC
2	CW/BW	○ CW: Concrete width for composite-I ○ BW: Width of bottom cover plate for TB
3	CD/BT	○ CD: Concrete density for composite-I ○ Thickness of bottom cover plate for TB

Returns:

Returns the assigned section property ID if successful else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.Property.CreateWideFlangePropertyFromTable(7, "HP10X42", "TB",
```

`DeleteAllControlDependentRelations()` [source]

Deletes all control/dependent joint specifications from model.

Returns:

Returns ‘0’ if OK successful deleted else ‘1’ if ERROR delete unsuccessful.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.Property.DeleteAllControlDependentRelations()
```

DeleteElementAttribute(*attribute_name: str, str_value: str*) [source]

Delete the element attribute by name.

Parameters:

- **attribute_name** (*str*) – Name of the attribute.
- **str_value** (*str*) – A string value

Returns:

Returns 0 if successful else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.DeleteElementAttribute("MEMBTYPE", "BRACE")
```

DeleteMaterial(*material_name: str*) [source]

Delete a material.

Parameters:

material_name (*str*) – Material Name

Returns:

'True' if succeeds 'else' False

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.DeleteMaterial("Q235")
```

DeleteMemberAttribute(*attribute_name: str, str_Value: str*) [source]

Delete the member attribute by name.

Parameters:

- **attribute_name** (*str*) – Name of the attribute.
- **str_Value** (*str*) – A string value

Returns:

Returns 0 if successful else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.DeleteMemberAttribute("MemberAttribute1", "A string va
```

DeleteMemberReleaseSpec(*beam_id: int*, *release_location: int*) [source]

Delete MEMBER RELEASE specification.

Parameters:

- **beam_id** (*int*) – The beam number ID.
- **release_location** (*int*) – The Release location at START (= 0) or END (= 1) of the member.

Returns:

Returns True if Delete Member Release Specification Successful. Returns False if Delete Member Release Specification failed

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> result = staad_obj.Property.DeleteMemberReleaseSpec(beamIds[0], 1)
```

DeleteMemberSpec(*spec_id: int*) [source]

Delete specification based on the specification number passed.

Parameters:

spec_id (*int*) – The specification number.

Returns:

Returns true delete specification successful, else false if delete specification failed.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> result = staad_obj.Property.DeleteMemberSpec(beamIds[0])
```

DeleteProperty(*property_id: int*) [source]

Delete property based on the property ID passed.

Parameters:

property_id (*int*) – Property ID.

Returns:

Returns true if successful else returns false if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.DeleteProperty(2)
```

GetAlphaAngleForSection(*spec_property_id: int*) [source]

Gets the angle between the principal axis and geometric axis of the section

Parameters:

spec_property_id (*int*) – The specified property ID.

Returns:

Returns a float for alpha angle (in Radian).

Return type:

float

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetAlphaAngleForSection(7)
```

GetAssignedAttributeByIndex(*attribute_index: int*) [source]

Gets assigned attribute at specified index

Parameters:

attribute_index (*int*) – The attribute index.

Returns:

tuple – Returns a tuple consisting of attribute name and a string value, respectively.

Return type:

tuple<string, string>L

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_count = staad_obj.Geometry.GetBeamList()
>>> if beam_count > 0:
>>>     beamIdList = staad_obj.Geometry.GetBeamList()
>>>     attributeCount = staad_obj.Property.GetAssignedAttributeCount(beamIdList[0])
>>>     attribute_name, str_value  = staad_obj.Property.GetAssignedAttributeByIndex(at
```

GetAssignedAttributeCount(*member_id: int*) [source]

Gets the number of attributes associated with beam or plate having the specified member ID (member_id).

Parameters:

member_id (*int*) – The number ID of member or plate.

Returns:

Returns the number of attributes associated with beam (if member with this ID exists) or plate (if member does not exist) having the specified member ID (member_id).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plateIds = staad_obj.Geometry.GetPlateList()
>>> result = staad_obj.Property.GetAssignedAttributeCount(plateIds[0])
```

GetBeamConstants(*beam_id: int*) [source]

Retrieve beta angle of the specified beam member.

Parameters:

beam_id (*int*) – The beam number ID

Returns:

Returns a tuple of Beam Constants found in following order Elasticity (Modulus of elasticity), Poisson (Poisson's ratio), Density (Weight density), Alpha (Coefficient of thermal expansion) and Damp (Damping ratio) respectively

Return type:

Tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> elasticity, poisson, density, alpha, damp = staad_obj.Property.GetBeamConstants(be
```

GetBeamMaterialName(*beam_id: int*) [source]

Get the material name assigned to a beam.

Parameters:

beam_id (*int*) – The beam number id.

Returns:

The material name.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> material_name = staad_obj.Property.GetBeamMaterialName(beam_ids[0])
```

GetBeamProperty(*beam_id: int*) [source]

Get a short member properties of the specified beam member.

Parameters:

beam_id (*int*) – The ID of the beam property.

Returns:

tuple – Tuple of short member properties consisting of width of the section, depth of the section, cross section area, shear area in local y-axis, shear area in local z-axis. moment of inertia about local z-axis, moment of inertia about local y-axis and torsional constant respectively.

If shear area in local y-axis & z-axis is zero, shear deformation is ignored in the analysis.

Return type:

tuple<float, float, float, float, float, float, float, float>

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> section_width, section_depth, cross_section_area, shear_area_y, shear_area_z, mome
```

GetBeamPropertyAll(*beam_id: int*) [source]

Gets long member properties of the specified beam member.

Parameters:

beam_id (*int*) – The ID of the beam property.

Returns:

tuple – Tuple of short member properties consisting of width of the section, depth of the section, cross section area, shear area in local y-axis, shear area in local z-axis. Moment of inertia about local z-axis, moment of inertia about local y-axis, torsional constant thickness of top flange and thickness of web respectively.
If shear area in local y-axis & z-axis is zero, shear deformation is ignored in the analysis.

Return type:

tuple<float, float, float, float, float, float, float, float, float, float>

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> section_width, section_depth, cross_section_area, shear_area_y, shear_area_z, mome
```

GetBeamSectionDisplayName(*beam_id: int*) [source]

This function returns the display section name of the specified beam.

Parameters:

beam_id (*int*) – The beam number ID

Returns:

Returns the section string name if successful else empty string if the specified beam is not found or property is not assigned to it. Refer to the table below for probable section names :

Sl No.	Section Type	In STD File	GetBeamSectionName
1	Standard Section from Steel Database	TABLE ST W36X925 TABLE D W36X925 SP 1 5 TABLE SD L20205 SP 1	W36X925 W36X925 L20205
2	Pipe and Tube definition	8 TABLE ST PIPE OD 2 ID 1 8 TABLE ST TUBE TH 1 WT 2 DT 3	PIPE TUBE
3	Prismatic	3 PRIS YD 1 ZD 2 YB 2 ZB 3 8 PRIS YD 3 ZD 1 ZB 2	Prismatic Tee Prismatic Trapezoid
4	Tapered	3 TAPERED 1 2 3 1 2 3 1	Taper
5	Assign Profile	3 ASSIGN ANGLE DOUBLE	Assign Double Angle
6	User Provided Table	14 TO 23 UPT 2 LANG40404	LANG40404

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> count = staad_obj.Property.GetBeamSectionDisplayName(beam_ids[0])
```

`GetBeamSectionName(beam_id: int)` [source]

Get beam section string name.

Parameters:

beam_id (*int*) – The beam number ID

Returns:

Returns the section string name. Refer to the table below for probable section names :

Sl No.	Section Type	In STD File	GetBeamSectionName
1	Standard Section from Steel Database	TABLE ST W36X925 TABLE D W36X925 SP 1 5 TABLE SD L20205 SP 1	W36X925 W36X925 L20205
2	Pipe and Tube definition	8 TABLE ST PIPE OD 2 ID 1 8 TABLE ST TUBE TH 1 WT 2 DT 3	PIPE TUBE
3	Prismatic	3 PRIS YD 1 ZD 2 YB 2 ZB 3 8 PRIS YD 3 ZD 1 ZB 2	Prismatic Tee Prismatic Trapezoid
4	Tapered	3 TAPERED 1 2 3 1 2 3 1	Taper
5	Assign Profile	3 ASSIGN ANGLE DOUBLE	Assign Double Angle
6	User Provided Table	14 TO 23 UPT 2 LANG40404	LANG40404

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> output = staad_obj.Property.GetBeamSectionName(beam_ids[0])
```

GetBeamSectionPropertyRefNo(*beam_id: int*) [source]

Returns the section property reference number of the specified beam.

Parameters:

beam_id (*int*) – The beam number ID

Returns:

Returns Section property ref number assigned to the specified beam. Zero if not found.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> result = staad_obj.Property.GetBeamSectionPropertyRefNo(beamIds[0])
```

GetBeamSectionPropertyTypeNo(*beam_id: int*) [source]

Get the section property type number of the specified beam.

Parameters:

beam_id (*int*) – The beam number ID

Returns:

Returns the section property type number if successful else returns '0' if error.

Section Type corresponding to property type number is shown in following table:

Section Type	Property Type Number
BEAM ST	610
BEAM D	616
BEAM TC	613
BEAM BC	614
BEAM TB	615
BEAM T	611
BEAM CM	612
CHANNEL ST	630
CHANNEL D	631
CHANNEL FR	633
ANGLE ST	640
ANGLE LD	642
ANGLE SD	643
ANGLE RA	641
ANGLE SA	646
PIPE ST	660
HSS RECTANGLE	654
HSS ROUND	655
CASTEL ST	656
TUBE ST	650
TEE ST	620
PLATE STRIP	666
ANGLE COLD ST	644
ANGLE COLD ST WITH LIPS	645
CHANNEL COLD ST	634
CHANNEL COLD ST WITH LIPS	635
ZEE COLD ST	662

Section Type	Property Type Number
ZEE COLD ST WITH LIPS	663
HAT COLD ST	664
TAPER	680
TAPERED TUBE	675
PRISMATIC CIRCLE	671
PRISMATIC RECT	672
PRISMATIC TRAP	674
PRISMATIC TEE	673
PRISMATIC GENERAL	676
SOLID ROUND	668
UPT PRISMATIC	699
UPT GENERAL	697
UPT WIDE FLANGE	690
UPT CHANNEL	691
UPT ANGLE	692
UPT DOUBLE ANGLE	693
UPT TEE	694
UPT PIPE	695
UPT TUBE	696
UPT ISECTION	698

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> output = staad_obj.Property.GetBeamSectionPropertyTypeNo(beam_ids[0])
```

`GetBeamSectionPropertyValuesEx(beam_id: int)` [source]

Returns the section property Values of the specified beam.

Parameters:

beam_id (*int*) – The beam number ID.

Returns:

tuple –

Returns a Tuple consisting of property type (proptype) number referring to below table, a float list for section property parameters respectively:

Section Type	propType	propValues
BEAM ST	610	Ax D Bf Tf Tw Iz Iy Ix
BEAM D	616	D Bf Tf Tw Iz Iy Ix SP
BEAM TC	613	Ax D Bf Tf Tw Iz Iy Ix WP TH
BEAM BC	614	Ax D Bf Tf Tw Iz Iy Ix WP TH
BEAM TB	615	Ax D Bf Tf Tw Iz Iy Ix WP TH BW BT
BEAM T	611	Ax D Bf Tf Tw Iz Iy Ix
BEAM CM	612	Ax D Bf Tf Tw Iz Iy Ix CT FC CW CD
CHANNEL ST	630	Ax D Bf Tf Tw Iz Iy Ix
CHANNEL D	631	Ax D Bf Tf Tw Iz Iy Ix SP
CHANNEL FR	633	Ax D Bf Tf Tw Iz Iy Ix FR
ANGLE ST	640	Ax D B T Iz Iy Ix
ANGLE LD	642	Ax D B T Iz Iy Ix LD
ANGLE SD	643	Ax D B T Iz Iy Ix SD
ANGLE RA	641	Ax D B T Iz Iy Ix
ANGLE SA	646	Ax D B T Iz Iy Ix
PIPE ST	660	Ax OD Tw Iz Iy Ix
HSS RECTANGLE	654	Ax D B T Iz Iy Ix
HSS ROUND	655	Ax OD Tw Iz Iy Ix
CASTEL ST	656	Ax D Bf Tf Tw Iz Iy Ix
TUBE ST	650	Ax D B T Iz Iy Ix
TEE ST	620	Ax D Bf Tf Tw Iz Iy Ix
PLATE STRIP	666	Ax D B Iz Iy Ix

Section Type	propType	propValues
ANGLE COLD ST	644	Ax D B T Iz Iy Ix R Ay Az
ANGLE COLD ST WITH LIPS	645	Ax D B T Iz Iy Ix R LIP Ay Az
CHANNEL COLD ST	634	Ax D Bf T R Iz Iy Ix Ay Az
CHANNEL COLD ST WITH LIPS	635	Ax D Bf T R Iz Iy Ix LIP Ay Az
ZEES COLD ST	662	Ax D B T R Iz Iy Ix Ay Az
ZEES COLD ST WITH LIPS	663	Ax D B T LIP LIP_Angle R Iz Iy Ix Ay Az
HAT COLD ST	664	Ax D B T BOT_F R Iz Iy Ix Ay Az
TAPER	680	F1 F2 F3 F4 F5 F6 F7
TAPERED TUBE	675	Ax Iz Iy Ix D1 D2 TH SECTION_TYPE SECTION_TYPE(1 TO 6 for Round,Hexdecagonal,Dodecagonal,Octagonal,Hexagonal,Square respectively)
PRISMATIC CIRCLE	671	Ax Iz Iy Ix YD
PRISMATIC RECT	672	Ax Iz Iy Ix YD ZD
PRISMATIC TRAP	674	Ax Iz Iy Ix YD ZD ZB
PRISMATIC TEE	673	Ax Iz Iy Ix YD ZD YB ZB
PRISMATIC GENERAL	676	Ax Ay Az Ix Iy Iz YD ZD YB ZB
SOLID ROUND	668	Ax OD Tw Iz Iy Ix Z

Section Type	propType	propValues
UPT PRISMATIC	699	Ax Iz Iy Ix Ay Az YD ZD
UPT GENERAL	697	Ax D Td B Tb Iz Iy Ix Sz Sy Ay Az Pz Py Hss Dee
UPT WIDE FLANGE	690	Ax D Tw Wf Tf Iz Iy Ix Ay Az Wf1 Tf1
UPT CHANNEL	691	Ax D Tw Wf Tf Iz Iy Ix Cz Ay Az
UPT ANGLE	692	Ax D Wf Tf R Ay Az Iz Iy Ix
UPT DOUBLE ANGLE	693	Ax D Wf Tf SP Iz Iy Ix Cy Ay Az
UPT TEE	694	Ax D Wf Tf Tw Iz Iy Ix Cy Ay Az
UPT PIPE	695	Ax OD ID Ay Az Iz Iy Ix
UPT TUBE	696	Ax D Wf Tf Iz Iy Ix Ay Az
UPT ISECTION	698	Dww Tww Dww1 Bff Tff Bff1 Tff1 Ayf Azf Xif

Return type:

Tuple(int, list)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> section_property_type, section_properties = staad_obj.Property.GetBeamSectionPrope
```

`GetBetaAngle(beam_id: int)` [source]

Retrieves beta angle of the specified beam member.

Parameters:

beam_id (*int*) – The beam number ID

Returns:

Returns Beta angle else returns status code '-3001' if it can't find member.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> output = staad_obj.Property.GetBetaAngle(beam_ids[0])
```

GetCentroidLocationForSection(*property_id: int*) [source]

Gets the location of the Centroid of the specified section.

Parameters:

property_id (*int*) – The specified property ID.

Returns:

Tuple – Returns a tuple consisting of offset value of centroid along Y axis and offset value of centroid along Z axis, respectively.

Return type:

tuple(int, int)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> section_list = staad_obj.Property.GetSectionPropertyList()
>>> for section_id in section_list:
>>>     y_axis_offset, z_axis_offset = staad_obj.Property.GetCentroidLocationForSectic
```

GetCountofSectionPropertyValuesEx() [source]

Returns the total count of Section Property values.

Returns:

Returns the total count of Section Property values.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetCountofSectionPropertyValuesEx()
```

GetCountryTableNo(*beam_id: int*) [source]

Get the country Code for the specified member.

Parameters:

beam_id (*int*) – The beam number ID

Returns:

Returns the country CODE if successful.

Else returns status code :

- -3001 : It is unable to find member.
- -6022 : No property is attached to the member/element.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> country_code = staad_obj.Property.GetCountryTableNo(beam_ids[0])
```

GetDefaultStandardProfileDBFolder() [source]

Gets standard profile default database folder path.

Returns:

Returns the standard profile default database folder path.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> folder_path = staad_obj.Property.GetDefaultStandardProfileDBFolder()
```

GetElementCountByAttribute(*attribute_name: str, str_value: str*) [source]

Returns the number of element(s) in specified attribute.

Parameters:

- **attribute_name** (*str*) – Name of the attribute.
- **str_value** (*str*) – A string value

Returns:

Returns number of elements in specified attribute if successful else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetElementCountByAttribute("MEMBTYPE", "BRACE")
```

GetElementGlobalOffSet(*plate_id: int*, *plate_node_index: int*) [source]

Get element offsets in all three local directions.

Parameters:

- **plate_id** (*int*) – The plate number ID.
- **plate_node_index** (*int*) – The Node Index at which the offset is to be applied (1/2/3/4).

Returns:

tuple – Returns a tuple consisting of the offset x coordinate (global), the offset y coordinate (global) and the offset z coordinate (global) respectively.

Return type:

Tuple(float, float, float)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plate_ids = staad_obj.Geometry.GetPlateIDs()
>>> offset_x, offset_y, offset_z = staad_obj.Property.GetElementGlobalOffSet(plate_ids
```

GetElementListByAttribute(*attribute_name: str*, *str_value: str*) [source]

Get element list by attribute.

Parameters:

- **attribute_name** (*str*) – Name of the attribute.
- **str_value** (*str*) – A string value

Returns:

Returns an elements number ID list.

Return type:

List of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetElementListByAttribute("MEMBTYPE", "BRACE")
```

GetElementLocalOffset(*plate_id: int, plate_node_index: int*) [source]

Get element offsets in all three local directions.

Parameters:

- **plate_id** (*int*) – The plate number ID.
- **plate_node_index** (*int*) – The Node Index at which the offset is to be applied (1/2/3/4).

Returns:

tuple – Returns a tuple consisting of the offset x coordinate, the offset y coordinate, the offset z coordinate, respectively.

Return type:

Tuple(float, float, float)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plate_ids = staad_obj.Geometry.GetPlateIDs()
>>> offset_x, offset_y, offset_z = staad_obj.Property.GetElementLocalOffset(plate_ids[
```

GetElementMaterialName(*element_id: int*) [source]

Get the material name assigned to an element.

Parameters:

element_id (*int*) – The element number Id.

Returns:

The material name.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> solid_ids = staad_obj.Geometry.GetSolidList()
>>> material_name = staad_obj.Property.GetSolidMaterialName(solid_ids[0])
```

GetElementOffSetSpec(*plate_id: int, plate_node_index: int*) [source]

Get Element offsets in all three global directions.

Parameters:

- **plate_id** (*int*) – The plate number ID.
- **plate_node_index** (*int*) – The Node Index at which the offset is to be applied (1/2/3/4).

Returns:

tuple – Returns a list consisting of the offset direction at Local (= 0) or Global (= 1) or Z-Offset (=2) of the member, the offset x coordinate (global), the offset y coordinate (global), the offset z coordinate and (global) respectively.

Return type:

Tuple(int, float, float, float)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plate_ids = staad_obj.Geometry.GetPlateIDs()
>>> direction, offset_x, offset_y, offset_z = staad_obj.Property.GetElementOffSetSpec(
```

GetElementOffsetSpecCount() [source]

Returns the total number of element offset specifications in the current model.

Returns:

Returns the total number of element offset specifications.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetElementOffsetSpecCount()
```

GetFireProofDataForBeam(*beam_id: int*) [source]

Get fire proofing data for beam.

Parameters:

beam_id (*int*) – The beam number.

Returns:

tuple – Returns a tuple consisting of type of fire proof [1 for BFP, 2 for CFP], thickness of fire proof and density of fire proof respectively.

Return type:

Tuple(int, float, float)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Property.GetBeamList()
>>> count = staad_obj.Property.GetFireProofDataForBeam(beam_ids[0])
```

GetFireProofedBeamCount() [source]

Returns count of beams which are fire proofed.

Returns:

Returns the total number of fire proofed beams in the current model.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetFireProofedBeamCount()
```

GetFireProofedBeamList() [source]

Returns a list of the member ids of all the fire proofed members in the current model.

Returns:

Returns for list of member number ids of all the members that are fire proofed.

Return type:

List of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> fire_proofed_beam_list = staad_obj.Property.GetFireProofedBeamList()
```

GetFireProofingSpecAssignedBeamCount(*index: int*) [source]

Get the count of beams assigned with a particular fire proofing specification.

Parameters:

index (*int*) – Non-zero based index of the fire proofing specification.

Returns:

Returns the number of beams assigned with a particular fire proofing specification.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetFireProofingSpecAssignedBeamCount(5)
```

GetFireProofingSpecAssignedBeamList(*index: int*) [source]

Populates a list of the member ID(s) of all the members assigned to a particular fire proofing specification.

Parameters:

index (*int*) – Non-zero based index of the fire proofing specification.

Returns:

Returns for list of member numbers IDs of all the members that are fire proofed with a particular fire proofing specification.

Return type:

List

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetFireProofingSpecAssignedBeamList(5)
```

GetFireProofingSpecCount() [source]

Returns the count of different fire proofing specifications in the model.

Returns:

Returns the total number of fire proofing specification.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetFireProofingSpecCount()
```

GetFireProofingSpecDetails(*index: int*) [source]

Get the details for the specified fire proofing specification number.

Parameters:

Index (*int*) – Non-zero based index of the fire proofing specification.

Returns:

tuple – Returns a tuple consisting of type of fire proof, thickness of fire proof, density of fire proof & number of beams assigned respectively.

Return type:

Tuple(int, float, float, int)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetFireProofingSpecDetails(1)
```

GetInactiveMemberCount()

[source]

Returns the total number of inactive members in the current model.

Returns:

Returns the total number of inactive members.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetInactiveMemberCount()
```

GetInactiveMemberList()

[source]

Populates a list of the member ids of all the inactive members in the current model.

Returns:

Returns a list for list of member number ids of inactive members.

Return type:

List of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> inactive_member_list = staad_obj.Property.GetInactiveMemberList()
```

GetIsotropicMaterialAssignedBeamCount(*material_name: int*)

[source]

Get isotropic material assigned beam count.

Parameters:

material_name (*int*) – Identification title of the material.

Returns:

Returns count of isotropic material assigned beams if successful.

Else returns -6023 if Material not found in material database.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> material_name = "Material123"
>>> count = staad_obj.Property.GetIsotropicMaterialAssignedBeamCount(material_name)
```

GetIsotropicMaterialAssignedBeamList(*material_name: str*) [source]

Get isotropic material assigned beam list.

Parameters:

material_name (*str*) – Identification title of the material.

Returns:

Returns a list of beam ids.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> material_name = "Material123"
>>> result = staad_obj.Property.GetIsotropicMaterialAssignedBeamList(material_name)
```

GetIsotropicMaterialAssignedPlateCount(*material_name: int*) [source]

Gets the count of plates assigned with the specific isotropic material.

Parameters:

material_name (*str*) – Material Name.

Returns:

Returns count of plates assigned with the specific isotropic material.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetIsotropicMaterialAssignedPlateCount("STEEL")
```

GetIsotropicMaterialAssignedPlateList(*material_name: str*) [source]

Gets the list of plate numbers which are assigned with the specified isotropic material.

Parameters:

material_name (*str*) – Material Name.

Returns:

Returns a list for plate id of plates which have material assigned to.

Return type:

List of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetIsotropicMaterialAssignedPlateList("Q235")
```

GetIsotropicMaterialAssignedSolidCount(*material_name: str*) [source]

Get the count of solids assigned with the specified isotropic material.

Parameters:

material_name (*str*) – Identification title of the material.

Returns:

Returns count of solids assigned with the specified isotropic material.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetIsotropicMaterialAssignedSolidCount("CONCRETE")
```

GetIsotropicMaterialAssignedSolidList(*material_name: str*) [source]

Get isotropic material assigned solid list.

Parameters:

material_name (*str*) – Identification title of the material.

Returns:

Returns a list of int for list of solid.

Return type:

List of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetIsotropicMaterialAssignedSolidList("STEEL")
```

GetIsotropicMaterialCount()

[source]

Gets the number of isotropic material present in the current structure.

Returns:

Returns the number of isotropic materials.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.GetIsotropicMaterialCount()
```

GetIsotropicMaterialProperties(*material_number: int*)

[source]

Get the properties for the specified isotropic material number.

Parameters:

material_number (*int*) – Zero based index of the material

Returns:

tuple – Returns a tuple consisting of Material Name, Modulus of elasticity, Poisson's ratio, Shear modulus, Weight density, Coefficient of thermal expansion and Damping ratio, respectively.

Return type:

Tuple <str, float, float, float, float, float, float>

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> isotropic_mat_no = staad_obj.Property.GetIsotropicMaterialCount()
>>> if isotropic_mat_no > 0:
>>>     material, elasticity, poisson, shear_mod, density, coef_thermal_exp, damp_rati
```

GetIsotropicMaterialPropertiesAssigned(*material_no: int*)

[source]

Gets isotropic material properties and if material assigned to element(s) or not.

Parameters:

material_no (*int*) – Material number ID

Returns:

tuple – Returns a Tuple consisting of material name, modulus of elasticity, poisson's ratio, shear modulus, weight density, coefficient of thermal expansion, damping ratio and material assigned to elements or not: unassigned(= 1), assigned(= 2) respectively.

Return type:

Tuple <str, float, float, float, float, float, float, int>

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> isotropic_mat_no = staad_obj.Property.GetIsotropicMaterialCount()
>>> if isotropic_mat_no > 0:
>>>     elasticity, poisson, shear_modulus, weight_density, thermal_expansion, damping
```

GetIsotropicMaterialPropertiesEx(*material_number: int*) [source]

Get the properties for the specified isotropic material number.

Parameters:

material_number (*int*) – Zero based index of the material

Returns:

tuple – Returns a Tuple consisting of Modulus of elasticity (E), Poisson's ratio (POI), Shear modulus (G), Weight density (DEN), Coefficient of thermal expansion (ALP), Damping ratio (DAMP), Yield stress (Fy), Tensile strength (Fu), Yield strength ratio (Ry), Tensile strength ratio (Rt) and Compressive strength (Fcu) respectively.

Return type:

Tuple(str, float, float, float, float, float, float, float, float, float, float, float)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> material_name, elasticity, poisson_ratio, shear_modulus, density, coef_thermal_exp
```

GetMaterialProperty(*MaterialName: str*) [source]

Get a specific material property.

Parameters:

MaterialName (*str*) – The Name of the material .

Returns:

Tuple consisting of elasticity, possion, density, alpha and damping ratio value, respectively.

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> elasticity, section_depth, density, alpha, damping_ratio = staad_obj.Property.GetM
```

GetMaterialPropertyEx(*material_name: str*) [source]

Get the properties for the specified isotropic material Name.

Parameters:

material_name (*str*) – Material name

Returns:

tuple – Returns a tuple consisting of Modulus of elasticity (E), Poisson's ratio (POI), Weight density (DEN), Coefficient of thermal expansion (ALP), Damping ratio (DAMP), Yield stress (Fy), Tensile strength (Fu), Yield strength ratio (Ry), Tensile strength ratio (Rt) and Compressive strength (Fcu) respectively.

Return type:

Tuple(float, float, float, float, float, float, float, float, float, float)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> elasticity, poisson_ratio, shear_modulus, density, coef_thermal_exp, cr_damp, yiel
```

GetMemberAttributeCount() [source]

Get the Count of Member Attribute.

Returns:

Returns Member Attribute Count

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetMemberAttributeCount()
```

GetMemberAttributeList() [source]

Get member attribute list.

Returns:

tuple – Returns a tuple consisting of attribute name list, the corresponding attribute value list and attribute count respectively.

Return type:

Tuple(list, list, int)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> attribute_name_list, attribute_value_list, attribute_count = staad_obj.Property.Ge
```

GetMemberCountByAttribute(*attribute_name: str, str_Value: str*) [source]

Return the number of member(s) in specified attribute.

Parameters:

- **attribute_name** (*str*) – Name of the attribute.
- **str_Value** (*str*) – A string value

Returns:

Returns 0 if successful else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetMemberCountByAttribute("MEMBTYPE", "BRACE")
```

GetMemberGlobalOffSet(*beam_id: int, member_offset_position: int*) [source]

Get beam end offsets in all three global directions.

Parameters:

- **beam_id** (*int*) – The beam number ID
- **member_offset_position** (*int*) – Member Start position (= 0); member End position (= 1).

Returns:

tuple – Returns a tuple consisting of member End position (= 1), the offset x in coordinate (local) and the offset y in coordinate (local) respectively.

Return type:

Tuple(float, float, float)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> offset_x, offset_y, offset_z = staad_obj.Property.GetMemberGlobalOffSet(beam_ids[0
```

GetMemberListByAttribute(*attribute_name: str, str_Value: str*) [source]

Get member list by attribute.

Parameters:

- **attribute_name** (*str*) – Name of the attribute.
- **str_Value** (*str*) – A string value

Returns:

Returns a list for Member(s) number ID list.

Return type:

List of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetMemberListByAttribute("MEMBTYPE", "BRACE")
```

GetMemberLocalOffSet(*beam_id: int, member_offset_position: int*) [source]

Get beam end offsets in all three local directions.

Parameters:

- **beam_id** (*int*) – The beam number ID
- **member_offset_position** (*int*) – Member Start position (= 0); member End position (= 1).

Returns:

Returns a List consisting of member End position (= 1), the offset x in coordinate (local) and the offset y in coordinate (local) respectively.

Return type:

List

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> offset_x, offset_y, offset_z = staad_obj.Property.GetMemberLocalOffSet(beam_ids[0]
```

GetMemberReleaseSpec(*member_no: int, end: int*) [source]

Get the release specification for a member at the specified end.

Parameters:

- **member_no** (*int*) – The member number.
- **end** (*int*) – Sets End at start if passed '0' else at the end if passed '1', for which you want release specification.

Returns:

Tuple – Tuple consisting of List of Release Values (6 elements for 6 DOFs. Element value: No release or spring = 0, release = 1, spring = -1, Only MP defined = -3 , MPX, MPY or MPZ defined = -2 .) & Spring Constant Values (Rotational releases float list with 6 elements for 6 DOFs. Element values Spring value or partial moment factor in floating point number) for the member (in same order).

Return type:

Tuple(List, List)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> release_values, spring_constant_values  = staad_obj.Property.GetMemberReleaseSpec(
```

GetMemberReleaseSpecEx(*beam_id: int, release_spec_position: int*) [source]

Get releases for the specified member at the specified end.

Parameters:

- **beam_id** (*int*) – Beam number ID.
- **release_spec_position** (*int*) – Member Start end (= 0); member End end (= 1).

Returns:

Returns a tuple consisting of following items respectively :

0. Translational release list with 6 elements for 6 DOFs. Element value: No release or spring = 0, release = 1, spring = -1 , Only MP defined = -3 , MPX, MPY or MPZ defined = -2.
1. Rotational releases list with 6 elements for 6 DOFs.
2. Element values Spring value or partial moment factor in floating point number, Partial moment release factor (same for MX, MY and MZ)
3. Rotational releases list with 3 elements for 3 rotational DOFs. Element values Spring value or partial moment factor in floating point number.

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_ids = staad_obj.Property.GetUserProvidedTableList()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> trans_release_list, rot_release_list, spring_const_list, rot_spring_const_list = s
```

GetMemberSpecCode(*member_id: int*) [source]

Get the type of specification attached to member with specified member ID (member_id).

Parameters:

member_id (*int*) – The member number ID.

Returns:

Returns value referring to type of member specification as per table below:

Value	Type of Member Specification
0	Truss Member
1	Tension-only Member
2	Compression-only Member
3	Cable-only Member
4	Joist Member
-1	Other

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> result = staad_obj.Property.GetMemberSpecCode(beamIds[0])
```

GetOrthotropic2DMaterialCount() [source]

Return the number of 2D orthotropic material present in the current structure.

Returns:

Returns the number of 2D orthotropic material.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> ortho_2d_mat_count = staad_obj.Property.GetOrthotropic2DMaterialCount()
```

GetOrthotropic2DMaterialProperties(*material_no: int*) [source]

Get the properties for the specified 2D orthotropic material.

Parameters:

material_no (*int*) – Material Number ID

Returns:

tuple – Returns a tuple consisting of Modulus of elasticity, Poisson's ratio, Shear modulus, Weight density, Coefficient of thermal expansion and Damping ratio, respectively.

Return type:

Tuple(float, float, float, float, float, float)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> ortho_2d_mat_count = staad_obj.Property.GetOrthotropic2DMaterialCount()
>>> if ortho_2d_mat_count > 0:
>>>     elasticity, poisson, shear_mod, density, coef_thermal_exp, damp_ratio = staad_
```

GetOrthotropic3DMaterialCount() [source]

Gets orthotropic 3D material count.

Returns:

Returns the orthotropic 3D material count.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> ortho_3d_mat_count = staad_obj.PropertyGetOrthotropic3DMaterialCount()
```

GetOrthotropic3DMaterialProperties(*material_no: int*) [source]

Get the properties for the specified 3D orthotropic material.

Parameters:

material_no (*int*) – Material Number ID

Returns:

tuple – Returns a tuple consisting of Modulus of elasticity, Poisson's ratio, Shear modulus, Weight density, Coefficient of thermal expansion and Damping ratio respectively.

Return type:

Tuple(float, float, float, float, float, float)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> ortho_3d_mat_count = staad_obj.Property.GetOrthotropic3DMaterialCount()
>>> if ortho_3d_mat_count > 0:
>>>     elasticity, poisson, shear_mod, density, coef_thermal_exp, damp_ratio = staad_
```

GetPlateMaterialName(*plate_id: int*) [source]

Get the material name assigned to a plate.

Parameters:

plate_id (*int*) – The plate number Id.

Returns:

The material name.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plate_ids = staad_obj.Geometry.GetPlateList()
>>> material_name = staad_obj.Property.GetPlateMaterialName(plate_ids[0])
```

GetPlateSectionPropertyRefNo(*PlateNo: int*) [source]

Get the assigned section property ID of specified plate.

Parameters:

PlateNo (*int*) – The plate number ID.

Returns:

Returns the assigned section property ID if successful, else returns status code:

- -4001 : Cannot find plate.
- -6022 : No property is attached to the plate.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plate_list = staad_obj.Geometry.GetPlateList()
>>> result = staad_obj.Property.GetPlateSectionPropertyRefNo(plate_list[0])
```

GetPlateThickness(*plate_no: int*) [source]

Get the thickness property of a plate.

Parameters:

plate_no (*int*) – The plate number.

Returns:

List – The thickness of nodes in the plate.

Return type:

Float list

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plate_ids = staad_obj.Geometry.GetPlateList()
>>> thickness_values = staad_obj.Property.GetPlateThickness(plate_ids[0])
```

GetPropertyUniqueID(*property_unique_id: int*) [source]

Get Property Unique ID.

Parameters:

property_unique_id (*int*) – Property number

Returns:

Returns property Unique ID.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> property_unique_id = staad_obj.Property.GetPropertyUniqueID(3)
```

GetPublishedProfileName(*staad_profile_name: str, country_code: int*) [source]

Get project published name by STAAD profile name.

Parameters:

- **staad_profile_name** (*str*) – STAAD profile name.
- **country_code** (*int*) – The value for the specified country.

Returns:

Returns published profile name if successful.

Returns NULL or empty string if unable to find any equivalent published name corresponding to STAAD profile name staad_profile_name.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetPublishedProfileName("STAADProfile1", 1)
```

GetRecordForSection(*country_code: int, section_name: str*) [source]

Get the Record No (Record No in Section database) on table with specific Country ID and specific Section Name.

Parameters:

- **country_code** (*int*) – Country id. (Refer OsProperty.CreateBeamPropertyFromTable for Country ID details).
- **section_name** (*str*) – Section Name(Type: String).

Returns:

Returns record number for specific section if successful else -1 if it encounters generate error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetRecordForSection(2, "HW200X200")
```

GetSTAADProfileName(*published_name: str, country_code: int*) [source]

Gets STAAD profile name by published profile name.

Parameters:

- **published_name** (*str*) – Published profile name.
- **country_code** (*int*) – The value for the specified country.

Returns:

Returns STAAD profile name if successful.

Returns NULL if Unable to find any equivalent STAAD name corresponding to profile name (published_name).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetSTAADProfileName("PublishedProfile1", 1)
```

GetSectionPropertyAssignedBeamCount(*prof_type: int*) [source]

Get section assigned beam count.

Parameters:

prof_type (*int*) –

Assign Profile Type:

Prof Type	Value
AssignAngle	0
AssignDoubleAngle	1
AssignBeam	2
AssignColumn	3
AssignChannel	4

Returns:

Returns the section table number if successful.

Returns -3001 if cannot find member beam number.

Returns -6004 if section not found in profile database.

Returns -6022 if no property is attached to the member/element.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetSectionPropertyAssignedBeamCount(1)
```

GetSectionPropertyAssignedBeamList(*prof_type: int*) [source]

Get section assigned beam list.

Parameters:

prof_type (*int*) –

Assign Profile Type:

Prof Type	Value
AssignAngle	0
AssignDoubleAngle	1
AssignBeam	2
AssignColumn	3
AssignChannel	4

Returns:

Returns a list of beam ids.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetSectionPropertyAssignedBeamList(2)
```

GetSectionPropertyCount() [source]

Returns total number of different sectional properties exist in the current STAAD file.

Returns:

Returns total number of different sectional properties.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetSectionPropertyCount()
```

GetSectionPropertyCountry(*sec_ref_no: int*) [source]

Returns the country reference number for the section property reference number specified.

Parameters:

sec_ref_no (*int*) – The assigned section property ID

Returns:

Returns country code else returns -6025 if no property is defined in the model.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> nAssignedSectionPropID = staad_obj.Property.CreateBeamPropertyFromTable(1, "W14X87
>>> output = staad_obj.Property.GetSectionPropertyCountry(nAssignedSectionPropID)
```

GetSectionPropertyList()

[source]

Gets the list of Section Property Reference IDs.

Returns:

Returns a List of Section Property reference IDs.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetSectionPropertyList()
```

GetSectionPropertyName(*sctn_prop_id: int*)

[source]

Get the property name for the specified section property reference number.

Parameters:

sctn_prop_id (*int*) – The assigned section property ID

Returns:

Returns a string for identification title of material.

Return type:

string

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> nAssignedSectionPropID = staad_obj.Property.CreateBeamPropertyFromTable(1, "W14X87
>>> output = staad_obj.Property.GetSectionPropertyName(nAssignedSectionPropID)
```

GetSectionPropertyType(*sec_ref_no: int*) [source]

Returns the section property type for the specified section property reference number.

Parameters:

sec_ref_no (*int*) – The assigned section property ID

Returns:

Returns number referring to section type code table.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> nAssignedSectionPropID = staad_obj.Property.CreateBeamPropertyFromTable(1, "W14X87
>>> output = staad_obj.Property.GetSectionPropertyType(nAssignedSectionPropID)
```

GetSectionPropertyValues(*prof_type: int*) [source]

Retrieve long member properties of the specified beam member.

Parameters:

prof_type (*int*) –

Assign Profile Type:

Prof Type	Value
AssignAngle	0
AssignDoubleAngle	1
AssignBeam	2
AssignColumn	3
AssignChannel	4

Returns:

tuple – Returns a Tuple consisting of Width of the section (WID), Depth of the section (DEP), Cross section area (Ax), Shear area in local y-axis (If zero, shear deformation is ignored in the analysis (Ay)), Shear area in local z-axis (If zero, shear deformation is

ignored in the analysis (Az)), Moment of inertia about local z-axis (Ix), Moment of inertia about local y-axis (Iy), Torsional constant (Iz) and Thickness of top flange (Tf) and Thickness of web (Tw) respectively.

Return type:

Tuple (float, float, float, float, float, float, float, float, float, float)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> section_width, section_depth, cross_section_area, shear_area_y, shear_area_z, mome
```

GetSectionPropertyValuesEx(*section_property_id: int*) [source]

Returns the section property Values of the specified beam.

Parameters:

section_property_id (*int*) – Section property ID.

Returns:

tuple – Returns a Tuple consisting of Number referring to below table, a float list for section property parameters respectively.

Return type:

Tuple(int, list)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> nAssignedSectionPropID = staad_obj.Property.CreateBeamPropertyFromTable(1, "W14X87
>>> result = staad_obj.Property.GetSectionPropertyValuesEx(nAssignedSectionPropID)
```

GetSectionTableNo(*beam_id: int*) [source]

Get section table number.

Parameters:

beam_id (*int*) – The beam number ID

Returns:

Returns the section table number if successful.

Else returns status code :

- -3001 : It is unable to find member.
- -6004 : Section not found in profile database.
- -6022 : No property is attached to the member/element.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> output = staad_obj.Property.GetSectionTableNo(beam_ids[0])
```

GetShapeCode(*country_code: int, section_name: str*) [source]

Get the Shape Code with specific Country and specific Section Name.

Parameters:

- **country_code** (*int*) –

  Country id according to the table below:

Country Code	Country
1	American
2	Australian
3	British
4	Canadian
5	Chinese
6	Dutch
7	European
8	French
9	German
10	Indian
11	Japanese
12	Russian
13	SouthAfrican
14	Spanish
15	Venezuelan
16	Korean

- **section_name** (*str*) – Section Name.

Returns:

Returns the Shape Code according to the table below else ‘-1’ if it encounters generate error:

Country	Shape Code
American	1 for “Wshape”, 2 for “MShape”, 3 for “SShape”, 4 for “HPShape”, 5 for “BShape”, 6 for “Channel”, 7 for “MCChannel”, 8 for “Angle”, 9 for “Tube”, 10 for “Pipe”, 11 for “HSSRectangle”, 12 for “HSSRound”, 13 for “CastellatedNonCompBeam”, 14 for “CastellatedCompBeam”, 15 for “RodShape”, 16 for “CableShape”, 23 for “HSSRectangleA1085”, 24 for “HSSRoundA1085”
Mexican	1 for “IEShape”, 2 for “IRShape”, 3 for “ISShape”, 4 for “CEChannel”, 5 for “LDAngle”, 6 for “LIAngle”, 7 for “OCPipe”, 8 for “ORTube”, 9 for “ORTubeR”
Australian	1 for “UBShape”, 2 for “UCShape”, 3 for “WBShape”, 4 for “WCShape”, 5 for “Channel”, 6 for “Angle”
British	1 for “UBShape”, 2 for “UCShape”, 3 for “UPShape”, 4 for “JOShape”, 5 for “Channel”, 6 for “Angle”, 7 for “Tube”, 8 for “Pipe”
Canadian	1 for “Wshape”,

Country	Shape Code
	2 for “MShape”, 3 for “SShape”, 4 for “HPShape”, 5 for “WWShape”, 6 for “Channel”, 7 for “MCChannel”, 8 for “Angle”, 9 for “Tube”, 10 for “Pipe”, 11 for “HSSRect”, 12 for “HSSRound”
Chinese	1 for “IShape”, 2 for “Channel”, 3 for “Angle”, 4 for “Tube”, 5 for “Pipe”, 6 for “TShape”, 7 for “HShape”
Dutch	1 for “HEShape”, 2 for “IPEShape”, 3 for “IPNShape”, 4 for “UPNChannel”, 5 for “Angle”, 6 for “Tube”, 7 for “Pipe”, 8 for “PlateStrip”, 9 for “SolidRound”, 10 for “SolidSquare”
European	1 for “IPEShape”, 2 for “HEShape”, 3 for “DILShape”, 4 for “IPNShape”, 5 for “UChannel”, 6 for “UPNChannel”, 7 for “Angle”, 8 for “Tube”, 9 for “Pipe”, 10 for “BulbFlat”, 11 for “FlatBar”, 12 for “HDShape”, 13 for “HLShape”, 14 for “HPShape”, 15 for “SolidSquare”,

Country	Shape Code
	16 for “UPEChannel”, 17 for “UAPChannel”, 18 for “Rhs”, 19 for “Shs”, 20 for “Chs”
French	1 for “IPEShape”, 2 for “HEShape”, 3 for “IPNShape”, 4 for “Channel”, 5 for “Angle”, 6 for “Tube”, 7 for “Pipe”
German	1 for “IPEShape”, 2 for “HEShape”, 3 for “IShape”, 4 for “UChannel”, 5 for “Angle”, 6 for “Tube”, 7 for “Pipe”
Indian	1 for “SShape”, 2 for “IShape”, 3 for “MShape”, 4 for “WShape”, 5 for “TShape”, 6 for “Channel”, 7 for “Angle”, 8 for “Tube”, 9 for “Pipe”, 10 for “WPBShape”, 11 for “NPBShape”
Brazilian	1 for “IShape”, 2 for “WHShape”, 3 for “WIShape”, 4 for “TShape”, 5 for “Channel”, 6 for “Angle”, 7 for “Rhs”, 8 for “Shs”, 9 for “Chs”, 10 for “Pipe”, 11 for “Cs”, 12 for “Cvs”,

Country	Shape Code
	13 for “Vs”, 14 for “SShape”
Japanese	1 for “HShape”, 2 for “IShape”, 3 for “TShape”, 4 for “Channel”, 5 for “Angle”, 6 for “Tube”, 7 for “Pipe”, 8 for “Rhs”, 9 for “Shs”, 10 for “Chs”, 11 for “CTShape”, 51 for “HShapeOld”, 52 for “TShapeOld”
Russian	1 for “BShape”, 2 for “SHShape”, 3 for “KShape”, 4 for “IShape”, 5 for “Channel”, 6 for “Angle”, 7 for “Tube”, 8 for “Pipe”
SouthAfrican	1 for “IShape”, 2 for “HShape”, 3 for “PGShape”, 4 for “CChannel”, 5 for “Angle”, 6 for “Tube”, 7 for “Pipe”
Spanish	1 for “IPEShape”, 2 for “HEShape”, 3 for “IPNShape”, 4 for “Channel”, 5 for “Angle”, 6 for “Tube”, 7 for “Pipe”

Country	Shape Code
Venezuelan	1 for “Beam”, 2 for “Channel”, 3 for “Angle”, 4 for “Tube”, 5 for “Pipe”
Korean	1 for “WShape”, 2 for “HShape”, 3 for “IShape”, 4 for “WTShape”, 5 for “Channel”, 6 for “Angle”, 7 for “Pipe”, 8 for “Tube”
Aluminum	1 for “AAStandardIBeams”, 2 for “HBeam”, 3 for “ArmyNavyIBeam”, 4 for “AmericanStandardIBeam”, 5 for “IBeam”, 6 for “AAStandardChannel”, 7 for “Channel”, 8 for “ArmyNavyChannel”, 9 for “SpecialChannel”, 10 for “AmericanStandardChannel”, 11 for “EqualLegAngle”, 12 for “SquareEndEqualLegAngle”, 13 for “UnequalLegAngle”, 14 for “SquareEndUnequalLegAngle”, 15 for “SquareTube”, 16 for “RectangularTube”, 17 for “RoundTube”
UserTable	1 for “WideFlange”, 2 for “Channel”, 3 for “Angle”, 4 for “DblAngle”, 5 for “Tee”, 6 for “Pipe”, 7 for “Tube”, 8 for “General”, 9 for “ISection”, 10 for “Prismatic”
AmericanColdFormed	1 for “Angle”, 2 for “AngleS”,

Country	Shape Code
	3 for “Channel”, 4 for “ChannelS”, 5 for “Zee”, 6 for “ZeeS”, 7 for “Hat”, 8 for “Pipe”, 9 for “Tube”
RCecoColdFormed (Reserved)	1 for “Angle”, 2 for “AngleS”, 3 for “Channel”, 4 for “ChannelS”, 5 for “Zee”, 6 for “ZeeS”, 7 for “Hat”, 8 for “Pipe”, 9 for “Tube”, 10 for “EaveStrut”(Reserved)
Lysaght	4 for “ChannelS”, 6 for “ZeeS”
IndianColdFormed	1 for “Angle”, 2 for “AngleS”, 3 for “Channel”, 4 for “ChannelS”, 5 for “Zee”, 6 for “ZeeS”, 7 for “Hat”
BritishColdFormed	1 for “Angle”, 2 for “AngleS”, 3 for “Channel”, 4 for “ChannelS”, 5 for “Zee”, 6 for “ZeeS”, 7 for “Hat”, 8 for “Pipe”, 9 for “Tube”
AustralianColdFormed	1 for “RHS”, 2 for “SHS”, 3 for “CHS”
EuropeanColdFormed	1 for “RHS”, 2 for “SHS”, 3 for “CHS”

Country	Shape Code
KingspanColdFormed	1 for “Angle”, 2 for “AngleS”, 3 for “Channel”, 4 for “ChannelS”, 5 for “Zee”, 6 for “ZeeS”, 7 for “Hat”, 8 for “Pipe”, 9 for “Tube”
JapaneseColdFormed	11 for “BCP”, 12 for “BCPT”, 13 for “BCR”
RusColdFormed	8 for “Pipe”
AITC-Timber	1 for “GluedLaminatedTimber”, 2 for “Aspen”, 3 for “BalsamFir”, 4 for “BeechBirchHickory”, 5 for “CoastSitkaSpruce”, 6 for “Cottonwood”, 7 for “DouglasFirLarch”, 8 for “DouglasFirLarchNorth”, 9 for “DouglasFirLarchSouth”, 10 for “EasternHemlock”, 11 for “EasternHemlockTamarack”, 12 for “EasternHemlockTamarackN”, 13 for “EasternSoftwoods”, 14 for “EasternSpruce”, 15 for “EasternWhitePine”, 16 for “HemFir”, 17 for “HemFirNorth”, 18 for “MixedMaple”, 19 for “MixedOak”, 20 for “MixedSouthernPine”, 21 for “MountainHemlock”, 22 for “NorthernPine”, 23 for “NorthernRedOak”, 24 for “NorthernSpecies”, 25 for “NorthernWhiteCedar”, 26 for “PonderosaPine”, 27 for “RedMaple”, 28 for “RedOak”, 29 for “RedPine”, 30 for “Redwood”,

Country	Shape Code
	31 for “SitkaSpruce”, 32 for “SouthernPine”, 33 for “SprucePineFir”, 34 for “SprucePineFirSouth”, 35 for “WesternCedars”, 36 for “WesternCedarsNorth”, 37 for “WesternHemlock”, 38 for “WesternHemlockNorth”, 39 for “WesternWhitePine”, 40 for “WesternWoods”, 41 for “WhiteOak”, 42 for “YellowPoplar”
American Steel Joist	1 for “Kjoist”, 2 for “KCSJoist”, 3 for “LHJoist”, 4 for “DLHJoist”, 5 for “JoistGirder”
Generic	1 for “WShape”, 2 for “TShape”, 3 for “Channel”, 4 for “Angle”, 5 for “Tube”, 6 for “Pipe”, 7 for “Rectangle”, 8 for “Round”, 9 for “Zee”, 20 for “General”
Canadian Timber	1 for “GluedLaminatedTimber”, 2 for “DouglasFirLarch”, 3 for “HemFir”, 4 for “NorthernSpecies”, 5 for “SprucePineFir”
Butler	4 for “EaveStrut”, 6 for “ZeePurlin”, 9 for “BoxStrut”, 10 for “WideFlange”, 11 for “TaperedWideFlange”, 12 for “SolidRound”
Jindal	1 for “UBShape”, 2 for “HEShape”, 3 for “IPEShape”, 4 for “UCShape”,

Country	Shape Code
	5 for “ISMCShape”, 6 for “WPBShape”, 7 for “NPBShape”
Tata Structura	1 for “Rhs”, 2 for “Shs”, 3 for “Chs”
APL Apollo Tubes	1 for “Rhs”, 2 for “Shs”, 3 for “Chs”

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetShapeCode(5, "HW200X200")
```

GetSolidMaterialName(*solid_id: int*) [source]

Get the material name of the specified solid.

Parameters:

solid_id (*int*) – The Solid number ID.

Returns:

Returns material name of the specified solid.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> solid_ids = staad_obj.Geometry.GetSolidList()
>>> result = staad_obj.Property.GetSolidMaterialName(solid_ids[0])
```

GetStandardProfileDBFolder() [source]

Gets standard profile default database folder path.

Returns:

Returns the standard profile database folder path.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetStandardProfileDBFolder()
```

GetStandardSectionDatabaseName(*section_property_id: int*) [source]

Gets standard section database name for the specified section property reference number.

Parameters:

section_property_id (*int*) – The section property reference ID.

Returns:

Returns <Non-Empty-String> if the standard section database name if successful else <Empty-String> if specified section property reference does not belong to Standard section database.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetStandardSectionDatabaseName(4)
```

GetStandardSectionName(*section_reference_id: int*) [source]

Get the section name from the standard section database and table for the specified standard section property reference number.

Parameters:

section_reference_id (*int*) – The section property reference ID.

Returns:

Returns <Non-Empty-String> if the standard section database name if successful else <Empty-String> if specified section property reference does not belong to Standard section database.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetStandardSectionName()
```

GetStandardSectionTableName(*section_reference_id: int*) [source]

Get the section name from the standard section database and table for the specified standard section property reference number.

Parameters:

section_reference_id (*int*) – The section property reference ID.

Returns:

Returns <Non-Empty-String> if the standard section database name if successful else <Empty-String> if specified section property reference does not belong to Standard section database.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.GetStandardSectionTableName(1)
```

GetThicknessPropertyAssignedPlateCount(*property_reference_id: int*) [source]

Gets the count of plates which are assigned with the specified Thickness Property reference ID.

Parameters:

property_reference_id (*int*) – Thickness Property reference ID.

Returns:

Returns count of plates which are assigned with the specified Thickness Property reference ID.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> thickness_props = staad_obj.Property.GetThicknessPropertyList()
>>> result = staad_obj.Property.GetThicknessPropertyAssignedPlateCount(thickness_props
```

GetThicknessPropertyAssignedPlateList(*property_reference_id: int*) [source]

Gets the list of plate numbers which are assigned with the specified Thickness Property reference ID.

Parameters:

property_reference_id (*int*) – The specific Thickness Property reference ID.

Returns:

Returns a list for plate number list.

Return type:

List of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> thickness_props = staad_obj.Property.GetThicknessPropertyList()
>>> result = staad_obj.Property.Property.GetThicknessPropertyAssignedPlateList(thickne
```

GetThicknessPropertyCount() [source]

Get Thickness Property Count.

Returns:

Returns total thickness properties count.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetThicknessPropertyCount()
```

GetThicknessPropertyList() [source]

Get Thickness Property ID list

Returns:

Returns a list of Thickness Property ID list.

Return type:

List of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetThicknessPropertyList()
```

GetThicknessPropertyValues(*property_reference_id: int*) [source]

Get Thickness Property Values

Parameters:

property_reference_id (*int*) – The specific Thickness Property reference ID

Returns:

Returns a list for thickness value list.

Return type:

List of floats

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> thickness_props = staad_obj.Property.GetThicknessPropertyList()
>>> result = staad_obj.Property.GetThicknessPropertyValues(thickness_props[0])
```

GetTypeForIsotropicMaterial(*material_name: str*) [source]

Get Type For the specified Isotropic Material.

Parameters:

material_name (*str*) – Identification title of the material.

Returns:

Returns an int for Material Type:

No.	Material Type
0	Not Specified
1	Steel
2	Concrete
3	Aluminum
4	Timber

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetTypeForIsotropicMaterial(strInput)
```

GetUptGeneralProfileBoundaryPoints(*table_number_id: int, section_name: str, is_inner: bool*) [source]

Get Profile Points coordinate from User Provided general section Table (UPT).

Parameters:

- **table_number_id** (*int*) – The existing table number ID.
- **section_name** (*str*) – UPT section string name.
- **is_inner** (*bool*) – (Reserved for inner points, set it to false)

Returns:

Returns a tuple consisting of profile points coordinate list in Z and profile points coordinate list in Y respectively.

Return type:

Tuple of float

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> point_cord_z_list, point_cord_y_list = staad_obj.Property.GetUptGeneralProfileBour
```

GetUptGeneralProfilePointsCount(*table_reference_id: int, section_name: str*) [source]

Get profile points count from user provided general section table (UPT).

Parameters:

- **table_reference_id** (*int*) – The existing table number ID.
- **section_name** (*str*) – UPT section string name.

Returns:

Returns a Tuple consisting of count of outer profile points and count of inner profile points(Reserved, not be used now) respectively.

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetUptGeneralProfilePointsCount(1, "AAAA")
```

GetUptGeneralStressLocationPoints(*table_reference_id: int, section_name: str*) [source]

Stress Location in local coordinate from User Provided general section Table (UPT).

Parameters:

- **TableRef** (*int*) – The existing table number ID.
- **SectionName** (*str*) – UPT section string name.

Returns:

Tuple of list – Returns a tuple consisting of list (of size 4) consisting of stress Location coordinate in Z and list (of size 4) consisting stress location coordinate in Y respectively.

Return type:

Tuple(list, list)

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> point_cord_z_list, point_cord_y_list = staad_obj.Property.GetUptGeneralStressLocat
```

GetUserProvidedTableCount() [source]

Get the number of UPT tables.

Returns:

Returns the number of UPT tables.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetUserProvidedTableCount()
```

GetUserProvidedTableList() [source]

Get the UPT table ID list.

Returns:

Returns UPT table ID list.

Return type:

List of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetUserProvidedTableList()
```

GetUserProvidedTableNo(*table_index: int*) [source]

Get section user provided table number ID by user table index.

Parameters:

table_index (*int*) – User Provided Table (UPT) index.

Returns:

Returns User Provided Table (UPT) number ID else returns -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_ids = staad_obj.Property.GetUserProvidedTableList()
>>> result = staad_obj.Property.GetUserProvidedTableNo(upt_ids[0])
```

GetUserProvidedTableSectionCount(*table_id: int*) [source]

Get the number of sections defined in specified User Provided Table (UPT).

Parameters:

table_id (*int*) – The User Provided Table (UPT) number ID.

Returns:

Returns number of section in a given UPT.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetUserProvidedTableSectionCount(10)
```

GetUserProvidedTableSectionList(*table_id: int*) [source]

Get the list of section names in specified User Provided Table (UPT).

Parameters:

table_id (*int*) – The User Provided Table (UPT) number ID.

Returns:

Returns a list of strings consisting of indexes and corresponding section string names included.

Return type:

list of strings

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetUserProvidedTableSectionList(10)
```

`GetUserProvidedTableSectionProperties(table_id: int, section_name: str, property_count: int = 24)` [source]

Get the section type and section properties of specified UPT section.

Parameters:

- **table_id** (*int*) – The User Provided Table (UPT) number ID.
- **section_name** (*str*) – UPT section string name given to this section property.
- **property_count** (*int*) – The number of properties present in section of UPT table (default is 24).

Returns:

Returns a tuple consisting of UPT Section Type from the below table and list of section property parameters respectively:

User Table Type	UPT Section Type	propValues
USER TABLE PRISMATIC	502	Ax Iz Iy Ix Ay Az YD ZD
USER TABLE GENERAL	482	Ax D Td B Tb Iz Iy Ix Sz Sy Ay Az Pz Py Hss Dee
USER TABLE WIDE FLANGE	412	Ax D Tw Wf Tf Iz Iy Ix Ay Az Wf1 Tf1
USER TABLE CHANNEL	422	Ax D Tw Wf Tf Iz Iy Ix Cz Ay Az
USER TABLE ANGLE	432	D Wf Tf R Ay Az
USER TABLE DOUBLE ANGLE	442	D Wf Tf SP Iz Iy Ix Cy Ay Az
USER TABLE TEE	452	Ax D Wf Tf Tw Iz Iy Ix Cy Ay Az
USER TABLE PIPE	462	OD ID Ay Az
USER TABLE TUBE	472	Ax D Wf Tf Iz Iy Ix Ay Az
USER TABLE ISECTION	492	Dww Tww Dww1 Bff Tff Bff1 Tff1 Ayf Azf Xif

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> section_type, section_properties = staad_obj.Property.GetUserProvidedTableSectionP
```

`GetUserProvidedTableSectionPropertyCount(upt_table_id: int, section_name: str)` [source]

Get the user provided table section property count in specified User Provided Table (UPT).

Parameters:

- **upt_table_id** (*int*) – The User Provided Table (UPT) number ID.
- **section_name** (*str*) – UPT section string name given to this section property.

Returns:

Returns the number of section(s) in given UPT.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.GetUserProvidedTableSectionPropertyCount(8, "UPT VJG20
```

GetUserProvidedTableSectionType(*table_id: int*) [source]

Get the user provided table section type in specified User Provided Table (UPT).

Parameters:

table_id (*int*) – User Provided Table (UPT) number ID.

Returns:

Returns an int for number referring to Section Type Code table:

User Table Type	UPT Section Type
USER TABLE PRISMATIC	502
USER TABLE GENERAL	482
USER TABLE WIDE FLANGE	412
USER TABLE CHANNEL	422
USER TABLE ANGLE	432
USER TABLE DOUBLE ANGLE	442
USER TABLE TEE	452
USER TABLE PIPE	462
USER TABLE TUBE	472
USER TABLE ISECTION	492

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_ids = staad_obj.Property.GetUserProvidedTableList()
>>> result = staad_obj.Property.GetUserProvidedTableSectionType(7)
```

IsStandardDatabaseSection(*section_reference_id: int*) [source]

Checks if the specified section property reference number is from standard section database source or not.

Parameters:

section_reference_id (*int*) – The section property reference ID.

Returns:

Returns 'True' if section source is standard database else 'False' if section source is other than standard database.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.IsStandardDatabaseSection(1)
```

RemoveAllElementNodeReleaseSpec() [source]

Remove all element node release specification from the model.

Returns:

Returns '1' if OK else '0' if no element release specification present.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.RemoveAllElementNodeReleaseSpec()
```

RemoveAllElementOffsetSpec() [source]

Removes all element node offset specifications from the model.

Returns:

Returns 1 if OK else 0 if no element offset specifications present.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.Property.RemoveAllElementOffsetSpec()
```

RemoveAttribute(*attribute_name: str, str_value: str, member_ids: list | int*)

Remove the member(s) from specified attribute. [source]

Parameters:

- **attribute_name** (*str*) – Name of the attribute.
- **str_value** (*str*) – A string value
- **member_ids** (*list of int or int*) – Member(s) number ID list.

Returns:

Returns 0 if successful else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> result = staad_obj.Property.RemoveAttribute("MEMBTYPE", "BRACE", beamIds[0:2])
```

RemoveElementIgnoreInplaneRotnSpecFromPlate(*plate_id: int*) [source]

Remove element ignore in plane rotation specification from plate.

Parameters:

plate_id (*int*) – The plate number ID.

Returns:

Returns 0 if OK else returns -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plateIds = staad_obj.Geometry.GetPlateList()
>>> result = staad_obj.Property.RemoveElementIgnoreInplaneRotnSpecFromPlate(plateIds[0
```

RemoveElementNodeReleaseSpecFromPlate(*plate_id: int, node_id: int*) [source]

Remove element node release specification from plate.

Parameters:

- **plate_id** (*int*) – The plate number ID.
- **node_id** (*int*) – The node number ID to be released.

Returns:

Returns 0 if OK else returns -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plateIds = staad_obj.Geometry.GetPlateList()
>>> nodeIds = staad_obj.Geometry.GetNodeList()
>>> result = staad_obj.Property.RemoveElementNodeReleaseSpecFromPlate(plateIds[0], noc
```

RemoveElementPlaneStressSpecFromPlate(*plate_id: int*) [source]

Remove element plane stress specification from plate.

Parameters:

plate_id (*int*) – The plate number ID.

Returns:

Returns 0 if OK else returns -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plateIds = staad_obj.Geometry.GetPlateList()
>>> result = staad_obj.Property.RemoveElementPlaneStressSpecFromPlate(plateIds[0])
```

RemoveMaterialFromBeam(*beam_id: int*) [source]

Remove the material assignment from a beam.

Parameters:

beam_id (*int*) – The Beam number ID.

Returns:

Returns 0 if Ok else returns -1 if it fails.

Return type:

Int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beam_ids = staad_obj.Geometry.GetBeamList()
>>> result = staad_obj.Property.RemoveMaterialFromBeam(beam_ids[0])
```

RemoveMaterialFromPlate(*plate_ids: list | int*) [source]

Remove the material assignment from a plate.

Parameters:

plate_ids (*list of int or int*) – The plate number.

Returns:

'True' if removes material else 'False' if it fails

Return type:

Bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plate_ids = staad_obj.Geometry.GetPlateList()
>>> result = staad_obj.Property.RemoveMaterialFromPlate(plate_ids[0])
```

RemoveMaterialFromSolid(*solid_id_list: list*) [source]

Remove Material From the specific Solids.

Parameters:

solid_id_list (*list of int*) – List of Solids IDs

Returns:

Returns 'True' if it succeeds in removing material from solids else 'False'.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.RemoveMaterialFromSolid([8, 5, 10, 3])
```

RemoveMemberCableSpecFromBeam(*beam_id: int, tension_or_length: int*) [source]

Removes the member cable specification from a particular member at the provided location type (Tension or Length).

Parameters:

- **beam_id** (*int*) – The beam number ID.
- **tension_or_length** (*int*) – The Cable location at Tension (= 0) or Length (= 1) of the member.

Returns:

Returns 'True' if remove member cable specification succeeded else 'False'.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> result = staad_obj.Property.RemoveMemberCableSpecFromBeam(beamIds[0], 0)
```

RemoveMemberCompressionSpecFromBeam(*beam_id: int*) [source]

Remove member compression specification from beam.

Parameters:

beam_id (*int*) – The beam number ID.

Returns:

Returns 0 if OK else returns -1 if general error

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> result = staad_obj.Property.RemoveMemberCompressionSpecFromBeam(beamIds[0])
```

RemoveMemberFireProofingSpecFromBeam(*beam_id: int*) [source]

Remove member fire proofing specification from beam.

Parameters:

beam_id (*int*) – The beam number ID.

Returns:

Returns 1 if fire proofing specification removed from beam else 0 if unable to remove fire proofing specification from beam.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.RemoveMemberFireProofingSpecFromBeam(1)
```

RemoveMemberIgnoreStiffSpecFromBeam(*beam_id: int*) [source]

Remove member ignore stiff specification from beam.

Parameters:

BeamNo (*int*) – The beam number ID.

Returns:

Returns 0 if OK else returns -1 if general error

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> result = staad_obj.Property.RemoveMemberTensionSpecFromBeam(beamIds[0])
```

RemoveMemberInactiveSpecFromBeam(*beam_id: int*) [source]

Remove member inactive specification from beam.

Parameters:

beam_id (*int*) – The beam number ID.

Returns:

Returns 0 if OK else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> staad_obj.Property.RemoveMemberInactiveSpecFromBeam(beamIds[0])
```

RemoveMemberOffsetSpecFromBeam(*beam_id: int, release_location: int*) [source]

Removes the member offset specification from a particular member at the provided location (Start or End).

Parameters:

- **beam_id** (*int*) – The beam number ID.
- **release_location** (*int*) – The Release location at START (= 0) or END (= 1) of the member.

Returns:

Returns true if successful else false if it fails.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> result = staad_obj.Property.RemoveMemberOffsetSpecFromBeam(beamIds[0], 0)
```

RemoveMemberReleaseSpecFromBeam(*beam_id: int, release_location: int*) [source]

Removes the member specification from a particular member at the provided location (Start or End).

Parameters:

- **beam_id** (*int*) – The beam number ID.
- **release_location** (*int*) – The Release location at START (= 0) or END (= 1) of the member.

Returns:

Returns true if successful else false if it fails.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> staad_obj.Property.RemoveMemberReleaseSpecFromBeam(beamIds[0], 0)
```

RemoveMemberTensionSpecFromBeam(*beam_id: int*) [source]

Remove member tension specification from beam.

Parameters:

beam_id (*int*) – The beam number ID.

Returns:

Returns 0 if OK else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> result = staad_obj.Property.RemoveMemberTensionSpecFromBeam(beamIds[0])
```

RemoveMemberTrussSpecFromBeam(*beam_id: int*) [source]

Remove member truss specification from beam.

Parameters:

beam_id (*int*) – The beam number ID.

Returns:

Returns 0 if OK else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> beamIds = staad_obj.Geometry.GetBeamList()
>>> result = staad_obj.Property.RemoveMemberTrussSpecFromBeam(beamIds[0])
```

RemovePropertyFromBeam(*beam_id: int*) [source]

Remove property from beam.

Parameters:

beam_id (*int*) – The beam number ID.

Returns:

Returns 0 if successful else -1 if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.RemovePropertyFromBeam(1)
```

RemovePropertyFromPlate(*plate_id: int*) [source]

Removes Thickness Property From the specific surface.

Parameters:

plate_id (*int*) – Plate Id for plate to remove thickness from

Returns:

Returns value of the assigned section property ID.

Else returns '-4001' if cannot find plate with specified plate id (plate_id).

Else returns '-6022' if no property is attached to the plate.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.RemovePropertyFromPlate(2)
```

RemovePropertyFromUPTTable(*table_reference_id: int, section_name: str*)

Remove a property from User Provided Table (UPT) if exist. [source]

Parameters:

- **table_reference_id** (*int*) – The existing table number ID.
- **section_name** (*str*) – UPT section string name.

Returns:

Returns 1 if successful. Returns 0 if general error. Returns -100 if invalid table number or section name.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> upt_num_id = staad_obj.Property.CreateUPTTable(7)
>>> result = staad_obj.Property.RemovePropertyFromUPTTable(upt_num_id, "VJG20")
```

RemoveUPTTable(*table_ref_id: int*)

[source]

Remove the whole User Provided Table (UPT) specified by table number ID.

Parameters:

table_ref_id (*int*) – The existing table number ID

Returns:

Returns 'True' if successful else 'False' if general error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.RemoveUPTTable(1)
```

SetMaterialName(*material_name: str*) [source]

Set the material name for a member.

Parameters:

material_name (*str*) – The material name to assign.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Property.SetMaterialName("UserDefineMaterial_1")
```

SetPropertyUniqueID(*property_number: int, property_unique_id: str*) [source]

Set Property Unique ID to specification property number.

Parameters:

- **property_number** (*int*) – Property number
- **property_unique_id** (*str*) – Property Unique ID

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.SetPropertyUniqueID(4, "EA8A58A7-FF56-4F25-A9A9-C6D079
```

SetStandardProfileDBFolder(*folder_name: str*) [source]

Sets standard profile database folder path.

Parameters:

folder_name (*str*) – Path of the folder.

Returns:

Returns 0 if successful else -1 if error (If path is empty or does not exist).

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.SetStandardProfileDBFolder("C:\Staad.Pro 2023\STAAD\Pro
```

SetTypeToIsotropicMaterial(*material_name: str, material_type: int*) [source]

Set Type To the specified Isotropic Material.

Parameters:

- **material_name** (*str*) – Identification title of the material.
- **material_type** (*int*) – Material Type.

Returns:

Returns 'True' if Set Type to Material successful else 'False' if generate error.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Property.SetTypeToIsotropicMaterial("TestMat", 4)
```

UpdatePropertiesToDesignSection() [source]

Updates all the section properties that have been designed with a SELECT MEMBER command.

Returns:

Returns 1 if assignment is successful else 0 if assignment is unsuccessful.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Property.UpdatePropertiesToDesignSection()
```

__init__(*staadObj*) [source]

Support

Contents

- OSSupport

class openstaadpy.os_analytical.ossupport.**OSSupport** [source]

Bases: object

AssignSupportToEntityList(*supportid*, *entitylist*) [source]

Assign the specified support to an entity list.

Parameters:

- **supportid** (*int*) – Support reference ID.
- **entitylist** (*list of int*) – List of node or plate numbers.

Returns:

True if successful.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.Support.AssignSupportToEntityList(3, [41, 42]
>>> print(status)
```

AssignSupportToNode(*NodeIDs: list | int*, *SupportID: int*)

Assign a support to one or more nodes. [source]

Parameters:

- **NodeIDs** (*list of int or int*) – List of node numbers or a single node number to assign the support to.

- **SupportID** (*int*) – Support reference number ID.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> fixed_id = staad_obj.Support.CreateSupportFixed()
>>> staad_obj.Support.AssignSupportToNode([1, 2, 3], fixed_id) // Pa
>>> staad_obj.Support.AssignSupportToNode(5, fixed_id) //Passing sup
```

CreateElasticFooting(*length, width, direction, subgrade*)

Create an elastic footing support. [source]

Parameters:

- **length** (*float*) – Length of footing.
- **width** (*float*) – Width of footing.
- **direction** (*int*) – Direction.
- **subgrade** (*float*) – Subgrade modulus.

Returns:

Support reference number ID.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> support_id = staad_obj.Support.CreateElasticFooting(5, 6, 2, 20)
>>> print(support_id)
```

CreateElasticMat(*direction, subgrade, printFlag, springType*)

Create an elastic mat support. [source]

Parameters:

- **direction** (*int*) – Direction.
- **subgrade** (*float*) – Subgrade modulus.
- **printFlag** (*int*) – Print flag.
- **springType** (*int*) – Spring type.

Returns:

Support reference number ID.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> support_id = staad_obj.Support.CreateElasticMat(5, 20, 0, 1)
>>> print(support_id)
```

`CreateInclinedSupport(inclinedType: int, refType: int, refNode: int, coord, releaseSpec: list, springSpec: list)` [source]

Create an inclined support.

Parameters:

- **inclinedType** (*int*) –

  Type of the inclined support:

Value	Inclined Type
1	Pinned
2	Fixed
3	FixedBut
4	Enforced
5	EnforcedBut

- **refType** (*int*) –

  Type of the reference point:

Value	Table Type
0	fRefX, fRefY, fRefY global distances from the joint to the reference point.
1	fRefX, fRefY, fRefY global coordinates of the reference point.
2	a joint number (vaRefNode) whose x, y, z global coordinates is the reference point.

- **refNode** (*int*) – Reference node number.
- **coord** (*list of float*) – Reference coordinates. [X, Y, Z]
- **releaseSpec** (*list of float*) – Release specification. Fixed (= 0) or Release (= 1) [FX, FY, FZ, MX, MY, MZ]
- **springSpec** (*list of float*) – Spring specification. [KFX, KFY, KFZ, KMX, KMY, KMZ]

Returns:

Support reference number ID.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> support_id = staad_obj.Support.CreateInclinedSupport(2, 2, 1, [0
>>> print(support_id)
```

`CreatePlateMat(direction: int, subgrades, printFlag: bool, springType: int)` [source]

Create a plate mat support.

Parameters:

- **direction** (*int*) –

Value	Inclined Type
0	X Direction
1	Y Direction
2	Z Direction
3	X Only Direction
4	Y Only Direction
5	Z Only Direction
6	All Direction

- **subgrades** (*list of float or float*) – Subgrade modulus value(s).
- **printFlag** (*bool*) – Print flag.
- **springType** (*int*) – Spring type.

Returns:

Support reference number ID. / 0 if error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> support_id = staad_obj.Support.CreatePlateMat(2, [20, 30, 40], F
>>> print(support_id)
```

`CreateSupportFixed()` [source]

Creates a fully fixed support.

Returns:

Support reference number ID. -1 indicates General Error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Support.CreateSupportFixed()
```

CreateSupportFixedBut(*ReleaseSpec: list, SpringSpec: list*)

[source]

Creates fixed support with releases in specified directions or a spring support with spring constants in specified directions.

Returns:

Support reference number ID. -1 indicates General Error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Support.CreateSupportFixedBut(ReleaseSpec, Spr
```

CreateSupportPinned()

[source]

Creates a pinned support (i.e., free to rotate about local y and z axis, fixed in all other degrees of freedom).

Returns:

Support reference number ID. -1 indicates General Error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Support.CreateSupportPinned()
```

DeleteSupport(*supportNo: int*) [source]

Remove a support item from the model.

Parameters:

supportNo (*int*) – Support item number.

Returns:

True if successful.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.Support.DeleteSupport(2)
>>> print(status)
```

GetCountOfElasticFooting() [source]

Get the total number of elastic footing supports.

Returns:

Number of elastic footing supports.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Support.GetCountOfElasticFooting()
>>> print(count)
```

GetCountOfElasticMat() [source]

Get the total number of ElasticMat supports.

Returns:

Number of ElasticMat supports.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> support_count = staad_obj.Support.GetCountOfElasticMat()
>>> print(support_count)
```

GetCountOfPlateMat() [source]

Get the total number of plate mat supports.

Returns:

Number of plate mat supports.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> support_count = staad_obj.Support.GetCountOfPlateMat()
>>> print(support_count)
```

GetElasticFootingAssignmentList(*supportid*) [source]

Get list of assigned node Ids for a specific elastic footing support Id.

Parameters:

supportid (*int*) – Support reference ID.

Returns:

List of node numbers.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> node_list = staad_obj.Support.GetElasticFootingAssignmentList(3)
>>> print(node_list)
```

GetElasticFootingDetail(*supportid*) [source]

Get elastic footing support information for a specific support Id.

Parameters:

supportid (*int*) – Support reference ID.

Returns:

(length, width, direction, subgrade, nodesCount)

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> details = staad_obj.Support.GetElasticFootingDetail(2)
>>> print(details)
```

GetElasticMatAssignmentList(*supportid*) [source]

Get elastic mat support entity list for a specific support Id.

Parameters:

supportid (*int*) – Support reference ID.

Returns:

List of node numbers.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> node_list = staad_obj.Support.GetElasticMatAssignmentList(4)
>>> print(node_list)
```

`GetElasticMatDetail(supportid)` [source]

Get elastic mat support information for a specific support Id.

Parameters:

supportid (*int*) – Support reference ID.

Returns:

(direction, subgrade, printFlag, springType, nodesCount)

- **direction : *int***

Value	Inclined Type
0	X Direction
1	Y Direction
2	Z Direction
3	X Only Direction
4	Y Only Direction
5	Z Only Direction

- **subgrade : *float***

  Subgrade modulus.

- **printFlag : *bool***

  Print flag. True if checked, False if not.

- **springType : *int***

Value	Spring Type
0	None
1	Compression only
2	Multi-linear

- **nodesCount : *int***

  Number of nodes assigned to this support.

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> details = staad_obj.Support.GetElasticMatDetail(4)
>>> print(details)
```

GetPlateMatAssignmentList(*plateMatNo*) [source]

Get plate mat support entity list for a specific support Id.

Parameters:

plateMatNo (*int*) – Plate mat support ID.

Returns:

List of plate numbers.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> plate_list = staad_obj.Support.GetPlateMatAssignmentList(1)
>>> print(plate_list)
```

GetPlateMatDetail(*plateMatNo*) [source]

Get plate mat support information for a specific support Id.

Parameters:

plateMatNo (*int*) – Plate mat support ID.

Returns:

(direction, subgrade1, subgrade2, subgrade3, printFlag, springType, nAssignedPlateCount)

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> details = staad_obj.Support.GetPlateMatDetail(1)
>>> print(details)
```

GetPlateMatSupportId(*plateMatIndex*) [source]

Get the plate mat support ID.

Parameters:

plateMatIndex (*int*) – PlateMat index (starting from 0).

Returns:

Plate mat support ID.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> support_id = staad_obj.Support.GetPlateMatSupportId(1)
>>> print(support_id)
```

GetSupportCount() [source]

Get the total number of supported nodes in the current structure.

Returns:

Number of supported nodes.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.Support.GetSupportCount()
```

GetSupportInformation(*nodeNo: int*) [source]

Get support information for the specified node.

Parameters:

nodeNo (*int*) – Node number.

Returns:

Returns a tuple consisting of support_type, list of release specifications.and list of spring specifications respectively.

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> stype, release, spring = staad_obj.Support.GetSupportInformation
```

GetSupportInformationEx(*nodeNo: int*) [source]

Get extended support information for the specified node.

Parameters:

nodeNo (*int*) – Node number.

Returns:

(supportNo, supportType, releaseSpec, springSpec)

- **supportNo**

  Support item number.

- **supportType**

  Support type code.

- **releaseSpec**

  List of release specifications. (= 1) or Fixed (= 0) or Spring (=-1) for [FX, FY, FZ, MX, MY, MZ]

- **springSpec**

  List of spring specifications. [KFX, KFY, KFZ, KMX, KMY, KMZ]

Return type:

tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Support.GetSupportInformationEx(1)
```

GetSupportName(*supportNo: int*) [source]

Get support string name.

Parameters:

supportNo (*int*) – Support item number.

Returns:

Support name.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Support.GetSupportName(2)
```

GetSupportNodes()

[source]

Get all supported node numbers.

Returns:

List of supported node numbers.

Return type:

list of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> nodes = staad_obj.Support.GetSupportNodes()
```

GetSupportType(*nodeNo: int*)

[source]

Get the support type for the specified node.

Parameters:

nodeNo (*int*) – Node number.

Returns:

Support type code.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Support.GetSupportType(1)
```

GetSupportUniqueID(*supportNo: int*)

[source]

Get unique ID GUID string for a support item.

Parameters:

supportNo (*int*) – Support item number.

Returns:

GUID string.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Support.GetSupportUniqueID(2)
```

RemoveElasticFooting(*supportid*) [source]

Remove elastic footing support for a specific support Id.

Parameters:

supportid (*int*) – Support reference ID.

Returns:

True if successful.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.Support.RemoveElasticFooting(3)
>>> print(status)
```

RemoveElasticFootingFromNode(*nodeid*) [source]

Remove elastic footing support from a specific node.

Parameters:

nodeid (*int*) – Node number.

Returns:

True if successful.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.Support.RemoveElasticFootingFromNode(2)
>>> print(status)
```

RemoveElasticMat(*supportid*) [source]

Remove elastic mat support for a specific support Id.

Parameters:

supportid (*int*) – Support reference ID.

Returns:

True if successful.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.Support.RemoveElasticMat(4)
>>> print(status)
```

RemoveElasticMatFromNode(*nodeid*) [source]

Remove elastic mat support from a specific node.

Parameters:

nodeid (*int*) – Node number.

Returns:

True if successful.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.Support.RemoveElasticMatFromNode(7)
>>> print(status)
```

RemovePlateMat(*supportId*) [source]

Remove plate mat support for a specific support Id.

Parameters:

supportId (*int*) – Plate mat support ID.

Returns:

True if successful.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.Support.RemovePlateMat(4)
>>> print(status)
```

RemovePlateMatFromPlate(*plateNo: int*) [source]

Remove plate mat support from a specific plate.

Parameters:

plateNo (*int*) – Plate number.

Returns:

True if successful.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.Support.RemovePlateMatFromPlate(56)
>>> print(status)
```

RemoveSupportFromNode(*NodeIDs: list*) [source]

Remove support from one or more nodes.

Parameters:

NodeIDs (*list of int*) – List of node numbers from which to remove the support.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Support.RemoveSupportFromNode([1, 2, 3])
```

SetSupportUniqueID(*supportNo: int, guid: str*) [source]

Set unique ID for a support item.

Parameters:

- **supportNo** (*int*) – Support item number.
- **guid** (*str*) – GUID string.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Support.SetSupportUniqueID(2, "supportId")
```

__init__(*staadObj*) [source]

View

Contents

- OSView

class openstaadpy.os_analytical.osview.**OSView** [source]

Bases: object

CloseActiveWindow() [source]

Closes the active graphic or table window, however there must be at least one graphic window remaining open. Note that window at position 1 can not be closed.

Returns:

Returns True if Window closed. Returns FALSE if Unsuccessful.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> output = staad_obj.View.CloseActiveWindow()
```

CopyPicture() [source]

Copy active view to clipbord and gives size of image in it's referance variables. It does not copy table views but works with Structure View, Graphs, 3d Rendered view.

Returns:

Returns a tuple containing xDim size of image in x direction (Length in Pixel) and yDim size of image in y direction (width in Pixel) respectively.

Return type:

Tuple

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.CopyPicture()
```

CreateNewViewForSelections() [source]

Creates a new view in new window for the selected objects displayed in the active window.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.CreateNewViewForSelections()
```

CreateNewViewForSelectionsEx(*windowOptions: int*) [source]

Creates a new view for the selected objects displayed in the active window based on specified options.

Parameters:

windowOptions (*int*) – 0 = Creates a new window for the view, 1 = Display the view in the active window (type - Long).

Returns:

Returns True Creation of new view is successful. Returns False Creation of new view is unsuccessful.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.CreateNewViewForSelectionsEx(1)
```

DetachView() [source]

Remove a view from the collection of saved views. The view to be removed must be open and the active window. Once the view is detached, the active window becomes <untitled>. To set the active window see the function OSViewUI::SetActiveWindow. Note that the active window view cannot be Whole Structure or untitled.

Returns:

Returns 1 if View successfully detached. Returns 0 if Unsuccessful Returns -1 if Generic Error

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.View.DetachView()
```

ExportView(*FileLocation: str, FileName: str, FileFormat: int, Overwrite: bool*) [source]

Used for exporting the information displayed in the active view window into a standard, graphical image format (e.g., Bitmap, JPEG, TIFF et.).

Parameters:

- **FileLocation** (*str*) – Location of the saved view file (Folder need to be present otherwise it will return -100) .
- **FileName** (*str*) – Name of the saved view file.
- **FileFormat** (*int*) – 0 = bmp, 1 = jpg, 2 = tga, 3 = tif - Create the view in the specific format.
- **Overwrite** (*bool*) –

  Boolean for provide option to overwrite an existing file.

  - True - Allow Overwrite
  - False - No overwrite

Returns:

Returns 1 if Export view is successful. Returns -1 if Generic Error.
Returns -100 if Invalid Argument. Returns -1003 if File already exist.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.ExportView(r"<folderPath>", "<fileName>", 1, True
```

GetApplicationDesktopSize()

[source]

Get the size of the application desktop.

Returns:

(width, height) of the application desktop.

Return type:

tuple of int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> width, height = staad_obj.View.GetApplicationDesktopSize()
```

GetBeamsInView(*nBeamList: list*)

[source]

Get Beams In View

Parameters:

nBeamList (*nBeamList*) – Collection of beam

Returns:

Returns number of beams present in view.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.GetBeamsInView([1, 2, 4])
```

GetInterfaceMode() [source]

This function returns the current visual mode in the STAAD.Pro environment.

Returns:

Returns 0 if Pre-processor or modeling mode. Returns 1 if Post-processing mode. Returns 2 if Interactive design mode for STAAD.etc interoperability. Returns 4 if Piping mode. Returns 5 if BEAVA (i.e., Bridge Deck) mode.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.GetInterfaceMode()
```

GetNoOfBeamsInView() [source]

Get No Of Beams In View

Returns:

Returns number of beams present in view.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.GetNoOfBeamsInView()
```

GetScaleCount() [source]

Returns the count of scales that are used in STAAD.Pro which can be read and set using the functions GetScaleValues and SetScaleValues.

Returns:

Returns Positive_Value Count of scales. Returns 0/Negative_Value Unsuccessful.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> scaleCount = staad_obj.View.GetScaleCount()
```

GetScaleValueByType(*scaleTypeId: int*) [source]

Obtain the value of the scale that is used to display a specified load or result diagram as defined in the Diagrams>Scales dialog depending upon the scale ID passed.

Parameters:

scaleTypeId (*int*) – The index of the required load or result type

Returns:

Returns value of scale type listed, in Base Units

Return type:

float

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> value = staad_obj.View.GetScaleValueByType(1)
>>> print(value)
```

GetScaleValues() [source]

Obtain the current set of scales used for displaying loads and results shown in the Diagrams > Scales dialog.

Returns:

Returns list of float type and size same as number of scale types. API gets all the values in this list. Scale type and ID in array will be as following:

ID	Type	Scale Items	Unit per length
0	Loads	Point Force	Force
1	Loads	Dist. Force	Force/length
2	Loads	Point Moment	Force*length
3	Loads	Dist. Moment	Force*length/length
4	Loads	Pressure	Force/length^2
5	Results	Bending Y	Force*length
6	Results	Bending Z	Force*length
7	Results	Shear Y	Force
8	Results	Shear Z	Force
9	Results	Axial	Force
10	Results	Torsion	Force*length
11	Results	Displacement	Length
12	Results	Beam Stress	Force/length^2
13	Results	Mode Shape	(none)

Return type:

list of float

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> list = staad_obj.View.GetScaleValues()
>>> print(list)
```

GetWindowCount()

[source]

Get the number of windows currently open. This includes both graphic windows and tables.

Returns:

Returns Positive_Number The count of open Window. Returns -1 if Error

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> count = staad_obj.View.GetWindowCount()
```

GetWindowTitle(*id: int*)

[source]

Returns the Title of the Window.

Parameters:

id (*int*) – The index of the required Window (Type: Long). Note that IDs start from 1. Window IDs depend on the creation time; that is the last created window will have the last ID. Windows which are listed under in View tab > Windows Dropdown are supported in this API.

Returns:

Returns the Window string title. Returns Empty_String Window id not found.

Return type:

str

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> title = staad_obj.View.GetWindowTitle(1)
```

HideAllMembers()

[source]

Hide all members in the STAAD view.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.HideAllMembers()
```

HideEntity(*nEntity: int*)

[source]

Hides the specified entity, which may be a Beam, Plate, Solid, or Surface.

Parameters:

nEntity (*int*) – Variable that holds an entity (i.e., Member, Plates etc.) number to be hidden.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.HideEntity(5)
```

HideMember(*IDMember*)

[source]

Hide a specific member in the STAAD view.

Parameters:

IDMember (*int*) – Member number to hide.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.HideMember(1)
```

HideMembers(*NMembers, NaMemberNos*) [source]

Hide specific members in the STAAD view.

Parameters:

- **NMembers** (*int*) – Number of members to hide.
- **NaMemberNos** (*list of int*) – List of member numbers to hide.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.HideMembers(2, [1, 2])
```

HidePlate(*nPlate: int*) [source]

Hide the specified plate.

Parameters:

nPlate (*int*) – Variable that holds plate number to be hidden.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.HidePlate(5)
```

HideSolid(*nSolid: int*) [source]

Hide the specified solid.

Parameters:

nSolid (*int*) – Variable that holds solid number to be hidden.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.HideSolid(5)
```

HideSurface(*nSurface: int*) [source]

Hide the specified surface.

Parameters:

nSurface (*int*) – Variable that holds surface number to be hidden.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.HideSurface(5)
```

OpenView(*viewName: str, windowOptions: bool*) [source]

Open a previously saved view in either the active window or create a new window which becomes the active window.

Parameters:

- **viewName** (*str*) – New name of the saved view
- **windowOptions** (*bool*) – False = Creates a new window for the view which becomes the active window, True / 1 = Display the view in the current active window

Returns:

Returns 1 if View Successfully opened. Returns 0 if Unsuccessful Returns 2 if View name does not exist. Returns -1 if Generic Error Returns -100 if Invalid Argument

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.View.OpenView("view1", True)
>>> print(status)
```

RefreshView()

[source]

Refresh the STAAD view window.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.RefreshView()
```

RenameView(*viewName: str*)

[source]

Renames a saved view. The view should be open and be the active window.

Parameters:

viewName (*str*) – New name of the saved view

Returns:

Returns 1 if Rename view is successful. Returns 0 if Unsuccessful
Returns 2 if View name already used. Returns -1 if Generic Error
Returns -100 if Invalid Argument

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.View.RenameView("view1")
```

RotateDown(*dDegrees: float*)

[source]

Rotates the structure through Degrees about the Global X-Axis.

Parameters:

dDegrees (*float*) – Variable providing the degree of rotation.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.RotateDown(30)
```

RotateLeft(*dDegrees: float*) [source]

Rotates the structure through Degrees about the Global Y-Axis.

Parameters:

dDegrees (*float*) – Variable providing the degree of rotation.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.RotateLeft(30)
```

RotateRight(*dDegrees: float*) [source]

Rotates the structure through Degrees about the Global Y-Axis.

Parameters:

dDegrees (*float*) – Variable providing the degree of rotation.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.RotateRight(30)
```

RotateUp(*dDegrees: float*) [source]

Rotates the structure through Degrees about the Global X-Axis.

Parameters:

dDegrees (*float*) – Variable providing the degree of rotation.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.RotateUp(30)
```

SaveView(*viewName: str, overWrite: bool*) [source]

Save the active graphic view to the collection of saved views which can be opened using the function OpenView.

Parameters:

- **viewName** (*str*) – New name for the view
- **overWrite** (*bool*) – Option to overwrite if the given viewName already exists. False = Do not overwrite., TRUE = Overwrite viewName if it exists.

Returns:

Returns 1 if Save view is successful. Returns 0 if Unsuccessful Returns 2 if View name already exist and overWrite is false. Returns -1 if Generic Error Returns -100 if Invalid Argument

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.View.SaveView("view1", True)
>>> print(status)
```

SelectByItemList(*entityType: int, nItems: int, itemList: list*)

Select entities as specified. [source]

Parameters:

- **entityType** (*int*) –

  Variable that holds entity type. Values may be as follows:

ID	Entity Type
1	Node
2	Beam
3	Plate
4	Solid
5	Surface

- **nItems** (*int*) – Variable that holds total number of entities needs to be selected.
- **itemList** (*list of int*) – List holds the entity nos, which need to be selected.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SelectByItemList(1, 2, [1, 2])
```

`SelectByMissingAttribute(attributeCode: int)` [source]

Select entity list for which specified entity is missing.

Parameters:

attributeCode (*int*) –

Variable that holds attribute type. Values may be as follows:

ID	Entity Type
1	Missing Property
2	Missing Modulus of Elasticity
3	Missing Density of Material
4	Missing Alpha (Coefficient of Thermal Expansion)
5	Missing Poisson Ratio

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SelectByMissingAttribute(5)
```

SelectEntitiesConnectedToMember(*entityType: int, memberNo: int*) [source]

Select entities as specified in type and connected with the specified Member.

Parameters:

- **entityType** (*int*) –

  Variable that holds entity type. Values may be as follows:

ID	Entity Type
0	Geometry
1	Beam
2	Plate
3	Solid

- **memberNo** (*int*) – Variable that holds Member numbers with which connected entities needs to be selected.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SelectEntitiesConnectedToMember(1, 2)
```

SelectEntitiesConnectedToNode(*entityType: int, nodeNo: int*)

Select entities as specified in type and connected with the specified node. [source]

Parameters:

- **entityType** (*int*) –

  Variable that holds entity type. Values may be as follows:

ID	Entity Type
0	Geometry
1	Beam
2	Plate
3	Solid

- **nodeNo** (*int*) – Variable that holds node numbers with which connected entities needs to be selected.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SelectEntitiesConnectedToNode(0, 1)
```

`SelectEntitiesConnectedToPlate(entityType: int, plateNo: int)`

Select entities as specified in type and connected with the specified Plate. [source]

Parameters:

- **entityType** (*int*) –

  Variable that holds entity type. Values may be as follows:

ID	Entity Type
0	Geometry
1	Beam
2	Plate
3	Solid

- **plateNo** (*int*) – Variable that holds Plate numbers with which connected entities needs to be selected.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SelectEntitiesConnectedToPlate(2, 3)
```

SelectEntitiesConnectedToSolid(*entityType: int, solidNo: int*)

[source]

Select entities as specified in type and connected with the specified Solid.

Parameters:

- **entityType** (*int*) –

  Variable that holds entity type. Values may be as follows:

ID	Entity Type
0	Geometry
1	Beam
2	Plate
3	Solid

- **solidNo** (*int*) – Variable that holds Solid numbers with which connected entities needs to be selected.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SelectEntitiesConnectedToSolid(3, 4)
```

SelectGroup(*bstrGroup: str*)

[source]

Select the relevant entities of the specified group.

Parameters:

bstrGroup (*str*) – A string variable that holds the group name.

Returns:

Returns True if successful Returns False if unsuccessful

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SelectGroup("grp_name")
```

SelectInverse(*entityType: int*) [source]

Inverse geometry selection for the specified entity.

Parameters:

entityType (*int*) –

Variable that holds entity type. Values may be as follows:

ID	Entity Type
1	Node
2	Beam
3	Plate
4	Solid
5	Surface

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SelectInverse(1)
```

SelectMembersParallelTo(*bstrAxis: str*) [source]

Select members parallel to the specified axis.

Parameters:

bstrAxis (*str*) –

Variable that holds the Axis ID. It may have three values:

ID	Axis
X	X-Axis
Y	Y-Axis
Z	Z-Axis

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SelectMembersParallelTo(5)
```

SetActiveWindow(*id: int*) [source]

Set a given window (active graphic or table window) with the provided id as the active window. The indexing starts from 1.

Parameters:

id (*int*) – The id of the window to be made the active window.

Returns:

Returns True if successful. Returns FALSE if unsuccessful.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SetActiveWindow(2)
```

`SetBeamAnnotationMode(Type: int, DWFlags: int, RefreshFlag: bool)` [source]

This function sets the current page mode in the STAAD.Pro environment.

Parameters:

- **Type** (*int*) –

  Variable controlling the annotation type. It may be one of the following values:

ID	Annotation Type
0	Axial Diagram
1	Torsion Diagram
2	Moment Diagram
3	Shear Diagram
4	Stress Diagram
5	Displacement Diagram

- **DWFlags** (*int*) –

  Variable controlling what values are to be shown for the annotationType. It may be one of the following values:

ID	Values
1	End Values
2	Max Absolute Values
3	Mid-span Values

- **RefreshFlag** (*int*) – Boolean variable (True or False). If True, STAAD.Pro viewing windows refresh with the current annotation mode.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SetBeamAnnotationMode(2, 1, True)
```

SetDesignResults(*utilization: int, color: bool, showValues: bool*) [source]

Sets Design Results to active view, this function replicates the setting of Design Results in the Diagrams>Design Results dialog.

Parameters:

- **Id** (*int*) – Value of type Long. (0 = None, 1 = Actual Ratio, 2 = Normalised Ratio.)
- **color** (*bool*) – Value of type Boolean. (False/0 = Basic Colored, True/1 = Detailed Colored.)
- **showValues** (*bool*) – Value of type Boolean. (False/0 = Do Not Show Values, True/1 = Show Values.)

Returns:

Returns 1 if Set Design Results is successful. Returns 0 if Unsuccessful. Returns -1 if Generic Error. Returns -2 if Design Results not Loaded. Returns -100 if Invalid Argument.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SetDesignResults(1, True, True)
```

SetDiagramMode(*which: int, showFlag: bool, refreshFlag: bool*)

Sets the label on the structure diagram on or off. [source]

Parameters:

- **which** (*int*) –

  Variable identifying the diagram type. It may be one of the following values

ID	Diagram Type
0	Load
1	Displacement
2	MY
3	MZ
4	FY
5	FZ
6	AX
7	TR
8	Structure
9	Full Section
10	Section Outline
11	Stress
12	Shrink
13	Perspective
14	Hide Structure
15	Fill Plates & Solids
16	Hide Plates & Solids
18	Hide Piping
19	Sort Geometry
20	Sort Nodes
21	Plate Stress
22	Solid Stress
23	Mode Shape

ID	Diagram Type
24	Stress Animation
25	Plate reinforcement
26	Deck Influence Diagram*
27	Deck Carriageways*
28	Deck Triangulation*
29	Deck Loads*
30	Deck Vehicles*
	(*) Requires the STAAD.beava component

- **showFlag** (*bool*) – Variable to set label mode on (True) or off (False).
- **refreshFlag** (*bool*) – Variable (True or False). If True, STAAD.Pro viewing windows refresh.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SetDiagramMode(1, True, True)
```

`SetInterfaceMode(interfaceMode: int)` [source]

This function sets the current visual mode in the STAAD.Pro environment.

Parameters:

interfaceMode (*int*) –

Variable to set the current visual mode in STAAD.Pro environment.

Followings are the valid values for mode:

ID	Mode Type
0	Pre-processor or modeling mode
1	Physical modeling mode
2	Building planner mode
3	Piping mode
5	Post Processing mode
6	FoundationDesign mode
7	ConnectionDesign mode
9	AdvancedConcreteDesign mode
10	AdvancedSlabDesign mode
11	Earthquake mode
12	SteelAutoDrafter mode
13	ChineseSteelDesign mode

Returns:

Returns 0 if Pre-processor or modeling mode. Returns 1 if Post-processing mode. Returns 2 if Interactive design mode for STAAD.etc interoperability. Returns 4 if Piping mode. Returns 5 if BEAVA (i.e., Bridge Deck) mode.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SetInterfaceMode(1)
```

`SetLabel(which: int, showFlag: bool)` [source]

Sets the label on the structure diagram on or off.

Parameters:

- **which** (*int*) –

  Variable identifying the diagram type. It may be one of the following values:

ID	Label Type
0	Node number label
1	Member number label
2	Member property reference label
3	Material property reference label
4	Support label
5	Member release label
6	Member orientation label
7	Member section label
8	Load value label
9	Axes label
10	Node position label
11	Member specification label
12	Member ends
13	Plate element number label
14	Plate element orientation label
15	Solid element number label
16	Dimension label
17	Floor load label
18	Floor load distribution diagram label
19	Wind load label
20	Wind load influence area diagram label
21	Diagram Info

- **showFlag** (*bool*) – Variable to set label mode on (True) or off (False).

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SetLabel(20, True)
```

`SetModeSectionPage(interfaceMode: int, sectionNumber: int, pageNumber: int)` [source]

This function sets the current page mode in the STAAD.Pro environment.

Parameters:

- **interfaceMode** (*int*) –

  Variable to set the current visual mode in STAAD.Pro environment. Followings are the valid values for mode:

ID	Interface Mode
0	Pre-processor or modeling mode
1	Post-processing mode
2	Interactive design mode for STAAD.etc interoperability
4	Piping mode
5	BEAVA (i.e., Bridge Deck) mode

- **sectionNumber** (*int*) –

  Variable to set the current main page (the tabs on the left-hand side of the screen) in the STAAD.Pro environment. The following are valid values for section:

ID	Main Page
1	Setup page
2	Geometry page
3	General page
5	Node Results page
6	Beam Result page
7	Plate Results page
8	Solid Results page

- **pageNumber** (*int*) –

  Variable to set the current sub page (within a particular main page - the tabs on the left-hand side of the screen) in the STAAD.Pro environment. The following are valid values for modeSubPage:

ID	Page Number
0	Job Info page
1	Beam page
4	Plate page
5	Solid page
6	Property page
7	Constant page
8	Material page
9	Support page
10	Member Specifications page
11	Load page
17	Reaction page
18	Displacement page
19	Failure page
20	Forces page
21	Beam Stress page
22	Plate Stress page
23	Solid Stress page

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SetModeSectionPage(1,6,20)
```

SetNodeAnnotationMode(*dFlag: bool, refreshFlag: bool*) [source]

Sets the node displacement annotation mode. This function works only in the post-processing mode of STAAD.Pro.

Parameters:

- **dFlag** (*bool*) –

  Variable controlling the annotation type. It may be one of the following values:

ID	Annotation Type
1	X Displacement
2	Y Displacement
3	Z Displacement
4	Resultant Displacement

- **refreshFlag** (*bool*) – Variable (True or False). If True, STAAD.Pro viewing windows refresh with the current annotation mode.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SetNodeAnnotationMode(1, True)
```

SetReactionAnnotationMode(*dFlag: bool, refreshFlag: bool*)

Sets the node displacement annotation mode. This function works [source] only in the post-processing mode of STAAD.Pro.

Parameters:

- **dFlag** (*bool*) –

  Variable controlling the annotation type. It may be one of the following values:

ID	Annotation Type
1	X Reaction
2	Y Reaction
3	Z Reaction
4	X Rotation
5	Y Rotation
6	Z Rotation
7	Reaction Value Only

- **refreshFlag** (*bool*) – Variable (True or False). If True, STAAD.Pro viewing windows refresh with the current annotation mode.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SetReactionAnnotationMode(1, True)
```

SetScaleValueByType(*scaleTypeId: int, value: float*) [source]

Set the scale used for displaying a chosen load or result diagram as shown in the Diagrams>Scales dialog.

Parameters:

- **scaleTypeId** (*int*) – The index of the required load or result type to be set.
- **value** (*float*) – Value of scale type to be used.

Returns:

Returns 1/TRUE Value was successfully updated. Returns 0/FALSE Value could not be updated.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> retValue = staad_obj.View.SetScaleValueByType(1, 1.2)
```

SetScaleValues(*ScalesList: list*) [source]

Set the scales used for displaying loads and results as shown in the Diagrams>Scales dialog.

Parameters:

ScalesList (*list of float*) – List of float type and size same as number of scale types. API sets the values in this list to scale types

Returns:

Returns 1 if Values were successfully updated. Returns 0 if Values could not be updated. Returns -1 if Error with defined array.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SetScaleValues([1.0, 2.5, 3.2])
```

SetSectionView(*plane: int, minVal: float, maxVal: float*)

Creates a section view of the structure. [source]

Parameters:

- **plane** (*int*) –

  Variable identifying the section plane. It may be one of the following values:

ID	Values for plane
0	XY Plane
1	YZ Plane
2	XZ Plane

- **minVal** (*float*) – Minimum range of the cutting plane.
- **maxVal** (*float*) – Maximum range of the cutting plane.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SetSectionView(1, 0.4, 0.6)
```

`SetUnits(uType: int, strUnit: str)` [source]

Set viewing unit for the active view.

Parameters:

- **uType** (*int*) –

  Variable that holds unit type. Values are as follows:

ID	Unit Type
0	Dimension
1	Displacement
2	SectionDimension
3	SectionArea
4	Inertia
5	Force
6	Moment
7	DistributedForce
8	DistributedMoment
9	Density
10	Acceleration
11	Spring
12	RotSpring
13	MaterialModulus
14	Stress
15	Alpha
16	Temperature
17	Mass
18	SectionModulus
19	RotationalDisplacement
20	SubgradeModulus
-1	NoUnit

- **strUnit** (*str*) – Variable array that holds the unit for the specified type. Like “cm”, “kns”, “feet”, “kn/cm” etc.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SetUnits(0, "cm")
```

SetWindowPosition(*xTop, yTop, xWindow, yWindow*) [source]

Set the position and size of the STAAD application window.

Parameters:

- **xTop** (*int*) – X coordinate of the top-left corner.
- **yTop** (*int*) – Y coordinate of the top-left corner.
- **xWindow** (*int*) – Width of the window.
- **yWindow** (*int*) – Height of the window.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SetWindowPosition(100, 100, 800, 600)
```

ShowAllMembers() [source]

Show all members in the STAAD view.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.ShowAllMembers()
```

ShowBack() [source]

Set the view to the back orientation.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.ShowBack()
```

ShowBottom() [source]

Set the view to the bottom orientation.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.ShowBottom()
```

ShowFront() [source]

Set the view to the front orientation.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.ShowFront()
```

ShowIsometric() [source]

Set the view to isometric orientation.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.ShowIsometric()
```

ShowLeft() [source]

Set the view to the left orientation.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.ShowLeft()
```

ShowMember(*nMember: int*) [source]

Show the specified member.

Parameters:

nMember (*int*) – Variable that holds member number to be shown.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.ShowMember(5)
```

ShowMembers(*NMembers, NaMemberNos*) [source]

Show specific members in the STAAD view.

Parameters:

- **NMembers** (*int*) – Number of members to show.
- **NaMemberNos** (*list of int*) – List of member numbers to show.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.ShowMembers(2, [1, 2])
```

ShowPlan() [source]

Set the view to the plan (top) orientation.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.ShowPlan()
```

ShowRight()

[source]

Set the view to the right orientation.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.ShowRight()
```

SpinLeft(*Degrees*)

[source]

Spin the view to the left by a specified number of degrees.

Parameters:

Degrees (*float or int*) – Number of degrees to spin left.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SpinLeft(15)
```

SpinRight(*Degrees*)

[source]

Spin the view to the right by a specified number of degrees.

Parameters:

Degrees (*float or int*) – Number of degrees to spin right.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.SpinRight(15)
```

ZoomAll()

[source]

Zoom to show all objects in the STAAD view.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.ZoomAll()
```

ZoomExtentsMainView()

[source]

Zoom to extents in the main STAAD view.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.View.ZoomExtentsMainView()
```

__init__(*staadObj*)

[source]

Command

Contents

- OSCommand

class openstaadpy.os_analytical.oscommand.**OSCommand** [source]

Bases: object

CreateSteelDesignCommand(*NDesignCode: int, NCommandNo: int, IntValues: list[int], FloatValues: list, StringValues: list, NAssignList: list[int]*) [source]

Creates a steel design command for the specified design code and command number.

Parameters:

NDesignCode (*int*) –
Index of the design code to be used. Supported codes include:

Code Index	Description
1001	AASHTO ASD
1002	AISC ASD
1003	AS 4100-1998
1004	BS 5950-1:2000
1005	BS 5400:Part 3:1982
1006	CAN/CSA-S16-01
1007	French CM66 1977
1008	DIN 18 800 Part 1
1009	IS 800 1984, ASD
1010	Japan AIJ 2002
1011	AISC LRFD
1012	Norway NS 3472 2001
1014	Norway NPD 1993
1016	API 2A-WSD
1020	ASCE 10-97
1025	Russia SNiP 2.23-81* 1990
1027	Canada S136-94
1028	IS 801 1975
1029	IS 802 1995
1030	Mexico NTC 1987
1032	IS 800 2007, LSD
1052	IS 800 2007, WSD
1033	BS 5950-5:1998

Code Index	Description
1034	South Africa SANS 10162-1:2011
1044	AASHTO LRFD
1045	ANSI/AISC 360-05
1046-1053	ASME NF 3000 (various years)
1060	ASCE 52
1061	ANSI/AISC 360-10
1062	Canadian S16_09
1063	Russia SP 16.13330.2011
1064	South Africa SANS10162-1:1993
1065	Canadian S16_14
1066	NZS3404_1997
1067	ANSI/AISC 360-16
1068	AISI S100-2016
1069	Canadian S16_19
1102	AISC CASTELLATED
1202	AISC N690 1994
1204	AISC N690 1984
1210	Japan AIJ 2005
1220	BS EN 1993-1-1:2005
1221	Russia SP 16.13330.2017

NCommandNo : *int*

Index of the specific design command to be applied.

- **Table 1/10:**

Design Parameter	NCommandNo		Design Parameter	NComma
AISC 360-05, ** **360-10,and 360-16			**AISC LRFD (2nd and 3 Ed.)**	
CODE	9010		CODE	9010
ALH	9674		AXIS	9892
BEAM	9380		BEAM	9380
BRC	9955		CAN	9883
CAN	9883		CB	9280
CB	9280		DFF	9210
CSPACING	9103		DJ1	9390
DFF	9210		DJ2	9400
DJ1	9390		DMAX	9160
DJ2	9400		DMIN	9170
DMAX	9160		FLX	9781
DMIN	9170		FYLD	9100
DUCT	9964		FU	9705
E5P	9133		KX	9235
FLX	9781		KY	9240
FRM	9954		KZ	9250
FU	9705		LX	9125
FYLD	9100		LY	9130
IMM	9877		LZ	9140
KX	9235		MAIN	9330

Design Parameter	NCommandNo		Design Parameter	NComma
INT	9957		NSF	9260
KY	9240		PROFILE	9520
KZ	9250		RATIO	9360
LBRC	9968		STIFF	9200
LEG	9500		STP	9894
LX	9125		TMAIN	9335
LY	9130		TRACK	9350
LZ	9140		UNB	9660
MAIN	9330		UNT	9650
METHOD	9923			
MTYP	9956			
NBRC	9972			
NSF	9260			
PROFILE	9520			
RATIO	9360			
SEISMIC	9953			
SGR	9460			
SLF	9922			
SNUG	9102			
SOE	9975			
SRT	9131			
STB	9973			

Design Parameter	NCommandNo		Design Parameter	NComma
STT	9974			
STIFF	9200			
STP	9894			
TBRC	9969			
TFA	9976			
TSL	9132			
TMAIN	9335			
TND	9959			
TORSION	9670			
TRACK	9350			
UNB	9660			
UNL	9150			
UNR	9970			
UNT	9650			
WTYPE	9958			

- **Table 2/10:**

Design Parameter	NCommandNo		Design Parameter	NComma
AISI 2016			**API 2A-WSD**	
CODE	9010		CODE	9010
AXIS	9892		BEAM	9380
BEAM	9380		CB	9280
CAN	9883		CMY	9310
DFF	9210		CMZ	9320
DJ1	9390		DMAX	9160
DJ2	9400		DMIN	9170
FLX	9781		FSJ	9503
FU	9705		FYLD	9100
FYLD	9100		KY	9240
KT	9805		KZ	9250
KY	9240		LY	9130
KZ	9250		LZ	9140
LT	9806		MAIN	9330
LY	9130		NSF	9260
LZ	9140		RATIO	9360
METHOD	9923		SSY	9290
NSF	9260		SSZ	9300
RATIO	9360		TRACK	9350
SSY	9290		UNF	9270
SSZ	9300		UNL	9150

Design Parameter	NCommandNo		Design Parameter	NComma
STIFF	9200		WELD	9370
TRACK	9350		WMIN	9180
TSA	9782		WSTR	9110
UNB	9660			
UNT	9650			

- **Table 3/10:**

Design Parameter	NCommandNo		Design Parameter	NComm
ANSI/AISC N690 1994			**ANSI/AISC N690 19**	
CODE	9010		CODE	9010
BEAM	9380		CAN	9883
CAN	9883		CB	9280
CB	9280		CMY	9310
CMY	9310		CMZ	9320
CMZ	9320		CT	9924
COMPOSITE	9710		DFF	9210
CONDIA	9715		DJ1	9390
CONHEIGHT	9720		DJ2	9400
CYCLES	9725		DMAX	9160
DFF	9210		DMIN	9170
DJ1	9390		FU	9705
DJ2	9400		FYLD	9100
DLR2	9735		KY	9240
DLRATIO	9730		KZ	9250
DMAX	9160		LY	9130
DMIN	9170		LZ	9140
EFFWIDTH	9740		MAIN	9330
FYLD	9100		NSF	9260
FPC	9745		PROFILE	9520
FSS	9882		RATIO	9360

Design Parameter	NCommandNo		Design Parameter	NComn
FU	9705		SFC	9942
FYLD	9100		SFT	9943
KX	9235		SMY	9945
KY	9240		SMZ	9944
KZ	9250		STIFF	9200
LX	9125		STYPE	9101
LY	9130		TMAIN	9335
LZ	9140		TRACK	9350
MAIN	9330		UNB	9660
NSF	9260		UNT	9650
OVR	9880			
PLTHICK	9755			
PLTWIDTH	9760			
PROFILE	9520			
RATIO	9360			
RIBHEIGHT	9765			
RIBWIDTH	9770			
SFC	9942			
SFT	9943			
SHE	9893			
SHORING	9775			
SLABTHICK	9780			

Design Parameter	NCommandNo		Design Parameter	NComn
SMY	9945			
SMZ	9944			
SSY	9290			
SSZ	9300			
STIFF	9200			
STYPE	9101			
TAPER	9868			
TMAIN	9335			
TORSION	9670			
TRACK	9350			
UNB	9660			
UNT	9650			
WELD	9370			
WMAX	9881			
WMIN	9180			
WSTR	9110			

- **Table 4/10:**

Design Parameter	NCommandNo		Design Parameter	NComma
BS 5950			**BS 5400**	
CODE	9010		CODE	9010
AD	9902		ESTIFF	9690
BEAM	9381		KY	9240
CAN	9883		KZ	9250
CB	9281		LY	9130
DFF	9210		LZ	9140
DJ1	9390		MAIN	9332
DJ2	9400		NSF	9260
DMAX	9160		PY	9120
DMIN	9170		RATIO	9360
ESTIFF	9690		SBLT	9472
KY	9240		SGR	9464
KZ	9250		TRACK	9352
LEG	9501		UNL	9150
LVV	9190		WET	9510
LY	9130			
LZ	9140			
MLT	9820			
MX	9840			
MY	9850			
MYX	9830			

Design Parameter	NCommandNo		Design Parameter	NComma
NSF	9260			
PNL	9700			
PY	9120			
MAIN	9331			
RATIO	9360			
SAME	9870			
SBLT	9472			
SWAY	9860			
SGR	9464			
TB	9530			
TRACK	9357			
UNF	9270			
UNL	9150			
WELD	9371			

- **Table 5/10:**

Design Parameter	NCommandNo		Design Parameter	NComma
Canada S136-94			**French**	
CODE	9010		CODE	9010
BEAM	9380		BEAM	9385
CMZ	9790		C1	9550
CMY	9795		C2	9560
CWY	9783		DFF	9210
DMAX	9160		DJ1	9390
DMIN	9170		DJ2	9400
FLX	9781		DMAX	9160
FU	9705		DMIN	9170
FYLD	9100		FYLD	9100
KT	9805		KY	9240
KY	9240		KZ	9250
KZ	9250		LY	9130
LT	9806		LZ	9140
LY	9130		NSF	9260
LZ	9140		RATIO	9360
NSF	9260		SAME	9870
STIFF	9200		TRACK	9355
TRACK	9350		UNF	9270
TSA	9782		UNL	9150

Design Parameter	NCommandNo		Design Parameter	NComma

- **Table 6/10:**

Design Parameter	NCommandNo		Design Parameter	NComma
Indian 800 1984			**Indian 801 1975**	
CODE	9010		CODE	9010
BEAM	9380		BEAM	9380
CMY	9310		CB	9280
CMZ	9320		CMY	9310
DFF	9210		CMZ	9320
DJ1	9390		CWY	9783
DJ2	9400		FLX	9781
DMAX	9160		FU	9705
DMIN	9170		FYLD	9100
FYLD	9100		KX	9235
KY	9240		KY	9240
KZ	9250		KZ	9250
LY	9130		LX	9125
LZ	9140		LY	9130
MAIN	9334		LZ	9140
NSF	9260		NSF	9260
PROFILE	9520		RATIO	9360
RATIO	9360		TRACK	9350
SSY	9290		TSA	9782
SSZ	9300		UNL	9270
TMAIN	9335			

Design Parameter	NCommandNo		Design Parameter	NComma
TRACK	9350			
UNF	9270			
UNL	9150			

- **Table 7/10:**

Design Parameter	NCommandNo		Design Parameter	NComma
Japanese AIJ 2002			**Mexian**	
CODE	9010		CODE	9010
BEAM	9380		BEAM	9380
CAN	9883		CB	9280
CB	9280		CMB	9878
DFF	9210		DFF	9210
DJ1	9390		DJ1	9390
DJ2	9400		DJ2	9400
DMAX	9160		DMAX	9160
DMIN	9170		DMIN	9170
FYLD	9100		DSD	9879
KY	9240		FU	9705
KZ	9250		FYLD	9100
LY	9130		IMM	9877
LZ	9140		INO	9876
MAIN	9336		IRR	9875
MBG	9823		KX	9235
MISES	9819		KY	9240
NSF	9260		KZ	9250
RATIO	9360		LDR	9874
SLF	9922		LX	9125

Design Parameter	NCommandNo		Design Parameter	NComma
TMAIN	9335		LY	9130
TRACK	9358		LZ	9140
UNF	9270		NSF	9260
UNL	9150		RATIO	9360
YNG	9824		STIFF	9200
			TRACK	9350
			UNB	9660
			UNT	9650

- **Table 8/10:**

Design Parameter	NCommandNo		Design Parameter	NComma
Norwegian NORSOK N-004			**Russian SNip 81**	
CODE	9010		CODE	9010
BEAM	9380		BEAM	9380
CMY	9310		CB	9281
CMZ	9320		CMM	9494
DFF	9210		CMN	9494
DJ1	9390		DFF	9210
DJ2	9400		DMAX	9160
DMAX	9160		DMIN	9170
DMIN	9170		GAMC1	9809
FYLD	9100		GAMC2	9810
FU	9705		GMF	9977
HYD	9940		KY	9240
KY	9240		KZ	9250
KZ	9250		LEG	9502
LX	9125		LY	9130
LY	9130		LZ	9140
LZ	9140		MAIN	9333
MAIN	9330		NSF	9260
PSD	9941		PY	9121
RATIO	9360		RATIO	9360
SGR	9466		SBLT	9471

Design Parameter	NCommandNo		Design Parameter	NComma
TMAIN	9335		SGR	9463
TRACK	9351		TB	9952
			TRACK	9356
			UNL	9151

- **Table 9/10:**

Design Parameter	NCommandNo		Design Parameter	NComma
EC3			**Australian**	
CODE	9010		CODE	9010
ALH	9674		ALB	9804
ALPHA	9926		ALM	9803
BEAM	9382		BEAM	9380
BETA	9382		DFF	9210
C1	9550		DJ1	9390
C2	9560		DJ2	9400
C3	9565		DMAX	9160
CAN	9883		DMIN	9170
CMM	9482		FU	9705
CMN	9492		FYLD	9100
CMT	9671		IST	9800
DFF	9210		KT	9805
DJ1	9390		KY	9240
DJ2	9400		KZ	9250
DMAX	9160		LHT	9938
DMIN	9170		LX	9125
EFT	9673		LY	9130
ELB	9697		LZ	9140
ESTIFF	9690		MAIN	9330
FAB	9508		NSC	9802

Design Parameter	NCommandNo		Design Parameter	NComma
FU	9705		NSF	9260
GM0	9504		PBRACE	9939
GM1	9505		PHI	9801
GM2	9506		RATIO	9360
GST	9676		SGR	9466
HGT	9720		SKL	9808
KC	9696		SKR	9809
KY	9240		SKT	9807
KZ	9250		TMAIN	9335
LEG	9501		TRACK	9350
LVV	9190		TSP	9932
LY	9130		UNB	9660
LZ	9140		UNT	9650
MTH	9677			
MU	9695			
NA	9465			
NSF	9260			
PLG	9930			
PY	9120			
RATIO	9360			
SBLT	9472			
SGR	9461			

Design Parameter	NCommandNo		Design Parameter	NComma
STIFF	9202			
TOM	9672			
TORSION	9688			
TRACK	9353			
UNF	9270			
UNL	9150			
ZG	9507			

- **Table 10/10:**

Design Parameter	NCommandNo		Design Parameter	NComma
Russian SP 2011			**Russian SP 2017**	
CODE	9010		CODE	9010
BEAM	9380		BMT	9951
BMT	9951		CB	9281
CB	9281		CMM	9494
CMM	9494		CMN	9494
CMN	9494		DFF	9210
DFF	9210		DJ1	9390
DJ1	9390		DJ2	9400
DJ2	9400		DMAX	9160
DMAX	9160		DMIN	9170
DMIN	9170		ENSGR	9818
ENMAIN	9817		FU	9705
ENSGR	9818		GAMC1	9809
GAMC1	9809		GAMC2	9810
GAMC2	9810		GAMM	9978
GAMM	9815		GMF	9977
GMF	9977		KY	9240
KY	9240		KZ	9250
KZ	9250		LEG	9502
LEG	9502		LY	9130
LY	9130		LZ	9140

Design Parameter	NCommandNo		Design Parameter	NComma
LZ	9140		NSF	9260
MAIN	9333		PY	9121
NSF	9260		SBLT	9471
PY	9121		STP	9894
RATIO	9360		TB	9952
SBLT	9471		TRACK	9356
SGR	9463			
TB	9952			
TRACK	9356			
UNL	9151			

IntValues : *list of int*

Integer parameter values for the design command.

FloatValues : *list of float*

Floating-point parameter values for the design command.

StringValues : *list of str*

String parameter values for the design command.

NAssignList : *list*

List of reference IDs of objects to which the design command should be assigned.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Command.CreateSteelDesignCommand(1, 9010, [9380, 9310,
```

DeleteAllAnalysisCommands() [source]

Delete all analysis commands from the current STAAD model.

Returns:

Status code ‘1’ for OK and ‘0’ if failed to delete.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Command.DeleteAllAnalysisCommands()
>>> print(result)
```

DeleteCheckIrregularitiesCommand() [source]

Delete the check irregularities command.

Returns:

Status code ‘1’ for OK and ‘0’ if failed to delete.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Command.DeleteCheckIrregularitiesCommand()
>>> print(result)
```

DeleteCheckSoftStoryCommand() [source]

Delete the check soft story command.

Returns:

Status code ‘1’ for OK and ‘0’ if failed to delete.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Command.DeleteCheckSoftStoryCommand()
>>> print(result)
```

DeleteFloorDiaphragmBaseCommand() [source]

Delete the floor diaphragm base command.

Returns:

Status code ‘1’ for OK and ‘0’ if failed to delete.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.Command.DeleteFloorDiaphragmBaseCommand()
>>> print(status)
```

PerformAnalysis(*printOption: int*) [source]

Perform a standard analysis on the current STAAD model.

Parameters:

printOption (*int*) –

The print option for the analysis results. Possible values are:

Value	Print Option
1	Print Load Data
2	Print Statics Check
3	Print Statics Load
4	Print Mode Shapes
5	Print Both
6	Print All
0	No Print

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Command.PerformAnalysis()
```

`PerformBucklingAnalysis(MaxNoOfIterations: int, PrintOption: int)` [source]

Creates the commands required to perform BUCKLING ANALYSIS.

Parameters:

- **iNoOfIterations** (*int*) – Maximum number of iterations desired.
- **iPrintOption** (*int*) –

  Option for specifying the print output. Choose from the following:

Value	Print Option
1	Print Load Data
2	Print Statics Check
3	Print Statics Load
5	Print Both
6	Print All
0	No Print

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Command.PerformBucklingAnalysis(5, 6)
```

`PerformBucklingAnalysisEx(Method: int, MaxNoOfIterations: int, PrintOption: int)` [source]

Assigns the commands required to perform a BUCKLING ANALYSIS on the model. This requires the presence of nonlinear cables in the structure.

Parameters:

- **Method** (*int*) –

  Buckling Analysis method to use.

Value	Method
0	BucklingAnalysisMethod.Iterative
1	BucklingAnalysisMethod.Eigen

- **MaxNoOfIterations** (*int*) – Maximum number of iterations desired. Default is 10; 15 is recommended. This is used only for the Basic Solver.

- **PrintOption** (*int*) –

  Option index for specifying the print output. Choose from the following:

Value	Print Option
1	Print Load Data
2	Print Statics Check
3	Print Statics Load
4	Reserved (do not use)
5	Print Both
6	Print All
0	No Print

Returns:

True if the command was successfully added or updated, *False* otherwise.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Command.PerformBucklingAnalysisEx(0, 15, 6)
```

PerformCableAnalysis(*NoOfIterations: int, PrintOption: int*)

Creates the commands required to perform a CABLE ANALYSIS. This requires the presence of cable members in the structure. [source]

> **Note**
>
> The advanced algorithm will be used only if an Advanced license is enabled. For further details, refer to TR.37.3 of the STAAD.Pro Help manual.

Parameters:

- **iNoOfIterations** (*int*) – Desired number of iterations.
- **iPrintOption** (*int*) –

  Option for specifying the print output. Choose from the following:

Value	Print Option
1	Print Load Data
2	Print Statics Check
3	Print Statics Load
5	Print Both
6	Print All
0	No Print

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Command.PerformCableAnalysis(25, 6)
```

`PerformCableAnalysisEx(AdvancedCableAnalysis: int, AdvOptions: list, Params: list, PrintOption: int)` [source]

Assigns the commands required to perform a CABLE ANALYSIS on the model. This requires the presence of nonlinear cables in the structure.

Parameters:

- **AdvancedCableAnalysis** (*int*) – Whether to perform Advanced Cable Analysis (*True* = 1 or *False* = 0).
- **AdvOptions** (*list of int*) –

  Additional options for Advanced Cable Analysis.

Array Index	Default	STAAD Command	Description
0	1	REFORM f11	Use Full Newton-Raphson method? 0 = False, 1 = True
1	0	KGEOM f12	Use Geometric Matrix (Kg)? 0 = False, 1 = True

  Notes: - If array size = 1, second value defaults to *False*. - If array size > 2, only the first two values are used.
- **Params** (*list of float*) –

  Additional parameters required for Cable Analysis. Some values represent integer flags.
  For **Basic Cable Analysis**:

Array Index	Default Value	STAAD Command	Description
0	145	STEPS f1	The number of load steps. The applied loads will be applied gradually in this many steps. Each step will be iterated to convergence. Should be in the range 5 to 145, with 145 as the default number.
1	300	EQITERATIONS f2	Maximum number of iterations permitted in each load step. Should be in the range of 10 to 500
2	1.0E-4	EQTOLERANCE f3	The convergence tolerance for the above iterations
3	0.0	SAGMINIMUM f4	Sag Minimum. Default is 0.0
4	1.0	STABILITY f5	A stiffness matrix value to be added to the global matrix at each translational direction for joints connected to cables and nonlinear trusses for the first (f6) Load Steps. Should be within the range of 0.0 to 1000.0
5	1	f6	The number of load steps over which the Stability stiffness matrix(f5) is gradually applied
6	0.0	KSMALL f7	A stiffness matrix value to be added to the global matrix at each translational

Array Index	Default Value	STAAD Command	Description
			direction for joints connected to cables and nonlinear trusses for every load step. Should be within the range of 0.0 and 1.0

For **Advanced Cable Analysis**:

Array Index	Default Value	STAAD Command	Description
0	1	STEPS f1	Number of load increments.
1	300	EQITERATIONS f2	Max iterations per increment.
2	1.0E-6	EQTOLERANCE f3	Convergence tolerance for residual force norm.

- **PrintOption** (*int*) –

  Option index for specifying the print output. Choose from the following:

Value	Print Option
1	Print Load Data
2	Print Statics Check
3	Print Statics Load
5	Print Both
6	Print All
0	No Print

Returns:

True if the command was successfully added or updated, *False* otherwise.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Command.PerformCableAnalysisEx(1, [0, 1], [12
>>> print(result)
```

`PerformDirectAnalysis(Option: int, Params: list, AddOptions: list, PrintOption: int)` [source]

Assigns the commands required to perform a DIRECT ANALYSIS for AISC on the model.

Parameters:

- **Option** (*DirectAnalysisOption*) –

  Direct Analysis method. Default is *DirectAnalysisOption.LRFD*.

Value	Method
1	DirectAnalysisOption.LRFD
2	DirectAnalysisOption.ASD

- **Params** (*list of float*) –

  Additional parameters required for Direct Analysis. Some values represent integer flags.

Array Index	Default Value	STAAD Command	Description
0	0.01	TAUTOL f1	Tau-b tolerance (typically 0.001 to 1.0).
1	0.01	DISPTOL f2	Displacement tolerance: - 0.01 in (displacement) - 0.01 radians (rotation) The value should not be too tight
2	1	ITERDIRECT i3	Max number of iterations (typically 1 to 10).
3	15	PDiter i5	Iterations for PDelta with SmallDelta (5 to 15) (max 15).

- **AddOptions** (*list of int*) –

  Additional boolean options for Direct Analysis.

Array Index	Default	STAAD Command	Description
0	0	REDUCEDEI i4	Use reduced EI (Tau-b * 0.8 * EI)? 0 = False, 1 = True
1	0	TBITER	Iterate Tau-b? 0 = False, 1 = True

  Notes: - If array size = 1, second value defaults to *False*. - If array size > 2, only the first two values are used.

- **PrintOption** (*int*) –

  Option index for specifying the print output. Choose from the following:

Value	Print Option
1	Print Load Data
2	Print Statics Check
3	Print Statics Load
4	Print Mode Shapes
5	Print Both
6	Print All
0	No Print

Returns:

True if the command was successfully added or updated, *False* otherwise.

Return type:

bool

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Command.PerformDirectAnalysis(1, [0.015, 0.02
>>> print(result)
```

PerformNonlinearAnalysisEx(*PrintOption: int, Arclength: float, NoOfIterations: int, Tolerance: float, Steps: int, Rebuild: int, AddGeometricStiffness: int, DispLimitData: list*) [source]

Assigns the commands required to perform NONLINEAR ANALYSIS of the current structure with the specified print option.

Parameters:

- **PrintOption** (*int*) –

  Option index for specifying the print output. Choose from the following:

Value	Print Option
1	Print Load Data
2	Print Statics Check
3	Print Statics Load
4	Print Mode Shapes
5	Print Both
6	Print All
0	No Print

- **Arclength** (*float*) – Absolute displacement limit for the first analysis step for Displacement Control. Set to 0 for no Displacement Control.
- **NoOfIterations** (*int*) – Maximum number of iterations to achieve equilibrium in the deformed position within the specified tolerance.
- **Tolerance** (*float*) – Tolerance value for determining convergence.
- **Steps** (*int*) – Number of load steps. The load is applied incrementally in these stages.
- **Rebuild** (*int*) –

  Frequency of rebuilding the Tangent or Stiffness Matrix (K) per load step and iteration:

  - 0 : Once per load step
  - 1 : Every load step and iteration
- **AddGeometricStiffness** (*int*) – Whether to add the geometric stiffness matrix to the stiffness matrix (K). Set to *1* to include, or *0* to exclude.
- **varDispLimitData** (*list of float*) –

  Displacement limit data to specify the target displacement. The array should follow this structure:

Array Index	Default	STAAD Command	Description
0	1	JOINT_TARGET i1	Joint being monitored in a displacement target analysis
1	1	i2	Global degree of freedom (1–6): 1: Global X, 2: Global Y, 3: Global Z, 4: Moment about Global X, 5: Moment about Global Y, 6: Moment about Global Z
2	0	f1	Displacement target value in current length units

Returns:

True if the command was successfully added or updated, *False* otherwise.

Return type:

Int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Command.PerformNonlinearAnalysisEx(6, 0, 5, 0
>>> print(result)
```

`PerformPDeltaAnalysisEx(NoOfIterations: int, PrintOption: int, bSmallDelta: int, AddGeometricStiffness: int)` [source]

Perform an extended P-Delta analysis with options.

Parameters:

- **NoOfIterations** (*int*) – Desired number of iterations.
- **PrintOption** (*int*) –

  Option index for specifying the print output. Choose from the following:

Value	Print Option
1	Print Load Data
2	Print Statics Check
3	Print Statics Load
4	Print Mode Shapes
5	Print Both
6	Print All
0	No Print

- **bSmallDelta** (*int*) – Set to 1 to include P-SMALL-Delta effect, or 0 to include P-LARGE-Delta effect.
- **AddGeometricStiffness** (*int*) – Set to 1 to add geometric stiffness, or 0 to exclude it.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Command.PerformPDeltaAnalysisEx(20, 5, 0, 1)
```

`PerformPDeltaAnalysisNoConverge(NoOfIterations: int, PrintOption: int)` [source]

Perform a P-Delta analysis without convergence check.

Parameters:

- **NoOfIterations** (*int*) – The number of iterations to perform.
- **PrintOption** (*int*) –

  The print option for the analysis results. Possible values are:

Value	Print Option
1	Print Load Data
2	Print Statics Check
3	Print Statics Load
4	Print Mode Shapes
5	Print Both
6	Print All
0	No Print

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Command.PerformPDeltaAnalysisNoConverge(5, 1)
```

SetCheckIrregularitiesCommand(*DesignCode*) [source]

Set a check irregularities command.

Parameters:

DesignCode (*int*) – Index number for specifying code

Returns:

Status code ‘1’ for OK and ‘0’ if failed to add or update.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Command.SetCheckIrregularitiesCommand(3)
>>> print(result)
```

SetCheckSoftStoryCommand(*DesignCode: int*) [source]

Set a check soft story command.

Parameters:

DesignCode (*int*) – Index number for specifying code

Returns:

Status code ‘1’ for OK and ‘0’ if failed to add or update

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> result = staad_obj.Command.SetCheckSoftStoryCommand(3)
>>> print(result)
```

SetFloorDiaphragmBaseCommand(*baseElevationValue: float*)

Set a floor diaphragm base command. [source]

Parameters:

baseElevationValue (*int*) – The diaphragm base command string.

Returns:

Status code ‘1’ for OK and ‘0’ if failed to add or update

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> status = staad_obj.Command.SetFloorDiaphragmBaseCommand(100.0)
>>> print(status)
```

__init__(*staadObj*) [source]

Output

Contents

- OSOutput

class openstaadpy.os_analytical.osoutput.OSOutput [source]

Bases: object

AreResultsAvailable() [source]

Check if analysis results are available or not.

Returns:

True if results are available, False otherwise.

Return type:

bool

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> are_results_available = staad_obj.Output.AreResultsAvailable()
```

GetAllPlateCenterForces(*plateNo: int, loadCaseNo: int*) [source]

Get the plate center stresses (Shear & Membrane) for the specified plate for specified load case.

Parameters:

- **plateNo** (*int*) – The plate number.
- **loadCaseNo** (*int*) – The load case number.

Returns:

list of plate center forces organized in following order:

List Index	Variable	Load Type
0	SQX	Shear stress on the local X face in the Z direction
1	SQY	Shear stress on the local Y face in the Z direction
2	SX	Axial stress in the local X direction
3	SY	Axial stress in the local Y direction
4	SXY	Shear stress in the local XY plane

Note :

- For additional information, please refer to Section: “Sign Convention of Plate Element Stresses and Moments” and Section 5.42 of the Technical Reference manual.

Return type:

list of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> plateList = staad_obj.Geometry.GetPlateList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> plateCenterForces = staad_obj.Output.GetAllPlateCenterForces(plateL
```

`GetAllPlateCenterMoments(plateNo: int, loadCaseNo: int)` [source]

Get the plate center stresses (Shear & Membrane) for the specified plate for specified load case.

Parameters:

- **plateNo** (*int*) – The plate number.
- **loadCaseNo** (*int*) – The load case number.

Returns:

list of plate center moments organized in following order:

List Index	Variable	Load Type
0	MX	Moment per unit width about the local X face
1	MY	Moment per unit width about the local Y face
2	MXY	Torsional Moment per unit width in the local X-Y plane

Note :

- For additional information, please refer to Section: “Sign Convention of Plate Element Stresses and Moments” and Section 5.42 of the Technical Reference manual.

Return type:

list of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> plateList = staad_obj.Geometry.GetPlateList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> plateCenterMoments = staad_obj.Output.GetAllPlateCenterMoments(plat
```

`GetAllPlateCenterPrincipalStressesAndAngles(plateNo: int, loadCaseNo: int)` [source]

Get all plate center principal stresses and angles.

Parameters:

- **plateNo** (*int*) – Plate number ID.

- **loadCaseNo** (*int*) – Case reference ID.

Returns:

List of float values orgainized according to below table:

Variable	Description
pdStresses[0]	Top-Maximum in-plane Principal stress
pdStresses[1]	Top-Minimum in-plane Principal stress
pdStresses[2]	Top-Maximum in-plane Shear stress
pdStresses[3]	Bottom-Maximum in-plane Principal stress
pdStresses[4]	Bottom-Minimum in-plane Principal stress
pdStresses[5]	Bottom-Maximum in-plane Shear stress
pdStresses[6]	Top-Angle which determines direction of maximum principal stress with respect to local X axis
pdStresses[7]	Bottom-Angle which determines direction of maximum principal stress with respect to local X axis

Return type:

list of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> plateList = staad_obj.Geometry.GetPlateList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> plateCenterPrincipalStressesAndAngles = staad_obj.Output.GetAllPlat
```

GetAllPlateCenterPrincipalStressesAndAnglesEx(*plateNo: int*, *loadCaseNo: int*) [source]

Get all plate center principal stresses and angles (extended).

Parameters:

- **plateNo** (*int*) – Plate number ID.
- **loadCaseNo** (*int*) – Load case number ID.

Returns:

Tuple of principal stresses values list and angle value list. They are organized following way

Principal Stresses list :

Variable	Description
pdStresses[0]	Top-Maximum in-plane Principal stress
pdStresses[1]	Top-Minimum in-plane Principal stress
pdStresses[2]	Top-Maximum in-plane Shear stress
pdStresses[3]	Bottom-Maximum in-plane Principal stress
pdStresses[4]	Bottom-Minimum in-plane Principal stress
pdStresses[5]	Bottom-Maximum in-plane Shear stress

Angles List :

Variable	Description
pdAngles[0]	Top-Angle which determines direction of maximum principal stress with respect to local X axis
pdAngles[1]	Bottom-Angle which determines direction of maximum principal stress with respect to local X axis

Return type:

tuple

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> plate_list = staad_obj.Geometry.GetPlateList()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> principal_stress_list, angle_list = staad_obj.Output.GetAllPlateCer
```

GetAllPlateCenterStressesAndMoments(*plateNo: int, loadCaseNo: int*) [source]

Gets plate center stresses and moments for the specified plate for specified load case.

Parameters:

- **plateNo** (*int*) – Plate number.
- **loadCaseNo** (*int*) – Load Case reference ID.

Returns:

list of plate center stresses and moments organized in following order:

List Index	Variable	Load Type
0	SQX	Shear stress on the local X face in the Z direction
1	SQY	Shear stress on the local Y face in the Z direction
2	MX	Moment per unit width about the local X face
3	MY	Moment per unit width about the local Y face
4	MXY	Torsional Moment per unit width in the local X-Y plane
5	SX	Axial stress in the local X direction
6	SY	Axial stress in the local Y direction
7	SXY	Shear stress in the local XY plane

Note :

- For additional information, please refer to Section: “Sign Convention of Plate Element Stresses and Moments” and Section 5.42 of the Technical Reference manual.

Return type:

list of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> plateList = staad_obj.Geometry.GetPlateList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> plateCenterStressesAndMoments = staad_obj.Output.GetAllPlateCenterS
```

GetAllSolidNormalStresses(*nSolidNo: int, nCorner: int, loadCaseNo: int*) [source]

Gets all solid normal stresses.

Parameters:

- **nSolidNo** (*int*) – Solid number ID.
- **nCorner** (*int*) – Corner of the solid.
- **loadCaseNo** (*int*) – The load case number.

Returns:

list with solid normal stresses SXX, SYY and SZZ (in order).

Return type:

list of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> solidList = staad_obj.Geometry.GetSolidList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> solidNormalStresses = staad_obj.Output.GetAllSolidNormalStresses(sc
```

GetAllSolidPrincipalStresses(*nSolidNo: int, nCorner: int, loadCaseNo: int*) [source]

Get all solid principal stresses.

Parameters:

- **nSolidNo** (*int*) – Solid number ID.
- **nCorner** (*int*) – Corner of the solid.
- **loadCaseNo** (*int*) – Load Case reference ID.

Returns:

List of principal stresses S_1, S_2, S_3 in same order.

Return type:

list of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> solidList = staad_obj.Geometry.GetSolidList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> principalStresses = staad_obj.Output.GetAllSolidPrincipalStresses(s
```

GetAllSolidShearStresses(*nSolidNo: int, nCorner: int, loadCaseNo: int*) [source]

Get all solid shear stresses.

Parameters:

- **nSolidNo** (*int*) – Solid number ID.
- **nCorner** (*int*) – Corner of the solid.
- **loadCaseNo** (*int*) – Load Case reference ID.

Returns:

List of shear stresses SXY, SYZ, SZX in same order.

Return type:

list of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> solidList = staad_obj.Geometry.GetSolidList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> shearStresses = staad_obj.Output.GetAllSolidShearStresses(solidList
```

GetAllSolidVonMisesStresses(*nSolidNo: int, nCorner: int, loadCaseNo: int*) [source]

Get all solid Von Mises stresses.

Parameters:

- **nSolidNo** (*int*) – Solid number ID.
- **nCorner** (*int*) – Corner of the solid.
- **loadCaseNo** (*int*) – Load Case reference ID.

Returns:

List of Von Mises stresses.

Return type:

list of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> solidList = staad_obj.Geometry.GetSolidList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> vonMisesStresses = staad_obj.Output.GetAllSolidVonMisesStresses(sol
```

GetBasePressures(*loadCaseNo: int, nodelist: list*) [source]

Gets base presure in X, Y and Z direction using Base Presure command.

Parameters:

- **loadCaseNo** (*int*) – Load Case reference ID.
- **nodelist** (*list*) – List of node numbers.

Returns:

Tuple of list of base pressures in X, Y and Z direction per load case reference ID.

Return type:

tuple of list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> node_list = staad_obj.Geometry.GetNodeList()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> base_pressures_x, base_pressures_y, base_pressures_z = staad_obj.Ou
```

GetBucklingFactor(*buckling_mode_no: int*) [source]

Gets the buckling factor for a specified buckling mode.

Parameters:

buckling_mode_no (*int*) – Buckling mode number

Returns:

Buckling factor.

Return type:

float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> buckling_factor = staad_obj.Output.GetBucklingFactor(1)
```

GetBucklingModeDisplacementAtNode(*buckling_mode_no: int*, *node_no: int*) [source]

Gets the modal displacement at a specified node number and mode.

Parameters:

- **buckling_mode_no** (*int*) – Buckling mode number.
- **node_no** (*int*) – Node number at which buckling analysis result is to be extracted.

Returns:

List of buckling mode displacements.

Return type:

list of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> node_list = staad_obj.Geometry.GetNodeList()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> buckling_mode_displacement_at_node = staad_obj.Output.GetBucklingMo
```

GetIntermediateDeflectionAtDistance(*memberNo: int, distance: int, loadCaseNo: int*) [source]

Get the intermediate section deflections for specified member number and load case.

Parameters:

- **memberNo** (*int*) – Member number ID.
- **distance** (*float*) – Distance from the starting end of the member.
- **loadCaseNo** (*int*) – Load Case reference ID.

Returns:

Tuple of displacement in Y & Z direction respectively.

Return type:

tuple

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> beam_list = staad_obj.Geometry.GetBeamList()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> deflections = staad_obj.Output.GetIntermediateDeflectionAtDistance(
```

GetIntermediateMemberAbsTransDisplacements(*memberNo: int, distance: float, loadCaseNo: int*) [source]

Gets section displacement (or relative displacements) of a beam section for specified member number, distance, and load case.

Parameters:

- **memberNo** (*int*) – Member number ID.
- **distance** (*float*) – Distance from starting end in terms of member length.
- **loadCaseNo** (*int*) – Load Case reference ID.

Returns:

List of relative displacements at specified section in terms of LOCAL X, Y, Z coordinates (in order).

Return type:

list of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> beam_list = staad_obj.Geometry.GetBeamList()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> intermediate_member_abs_trans_displacements = staad_obj.Output.Get
```

GetIntermediateMemberForcesAtDistance(*memberNo: int, distance: float, loadCaseNo: int*) [source]

Gets sectional forces and moments for specified member number, distance, and load case.

Parameters:

- **memberNo** (*int*) – Member number ID.
- **distance** (*float*) – Distance from the starting end of the member.
- **loadCaseNo** (*int*) – Load Case reference ID.

Returns:

List of Section axial force, Shear force in LOCAL Y & Z direction, Torsion and Bending moment in Local MY & MZ direction respectively.

Return type:

list of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> beamList = staad_obj.Geometry.GetBeamList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> memberForces = staad_obj.Output.GetIntermediateMemberForcesAtDistar
```

GetIntermediateMemberTransDisplacements(*memberNo: int, distance: int, loadCaseNo: int*) [source]

Get section displacement (or relative displacements) of a beam section for specified member number, distance, and load case.

Parameters:

- **memberNo** (*int*) – Member number ID.
- **distance** (*int*) – Distance from starting end in terms of member length.
- **loadCaseNo** (*int*) – Load Case reference ID.

Returns:

List of relative displacements at specified section in terms of LOCAL X, Y, Z coordinates (in order).

Return type:

list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> beamList = staad_obj.Geometry.GetBeamList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> intermediateMemberTransDisplacements = staad_obj.Output.GetIntermed
```

GetMatInfluenceAreas(*nodelist: list*) [source]

Gets the mat influence areas for nodes supported using ELASTIC MAT command.

Parameters:

nodelist (*list*) – List of node numbers.

Returns:

Tuple of lists containing influence areas in YZ, ZX & XY place ordered per node number in node list .

Return type:

tuple of list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> node_list = staad_obj.Geometry.GetNodeList()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> mat_influence_areas = staad_obj.Output.GetMatInfluenceAreas(node_li
```

GetMaxBeamStresses(*beamNo: int, loadCaseNo: int*) [source]

Gets the maximum beam Stresses for Beam.

Parameters:

- **beamNo** (*int*) – Beam number ID.
- **loadCaseNo** (*int*) – The load case number.

Returns:

tuple with value of maximum compressive stress, integer value (0 for non-corner-section material (eg. prismatic circle and pipe), 1 to 4 for section corner where maximum compressive stress was found), value of maximum tensile stress and integer value (0 for non-corner-section material (eg. prismatic circle and pipe), 1 to 4 for section corner where maximum tensile stress was found) (in order).

Return type:

tuple

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> beamList = staad_obj.Geometry.GetBeamList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> maxBeamStresses = staad_obj.Output.GetMaxBeamStresses(beamList[0],
```

GetMaxSectionDisplacement(*memberNo: int, dir: str, loadCaseNo: int*) [source]

Gets the maximum section displacements for specified member number, direction, and load case.

Parameters:

- **memberNo** (*int*) – Member number ID.
- **dir** (*str*) – Direction in GLOBAL: X, Y or Z
- **loadCaseNo** (*int*) – The load case number.

Returns:

tuple with maximum section displacement in specified direction and the location along the length of the member where the maximum section displacement is located (in order).

Return type:

tuple of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> beamList = staad_obj.Geometry.GetBeamList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> maxSectionDisplacement = staad_obj.Output.GetMaxSectionDisplacement
```

GetMemberDesignSectionName(*beamNo: int*) [source]

Get the design section name for specified member.

Parameters:

beamNo (*int*) – Beam number ID.

Returns:

Returns design section name for the specified member. Returns empty string if not found.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> beam_list = staad_obj.Geometry.GetBeamList()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> design_section_name = staad_obj.Output.GetMemberDesignSectionName(b
```

GetMemberEndDisplacements(*memberNo: int, end: int, loadCaseNo: int*) [source]

Get the end displacements for a given member.

Parameters:

- **memberNo** (*int*) – The member number.
- **end** (*int*) – The end number (0 for starting and 1 for ending).

- **loadCaseNo** (*int*) – The load case number.

Returns:

List with end displacements of Member End in terms of X, Y, Z (in order).

Return type:

list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> beamList = staad_obj.Geometry.GetBeamList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> memberEndDisplacements = staad_obj.Output.GetMemberEndDisplacements
```

`GetMemberEndForces(memberNo: int, end: int, loadCaseNo: int, LocalOrGlobal: int)` [source]

Get the end forces for a given member.

Parameters:

- **memberNo** (*int*) – The member number.
- **end** (*int*) – The end number (0 for starting and 1 for ending).
- **loadCaseNo** (*int*) – The load case number.
- **LocalOrGlobal** (*int*) – Local Or Global direction (0 for Local and 1 for Global).

Returns:

list with end force values FX, FY, FZ, MX, MY and MZ (in order).

Return type:

list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> beamList = staad_obj.Geometry.GetBeamList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> memberEndForces = staad_obj.Output.GetMemberEndForces(beamList[0],
```

GetMemberSteelDesignMaxFailureRatio() [source]

Gets the maximum failure ratio across all beams in the model.

Returns:

The maximum failure ratio.

Return type:

float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> member_steel_design_max_failure_ratio = staad_obj.Output.GetMemberS
```

GetMemberSteelDesignMinFailureRatio() [source]

Gets the minimum failure ratio across all beams in the model.

Returns:

The minimum failure ratio.

Return type:

float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> member_steel_design_min_failure_ratio = staad_obj.Output.GetMemberS
```

GetMemberSteelDesignRatio(*beamNo: int*) [source]

Gets the critical steel design ratio for a steel member. This method will return the results from the last parameter block for which the beam has been designed.

Parameters:

beamNo (*int*) – The beam number ID.

Returns:

Returns the critical steel design ratio. Returns -999 if analysis is performed but the member is not designed. Returns -1 if analysis is not performed.

Return type:

float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> beamList = staad_obj.Geometry.GetBeamList()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> memberSteelDesignRatio = staad_obj.Output.GetMemberSteelDesignRatio
```

GetMemberSteelDesignResults(*beamNo: int*) [source]

Gets steel design results for the specified member. This method will return the results from the last parameter block for which the beam has been designed.

Parameters:

beamNo (*int*) – Id of the member for which design results should be retrieved.

Returns:

Tuple of steel design results containing the design code name, the design status (pass or fail will be returned), the design utilization ratio, the allowable design utilization ratio, the critical design load case number, the critical section location from the start of member in current base unit, the critical design clause, the design section name, array of size 3 and type double to hold critical section forces. returns fx, my and mz values at 0, 1, & 2 index respectively in current base units and kl/r ratio of the specified member. (in same order)

Return type:

tuple

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> beam_list = staad_obj.Geometry.GetBeamList()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> design_code_name, design_status, critical_ratio, allowable_ratio, c
```

GetMinMaxAxialForce(*memberNo: int, loadCaseNo: int*) [source]

Gets the maximum and minimum axial forces and their locations for specified member number and load case.

Parameters:

- **memberNo** (*int*) – Member number ID.
- **loadCaseNo** (*int*) – The load case number.

Returns:

tuple with minimum axial force, the location along the length of the member where the minimum axial force is located, maximum axial force and the location along the length of the member where the maximum axial force is located (in order).

Return type:

tuple of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> beamList = staad_obj.Geometry.GetBeamList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> minMaxAxialForce = staad_obj.Output.GetMinMaxAxialForce(beamList[0]
```

GetMinMaxBendingMoment(*memberNo: int, dir: str, loadCaseNo: int*)

[source]

Gets the maximum and minimum bending moments and their locations for specified member number, load case, and bending direction.

Parameters:

- **memberNo** (*int*) – Member number ID.
- **dir** (*str*) – Bending direction in LOCAL coordinate: MY or MZ.
- **loadCaseNo** (*int*) – The load case number.

Returns:

tuple with minimum bending moment, the location along the length of the member where the minimum bending moment is located, maximum bending moment and the location along the length of the member where the maximum bending moment is located (in order).

Return type:

tuple of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> beamList = staad_obj.Geometry.GetBeamList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> minMaxBendingMoment = staad_obj.Output.GetMinMaxBendingMoment(beaml
```

GetMinMaxShearForce(*memberNo: int, dir: str, loadCaseNo: int*)

[source]

Gets the maximum and minimum shear forces and their locations for specified member number, load case, and force direction.

Parameters:

- **memberNo** (*int*) – Member number ID.
- **dir** (*str*) – Force direction in LOCAL coordinate: FY or FZ.
- **loadCaseNo** (*int*) – The load case number.

Returns:

tuple with minimum bending shear, the location along the length of the member where the minimum bending shear is located, maximum bending shear and the location along the length of the member where the maximum bending shear is located (in order).

Return type:

tuple of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> beamList = staad_obj.Geometry.GetBeamList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> minMaxShearForce = staad_obj.Output.GetMinMaxShearForce(beamList[0]
```

GetModalDisplacementAtNode(*modeNo: int, nodeNo: int*) [source]

Gets the modal displacement at a specified node number and mode.

Parameters:

- **modeNo** (*int*) – Mode number.
- **nodeNo** (*int*) – Node number ID.

Returns:

List of nodal displacements.

Return type:

list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> node_list = staad_obj.Geometry.GetNodeList()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> mode_displacement_at_node = staad_obj.Output.GetModalDisplacementAt
```

GetModalMassParticipationFactors(*modeNo: int*) [source]

Gets the modal participation factors for a specified mode number.

Parameters:

modeNo (*int*) – Mode number.

Returns:

Tuple of modal mass participation factor for X, Y & Z direction respectively.

Return type:

tuple

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> mode_mass_participation_factors = staad_obj.Output.GetModalMassPart
```

GetModeFrequency(*modeNo: int*) [source]

Get the natural frequency (Hz) for a specified mode.

Parameters:

modeNo (*int*) – The mode number.

Returns:

Natural Frequency value for a specified mode.

Return type:

float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> mode_frequency = staad_obj.Output.GetModeFrequency(4)
```

GetMultipleMemberSteelDesignMaxRatio(*beamNo: int*) [source]

Gets the maximum critical steel design ratio across all parameter blocks for a steel member.

Parameters:

beamNo (*int*) – Beam number ID.

Returns:

Returns the maximum critical steel design ratio.

Return type:

float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> beam_list = staad_obj.Geometry.GetBeamList()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> multiple_member_steel_design_max_ratio = staad_obj.Output.GetMultip
```

GetMultipleMemberSteelDesignRatio(*param_blk_name: str, beam_no: int*) [source]

Gets the critical steel design ratio for a steel member. This function is for AISC 360-16 code only.

Parameters:

- **param_blk_name** (*str*) – Steel design parameter block name.
- **beam_no** (*int*) – Beam number ID.

Returns:

Returns the critical steel design ratio.

Return type:

float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> countOfParamBlocks = staad_obj.Output.GetSteelDesignParameterBlockC
>>> if (countOfParamBlocks > 0) :
>>>     param_blk_name = staad_obj.Output.GetSteelDesignParameterBlockN
>>>     beam_list = staad_obj.Geometry.GetBeamList()
>>>     staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>>     multiple_member_steel_design_ratio = staad_obj.Output.GetMultip
```

GetMultipleMemberSteelDesignResults(*param_blk_name: str*, *beam_no: int*) [source]

Gets the critical steel design result information for a steel member.

Parameters:

- **param_blk_name** (*str*) – Steel design parameter block name.
- **beam_no** (*int*) – Beam number ID.

Returns:

Tuple consisting of the design code name, the design status (pass or fail will be returned), the design utilization ratio., the allowable design utilization ratio., the critical design load case number., the critical design clause.and the design section name. (in same order)

Return type:

tuple

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> countOfParamBlocks = staad_obj.Output.GetSteelDesignParameterBlockC
>>> if (countOfParamBlocks > 0) :
>>>     param_blk_name = staad_obj.Output.GetSteelDesignParameterBlockN
>>>     beam_list = staad_obj.Geometry.GetBeamList()
>>>     staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>>     multiple_member_steel_design_results = staad_obj.Output.GetMult
```

GetNLLoadStep(*loadCaseNo: int*) [source]

Gets the Load Step value used for nonlinear analysis for specified Load Case.

Parameters:

loadCaseNo (*int*) – Load Case reference ID.

Returns:

Returns Load Step value. Returns -1 General error. Returns -8002 Load Case nLC not found. Returns -9911 Nonlinear result set is not available.

Return type:

int

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> load_step_value = staad_obj.Output.GetNLLoadStep(load_cases[0])
```

GetNLNodeDisplacements(*nodeNo: int, loadCaseNo: int, loadStep: int*) [source]

Get the Load Level value and nodal displacements for specified node, Load Case and Load Step.

Parameters:

- **nodeNo** (*int*) – Node number ID.
- **loadCaseNo** (*int*) – Load Case reference ID.

- **loadStep** (*int*) – Load step number.

Returns:

Tuple containing value of load level and list of nodal displacements in Global (X, Y, Z, rX, rY, rZ) in same order.

Return type:

tuple

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> node_list = staad_obj.Geometry.GetNodeList()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> nl_node_displacement = staad_obj.Output.GetNLNodeDisplacements(node
```

GetNoOfBucklingFactors() [source]

Gets the number of buckling factors computed.

Returns:

The number of buckling factor(s) extracted by the eigen analysis.

Return type:

int

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> number_of_buckling_factors = staad_obj.Output.GetNoOfBucklingFactor
```

GetNoOfModesExtracted() [source]

Gets the number of modes extracted by a dynamic analysis.

Returns:

Number of modes extracted by a dynamic analysis.

Return type:

int

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> no_of_modes_extracted = staad_obj.Output.GetNoOfModesExtracted()
```

GetNodeDisplacements(*nodeNo: int, loadCaseNo: int*) [source]

Get the displacements for a given node.

Parameters:

- **nodeNo** (*int*) – The node number.
- **loadCaseNo** (*int*) – The load case number.

Returns:

List with nodal translational displacements in X, Y and Z directions, rotation about X, Y and Z directions, respectively

Return type:

list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> nodeList = staad_obj.Geometry.GetNodeList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 1, 1)
>>>
>>> displacements = staad_obj.Output.GetNodeDisplacements(nodeList[3],
```

GetOutputUnitForDensity() [source]

Get the output unit for density.

Returns:

The unit for density.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> unit = staad_obj.Output.GetOutputUnitForDensity()
```

GetOutputUnitForDimension() [source]

Get the output unit for dimension.

Returns:

The unit for dimension.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> unit = staad_obj.Output.GetOutputUnitForDimension()
```

GetOutputUnitForDisplacement() [source]

Get the output unit for displacement.

Returns:

The unit for displacement.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> unit = staad_obj.Output.GetOutputUnitForDisplacement()
```

GetOutputUnitForDistForce()

[source]

Get the output unit for distributed force.

Returns:

The unit for distributed force.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> unit = staad_obj.Output.GetOutputUnitForDistForce()
```

GetOutputUnitForDistMoment()

[source]

Get the output unit for distributed moment.

Returns:

The unit for distributed moment.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> unit = staad_obj.Output.GetOutputUnitForDistMoment()
```

GetOutputUnitForForce() [source]

Get the output unit for force.

Returns:

The unit for force.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> unit = staad_obj.Output.GetOutputUnitForForce()
```

GetOutputUnitForMoment() [source]

Get the output unit for moment.

Returns:

The unit for moment.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> unit = staad_obj.Output.GetOutputUnitForMoment()
```

GetOutputUnitForRotation() [source]

Get the output unit for rotation.

Returns:

The unit for rotation.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> unit = staad_obj.Output.GetOutputUnitForRotation()
```

GetOutputUnitForSectArea() [source]

Get the output unit for section area.

Returns:

The unit for section area.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> unit = staad_obj.Output.GetOutputUnitForSectArea()
```

GetOutputUnitForSectDimension() [source]

Get the output unit for section dimension.

Returns:

The unit for section dimension.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> unit = staad_obj.Output.GetOutputUnitForSectDimension()
```

GetOutputUnitForSectInertia() [source]

Get the output unit for section inertia.

Returns:

The unit for section inertia.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> unit = staad_obj.Output.GetOutputUnitForSectInertia()
```

GetOutputUnitForSectModulus() [source]

Get the output unit for section modulus.

Returns:

The unit for section modulus.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> unit = staad_obj.Output.GetOutputUnitForSectModulus()
```

GetOutputUnitForStress() [source]

Get the output unit for stress.

Returns:

The unit for stress.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> unit = staad_obj.Output.GetOutputUnitForStress()
```

GetPMemberEndForces(*memberNo: int, end: int, loadCaseNo: int, LocalOrGlobal: int*) [source]

Gets member end forces for specified physical member number, member end and load case.

Parameters:

- **memberNo** (*int*) – Member number ID.
- **end** (*int*) – Member End (0 for starting and 1 for ending).
- **loadCaseNo** (*int*) – Load Case reference ID.
- **LocalOrGlobal** (*int*) – Results returned in either local or global axes. 0= Local, 1= Global direction.

Returns:

List of force of Member End in LOCAL coordinates in terms of FX, FY, FZ, MX, MY and MZ (in order).

Return type:

list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> plate_list = staad_obj.Geometry.GetPlateList()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> p_member_end_forces = staad_obj.Output.GetPMemberEndForces(plate_li
```

GetPMemberIntermediateForcesAtDistance(*memberNo: int, distance: int, loadCaseNo: int*) [source]

Gets sectional forces and moments for specified physical member number, distance, and load case.

Parameters:

- **memberNo** (*int*) – Physical Member number ID.
- **distance** (*int*) – Distance from the starting end of the member.
- **loadCaseNo** (*int*) – Load Case reference ID.

Returns:

List of 6 elements consisting of Section axial force, Shear force in LOCAL Y & Z direction, Torsion and Bending moment in Local MY & MZ direction.

Return type:

list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> plate_list = staad_obj.Geometry.GetPlateList()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> p_member_end_forces = staad_obj.Output.GetPMemberIntermediateForces
```

GetPlateCenterNormalPrincipalStresses(*plateNo: int, loadCaseNo: int*) [source]

Get principal stresses of specified plate.

Parameters:

- **plateNo** (*int*) – The plate number.
- **loadCaseNo** (*int*) – The load case number.

Returns:

Tuple containing maximum in-plane Principal Stress at Top surface of plate, minimum in plane Principal Stress at Top surface of plate, maximum in-plane Principal Stress at Bottom surface of plate and minimum in-plane Principal Stress at Bottom surface of plate respectively.

Return type:

Tuple

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> beamList = staad_obj.Geometry.GetPlateList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> plateCenterNormalPrincipalStresses = staad_obj.Output.GetPlateCente
```

GetPlateCenterVonMisesStresses(*plateNo: int, loadCaseNo: int*)

Gets Von Mises stresses at center of specified plate for specified load case. [source]

Parameters:

- **plateNo** (*int*) – Plate number ID.
- **loadCaseNo** (*int*) – Load Case reference ID.

Returns:

Tuple of Von Mises stress on the top surface of the plate and Von Mises stress on the bottom surface of the plate respectively

Return type:

tuple

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> plateList = staad_obj.Geometry.GetPlateList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> vonMisesStresses = staad_obj.Output.GetPlateCenterVonMisesStresses(
```

GetPlateCornerForces(*plateNo: int, cornerCode: int, loadCaseNo: int*) [source]

Get nodal forces at 4 corners of specified plate at load case.

Parameters:

- **plateNo** (*int*) – Plate number ID.
- **cornerCode** (*int*) – Corner Node No.
- **loadCaseNo** (*int*) – Load case number.

Returns:

List of nodal forces at 4 corners of specified plate at load case.

Return type:

list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> plate_list = staad_obj.Geometry.GetPlateList()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> forces = staad_obj.Output.GetPlateCornerForces(plate_list[0], 1, lc
```

GetPlateStressAtPoint(*plateNo: int, loadNo: int, stressPoint: list, facingPoint: int*) [source]

Get stresses values at a point on a specified plate.

Parameters:

- **plateNo** (*int*) – Number of the plate.
- **loadNo** (*int*) – Load number for which stress is requested
- **stressPoint** (*list*) – The coordinate at which the stress is required in global axes as an array size of 3 doubles. The values of x, y, z values of the point defined in the array index (0), (1) and (2).
- **facingPoint** (*int*) – x, y, z values of the facing node at indexes 0, 1 and 2. API always expects an array size of 3 doubles. Definition of facingPoint: It is the node which sits on the tip of a vector which is orthogonal to the vector stressPoint -> facingPoint and lies in the same plane as that of the plates through which the cut line passes.

Returns:

List of stresses values at a point on a specified plate containing following 32 values:

Array Index	stress Type
0	None
1	MaxAbs
2	TopMax
3	TopMin
4	TopTauMax
5	BotMax
6	BotMin
7	BotTauMax
8	MaxVM
9	VMTopMax
10	VMBotMax
11	MaxTresca
12	TopTresca
13	BotTresca
14	FX
15	FY
16	FXY
17	MX
18	MY
19	MZ
20	QX
21	QY
22	Global

Array Index	stress Type
23	GlobalMembraneStresses
24	GlobalshearStresses
25	BasePres
26	CombXTop
27	CombYTop
28	CombXYTop
29	CombXBot
30	CombYBot
31	CombXYBot

Return type:

list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> surface_name = "WALL 1"
>>> plate_list = staad_obj.Geometry.GetPlateList()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> plate_stress_at_point = staad_obj.Output.GetPlateStressAtPoint(plat
```

`GetResultantForceAlongLineForParametricSurface(parametricSurfaceName: str, nplates: int, loadId: int, startNode: list, endNode: list, facingNode: list, isTransformForceToGlobal: int, firstNode: int, secondNode: int, thirdNode: int)` [source]

Gets forces and moments along the cut line for a particular load case.

Parameters:

- **parametricSurfaceName** (*str*) – Name of the parametric surface.
- **nplates** (*int*) – Number of plates in plateList.
- **loadId** (*int*) – The load case for plate analysis.
- **startNode** (*list*) – List of x, y, z values of the start node at indexes 0, 1 and 2.
- **endNode** (*list*) – List of x, y, z values of the end node at indexes 0, 1 and 2.
- **facingNode** (*list*) – List of x, y, z values of the facing node at indexes 0, 1 and 2.
- **isTransformForceToGlobal** (*int*) – 1: return force in Global System, 0: return forces in local system of cut line.
- **firstNode** (*int*) – Node no representing the origin point for building the surface axis system.
- **secondNode** (*int*) – Node no representing the second point for building the surface axis system.
- **thirdNode** (*int*) – Node no representing the third point for building the surface axis system.

Returns:

List of resultant forces consisting Fx, Fy, Fz, Mx, My, Mz (in same order).

Return type:

list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> surface_name = "WALL 1"
>>> plate_list = staad_obj.Geometry.GetPlateList()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> result = staad_obj.Output.GetResultantForceAlongLineForParametricSu
```

GetResultantForceAlongLineForPlateList(*plateList: list, nplates: int, loadIdList: list, startNode: list, endNode: list, isTransformForceToGlobal: int, firstNode: int, secondNode: int, thirdNode: int*) [source]

Gets forces and moments along the cut line for a particular load case.

Parameters:

- **plateList** (*list*) – List of plates IDs. a) All plates in model, b) plates through which the cut line crosses (both would work but 'a' is computationally expensive)
- **nplates** (*int*) – No of plates in plateList
- **loadIdList** (*list*) – The load cases for plate analysis
- **startNode** (*list*) – List of x, y, z values of the start node at indexes 0, 1 and 2.
- **endNode** (*list*) – List of x, y, z values of the end node at indexes 0, 1 and 2.
- **isTransformForceToGlobal** (*int*) – 1: return force in Global System 0: return forces in local system of cut line
- **firstNode** (*int*) – Node no representing the origin point for building the surface axis system
- **secondNode** (*int*) – Node no representing the second point for building the surface axis system
- **thirdNode** (*int*) – Node no representing the third point for building the surface axis system

Returns:

List of resultant forces consisting Fx, Fy, Fz, Mx, My, Mz (in same order).

Return type:

list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> plate_list = staad_obj.Geometry.GetPlateList()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> resultant_force_along_line_for_plate_list = staad_obj.Output.GetRes
```

GetStaticCheckResult(*LoadCaseNo: int*) [source]

Gets the statics check result containing loads and reactions for specified load case.

Parameters:

loadCaseNo (*int*) – Load Case reference ID.

Returns:

Tuple of list of loads (FX, FY, FZ, MZ, MY, MZ (in same order)) and list of reactions (FX, FY, FZ, MZ, MY, MZ (in same order)).

Return type:

tuple of list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> static_check_result = staad_obj.Output.GetStaticCheckResult(load_ca
```

`GetSteelDesignParameterBlockCount()` [source]

Gets the count of steel design parameter blocks in the model. This function is for AISC 360-16 code only.

Returns:

Returns the count of steel design parameter blocks.

Return type:

int

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> steel_design_parameter_block_count = staad_obj.Output.GetSteelDesig
```

`GetSteelDesignParameterBlockNameByIndex(index: int)` [source]

Gets steel design parameter name at the specified index. This function is for AISC 360-16 code only. :param index: The index value of steel design parameter block list. Note, the index is zero based. :type index: int

Returns:

Steel design parameter block name.

Return type:

str

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> countOfParamBlocks = staad_obj.Output.GetSteelDesignParameterBlockC
>>> if (countOfParamBlocks > 0) :
>>>     param_blk_name = staad_obj.Output.GetSteelDesignParameterBlockN
```

GetSupportReactions(*nodeNo: int, loadCaseNo: int*) [source]

Get the support reactions for a given support.

Parameters:

- **nodeNo** (*int*) – The node number.
- **loadCaseNo** (*int*) – The load case number.

Returns:

List with Reaction in GLOBAL direction:[FX, FY, FZ, MX, MY, MZ]

Return type:

list

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> nodeList = staad_obj.Geometry.GetNodeList()
>>> loadCases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> supportReactions = staad_obj.Output.GetSupportReactions(nodeList[3]
```

GetTimeHistoryIntegrationStepInfo() [source]

Gets the time-step (secs) used for time-history integration.

Returns:

Time step used for the integration in seconds.

Return type:

float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>>
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> delta, nsteps = staad_obj.Output.GetTimeHistoryIntegrationStepInfo(
```

GetTimeHistoryResponse(*load_case: int, node_no: int, dof_no: int, response_type: int*) [source]

Gets the time-history responses of DOF at specified node in the VARIANT array responses

Parameters:

- **load_case** (*int*) – Load case number for future use. Use 0 at present.
- **node_no** (*int*) – Node number where the response is sought.
- **dof_no** (*int*) –

  Degrees of freedom define as DegreesOfFreedom enum:

Value	Degrees Of Freedom
Fx = 1	DegreesOfFreedom.Fx
Fy = 2	DegreesOfFreedom.Fy
Fz = 3	DegreesOfFreedom.Fz
Mx = 4	DegreesOfFreedom.Mx
My = 5	DegreesOfFreedom.My
Mz = 6	DegreesOfFreedom.Mz

- **response_type** (*int*) –

  Response type, i.e., displacement, velocity, or acceleration defined in TimeHistoryResponseType enum:

Value	Response Type
displacement = 0	TimeHistoryResponseType.dispResponse
velocity = 1	TimeHistoryResponseType.velResponse
acceleration = 2	TimeHistoryResponseType.acclResponse

Returns:

Response value.

Return type:

float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> node_list = staad_obj.Geometry.GetNodeList()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> time_history_response = staad_obj.Output.GetTimeHistoryResponse(loa
```

`GetTimeHistoryResponseAtTime(load_case: int, node_no: int, dof_no: int, response_type: int, at_time: float)` [source]

Gets the response at a specific time within the integration time span at a specified node for a given DOF.

Parameters:

- **load_case** (*int*) – Load case number for future use. Use 0 at present.
- **node_no** (*int*) – Node number where the response is sought.
- **dof_no** (*int*) –

  Degrees of freedom define as DegreesOfFreedom enum:

Value	Degrees Of Freedom
Fx = 1	DegreesOfFreedom.Fx
Fy = 2	DegreesOfFreedom.Fy
Fz = 3	DegreesOfFreedom.Fz
Mx = 4	DegreesOfFreedom.Mx
My = 5	DegreesOfFreedom.My
Mz = 6	DegreesOfFreedom.Mz

- **response_type** (*int*) –

  Response type, i.e., displacement, velocity, or acceleration defined in TimeHistoryResponseType enum:

Value	Response Type
displacement = 0	TimeHistoryResponseType.dispResponse
velocity = 1	TimeHistoryResponseType.velResponse
acceleration = 2	TimeHistoryResponseType.acclResponse

- **at_time** (*int*) – Time in seconds for which the response is sought.

Returns:

List returning the responses at the integration steps. The size of the list is (no of integration steps), the first location is for the response at time = 0.0.

Return type:

list of float

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> node_list = staad_obj.Geometry.GetNodeList()
>>> load_cases = staad_obj.Load.GetPrimaryLoadCaseNumbers()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> time_history_response_at_time = staad_obj.Output.GetTimeHistoryResp
```

GetTimeHistoryResponseMinMax(*load_case: int, node_no: int, dof_no: int, response_type: int*) [source]

Gets the min/max time-history responses of DOF at specified node.

Parameters:

- **load_case** (*int*) – Use 0 at present.
- **node_no** (*int*) – Node number where the response is sought.
- **dof_no** (*int*) –

  Degrees of freedom define as DegreesOfFreedom enum:

Value	Degrees Of Freedom
Fx = 1	DegreesOfFreedom.Fx
Fy = 2	DegreesOfFreedom.Fy
Fz = 3	DegreesOfFreedom.Fz

- **response_type** (*int*) –

  Response type, i.e., displacement, velocity, or acceleration defined in TimeHistoryResponseType enum:

Value	Response Type
displacement = 0	TimeHistoryResponseType.dispResponse
velocity = 1	TimeHistoryResponseType.velResponse
acceleration = 2	TimeHistoryResponseType.acclResponse

Returns:

Tuple constisiting of maximum response, the time when the maximum response occurs, minimum response and the time when the minimum response occurs (in same order).

Return type:

tuple

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> node_list = staad_obj.Geometry.GetNodeList()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> time_min_max = staad_obj.Output.GetTimeHistoryResponseMinMax(0, noc
```

`IsBucklingAnalysisResultsAvailable()` [source]

Determines whether buckling results are available

Returns:

True if buckling analysis results are available, False otherwise.

Return type:

bool

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> buckling_analysis_results_available = staad_obj.Output.IsBucklingAr
```

IsMultipleMemberSteelDesignResultsAvailable() [source]

Checks whether steel design results from multiple design block can be extracted or not. If true, then relevant multiple steel design parameters like GetMultipleMemberSteelDesignRatio or GetMultipleMemberSteelDesignResults can be used. Currently, this facility is limited to AISC 360-16 code only. For further details, please check RR 22.02.00-4.2 of STAAD.Pro Help manual.

Returns:

returns true (for boolean variable, for long variable, return value is 1) if result extraction from multiple steel design block is possible (i.e. aisc 360-16 code is used). else the return value will be false (0 for long variable).

Return type:

bool

Example

```
>>> from openstaadpy import os_analytical
>>>
>>> staad_obj = os_analytical.connect()
>>> staad_obj.AnalyzeEx(1, 0, 1)
>>>
>>> is_multiple_member_steel_design_results_available = staad_obj.Outpu
```

__init__(*staad_obj*) [source]

Errors

Error Class	Description	Error Code
OsErrorBase	Base class for all custom OpenSTAAD-related errors.	(base class, no code)
OsError	Raised for a general, unspecified OpenSTAAD error.	-1
OsUnableToCreateProp	Raised when a property cannot be created in OpenSTAAD.	0
OsInvalidArgument	Raised when an invalid argument is provided to an OpenSTAAD function.	-100
OsMultidimArrayExpected	Raised when a multidimensional array is expected but not provided.	-106
OsArrayExpected	Raised when an array is expected but not provided.	-107
OsArraySizeSmall	Raised when an array is too small for the operation.	-108
OsArraySizeZero	Raised when an array of size zero is encountered.	-109
OsNoBeamPlateSolidSelected	Raised when no beam, plate, or solid is selected in the model.	-110
OsDoubleExpected	Raised when a double (float) value is expected but not provided.	-112
OsIntegerExpected	Raised when an integer value is expected but not provided.	-113
OsArraySizeLessThanReqd	Raised when an array is smaller than the required size.	-125
OsFileAlreadyExists	Raised when a file already exists and the operation cannot proceed.	-1003

Error Class	Description	Error Code
OsNodeNotFound	Raised when a node is not found in the model.	-2001
OsErrorAddNode	Raised when an error occurs while adding a node.	-2004
OsNoNodeSelected	Raised when no node is selected in the model.	-2005
OsInvalidNodeNo	Raised when an invalid node number is provided.	-2006
OsCannotFindMember	Raised when a member (beam) cannot be found in the model.	0
OsBeamNotFound	Raised when a beam is not found in the model.	-3001
OsBeamAlreadyExists	Raised when a beam already exists and the operation cannot proceed.	-3002
OsIdenticalBeamAlreadyExists	Raised when an identical beam already exists in the model.	-3003
OsErrorAddBeam	Raised when an error occurs while adding a beam.	-3004
OsNoBeamSelected	Raised when no beam is selected in the model.	-3005
OsPlateNotFound	Raised when a plate is not found in the model.	-4001
OsErrorAddPlate	Raised when an error occurs while adding a plate.	-4004
OsNoPlateSelected	Raised when no plate is selected in the model.	-4005
OsInvalidPlateNo	Raised when an invalid plate number is provided.	-4006

Error Class	Description	Error Code
OsInvalidPlateNoFound	Raised when an invalid plate number is found in the model.	-4008
OsNoValidPlateNoFound	Raised when no valid plate number is found in the model.	-4009
OsSolidNotFound	Raised when a solid is not found in the model.	-5001
OsErrorAddSolid	Raised when an error occurs while adding a solid.	-5004
OsNoSolidSelected	Raised when no solid is selected in the model.	-5005
OsMeshNotFound	Raised when a mesh is not found in the model.	-5603
OsInvalidPropRef	Raised when an invalid property reference is provided.	-6001
OsLibErrorCreateProp	Raised when a library error occurs while creating a property.	-6003
OsProfileNotFound	Raised when a profile is not found in the property library.	-6004
OsProfileDataNotFound	Raised when profile data is not found in the property library.	-6005
OsInvalidSectionType	Raised when an invalid section type is provided.	-6006
OsInvalidAssignType	Raised when an invalid assign type is provided.	-6008
OsLibErrorBetaAssign	Raised when a library error occurs while assigning beta.	-6009
OsNoPropAttached	Raised when no property is attached to an element.	-6022

Error Class	Description	Error Code
OsMaterialNotFound	Raised when a material is not found in the property library.	-6023
OsNoPropDefined	Raised when no property is defined in the model.	-6025
OsUptCreateFailed	Raised when UPT (User Provided Table) creation fails.	-6031
OsAddUptSectionFailed	Raised when adding a UPT section fails.	-6032
OsUptSectionExists	Raised when a UPT section already exists.	-6045
OsGroupAlreadyExists	Raised when a group already exists in the model.	-7001
OsInvalidLoadDirection	Raised when an invalid load direction is provided.	-8001
OsLoadCaseNotFound	Raised when a load case is not found in the model.	-8002
OsCreateLoadFailed	Raised when creating a load fails.	-8004
OsLoadExists	Raised when a load already exists in the model.	-8029
OsSeismicCodeNotFound	Raised when a seismic code is not found in the model.	-8034
OsInvalidLoadDefId	Raised when an invalid load definition ID is provided.	-8039
OsInvalidLoadCombName	Raised when an invalid load combination name is provided.	-8040
OsInvalidLoadCombCategory	Raised when an invalid load combination category is provided.	-8041

Error Class	Description	Error Code
OsBeamForcesNotLoaded	Raised when beam forces are not loaded in the results.	-9004
OsNoGnlResultSet	Raised when no GNL (Geometric Nonlinear) result set is found.	-9911
OsMemberUpdated	Raised when a member is updated in the model.	3007
OsPmemberNotFound	Raised when a physical member is not found in the model.	-5701

Table

Contents

- OSTable

class openstaadpy.os_analytical.ostable.**OSTable** [source]

Bases: object

AddTable(*report_no: int, table_name: str, row_count: int, col_count: int*) [source]

Add or create a report data table and returns the table no for a specified table.

Parameters:

- **report_no** (*int*) – Report number.
- **table_name** (*str*) – Report Table Name.
- **row_count** (*int*) – row count of the specified table.
- **col_count** (*int*) – column count of the specified table.

Returns:

Return table number

Return 0 if Create table error.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
```

CreateReport(*report_title: str*) [source]

Creates a report with the specified title.

Parameters:

report_title (*str*) – A string containing the title of the report.

Returns:

Return report number

Return 0 if Create Report error.

Return type:

int

Example

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
```

DeleteReport(*report_no: int*) [source]

Delete an existing report.

Parameters:

report_no (*int*) – Report number.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.DeleteReport(report_no)
```

DeleteTable(*report_no: int, table_no: int*) [source]

Delete an existing table from report data.

Parameters:

- **report_no** (*int*) – Report number.
- **table_no** (*int*) – Table number in specified report.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.DeleteTable(report_no, table_number)
```

GetCellValue(*report_no: int*, *table_no: int*, *row_no: int*, *col_no: int*) [source]

Get cell values for a specified table.

Parameters:

- **report_no** (*int*) – Report number.
- **table_no** (*int*) – Table number in specified report.
- **row_no** (*int*) – Row number for specified table, start from 1.
- **col_no** (*int*) – Column number for specified table, start from 1.

Returns:

Returns cell value for specified cell in the table

Return type:

string

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> table_count = staad_obj.Table.SetCellValue(report_no, table_numb
>>> cell_value = staad_obj.Table.GetCellValue(report_no, table_numbe
```

GetReportCount() [source]

Returns the number of reports created.

Returns:

Return the number of reports created.

Return type:

int

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_count = staad_obj.Table.GetReportCount()
```

GetTableCount(*report_no: int*) [source]

Get table count in a specified report

Parameters:

report_no (*int*) – Report number.

Returns:

Returns table count in a specified report

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> table_count = staad_obj.Table.GetTableCount(report_no)
```

RenameTable(*report_no: int, table_no: int, table_name: str*)

Rename an existing report table. [source]

Parameters:

- **report_no** (*int*) – Report number.

- **table_no** (*int*) – Table number in specified report.
- **table_name** (*str*) – Report Table Name.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.RenameTable(report_no, table_number, "Table2")
```

ResizeTable(*report_no: int, table_no: int, row_nos: int, col_nos: int*) [source]

Resize existing table by increasing numbers of rows and columns from report data.

Parameters:

- **report_no** (*int*) – Report number.
- **table_no** (*int*) – Table number in specified report.
- **row_nos** (*int*) – row count for specified table.
- **col_nos** (*int*) – column count for specified table.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.ResizeTable(report_no, table_number, 12, 6)
```

SaveReport(*report_no: int*) [source]

Saves the specified report along with all its tables.

Parameters:

report_no (*int*) – Providing the report number identifying the STAAD report which is to be saved

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> staad_obj.Table.SaveReport(report_no)
```

SaveReportAll() [source]

Saves all the reports created.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> staad_obj.Table.SaveReportAll()
```

SaveTable(*report_no: int, table_no: int*) [source]

Save a report table.

Parameters:

- **report_no** (*int*) – Report number.
- **TableNo** (*int*) – Table number in specified report.

Return type:

None

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.SaveTable(report_no, table_number)
```

SetCellTextBold(*report_no: int, table_no: int, row_no: int, col_no: int*) [source]

Sets the text in a given row and column to bold.

Parameters:

- **report_no** (*int*) – Report number.
- **table_no** (*int*) – Table number in specified report.
- **row_no** (*int*) – Row number for specified table, start from 1.
- **col_no** (*int*) – Column number for specified table, start from 1.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.SetCellValue(report_no, table_number, 1, 5, "abc
>>> staad_obj.Table.SetCellTextBold(report_no, table_number, 1, 5)
```

SetCellTextColor(*report_no: int, table_no: int, row_no: int, col_no: int, red: int, green: int, blue: int*) [source]

Sets the color of the text to be displayed in a particular cell. By default, the color is always set to black.

Parameters:

- **report_no** (*int*) – Report number.
- **table_no** (*int*) – Table number in specified report.
- **row_no** (*int*) – Row number for specified table, start from 1.
- **col_no** (*int*) – Column number for specified table, start from 1.

- **red** (*int*) – An integer between 0 and 255 that represents the intensity of red in the color for the text.
- **green** (*int*) – An integer between 0 and 255 that represents the intensity of green in the color for the text.
- **blue** (*int*) – An integer between 0 and 255 that represents the intensity of blue in the color for the text.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.SetCellValue(report_no, table_number, 1, 5, "abc
>>> staad_obj.Table.SetCellTextColor(report_no, table_number, 1, 5,
```

SetCellTextHorzAlignment(*report_no: int, table_no: int, row_no: int, col_no: int, align: int*) [source]

Sets the text in a particular row and column to a specified horizontal alignment. By default, all the text is right justified.

Parameters:

- **report_no** (*int*) – Report number.
- **table_no** (*int*) – Table number in specified report.
- **row_no** (*int*) – Row number for specified table, start from 1.
- **col_no** (*int*) – Column number for specified table, start from 1.
- **align** (*int*) – align Sets the text in a particular row and column to a specified horizontal alignment. The possible values are:0 = left; 1 = center; 2 = right

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.SetCellValue(report_no, table_number, 1, 5, "abc
>>> staad_obj.Table.SetCellTextHorzAlignment(report_no, table_number
```

SetCellTextItalic(*report_no: int, table_no: int, row_no: int, col_no: int*) [source]

Italicizes the text in a given row and column.

Parameters:

- **report_no** (*int*) – Report number.
- **table_no** (*int*) – Table number in specified report.
- **row_no** (*int*) – Row number for specified table, start from 1.
- **col_no** (*int*) – Column number for specified table, start from 1.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.SetCellValue(report_no, table_number, 1, 5, "abc
>>> staad_obj.Table.SetCellTextItalic(report_no, table_number, 1, 5)
```

SetCellTextSize(*report_no: int, table_no: int, row_no: int, col_no: int, size: float*) [source]

Sets the text in a particular row and column to a certain font size. The font sizes are equivalent to the ones used in Microsoft Word.

Parameters:

- **report_no** (*int*) – Report number.
- **table_no** (*int*) – Table number in specified report.
- **row_no** (*int*) – Row number for specified table, start from 1.
- **col_no** (*int*) – Column number for specified table, start from 1.
- **size** (*float*) – Containing the size of the font to set the text to.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.SetCellValue(report_no, table_number, 1, 5, "abc
>>> staad_obj.Table.SetCellTextSize(report_no, table_number, 1, 5, 1
```

SetCellTextSizeAll(*report_no: int, table_no: int, size: float*)

[source]

Sets the text in the entire table to FontSize. The font sizes are equivalent to the ones used in Microsoft Word.

Parameters:

- **report_no** (*int*) – Report number.
- **table_no** (*int*) – Table number in specified report.
- **size** (*float*) – Containing the size of the font to set the text to.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 11,
>>> staad_obj.Table.SetCellTextSizeAll(report_no, table_number, 5)
```

SetCellTextUnderline(*report_no: int, table_no: int, row_no: int, col_no: int*)

[source]

Underlines the text in a given row and column.

Parameters:

- **report_no** (*int*) – Report number.
- **table_no** (*int*) – Table number in specified report.
- **row_no** (*int*) – Row number for specified table, start from 1.
- **col_no** (*int*) – Column number for specified table, start from 1.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.SetCellValue(report_no, table_number, 1, 5, "abc
>>> staad_obj.Table.SetCellTextUnderline(report_no, table_number, 1,
```

SetCellTextVertAlignment(*report_no: int, table_no: int, row_no: int, col_no: int, align: int*) [source]

Sets the text in a particular row and column to a specified vertical alignment. By default, all the text is top justified.

Parameters:

- **report_no** (*int*) – Report number.
- **table_no** (*int*) – Table number in specified report.
- **row_no** (*int*) – Row number for specified table, start from 1.
- **col_no** (*int*) – Column number for specified table, start from 1.
- **align** (*int*) – align Sets the text in a particular row and column to a specified vertical alignment. The possible values are: 0 = top; 4 = center; 8 = bottom

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.SetCellValue(report_no, table_number, 1, 5, "abc
>>> staad_obj.Table.SetCellTextVertAlignment(report_no, table_number
```

SetCellValue(*report_no: int, table_no: int, row_no: int, col_no: int, value: str*) [source]

Set cell values for a specified table.

Parameters:

- **report_no** (*int*) – Report number.

- **table_no** (*int*) – Table number in specified report.
- **row_no** (*int*) – Row number for specified table, start from 1.
- **col_no** (*int*) – Column number for specified table, start from 1.
- **value** (*str*) – Set cell value for specified cell in the table.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.SetCellValue(report_no, table_number, 1, 5, "abc
```

SetColumnHeader(*report_no: int, table_no: int, col_no: int, header: str*) [source]

Sets column header for a specified table.

Parameters:

- **report_no** (*int*) – Report number.
- **table_no** (*int*) – Table number in specified report.
- **col_no** (*int*) – Column number for specified table, start from 1.
- **header** (*str*) – Column header for a specified column in table.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.SetColumnHeader(report_no, table_number, 1, "hea
```

SetColumnUnitString(*report_no: int, table_no: int, col_no: int, unit_string: str*) [source]

Sets unit for the specific column of a specified table.

Parameters:

- **report_no** (*int*) – Report number.
- **table_no** (*int*) – Table number in specified report.
- **col_no** (*int*) – Column number for specified table, start from 1.
- **unit_string** (*str*) – Set unit for the specific column of a specified table.

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.SetColumnUnitString(report_no, table_number, 1,
```

SetRowHeader(*report_no: int, table_no: int, row_no: int, header: str*) [source]

Sets row header for the specific row of a specified table.

Parameters:

- **report_no** (*int*) – Report number.
- **table_no** (*int*) – Table number in specified report.
- **row_no** (*int*) – Row number for specified table, start from 1
- **header** (*str*) – Set header for the specific row of a specified table

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> report_no = staad_obj.Table.CreateReport("testreport")
>>> table_number = staad_obj.Table.AddTable(report_no, "Table1", 10,
>>> staad_obj.Table.SetRowHeader(report_no, table_number, 1, "row1")
```

__init__(*staadObj*) [source]

Design

Contents

- OSDesign

class openstaadpy.os_analytical.osdesign.OSDesign [source]

Bases: object

AssignDesignCommand(*design_ref_id: int, design_command_name: str, design_command_value: str, member_ids: list | int*) [source]

Assign a design command to specified members in a design brief.

Parameters:

- **design_ref_id** (*int*) – Design brief reference ID.
- **design_command_name** (*str*) – Name of the design command.
- **design_command_value** (*str*) – Value for the design command.
- **member_ids** (*list of int*) – List of member numbers.

Returns:

0 if successful, -1 otherwise.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> ref_id = staad_obj.Design.CreateDesignBrief(1001)
>>> result = staad_obj.Design.AssignDesignCommand(ref_id, "CHECK COD
```

AssignDesignGroup(*design_ref_id: int, design_group_name: str, design_group_value: str, same_as_member: int, member_ids: list | int*) [source]

Assign physical members to a design group using a design command.

Parameters:

- **design_ref_id** (*int*) – Design brief reference ID.
- **design_group_name** (*str*) – Name of the design group.
- **design_group_value** (*str*) – Value for the design group.
- **same_as_member** (*int*) – Reference member for the group.
- **member_ids** (*list of int*) – List of member numbers.

Returns:

0 if successful, -1 otherwise.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> ref_id = staad_obj.Design.CreateDesignBrief(1001)
>>> result = staad_obj.Design.AssignDesignGroup(ref_id, "scSteelGrou
```

`AssignDesignParameter(design_ref_id: int, design_param: str, design_param_value: str, member_ids: list | int)` [source]

Assign design parameters to a specified design brief.

Parameters:

- **design_ref_id** (*int*) – Design brief reference ID.
- **design_param** (*str*) – Name of the design parameter.
- **design_param_value** (*str*) – Value for the design parameter.
- **member_ids** (*list of int or int*) – List of member numbers.

Returns:

0 if successful, -1 otherwise.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> ref_id = staad_obj.Design.CreateDesignBrief(1001)
>>> result = staad_obj.Design.AssignDesignParameter(ref_id, "BEAM",
```

CreateDesignBrief(*design_code: int*) [source]

Create a new design brief with the specified design code.

Parameters:

design_code (*int*) – Design code index.

Returns:

Reference ID of the created design brief.

Return type:

int

Example

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> ref_id = staad_obj.Design.CreateDesignBrief(1001)
```

GetDesignBriefCode(*design_ref_id: int*) [source]

Get the design code for a specified design brief.

Parameters:

design_ref_id (*int*) – Design brief reference ID.

Returns:

Design code.

Return type:

int

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> ref_id = staad_obj.Design.CreateDesignBrief(1001)
>>> result = staad_obj.Design.GetDesignBriefCode(ref_id)
```

GetMemberDesignParameters(*design_ref_id: int*, *member_no: int*)

Get the design parameters for a specified member in a design brief. [source]

Parameters:

- **design_ref_id** (*int*) – Design brief reference ID.
- **member_no** (*int*) – Member number.

Returns:

Dictionary containing:

status : *int*

Return code from COM (0 success, -1 failure or other).

count : *int | None*

Number of parameters available for the member (from COM object's Count property) or None.

_raw : *COM object*

Original COM object (StaadPro.MembSteelDgnParams).

parameters : *dict[str, list]*

Mapping of parameter name -> [value, unit, description, default]. If any component is unavailable it is set to None. Parameter names are returned exactly as provided by COM.

Return type:

dict

Examples

```
>>> from openstaadpy import os_analytical
>>> staad_obj = os_analytical.connect()
>>> brief_id = 1
>>> params = staad_obj.Design.GetMemberDesignParameters(brief_id, 1)
```

__init__(*staadObj*) [source]